

```
function 할일카드({ 제목, 끝났나, 눌렀을때 }) {  
  const [열림, 열기] = useState(false);  
  useEffect(() => { fetch("/api/할일").then((r) => r.json()); }, []);  
  return (  
    <li className={끝났나 ? "끝" : ""} onClick={눌렀을때}>  
      {제목}  
    </li>  
  );  
}
```

1 권 · 화면 그리기

React

화면을 컴포넌트로 쪼개고, state 로 바꾸고,
서버에서 받아 옵니다.

브라우저 화면이 어떻게 바뀌는지를 매번 적어 두었습니다.

6

단원

26

개념

127

직접 해보기

70

문제

차례

00	개발환경 만들기	11
01	설치하고 켜기	12
01	React 시작하기	31
01	왜 React를 쓰나	32
02	첫 화면 띄우기	39
03	화면이 다시 그려진다	47
04	에러와 경고 읽기	54
02	JSX	71
01	JSX란 무엇인가	72
02	중괄호로 값 넣기	83
03	속성 넣기	95
04	태그 규칙	109
05	여러 줄로 쓰기	120
03	컴포넌트와 props	141
01	컴포넌트 만들기	142
02	props 전달하기	151
03	props 구조분해와 기본값	162
04	children	172
05	컴포넌트 쪼개기	181
04	state와 이벤트	201
01	이벤트 붙이기	202
02	useState 시작	213
03	state가 바뀌면 다시 그린다	224
04	여러 state 다루기	230
05	이전 값으로 갱신하기	232

06	state 규칙	243
05	조건부 렌더링과 리스트	265
01	조건부 렌더링	266
02	리스트 그리기	272
03	key	282
04	걸러서 그리기	296
05	비었을 때와 falsy 함정	310
부록 가	연습문제·종합연습 정답	323
부록 나	심화문제 정답	353

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

준비물 — Node.js 하나

이 자료는 처음부터 끝까지 **실제 개발에서 쓰는 방식(Vite 프로젝트)** 으로 진행합니다. 그래서 첫 시간에 Node.js 를 확인하고 프로젝트를 한 번 켜는 것으로 시작합니다.

JS자료에서 이미 설치했다면 그대로 쓰면 됩니다. 없다면 <https://nodejs.org> 에서 **왼쪽 LTS** 버튼으로 받아 설치하세요. 설치 후 **터미널을 전부 닫고 새로 여세요.**

확인하는 법 — 터미널에 이렇게 칩니다.

```
node -v
npm -v
```

v22.14.0 / 11.16.0 처럼 숫자가 나오면 됩니다.

★ **Node 는 22 이상이어야 합니다.** 앞의 숫자가 22보다 작으면(예: v18.x, v20.x) 이 자료의 도구가 안 됩니다. <https://nodejs.org> 에서 LTS 를 새로 받아 설치하세요.

시작하는 다섯 줄

1. 터미널을 연다
2. React자료/실습프로젝트 폴더로 간다
3. npm install 라고 친다 ← 맨 처음 한 번만. 10초쯤 걸립니다
4. npm run dev 라고 친다
5. 브라우저에서 터미널에 뜬 주소(<http://localhost:5173/>)를 연다

3번은 맨 처음 한 번만 하면 됩니다. 이 자료가 쓰는 도구들을 인터넷에서 받아 오는 명령입니다. 받고 나면 node_modules 라는 폴더가 생기는데, 그게 도구 창고입니다. 그 뒤로는 4번(npm run dev)만 하면 됩니다.

인터넷이 필요합니다. 실습실에서 안 되면 강사에게 말하세요.

막히면 [실습프로젝트/src/00_개발환경_만들기/개념01_설치하고_켜기.md](#) 를 보세요. 터미널을 한 번도 안 써 본 사람 기준으로 썼습니다.

VS Code

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받으세요. **파일 → 폴더 열기**로 실습프로젝트 폴더를 여는 편이 훨씬 편합니다. **Ctrl + `** 로 터미널을 열면 이미 그 폴더에 서 있습니다.

3분만 만져 보고 오세요

화면이 떴으면 왼쪽 목록에서 [01_React_시작하기 / 개념01 왜 React를 쓰나](#) 를 고르세요.

같은 화면이 두 개 있습니다.

- ① 은 여러분이 JS자료에서 하던 방식으로 만든 것
- ② 는 React 로 만든 것

두 '답기' 버튼을 번갈아 눌러 보세요. **화면은 똑같이 움직입니다.** 그런데 F12 → Console 을 보면 이렇게 찍힙니다.

[JS 방식] 내가 직접 고친 곳: 3군데
[React 방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)

이 차이가 이 자료의 전부입니다. 지금 코드가 하나도 안 읽혀도 괜찮습니다.

그리고 마지막(13단원)에는 **할 일 목록·장바구니·사용자 검색·메모장** 네 개를 직접 만듭니다. JS자료 13단원에서 만들었던 할 일 목록을, 이번에는 React 로 다시 만듭니다.

폴더 순서

단원	제목	어디에
00	개발환경 만들기	실습프로젝트 (읽는 문서 .md)
01	React 시작하기	실습프로젝트
02	JSX	실습프로젝트
03	컴포넌트와 props	실습프로젝트
04	state와 이벤트	실습프로젝트
05	조건부 렌더링과 리스트	실습프로젝트
06	폼과 입력	실습프로젝트
07	state 설계와 불변성	실습프로젝트
08	파일 나누기와 스타일링	실습프로젝트 (개념01은 읽는 문서)

09	useEffect와 API	실습프로젝트
10	useRef와 커스텀 훅	실습프로젝트
11	React Router	실습프로젝트
12	Context와 useReducer	실습프로젝트
13	종합 프로젝트	실습프로젝트
14	서버와 연동	실습프로젝트 (터미널 두 개)
15	부록 — 타입스크립트	실습프로젝트 (안 해도 됩니다)

앞 단원을 건너뛰면 뒤 단원이 안 읽힙니다. 순서대로 가세요.

14단원은 서버를 같이 켭니다

`useForm` 으로 폼을 만들고, `FormData` 로 파일을 올리고, 내가 만든 서버에 붙습니다. **PART 3(백엔드)로 넘어가는 다리입니다.**

```
터미널 ①  cd 실습프로젝트          → npm run dev
터미널 ②  cd 실습프로젝트/14단원_서버 → node 서버.js
```

서버는 설치할 것이 없습니다. Node 만 있으면 `node 서버.js` 로 바로 돍니다. 서버 코드는 읽지 않아도 됩니다. 만드는 법은 PART 3 에서 배웁니다.

자료는 전부 실습프로젝트/src 안에 있습니다

```
React자료/
├─ _검증도구/           ← 강사용. 학생은 볼 필요 없습니다
├─ 실습프로젝트/       ← ★ 자료는 전부 여기
│  ├─ 14단원_서버/     ← 14단원에서 같이 켜는 연습용 서버
│  └─ src/
│     ├─ _ui/           ← 자료가 쓰는 부품 (읽지 않아도 됩니다)
│     ├─ 00_개발환경_만들기/ ← 첫 시간에 읽는 문서 한 장 (.md)
│     ├─ 01_React_시작하기/
│     ├─ 02_JSX/
│     ├─ ...
│     └─ 15_부록_타입스크립트/
```

왼쪽 목록에 저절로 나타납니다. 단원 폴더에 `.jsx` 를 만들면 등록 없이 잡힙니다.

이렇게 진행합니다

[터미널] npm run dev 를 켜 둔 채로 그대로 둡니다 ← 건드리지 않습니다
[VS Code] src 안의 예제 파일을 열어 읽고 고칩니다
[브라우저] 왼쪽에서 예제를 고르고, 저장하면 저절로 바뀌는 화면을 봅니다

- **새로고침(F5)은 누르지 마세요.** 왼쪽에서 고른 예제가 풀립니다. 저장이면 충분합니다.
- **화면이 안 바뀌면 터미널부터 보세요.** 문법이 틀리면 브라우저까지 가지도 못합니다.

분량

단원	16개 (00 준비 + 01~14 + 부록 1개)
수업 파일	107개
개념 섹션	439개
 직접 해보기	367개
연습문제	220문항
표준 진행 시간	71.5시간

자세한 시간표와 강사용 안내는 [수업_진행_가이드.md](#) 에 있습니다.

OPEN 00_개발환경_만들기

- ## 01 설치하고 켜기

설치하고 켜기

이 자료의 첫 문서입니다. 그리고 가장 많이 막히는 곳이기도 합니다. 터미널을 한 번도 안 써 본 사람 기준으로 썼습니다. 천천히 따라오세요.

여기만 넘기면 그 다음부터는 계속 코드만 봅니다. 화면이 한 번 뜨고 나면 이 문서는 다시 볼 일이 거의 없습니다. (막힐 때 5부 '자주 나는 에러' 만 다시 펴게 됩니다)

먼저 결론부터

섹션 1

여러분이 할 일은 다섯 줄입니다.

1. 터미널을 연다
2. 실습프로젝트 폴더로 간다
3. `npm install` 라고 친다 ← 맨 처음 한 번만
4. `npm run dev` 라고 친다
5. 브라우저에서 `http://localhost:5173/` 을 연다

자료에 실습프로젝트 폴더는 이미 만들어져 있습니다. 그래서 프로젝트를 만들 필요는 없습니다. 다만 이 자료가 쓰는 도구들은 인터넷에서 받아야 해서, 맨 처음 한 번 `npm install` 을 칩니다(10초쯤). 그 뒤로는 `npm run dev` 만 하면 됩니다. 자세한 이야기는 3부에 있습니다.

“직접 처음부터 만들어 보고 싶다” 는 분을 위한 안내는 4부에 따로 있습니다. 안 해도 됩니다. 궁금할 때만 보세요.

0부 — 준비: Node.js 가 있는지 확인

섹션 2

JS자료를 하면서 Node.js 를 이미 설치했습니다. 그대로 씁니다. 없다면 <<https://nodejs.org>> 에서 왼쪽 LTS 버튼으로 받아 설치하세요. 설치가 끝나면 열려 있던 터미널을 전부 닫고 새로 여세요. (안 그러면 못 찾습니다)

터미널을 열고 아래 두 줄을 쳐서 확인합니다.

```
node -v
npm -v
```

이 자료를 만든 컴퓨터에서는 이렇게 나왔습니다.

v24.18.0
11.16.0

★ 앞의 숫자가 22 이상이면 됩니다. v18 이나 v20 이 나오면 이 자료의 도구가 안 도니,
<<https://nodejs.org>> 에서 LTS 를 새로 받아 설치하고 터미널을 다시 여세요.

여러분 것과 숫자가 달라도 괜찮습니다. 다만 위에 적은 대로 앞 숫자는 22 이상이어야 합니다. npm 은
Node.js 를 설치하면 같이 딸려 옵니다. 따로 설치하는 게 아닙니다.

숫자가 아니라 빨간 글씨가 나온다면 5부의 [에러 ㉠] 을 보세요.

1부 — 터미널이 무엇인가

섹션 3

터미널이란

컴퓨터에 글자로 명령을 내리는 창입니다. 마우스로 아이콘을 누르는 대신, 글자를 치고 Enter 를 누릅니다.

JS자료 01~09단원에서 node 개념01_....js 를 썼던 그 창이 맞습니다. 이미 써 본 적이 있습니다. 새로운
물건이 아닙니다.

터미널에는 늘 “지금 어느 폴더에 있는가” 라는 것이 있습니다. 이게 터미널의 전부라고 해도 됩니다.

C:\Users\이름\Desktop\React자료\실습프로젝트>

이렇게 폴더 경로가 보이고 그 뒤에 커서가 깜빡입니다. 지금 이 폴더 안에 서 있다 는 뜻입니다. 명령을 치면
그 폴더를 기준으로 실행됩니다.

그래서 터미널에서 실수하는 것의 절반은 엉뚱한 폴더에서 명령을 친 것입니다. 명령이 안 먹으면 맨 앞에
보이는 폴더 경로부터 확인하세요.

여는 방법 세 가지

[방법 A] VS Code 에서 열기 — 가장 권합니다

- 1 VS Code 를 켜다
- 2 파일 → 폴더 열기 → 실습프로젝트 폴더를 고른다
- 3 Ctrl + ` 를 누른다 (숫자 1 왼쪽의 물결 키)

아래쪽에 터미널이 열립니다. 이미 실습프로젝트 폴더에 서 있는 상태입니다. 폴더를 옮길 필요가 없습니다.
코드 고치는 곳과 터미널이 한 창에 있어서 제일 편합니다.

> Ctrl + ` 가 안 먹으면 상단 메뉴 터미널 → 새 터미널 을 쓰세요.

[방법 B] 탐색기에서 열기

- 1 탐색기에서 실습프로젝트 폴더 안으로 들어간다
- 2 빈 곳에 **Shift + 마우스 오른쪽 클릭**
- 3 “여기에 PowerShell 창 열기” 또는 “터미널에서 열기” 를 고른다

이것도 그 폴더에 서 있는 상태로 열립니다.

[방법 C] 시작 메뉴에서 열고 직접 이동하기

시작 메뉴에 터미널 또는 PowerShell 을 쳐서 엽니다. 이렇게 열면 엉뚱한 폴더(보통 C:\Users\이름)에 서 있습니다. 직접 옮겨야 합니다.

```
cd C:\Users\이름\Desktop\React자료\실습프로젝트
```

cd 는 change directory, “그 폴더로 가라” 는 뜻입니다. (JS자료 README 에서 쓴 그것입니다)

> 경로를 손으로 치지 마세요. 탐색기 주소창을 한 번 클릭하면 경로가 글자로 바뀝니다. > 그걸 Ctrl+C 로 복사해서 cd 뒤에 붙여 넣으면 됩니다. > 터미널에 붙여 넣기는 마우스 오른쪽 클릭 또는 Ctrl+V 입니다.

> 폴더 이름에 공백이 있으면 따옴표로 감싸세요. > cd “C:\내 문서\React자료\실습프로젝트” > 안 감싸면 공백 앞까지만 폴더 이름으로 읽습니다.

> 경로에 한글이 있어도 괜찮습니다. 이 자료의 경로에는 React자료, 실습프로젝트 > 처럼 한글이 들어 있는데, 실제로 확인해 보니 아무 문제 없이 돌아갔습니다.

제대로 왔는지 확인하기

이동했으면 지금 폴더에 무엇이 있는지 봅니다.

```
dir
```

(PowerShell 에서는 ls 도 됩니다)

목록에 package.json 이 보이면 제대로 온 것입니다. 안 보이면 아직 다른 폴더에 있는 것입니다. 이게 5부 [에러 ②] 의 원인입니다.

2부 — 실습프로젝트 켜기

섹션 4

```
npm run dev
```

터미널에 이렇게 칩니다.

```
npm run dev
```

이렇게 나옵니다. 실제로 돌려서 그대로 옮긴 것입니다.

```
> -@0.0.0 dev
> vite

VITE v8.2.1 ready in 354 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

한 줄씩 무슨 뜻인지 봅시다.

| 줄 | 뜻 | |---|---| | > -@0.0.0 dev | 지금 dev 라는 명령을 실행한다. - 는 이 프로젝트의 이름입니다 (package.json 에 그렇게 적혀 있습니다). 0.0.0 은 버전입니다. | | > vite | dev 의 실제 내용은 vite 를 켜는 것이다 | | VITE v8.2.1 ready in 354 ms | Vite 8.2.1 이 0.354초 만에 준비됐다 | | Local: http://localhost:5173/ | 이 주소를 브라우저에 치세요 | | Network: use --host to expose | 같은 공유기에 있는 다른 기기(휴대폰 등)에서도 보고 싶으면 옵션이 필요하다는 안내. 지금은 필요 없습니다. | ready in ... 의 숫자와 v8.2.1 의 숫자는 여러분 것과 다를 수 있습니다. 정상입니다.

> > -@0.0.0 dev 의 - 가 이상해 보일 수 있습니다. > 이 실습프로젝트의 이름을 그냥 - 로 지어 둔 것뿐입니다. 고장이 아닙니다.

화면 보기

터미널에 나온 주소를 브라우저 주소창에 칩니다.

```
http://localhost:5173/
```

> **localhost 가 뭔가요?** > “이 컴퓨터” 라는 뜻의 주소입니다. 인터넷 어딘가가 아니라 **여러분 컴퓨터 안**을 가리킵니다. > 뒤의 5173 은 **포트 번호**입니다. 한 컴퓨터 안에서 여러 서버가 동시에 돌 수 있어서, > 번호로 구분합니다. 5173 은 Vite 가 기본으로 쓰는 번호입니다. > 이 주소는 여러분 컴퓨터에서만 열립니다. 인터넷에 올라간 것이 아닙니다.

> **더 쉬운 방법:** 터미널에 뜬 http://localhost:5173/ 글자 위에서 > Ctrl 을 누른 채 클릭하면 브라우저가 바로 열립니다. (VS Code 터미널에서 됩니다)

화면이 이렇게 나옵니다.

- **왼쪽:** 단원별 예제 목록 - **오른쪽:** 고른 예제의 내용

왼쪽 목록에서 예제를 누르면 오른쪽이 바뀝니다. 앞으로 01~15단원의 모든 예제를 이 화면에서 고릅니다.

> 단원 폴더에 .jsx 파일을 새로 만들면 왼쪽 목록에 **저절로** 나타납니다. > 등록하는 작업은 없습니다.

F5 를 안 눌러도 됩니다 — 직접 확인해 보세요

고친 뒤에 새로고침(F5)을 누를 필요가 없습니다. 직접 확인해 보는 게 제일 빠릅니다.

- 1 브라우저는 열어 둔 채로
- 2 VS Code 에서 `src/01_React_시작하기/` 의 아무 예제 파일을 연다
- 3 화면에 보이는 글자 하나를 아무렇게나 고친다
- 4 `Ctrl + S` 로 저장한다
- 5 **브라우저를 보세요.** 손도 안 댔는데 바뀌어 있습니다.

이것을 HMR(Hot Module Replacement) 이라고 부릅니다. 이름은 몰라도 됩니다. “저장하면 바뀐 곳만 갈아 끼운다” 가 전부입니다.

오히려 F5 는 누르지 마세요. 왼쪽에서 고른 예제가 풀려서 맨 앞으로 돌아갑니다.

끄는 법 — `Ctrl + C`

터미널 창을 클릭해서 고른 다음 `Ctrl + C` 를 누릅니다.

터미널에 따라 이런 것이 나올 수 있습니다.

```
일괄 작업을 끝내시겠습니까 (Y/N)?
```

(영어 설정이면 `Terminate batch job (Y/N)?`)

Y 를 치고 **Enter** 를 누르면 꺼집니다. 아무것도 안 물어보고 그냥 커서가 돌아오는 터미널도 있습니다. 그것도 정상입니다.

꺼졌는지 확인하려면 브라우저에서 **F5** 를 눌러 보세요. “**사이트에 연결할 수 없음**” 이 나오면 제대로 꺼진 것입니다.

터미널을 닫으면 화면도 안 뜹니다

이걸 꼭 기억하세요.

`npm run dev` 는 명령을 한 번 실행하고 끝나는 것이 아닙니다. **터미널이 그 서버를 계속 붙들고 있는** 것입니다. 그래서 터미널을 닫으면 서버도 같이 꺼집니다.

- 터미널 창을 닫는다 → 서버가 꺼진다 → 브라우저에서 “**사이트에 연결할 수 없음**” - 컴퓨터를 껐다 켜다
→ 당연히 꺼져 있습니다. **매번 다시 npm run dev** 를 해야 합니다.

공부하는 동안의 모양은 이렇습니다.

```
[터미널] npm run dev 를 켜 둔 채로 그대로 둡니다 ← 건드리지 않습니다
[VS Code] 코드를 고치고 Ctrl+S 로 저장합니다
[브라우저] 저절로 바뀐 화면을 봅니다
```

터미널에서는 아무것도 안 칩니다. 처음 한 번 `npm run dev` 만 치면 끝입니다.

> 터미널에 뭘 더 치고 싶은데 커서가 안 움직인다면, 서버가 그 창을 쓰고 있어서 그렇습니다. > 터미널 창을 하나 더 여세요. VS Code 터미널 오른쪽 위의 + 버튼입니다.

3부 — 폴더 안을 하나씩

섹션 5

실습프로젝트 폴더를 열면 이렇게 생겼습니다.

```

실습프로젝트/
├─ index.html          ← 모든 것의 시작점
├─ package.json        ← 이 프로젝트의 설명서
├─ package-lock.json   ← 설치된 정확한 버전 기록
├─ vite.config.js      ← Vite 설정
├─ .gitignore          ← 남에게 줄 때 빼고 줄 것들의 목록
├─ .oxlintrc.json      ← 코드 검사 도구 설정 (안 봐도 됩니다)
├─ tsconfig.json       ← 타입스크립트 설정 (15단원 부록에서만 씁니다)
├─ README.md           ← Vite 가 만들어 준 안내문 (영어입니다. 안 봐도 됩니다)
├─ node_modules/       ← 받아 온 도구들 (열어 볼 필요 없음)
├─ public/             ← 그대로 내보낼 파일들
├─ dist/               ← npm run build 결과물 (지금은 없어도 됩니다)
├─ src/                ← ★ 여러분이 만지는 곳
│  └─ main.jsx          ← 우리 코드의 첫 줄
│  └─ App.jsx           ← 예제 고르기 화면
│  └─ index.css         ← 공용 스타일
│  └─ _ui/              ← 자료가 쓰는 부품 (Summary, ErrorBox)
│  └─ 01_React_시작하기/ ← ★ 바로 다음에 볼 곳
│  └─ 09_useEffect와_API/
└─ ...
  
```

이 중에서 여러분이 실제로 여는 것은 **src** 안의 단원 폴더뿐입니다. 나머지는 “이런 게 있구나” 정도로 한 번 읽고 넘어가면 됩니다. 아래에서 하나씩 봅니다.

index.html — 모든 것의 시작점

브라우저가 가장 먼저 읽는 파일입니다. 열어 보면 이렇습니다.

```

<body>
  <div id="root"></div> <script type="module" src="/src/main.jsx"></script>
</body>
  
```

이 두 줄이 화면이 시작되는 곳입니다. **01단원 개념02**에서 자세히 봅니다.

- <div id="root"></div> — React 가 그림을 그릴 빈 자리 - <script ...> 한 줄 — 그 자리를 채울 코드를 부르는 줄

눈여겨볼 것 두 가지입니다.

1 script 가 딱 한 줄입니다. React 를 불러오는 줄도, JSX 를 번역하는 줄도 여기 없습니다. 그 일은 전부 Vite 가 합니다. 여러분이 화면에서 보는 모든 글자는 이 한 줄에서 시작합니다.

2 type="module" 이 붙었습니다. "이 파일은 import 를 쓰는 파일이다" 라는 표시입니다. import 가 정확히 무엇인지는 08단원에서 배웁니다. 그때까지는 "다른 파일에서 가져온다" 정도로 보면 됩니다.

> 이 파일은 src 폴더 밖에 있습니다. 이상해 보이지만 Vite 의 규칙입니다. > Vite 는 프로젝트 맨 위의 index.html 부터 읽기 시작합니다.

src/main.jsx — 우리 코드의 첫 줄

index.html 이 부르는 파일입니다. 여기서부터가 우리 코드입니다.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.querySelector("#root")).render(
  <StrictMode> <App /> </StrictMode>
);
```

createRoot(...).render(...) — 01단원 개념02에서 배울 그 두 줄입니다. ReactDOM. 이 앞에 안 붙은 것만 다릅니다. import 로 직접 꺼내 왔기 때문입니다.

<StrictMode> 는 처음 보는 것입니다. 개발 중에만 켜지는 검사 모드입니다. 문제가 될 만한 코드를 미리 찾아 주려고 React 가 넣어 둔 것입니다.

> ★ 중요 — 콘솔에 같은 줄이 두 번 찍히는 이유가 이것입니다. > StrictMode 는 문제를 찾아내려고 컴포넌트를 일부러 두 번 실행합니다. > 그래서 console.log 가 두 번 찍힙니다. 정상입니다. 여러분이 잘못된 게 아닙니다. > 실제 사용자에게 내보낼 때(npm run build)는 한 번만 실행됩니다. > 자세한 이야기는 09단원 개념02에서 합니다.

이 파일은 고치지 마세요. 자료가 미리 만들어 둔 틀입니다.

src/App.jsx — 예제 고르기 화면

왼쪽 목록과 오른쪽 내용을 만드는 파일입니다.

이 파일이 src/08_파일_나누기와_스타일링/ 같은 단원 폴더를 자동으로 훑어서 목록을 만듭니다. 그래서 파일을 만들기만 하면 목록에 저절로 나타납니다. 어디에 등록하는 작업이 없습니다.

메뉴에 뜨는 이름은 파일 이름에서 _ 를 공백으로 바꾼 것입니다.

개념03_import와_export.jsx → 개념03 import와 export

규칙이 두 가지 있습니다.

1 단원 폴더 바로 아래의 `.jsx` 만 목록에 잡힙니다. `src/08_파일_나누기와_스타일링/_부품/Card.jsx` 처럼 하위 폴더에 넣은 것은 안 잡힙니다. `import` 해서 쓸 부품 파일은 하위 폴더에 두면 됩니다. 개념04에서 그렇게 합니다.

2 파일이 `export default` 로 컴포넌트를 하나 내보내야 합니다. (개념03)

이 파일도 고치지 마세요.

`src/index.css` — 공용 스타일

예제의 모양(회색 상자·흰 칸·노란 안내문·파란 정리 상자)을 정하는 파일입니다. `.demo`, `.output`, `.guide`, `.summary` 같은 이름이 들어 있습니다.

모든 예제가 이 한 파일을 함께 씁니다. 버튼 색을 바꾸고 싶으면 여기 한 곳만 고치면 전부 바뀝니다. 이것이 08단원에서 다시 이야기할 “고칠 곳이 한 곳” 입니다.

CSS 는 이 자료에서 가르치지 않습니다. 예제에서 `className="demo"` 라고만 쓰면 같은 모양이 나온다는 것만 알면 됩니다.

이 파일은 `main.jsx` 의 `import "./index.css"`; 한 줄로 불러 옵니다. 그래서 모든 예제에 자동으로 적용됩니다. 각 예제에서 다시 `import` 할 필요가 없습니다. CSS 를 넣는 방법은 개념05에서 배웁니다.

이 파일도 고치지 마세요.

`src/_ui/` — 자료가 쓰는 부품

- `Summary.jsx` — 각 예제 맨 아래의 파란 정리 상자 - `ErrorBox.jsx` — 예제 하나가 터져도 나머지는 볼 수 있게 감싸 주는 상자

폴더 이름이 `_` 로 시작해서 숫자 두 자리로 시작하는 단원 폴더가 아니므로 왼쪽 목록에 안 잡힙니다.

`Summary` 는 여러분도 씁니다. 개념03부터 예제 맨 아래에서 쓰게 됩니다.

이 폴더도 고치지 마세요.

`package.json` — 이 프로젝트의 설명서

프로젝트에서 가장 중요한 파일입니다. 열어 보면 이렇습니다.

아래 목록에 나오는 이름 중 뒤의 둘은 08단원·11단원에서 배웁니다. 지금은 이름만 보세요.

```
{
  "name": "-",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
```

```

    },
    "dependencies": {
      "react": "^19.2.8",
      "react-dom": "^19.2.8",
      "react-router-dom": "^7.18.2",
      "styled-components": "^6.5.3"
    },
    "devDependencies": {
      "vite": "^8.2.0",
      ...
    }
  }
}

```

세 부분만 알면 됩니다.

scripts — 짧게 부를 이름을 정해 두는 곳

“dev”: “vite” 라고 적혀 있으면, `npm run dev` 라고 쳤을 때 `vite` 가 실행됩니다. **긴 명령에 별명을 붙여 두는 것입니다.**

이 프로젝트에는 이런 것들이 있습니다.

| 명령 | 하는 일 | ---|---| | `npm run dev` | 개발용 서버를 켜다 < 이것만 쓰면 됩니다 | | `npm run build` | 다 만든 앱을 `dist` 폴더로 묶는다 | | `npm run preview` | 묶은 결과를 미리 본다 | | `npm run lint` | 코드에 이상한 곳이 없는지 훑어본다 (안 써도 됩니다) | | `npm run typecheck` | 타입을 검사한다 (15단원 부록에서만 씁니다) |

`npm run` 뒤에 오는 이름은 **scripts 에 적힌 것만** 됩니다. 없는 이름을 치면 `Missing script: “...”` 가 나옵니다.

dependencies — 앱이 돌아가는 데 꼭 필요한 도구들

React 19.2.8, react-dom 19.2.8, react-router-dom 7.18.2, styled-components 6.5.3 이 적혀 있습니다. 위의 둘은 각각 11단원과 08단원에서 배웁니다. 지금은 이름만 보세요.

- `react-router-dom` 은 화면 여러 개를 오갈 때 쓰는 도구입니다. **11단원**에서 배웁니다. - `styled-components` 는 CSS 를 쓰는 방법 중 하나입니다. **08단원 개념05**에서 잠깐 봅니다.

둘 다 `package.json` 에 적혀 있으니 맨 처음 `npm install` 할 때 함께 받아옵니다. 그때 따로 설치하지 않아도 됩니다. ★ `create-vite` 로 새로 만든 프로젝트에는 이 둘이 **없습니다**. 우리 자료가 넣어 둔 것입니다. 이 둘은 11단원과 08단원에서 배웁니다. 여러분이 만든 프로젝트에서 쓰려면 `npm i react-router-dom styled-components` 를 해야 합니다. **적혀 있다는 것과 받아들여 있다는 것은 다릅니다**. 이걸 “필요하다” 는 목록일 뿐이고, 실제 파일은 `node_modules` 에 있습니다. 그 목록대로 받아 오는 명령이 `npm install` 입니다.

> ^19.2.8 앞의 ^ 는 “19.2.8 이상, 20 미만이면 된다” 는 뜻입니다. > 그래서 나중에 설치하면 19.2.9 가 깔릴 수도 있습니다. 정상입니다.

devDependencies — 만드는 동안만 필요한 도구들

Vite 가 여기 있습니다. Vite 는 개발할 때만 쓰고, 완성된 앱에는 안 들어갑니다. `dependencies` 와 나뉘어 있는 이유가 그것입니다.

package-lock.json — 정확한 버전 기록

`package.json` 이 `^19.2.8`(범위)이라면, 이 파일은 **실제로 받은 정확한 버전**을 적어 둡니다. 다른 컴퓨터에서 `npm install` 을 해도 **똑같은 버전**이 깔리게 해 줍니다.

손으로 고치지 않습니다. `npm` 이 알아서 씁니다. 열어 볼 일도 없습니다.

node_modules/ — 열어 볼 필요 없습니다

`npm install` 이 받아 온 도구들이 전부 들어 있는 폴더입니다.

React 하나만 쓰는 가장 작은 프로젝트를 실제로 만들어서 재 봤더니 이랬습니다.

||| |---|---| |폴더 개수 | 22개 | |파일 개수 | 437개 | |용량 | 약 62 MB |

우리 실습프로젝트는 `react-router-dom`(11단원)과 `TypeScript`(15단원)까지 들어 있어서 더 큼니다.

세 가지만 기억하세요.

- 1** **열어 볼 필요가 없습니다.** 남이 만든 코드가 들어 있는 창고입니다.
- 2** **손으로 고치지 마세요.** 고쳐도 `npm install` 을 다시 하면 원래대로 돌아갑니다.
- 3** **지워도 됩니다.** `npm install` 을 다시 하면 그대로 다시 만들어집니다. 그래서 다른 사람에게 프로젝트를 줄 때는 이 폴더를 빼고 줍니다. `.gitignore` 에 `node_modules` 가 적혀 있는 것이 그 이유입니다.

vite.config.js — Vite 설정

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
});
```

`plugins: [react()]` 한 줄이 전부입니다. “이 프로젝트는 React 를 쓰니 JSX 를 번역해 줘” 라는 뜻입니다. 브라우저는 JSX 를 모릅니다. 이 한 줄이 “브라우저에 보내기 전에 평범한 자바스크립트로 바꿔 놓아라” 를 말합니다. 02단원에서 그 바뀐 모습을 직접 봅니다.

이 자료에서는 이 파일을 손댈 일이 없습니다.

public/ — 그대로 내보낼 파일들

여기 있는 파일은 **번역이나 묶기 없이 그대로** 브라우저가 받아 갑니다. `favicon.svg`(탭에 뜨는 작은 아이콘)와 `오프라인_대체.js`가 들어 있습니다.

`public/오프라인_대체.js`는 실습실 인터넷이 막혀 09단원 데이터 예제가 안 될 때 쓰는 것입니다. `index.html`의 그 줄을 감싼 주석만 지우면 켜집니다. 자세한 것은 09단원에서 안내합니다.

dist/ — 빌드 결과

`npm run build`를 하면 생기는 폴더입니다. 안 만들어져 있어도 정상입니다. 인터넷에 올릴 때 이 폴더를 통째로 올립니다. 지금은 신경 쓰지 않아도 됩니다.

열리는 순서 정리

파일이 이 순서로 이어집니다. 이 흐름만 잡으면 프로젝트 구조는 다 본 것입니다.

```

브라우저가 localhost:5173 을 연다
↓
index.html 을 읽는다          ← <div id="root"> 와 script 한 줄
↓
src/main.jsx 를 읽는다        ← createRoot(...).render(<App />)
↓
src/App.jsx 를 그린다         ← 왼쪽 목록 + 오른쪽 내용
↓
목록에서 고른 예제 파일을 그린다 ← 여러분이 만드는 파일

```

여러분이 만지는 곳은 맨 아래 한 칸뿐입니다. 위 세 칸은 그대로 둡니다.

4부 — 직접 만들어 보고 싶다면 (안 해도 됩니다)

섹션 6

이 부분은 건너뛰어도 됩니다. 실습프로젝트가 이미 준비돼 있기 때문입니다. “나중에 내 프로젝트를 만들려면 어떻게 하나”가 궁금할 때 보세요.

명령은 세 줄입니다.

```

npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev

```

★ 실습프로젝트 폴더 안에서는 하지 마세요. 바탕화면 같은 다른 곳에서 하세요.

첫 줄 — `npm create vite@latest`

```

npm create vite@latest my-react-app -- --template react

```

한 토막씩 뜯어보면 이렇습니다.

| 토막 | 뜻 | |---|---| | `npm create` | “이런 프로젝트를 만들어 주는 도구를 받아서 실행해 줘” | |
`vite@latest` | 그 도구는 `create-vite` 이고, **최신 버전**을 쓴다 | | `my-react-app` | 만들 폴더 이름 (원하는
이름으로 바뀌어도 됩니다) | | `--` | 여기부터는 `npm` 이 아니라 **뒤 도구에게 주는 옵션**이라는 표시 | | `--`
`template react` | React 짜임새로 만들어 달라 |

`--template react` 를 빼면 “어떤 걸로 만들까요?” 하고 질문이 하나씩 나옵니다. 방향키로 골라도 되지만,
처음에는 옵션을 다 적어 주는 편이 헛갈리지 않습니다.

실제로 돌린 결과입니다.

```
> npx
> create-vite my-react-app --template react

|
◇ Scaffolding project in C:\Users\이름\Desktop\my-react-app...
|
└─ Done. Now run:

    cd my-react-app
    npm install
    npm run dev
```

`Scaffolding` 은 “뼈대를 세운다” 는 뜻입니다. 그리고 **다음에 뭘 하면 되는지 친절하게 알려 줍니다.** 그대로
따라 하면 됩니다.

> **처음 실행하면 이런 질문이 나올 수 있습니다.** > `` ` > Need to install the following packages: >
`create-vite@9.1.2` > Ok to proceed? (y) > `` ` > `create-vite` 라는 도구를 받아도 되냐고 묻는
것입니다. `y` 를 치고 Enter 를 누르세요.

만들어진 파일 목록입니다. 실제로 만들어서 확인했습니다.

```
my-react-app/
├─ .gitignore
├─ .oxlintrc.json
├─ index.html
├─ package.json
├─ README.md
├─ vite.config.js
├─ public/
│   └─ favicon.svg
│   └─ icons.svg
└─ src/
    └─ App.css
    └─ App.jsx
    └─ index.css
    └─ main.jsx
```

```
└─ assets/
  └─ hero.png
  └─ react.svg
  └─ vite.svg
```

`node_modules` 는 아직 없습니다. 다음 줄이 그걸 만듭니다.

> ★ 이 목록은 도구 버전에 따라 조금씩 다를 수 있습니다. > Vite 는 자주 바뀝니다. 여러분이 할 때는 파일이 몇 개 더 있거나 없을 수 있고, > `.oxlintrc.json` 대신 `eslint.config.js` 가 있을 수도 있습니다. > 파일 이름이 조금 달라도 놀라지 마세요. > `index.html`, `package.json`, `src/main.jsx`, `src/App.jsx` 이 네 개는 항상 있습니다.

둘째 줄 — `cd` 하고 `npm install`

```
cd my-react-app
npm install
```

`npm install` 은 `package.json` 에 적힌 목록대로 도구를 받아 오는 명령입니다. 받은 것은 `node_modules` 폴더에 들어갑니다. 인터넷이 필요합니다.

실제 결과입니다.

```
added 24 packages, and audited 25 packages in 9s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- `added 24 packages` — 24개를 받았다 - `9 packages are looking for funding` — 만든 사람들이 후원을 받고 있다는 안내. **에러가 아닙니다.** 무시해도 됩니다. - `found 0 vulnerabilities` — 알려진 보안 문제 0건

> 시간과 개수는 인터넷 속도에 따라 다릅니다. 30초 넘게 걸려도 정상입니다. > `npm warn ...` 같은 노란 줄이 몇 개 나와도 마지막에 `added ...` 가 나오면 성공입니다.

`npm install` 은 처음 한 번만 하면 됩니다. 그 뒤로는 `npm run dev` 만 하면 됩니다. (`package.json` 에 도구를 추가했을 때만 다시 합니다)

셋째 줄 — `npm run dev`

```
npm run dev
```



```
> my-react-app@0.0.0 dev
> vite

VITE v8.2.1 ready in 1016 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

주소를 열면 Vite 기본 화면이 나옵니다. 여기까지 나오면 성공입니다. 끄는 것은 똑같이 **Ctrl + C** 입니다.

확인된 버전 — 그리고 다룰 수 있다는 것

이 자료를 만든 날, 위 과정을 실제로 돌려서 나온 버전입니다.

```
|| 버전 || ---|---| Node.js | 24.18.0 || npm | 11.16.0 || create-vite | 9.1.2 || Vite | 8.2.1
(package.json 표기는 ^8.2.0) || React / react-dom | 19.2.8 || styled-components | 6.5.3 ||
@vitejs/plugin-react | 6.0.5 |
```

> ★ 이 숫자들은 계속 바뀝니다. > @latest 는 “그때그때 가장 새것” 이라는 뜻이라, 여러분이 할 때는 다른 숫자가 나옵니다. > 버전이 다르다고 뭘 잘못된 게 아닙니다. 화면이 나오면 성공입니다. > 자료에 적힌 숫자와 여러분 화면의 숫자가 다른 것은 정상입니다. > > 그리고 이 실습프로젝트는 직접 만든 것이 아니라 자료가 준 것이므로, > 09~15단원을 진행하는 동안에는 버전이 바뀌지 않습니다. 안심하고 쓰세요.

5부 — 자주 나는 에러 대처

섹션 7

여기가 실제로 학생이 멈추는 지점들입니다. 순서대로 확인하세요.

[에러 ①] npm 을 못 찾습니다

터미널에 이런 게 나옵니다.

```
'npm'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는
배치 파일이 아닙니다.
```

영어 설정이면 이렇게 나옵니다.

```
The term 'npm' is not recognized as a name of a cmdlet, function,
script file, or executable program.
```

뜻: “npm 이라는 게 뭔지 모르겠다” — 즉 컴퓨터가 npm 을 못 찾은 것입니다.

차례로 확인하세요.

- 1 **터미널을 닫고 새로 여세요.** 이게 제일 혼합니다. Node.js 를 설치하기 **전에** 열어 둔 터미널은 설치 사실을 모릅니다. VS Code 를 쓴다면 VS Code 자체를 껐다 켜세요.
- 2 `node -v` 도 같이 안 되면 **Node.js 가 아직 없는 것**입니다. <<https://nodejs.org>> 에서 왼쪽 **LTS** 를 받아 설치하고, 터미널을 새로 여세요.
- 3 `node -v` 는 되는데 `npm -v` 만 안 되는 경우는 드뭅니다. 그때는 Node.js 를 지웠다가 다시 설치하는 것이 가장 빠릅니다.

[에러 ②] `package.json` 을 못 찾습니다

```
npm error code ENOENT
npm error syscall open
npm error path C:\Users\이름\Desktop\package.json
npm error enoent Could not read package.json: Error: ENOENT: no such file or
directory, open 'C:\Users\이름\Desktop\package.json'
```

뜻: **폴더가 틀렸습니다.** 지금 서 있는 폴더에 `package.json` 이 없습니다.

`ENOENT` 는 “그런 파일이나 폴더가 없다”(Error NO ENTry) 는 뜻입니다. `npm error path` 줄을 보세요.

npm 이 어디를 뒤졌는지 그대로 적혀 있습니다. 위 예에서는 `Desktop` 을 뒤졌습니다. **실습프로젝트** 안으로 안 들어간 것입니다.

해결

```
cd 실습프로젝트
dir
```

`dir` 목록에 `package.json` 이 보이면 제대로 온 것입니다. 그다음 `npm run dev`.

> 1부 [방법 A] 처럼 **VS Code** 로 **실습프로젝트** 폴더를 열고 터미널을 열면 > 이 에러는 아예 안 납니다. 가장 확실한 예방책입니다.

[에러 ③] `vite` 를 못 찾습니다

```
> my-react-app@0.0.0 dev
> vite

'vite'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는
배치 파일이 아닙니다.
```

뜻: `package.json` 은 찾았는데 `npm install` 을 안 했습니다. `node_modules` 가 없어서 `vite` 라는 도구 자체가 없는 것입니다.

해결: `npm install` 을 먼저 하고 `npm run dev`.

> 자료의 실습프로젝트는 이미 설치돼 있어서 이 에러가 안 납니다. > 4부에서 직접 만들었을 때 `npm install` 을 건너뛰면 이게 납니다.

[에러 ④] 포트가 이미 쓰이고 있습니다

```
Port 5173 is in use, trying another one...

VITE v8.2.1 ready in 236 ms

→ Local: http://localhost:5174/
```

이건 에러가 아닙니다. Vite 가 알아서 옆 번호(5174)로 옮겨 준 것입니다.

해야 할 일: 5173 이 아니라 터미널에 실제로 뜬 주소를 여세요. 브라우저 주소창에 5173 이 남아 있으면 옛날 서버나 빈 화면이 보입니다.

왜 이렇게 되나: `npm run dev` 를 켜 둔 터미널을 잊고 또 켜는 것입니다. 서버가 두 개 돌고 있습니다. 정리하려면:

1 예전 터미널 창을 찾아 `Ctrl + C` 로 끕니다

2 그래도 모르겠으면 터미널 창을 전부 닫고 다시 하나만 켜세요

> 5173, 5174 로 계속 늘어난다면 서버가 여러 개 떠 있는 것입니다. > 컴퓨터를 껐다 켜면 전부 정리됩니다.

[에러 ⑤] `Ctrl + C` 로 안 꺼집니다

먼저 확인할 것: 터미널 창을 한 번 클릭했나요? 브라우저나 VS Code 편집기 쪽에 커서가 있으면 `Ctrl + C` 는 그냥 “복사” 가 됩니다. 터미널 안을 클릭해서 고른 다음 눌러야 합니다.

그래도 안 꺼지면 순서대로 해 보세요.

1 `Ctrl + C` 를 한번 더 누릅니다

2 일괄 작업을 끝내시겠습니까 (Y/N)? 가 떠 있다면 Y 를 치고 Enter (이걸 안 하면 안 꺼진 채로 남아 있습니다)

3 그래도 안 되면 터미널 창을 그냥 닫습니다. 서버도 같이 꺼집니다. VS Code 터미널이면 오른쪽 위 휴지통 아이콘을 누르세요. (X 는 숨기기일 뿐이라 서버가 계속 돛니다. 휴지통이 진짜 끄기입니다)

4 최후의 수단은 컴퓨터를 껐다 켜는 것입니다. 남은 서버가 전부 정리됩니다.

꺼졌는지 확인: 브라우저에서 F5 → “사이트에 연결할 수 없음” 이 나오면 꺼진 것입니다.

[에러 ⑥] 경로에 한글이나 공백이 있습니다

한글은 괜찮습니다. 이 자료의 경로에는 실습프로젝트, 09_useEffect와_API 처럼 한글이 들어 있고, 실제로 확인해 보니 `cd` 도 `npm run dev` 도 아무 문제 없이 돌아갔습니다.

문제가 되는 것은 이런 경우입니다.

- 1 공백이 있는 경로를 따옴표 없이 `cd` 했다 `` cd C:\내 문서\실습프로젝트 ← 안 됩니다 (“내” 까지만 읽습니다) cd “C:\내 문서\실습프로젝트” ← 이렇게 감싸세요 ``
- 2 터미널에서 한글이 ????? 나 깨진 글자로 보인다 폴더는 잘 찾아갑니다. 보이는 것만 깨진 것입니다. 그래도 신경 쓰이면 터미널에 `chcp 65001` 을 한 번 치세요.
- 3 경로에 # 이나 % 같은 기호가 있다 드물게 문제가 됩니다. 폴더 이름을 바꾸거나, 프로젝트를 C:\react 연습 처럼 짧고 단순한 경로로 옮기는 것이 가장 빠릅니다.

> 가장 확실한 예방책: 경로를 손으로 치지 말고 > 탐색기에서 폴더를 우클릭 → 터미널에서 열기 를 쓰세요. `cd` 자체가 필요 없어집니다.

[에러 ㉠] 브라우저에 아무것도 안 나옵니다

“사이트에 연결할 수 없음” 이 나온다면

서버가 안 켜져 있는 것입니다. 터미널을 보세요.

- 터미널을 닫았다 → 다시 열고 `npm run dev` - Ctrl + C 로 꺾다 → 다시 `npm run dev` - 컴퓨터를 꺾다 꺾다 → 다시 `npm run dev`

하얀 화면만 나온다면

F12 를 눌러 **Console** 탭의 맨 위 빨간 줄을 보세요. 대부분 코드 문제입니다. 예제 하나가 터진 경우라면 그 예제만 빨간 상자가 되고 나머지는 멀쩡합니다.

주소가 다른 경우

브라우저 주소창이 5173 인데 터미널에는 5174 라고 떠 있는 경우입니다.

에러 ㉡ · 를 보세요.

[에러 ㉢] 저장했는데 화면이 안 바뀝니다

- 1 정말 저장했나요? VS Code 탭 제목 옆에 점(●)이 남아 있으면 저장 전입니다.
- 2 터미널이 아직 살아 있나요? 꺼졌으면 아무리 저장해도 안 바뀝니다.
- 3 터미널에 빨간 글씨가 떴나요? 문법이 깨지면 Vite 가 거기서 멈춥니다. 고치고 저장하면 저절로 다시 돌아갑니다.
- 4 그래도 안 바뀌면 브라우저에서 F5 를 한 번 눌러 보세요. HMR 이 가끔 못 따라오는 경우가 있습니다. 흔하지는 않습니다.

그래도 안 될 때 — 마지막 순서

1. 터미널 창을 전부 닫는다
2. VS Code 를 켜다 켜다
3. 파일 → 폴더 열기 → 실습프로젝트 를 연다
4. `Ctrl + `` 로 터미널을 연다
5. `dir` 로 `package.json` 이 보이는지 확인한다
6. `npm run dev`

이 순서로 하면 위 에러의 대부분이 해결됩니다.

다음에 할 일

섹션 8

여기까지 하면 화면이 떴을 것입니다. 이제 코드를 봅니다.

이 문서 다음부터는 `.md` 가 아니라 `.jsx` 파일입니다. 실습프로젝트/src/01_React_시작하기/ 폴더 안에 있고, 왼쪽 목록에서 골라서 봅니다.

`npm run dev` 를 켜 둔 채로 **01단원 개념01** 을 여세요. 이 터미널은 수업이 끝날 때까지 켜 둡니다.

정리

섹션 9

- 1 여러분이 할 일은 다섯 줄입니다. 터미널 열기 → 실습프로젝트 로 가기 → `npm install`(맨 처음 한 번) → `npm run dev` → `http://localhost:5173/` 열기. 프로젝트 폴더는 이미 만들어져 있습니다.
- 2 터미널은 “지금 어느 폴더에 서 있는가” 가 전부입니다. VS Code 로 실습프로젝트 폴더를 열고 `Ctrl + ``` 로 터미널을 여는 것이 가장 안전합니다. `cd`` 를 안 쳐도 됩니다.
- 3 `npm run dev` 는 끝나는 명령이 아니라 서버를 켜 두는 명령입니다. 끄는 것은 `Ctrl + C`(터미널을 먼저 클릭). 터미널을 닫으면 화면도 안 뜹니다. 컴퓨터를 껐다 켜면 매번 다시 켜야 합니다.
- 4 저장하면 F5 없이 화면이 바뀝니다. state 도 살아남습니다.
- 5 `index.html` 이 시작점이고, 거기서 `src/main.jsx` 를 부르고, `main.jsx` 가 `src/App.jsx`(예제 고르기 화면)를 그리고, 그 안에 여러분이 만든 예제가 들어갑니다. 여러분이 만지는 곳은 맨 아래 한 칸뿐입니다.
- 6 `package.json` 은 프로젝트 설명서입니다. `scripts` 는 명령의 별명(`npm run dev` → `vite`), `dependencies` 는 필요한 도구 목록입니다. `node_modules` 는 그 목록대로 받아 온 창고(약 62 MB)이고 열어 볼 필요도, 고칠 필요도 없습니다.
- 7 `main.jsx` 의 `<StrictMode>` 때문에 개발 중에는 컴포넌트가 두 번 실행되어 `console.log` 가 두 번 찍힙니다. 정상입니다.

- 00
- 8
- src/_ui/ 안의 Summary·ErrorBox·그려진 뒤 는 자료가 미리 만들어 둔 부품입니다. 예제에서 불러다 쓰지만 고칠 일은 없습니다.
- 9
- 직접 만들려면 `npm create vite@latest 이름 -- --template react` → `cd 이름` → `npm install` → `npm run dev`. 버전과 파일 목록은 계속 바뀌므로 자료와 달라도 정상입니다.
- 10
- 막히면 5부를 보세요. `npm` 못 찾을 → 터미널 새로 열기. `package.json` 못 찾을 → 폴더가 틀림. `vite` 못 찾을 → `npm install` 안 함. 포트 사용 중 → 터미널에 뜬 주소를 열기.

00단원 마무리 개발환경 만들기

자주 넘어지는 자리

- 1
- `Ctrl + C` 로 dev 서버 끄기 / 포트가 이미 쓰일 때 대처 |

단 원 01

React 시작하기

OPEN 01_React_시작하기

```
const jsItems = [];  
그려진뒤("#js자리", (js자리) => {  
  js자리.innerHTML =  
  '<span id="jsCount">0</span>개' +
```

01 왜 React를 쓰나

02 첫 화면 띄우기

03 화면이 다시 그려진다

04 에러와 경고 읽기

- 연습문제

왜 React를 쓰나

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

여러분은 JS자료 13단원에서 할 일 목록을 직접 만들었습니다. 그때 이런 코드를 계속 썼습니다.

```
const li = document.createElement("li");
li.textContent = 할일;
list.appendChild(li);
countBox.textContent = todos.length;
```

값이 하나 바뀔 때마다 “화면의 어느 부분을 어떻게 고칠지”를 여러분이 전부 직접 적어 왔습니다.

React 는 그 일을 대신해 줍니다. 여러분은 “데이터가 이러면 화면은 이렇게 생겼다” 만 적습니다.

이 파일에서는 그 차이만 눈으로 봅니다. ★ 코드는 아직 이해하지 않아도 됩니다. 02단원부터 하나씩 배웁니다.

지금까지 하던 방식

섹션 1

데모 ① 의 안쪽을 만드는 코드입니다. React 를 한 줄도 쓰지 않습니다. 버튼도 숫자도 목록도 전부 손으로 만듭니다. JS자료에서 배운 것만 씁니다.

useState 는 04단원에서 배웁니다. 지금은 결과만 보세요.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";
import 그려진뒤 from "../_ui/그려진뒤.js";

const jsItems = [];
```

그려진뒤 는 이 자료가 미리 만들어 둔 도우미입니다. 읽지 않아도 됩니다. React 가 ① 상자를 다 그린 뒤에 이 안을 한 번 실행해 줍니다. (왜 기다려야 하는지는 개념02 섹션 4에서 다룹니다)

```
그려진뒤("#js자리", (js자리) => {
  js자리.innerHTML =
    '<button id="jsBtn">담기</button> ' +
    '<span id="jsCount">0</span>개' +
    '<ul id="jsList"></ul>';
```


★ 이 파일에서 React 없이 만든 화면은 위 네 줄이 전부입니다. “이 자리에 이런 태그를 넣어라” 를 내가 직접 적었습니다. 상자 제목(“① 지금까지 하던 방식”)은 여기서도 React 가 그렸습니다. 아래 섹션 2와 비교해 보세요.

```
const jsBtn = document.querySelector("#jsBtn");
const jsCount = document.querySelector("#jsCount");
const jsList = document.querySelector("#jsList");

jsBtn.addEventListener("click", () => {
  jsItems.push("아메리카노");
});
```

화면을 고치는 일을 ‘내가’ 한다 — 세 군데를 직접 손봐야 합니다.

```
const li = document.createElement("li"); // (1) 새 항목 만들고
li.textContent = jsItems.length + "번째 아메리카노";
jsList.appendChild(li); // (2) 목록에 붙이고
jsCount.textContent = jsItems.length; // (3) 개수도 따로 고치고 console.log("[JS
방식] 내가 직접 고친 곳: 3군데");
});
});
```

지금은 잘 돌아갑니다. 그런데 여기에 요구사항이 하나만 더 붙는다고 해 봅시다.

“3개가 넘으면 버튼을 잠그고, 목록 위에 ‘많이 담았습니다’ 를 띄워 주세요”

그러면 위 콜백 안에 고칠 곳이 3군데에서 5군데로 늘어납니다. 그리고 ‘빼기’ 버튼을 만들면, 그 안에도 똑같은 5군데를 또 적어야 합니다.

화면을 고치는 코드가 여기저기 흩어지면서 “지금 화면이 어떤 상태인지” 를 코드만 보고는 알 수 없게 됩니다. JS자료 13단원 종합03에서 이미 조금 느껴 봤을 것입니다.

▹ 직접 해보기

1

데모 ① 의 담기 버튼을 다섯 번 눌러 보세요. 콘솔에 “[JS 방식] ...” 이 다섯 번 찍힙니다. 화면을 고치기 위해 몇 군데를 손댔는지 세어 보세요.

React 방식

섹션 2

아래가 데모 ② 를 만드는 코드입니다. 같은 화면입니다. 문법은 02~05단원에서 배웁니다. 지금은 ‘모양’ 만 보세요.

```
function CoffeeCounter() {
```

items 는 ‘지금 담긴 목록’ 이라는 데이터입니다. useState 는 04단원에서 배웁니다. 지금은 ‘데이터를 담아 두는 것’ 정도로 보세요.

```
const [items, setItems] = useState([]);

function handleClick() {
  setItems([...items, "아메리카노"]); // 데이터만 바꿉니다 console.log("[React
  방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)");
}
```

아래는 “데이터가 이러면 화면은 이렇게 생겼다” 를 적은 것입니다. 화면을 ‘고치는’ 코드가 아니라, 화면을 ‘그리는’ 코드입니다.

```
return (
  <div>
    { /* onClick 은 04단원에서 배웁니다. "누르면 이 함수" 라는 뜻입니다. */ }
    <button onClick={handleClick}>담기</button>{" "}
    { /* ↑ {" "} 는 '여기 띄어쓰기 한 칸' 이라는 뜻입니다. 02단원에서 배웁니다.
      JSX 는 태그 옆의 줄바꿈을 지워 버려서, 이게 없으면 "담기0개" 로 붙습니다.
      데모 ① 은 HTML 이라 줄바꿈이 그대로 한 칸이 됩니다. 그 차이를 맞추는
      것입니다. */ }
    <span>{items.length}</span>개
    <ul>
      {items.map((item, index) => (
        <li key={index}>
          {index + 1}번째 {item}
        </li>
      ))}
    </ul> </div>
  );
}
```

섹션 1에서는 innerHTML 로 태그를 손수 적었습니다. 여기서는 “이 자리에 CoffeeCounter 가 들어간다” 라고 말하기만 합니다.

두 데모의 버튼을 번갈아 눌러 보세요. 화면은 똑같이 동작합니다.

```
F12 콘솔    [JS 방식] 내가 직접 고친 곳: 3군데
             [React 방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)
```

아까 그 요구사항(“3개가 넘으면 ...”)을 React 에 붙이면 어떻게 될까요? `return` 안의 화면 설명에 한 줄만 더 적으면 끝입니다. ‘담기’ 든 ‘빼기’ 든 어디서 데이터를 바꿔도 화면은 알아서 맞춰줍니다.

▫ 직접 해보기 2

데모 ② 의 담기 버튼도 다섯 번 눌러 보세요. 데모 ① 과 화면이 같게 나오는지 확인하세요.

명령형과 선언형

섹션 3

두 방식을 부르는 이름이 있습니다.

명령형 · “어떻게 고칠지” 를 순서대로 시킨다 ← 지금까지의 방식

li 를 만들어라 → 글자를 넣어라 → 목록에 붙여라 → 개수도 고쳐라

선언형 · “무엇이 보여야 하는지” 만 적는다 ← React 의 방식

담긴 게 3개면, 화면에는 3줄과 숫자 3이 보인다

식당에 비유하면 이렇습니다. 명령형 — 주방에 들어가 불을 켜고 물을 올리고 면을 넣으라고 일일이 시킨다
선언형 — “라면 하나요” 라고 말한다

비유는 여기까지입니다. 실제로 일어나는 일은 이렇습니다. 여러분이 데이터를 바꾸면 → React 가 화면 설명을 다시 읽고 → 바뀐 부분만 찾아서 → 그 부분만 실제 화면에 반영합니다.

‘바뀐 부분만 찾는’ 일을 React 가 대신하기 때문에, 여러분은 화면을 고치는 코드를 한 줄도 쓰지 않습니다.

섹션 1과 섹션 2를 다시 비교해 보세요. 섹션 1 — 화면을 만드는 코드 3줄 + 바뀔 때마다 고치는 코드 4줄
섹션 2 — 화면을 만드는 코드(return 안) 있고, 고치는 코드 0줄 처음 만드는 수고는 비슷합니다. 차이는 ‘그 뒤’ 에 생깁니다.

➤ 직접 해보기

3

아래 문장이 명령형인지 선언형인지 생각해 보세요. “장바구니가 비어 있으면 ‘텅 비었습니다’ 가 보인다”
(정답은 파일 맨 아래에)

React 가 대신해 주지 않는 것

섹션 4

React 를 쓴다고 자바스크립트를 덜 쓰지 않습니다. 오히려 더 씁니다. 방금 위 코드에도 JS자료에서 배운 것이 그대로 들어 있습니다.

```
const menu = ["아메리카노", "라떼", "케이크"];

console.log(menu.map((name) => name.length));
```

F12 콘솔 [5, 2, 3]

← 08단원 map 입니다. React 의 목록 그리기가 바로 이 map 입니다.

```
console.log([...menu, "삼각김밥"]);
```

F12 콘솔 ['아메리카노', '라떼', '케이크', '삼각김밥']

← 09단원 스프레드입니다. 위 handleClick 에서 쓴 것과 같은 문법입니다.

```
const first = menu[0];
console.log(`오늘의 메뉴는 ${first} 입니다`);
```

F12 콘솔 오늘의 메뉴는 아메리카노 입니다

← 02단원 템플릿 리터럴입니다.

즉 React 는 자바스크립트를 대신하는 것이 아니라, 자바스크립트 위에 얹어 쓰는 도구입니다. JS자료에서 배운 map · filter · 구조분해 · 스프레드를 이 자료에서 계속 씁니다.

▸ 직접 해보기 4

menu 에서 이름이 3글자인 것만 걸러 콘솔에 찍어 보세요. (08단원 filter)

이 자료를 어떻게 진행하나

섹션 5

00단원 · 이미 끝냈습니다. Node 를 깔고 이 프로젝트를 켜습니다.

01~07단원 · React 자체를 배웁니다. 화면 그리기 · JSX · 컴포넌트 · state · 폼.

전부 지금처럼 왼쪽 목록에서 예제를 골라 봅니다.

08단원 · 파일을 나누고 CSS 를 붙입니다. 그리고 지금 쓰는 도구(Vite)가

그동안 무엇을 대신해 주고 있었는지 밝힙니다.

09~13단원 · 데이터 받아오기, 화면 여러 개, 종합 프로젝트까지 갑니다.

14단원 · ★ 내가 만든 서버에 붙입니다. 폼·파일 업로드까지 갑니다.

여기서 프론트와 백엔드가 처음으로 만납니다.

15단원 · 부록입니다. 타입스크립트를 맛만 봅니다. 안 해도 됩니다.

```
console.log("이 파일은 여기까지입니다. 개념02로 가세요.");
```

F12 콘솔 이 파일은 여기까지입니다. 개념02로 가세요.

직접 해보기 5

F12 → Console 을 열어 이 파일이 찍은 줄을 전부 세어 보세요. (버튼을 누르기 전 기준입니다)

자주 하는 오해

섹션 6

이 파일은 아직 문법을 배우기 전이라 '실수' 대신 '오해' 를 모았습니다.

오해 1 · “React 를 쓰면 자바스크립트를 몰라도 된다”

→ 반대입니다. 섹션 4에서 본 것처럼 map · 스프레드 · 구조분해를 계속 씁니다.

JS자료가 잘 기억나지 않으면 08·09단원을 한 번 다시 보고 오세요.

오해 2 · “React 가 더 빠르다”

→ 화면을 바꾸는 속도만 보면 직접 고치는 쪽이 더 빠를 때도 많습니다.

React 의 이득은 속도가 아니라 '고칠 곳이 한 곳으로 모인다' 는 데 있습니다.
화면이 복잡해질수록 이득이 커집니다.

오해 3 · “React 는 HTML 안에 자바스크립트를 넣은 것이다”

→ 순서가 반대입니다. 자바스크립트 안에 화면 설명을 넣은 것입니다.

이 문법을 JSX 라고 하며 02단원에서 배웁니다.

오해 4 · “콘솔에 노란 경고가 있으니 뭔가 잘못됐다”

→ 이 자료에서는 아래 줄이 나옵니다. 정상입니다. 무시하세요.

Download the React DevTools ... ← 늘 나옵니다
You are using the in-browser Babel ... ← 02단원 예제를 한 번이라도 열면 나옵니다
이유는 개념04에서 설명합니다.

직접 해보기 정답

1 3군데입니다. li 만들기 / 목록에 붙이기 / 개수 고치기. 콘솔에 “[JS 방식] 내가 직접 고친 곳: 3군데” 가 다섯 번 찍힙니다.

2 화면 모양과 동작이 같습니다.

F12 콘솔 [React 방식] 내가 직접 고친 곳: 0군데 (데이터만 바꿈)

→ 화면이 같아도 코드가 다릅니다. React 쪽에는

`createElement · appendChild · textContent` 가 한 번도 안 나옵니다.

3 선언형입니다. “비어 있으면 이렇게 보인다” 는 결과를 말한 것입니다. 명령형으로 바꾸면 “비면 문구를 만들어 붙이고, 안 비면 지워라” 가 됩니다.

4 `console.log(menu.filter((name) => name.length === 3));` // 콘솔: ['케이크']

→ menu 는 ["아메리카노", "라떼", "케이크"] 세 개뿐이라 “삼각김밥” 은 애초에 없습니다.

길이가 3인 것만 걸러도 남는 것은 “케이크” 하나입니다.

5) 4줄입니다.

— 5, 2, 3

— ‘아메리카노’, ‘라떼’, ‘케이크’, ‘삼각김밥’

오늘의 메뉴는 아메리카노 입니다 이 파일은 여기까지입니다. 개념02로 가세요. → 그 위에 회색 글씨로 React DevTools 안내와 Babel 경고가 함께 나옵니다.

그 둘은 이 파일이 찍은 것이 아니라 도구가 찍은 것이라 세지 않았습니다.

정리 —————

- 1 지금까지는 화면을 **어떻게 고칠지** 직접 적었습니다(명령형).
- 2 React 는 **데이터가 이러면 화면은 이렇게 생겼다**만 적습니다(선언형).
- 3 데이터를 바꾸면 React 가 바뀐 부분을 찾아 화면에 반영합니다.
- 4 화면을 고치는 코드가 흩어지지 않아서, 화면이 복잡해질수록 이득이 커집니다.
- 5 React 는 자바스크립트를 대신하지 않습니다. `map·filter·스프레드`를 계속 씁니다.
- 6 이 자료는 처음부터 끝까지 실제 개발에서 쓰는 방식(Vite 프로젝트)으로 진행합니다.

```
export default function Concept01() {
  return (
    <div> <h1>개념 01 – 왜 React를 쓰나</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다.{" "} <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br /> <strong>이 파일의 <body>에는 글자가 한 자도 없습니다.
      </strong> 빈 칸과 회색 상자 테두리뿐입니다. 화면에 보이는 글자는 전부 React 가 그린
      것입니다 -{" "} <strong>① 상자의 안쪽만 빼고</strong>. 그 안쪽이 왜 혼자 예외인지가
      이 파일의 이야기입니다.
```

첫 화면 띄우기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

01

F12 → Console 도 함께 보세요.

개념01에서는 React 방식이 어떻게 생겼는지 눈으로만 봤습니다. 이 파일부터는 “그 화면이 어떻게 거기 나타났는지” 를 따라갑니다.

지금 여러분 브라우저에 보이는 이 글자들은 저절로 생긴 것이 아닙니다. 파일 네 개를 거쳐서 왔습니다.

`index.html` → `src/main.jsx` → `src/App.jsx` → 지금 이 파일

이 파일에서는 그 네 단계를 하나씩 열어 봅니다. ★ 앞의 세 개는 자료가 미리 만들어 둔 것입니다. 고칠 일이 없습니다. 그래도 한 번은 읽어 두세요. 화면이 안 나올 때 볼 곳이 거기입니다.

```
import Summary from "../_ui/Summary.jsx";
import 그려진뒤 from "../_ui/그려진뒤.js";
```

index.html — 시작하는 한 장

섹션 1

프로젝트 폴더 맨 위에 `index.html` 이 한 장 있습니다. 열어 보세요. 길지 않습니다. 중요한 것은 두 줄뿐입니다.

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

첫째 줄은 ‘빈 자리’ 입니다. 안에 아무것도 없습니다. 둘째 줄은 “이 자바스크립트 파일부터 실행해라” 입니다.

여러분이 `npm run dev` 로 띄운 주소(`http://localhost:5173` 같은 것)를 열면 브라우저가 받는 것은 이 `index.html` 한 장입니다. 그 순간 화면에는 아무것도 없습니다. 빈 `div` 하나뿐이니까요.

★ 페이지 소스 보기(`Ctrl+U`)로 확인할 수 있습니다. 지금 화면에는 글자가 잔뜩 있는데, 소스에는 그 글자가 하나도 없습니다. 화면에 보이는 것은 전부 자바스크립트가 나중에 만들어 넣은 것입니다.

▹ 직접 해보기 1

`Ctrl+U` 로 페이지 소스를 열어 보세요. “개념 02” 라는 글자가 소스 안에 있는지 찾아보세요. (정답은 파일 맨 아래에)

그 빈 div 가 React 가 그림을 그릴 자리입니다.

```
<div id="root"></div>
```

React 도 결국 브라우저 화면(DOM)에 그려야 하는데, “어디에” 그릴지는 React 가 알 수 없습니다. 우리가 알려 줘야 합니다. JS자료 10단원에서 목록을 만들 때도 항상 이렇게 시작했습니다.

```
const list = document.querySelector("#list");
```

같은 일입니다. 먼저 자리를 찾고, 그 자리에 그립니다.

id 가 꼭 root 여야 할까요? 아닙니다. 아무 이름이나 됩니다. 다만 거의 모든 React 프로젝트가 root 를 쓰기 때문에 그대로 따랐습니다.

한 가지만 기억하세요. 그 안은 이제 React 가 통째로 말합니다. 그래서 보통은 `<div id="root"></div>` 처럼 비워 둡니다.

코드로 확인해 봅시다. 그 자리는 지금도 페이지에 있습니다.

```
const 뿌리자리 = document.querySelector("#root");
```

```
console.log(뿌리자리.tagName);
```

F12 콘솔 DIV

```
console.log(뿌리자리.id);
```

F12 콘솔 root

안이 비어 있지 않습니다. React 가 이미 잔뜩 그려 넣었기 때문입니다.

```
console.log(뿌리자리.children.length > 0);
```

F12 콘솔 true

▹ 직접 해보기 2

위 세 줄 아래에 이 줄을 더해 보세요. `console.log(뿌리자리.parentElement.tagName);` 무엇이 찍힐지 먼저 예상해 보세요.

createRoot 와 render — 프로젝트 전체에 딱 한 번

자리를 찾았으니 이제 그리라고 말해야 합니다. 두 줄이면 됩니다.


```
const root = createRoot(자리);
root.render(화면설명);
```

두 줄이 각각 무슨 일을 하는지 나눠 봅시다.

— createRoot(자리)

“이 자리는 이제부터 내가 맡겠다” 고 React 에게 알려 주는 줄입니다. 한 자리에 대해 딱 한 번만 부릅니다. 두 번 부르면 경고가 납니다. 돌려주는 것은 root 라는 물건입니다. 이 안에 render 가 들어 있습니다.

— root.render(화면설명)

실제로 그려 달라고 부탁하는 줄입니다. 괄호 안에 “무엇이 보여야 하는지” 를 넣습니다.

이 두 줄은 이미 쓰여 있습니다. src/main.jsx 를 열어 보세요.

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.querySelector("#root")).render(
  <StrictMode> <App /> </StrictMode>
);
```

★ 이 두 줄은 프로젝트 전체에서 딱 한 번만 나옵니다. 화면이 아무리 많아져도 createRoot 는 늘 한 번입니다. 09단원까지 가도, 13단원 종합 프로젝트에서도 여기 이 한 줄뿐입니다.

그래서 여러분이 만드는 파일에는 createRoot 가 나오지 않습니다. 여러분은 “무엇이 보여야 하는지” 만 적고, 그리는 일은 main.jsx 가 맡습니다.

▹ 직접 해보기 3

src/main.jsx 를 열어 createRoot 가 몇 번 나오는지 세어 보세요. 그리고 이 프로젝트 전체(src 폴더)에서 몇 번 나오는지도요.

render 는 부탁이지 명령이 아닙니다

섹션 4

여기서 꼭 짚고 갈 것이 하나 있습니다.

render 는 “이렇게 그려 주세요” 하고 부탁만 하고 바로 다음 줄로 넘어갑니다. 실제로 화면이 만들어지는 것은 아주 잠깐 뒤입니다.

그래서 render 를 부른 바로 다음 줄에서 그 자리를 읽으면 아직 비어 있습니다. 이것 때문에 처음에 많이 헤맸습니다. 눈으로 확인해 봅시다.

```
const 새자리 = document.createElement("div");
새자리.id = "잠깐시험";
document.body.appendChild(새자리);

console.log("만든 직후 안쪽:", JSON.stringify(새자리.innerHTML));
```

F12 콘솔 만든 직후 안쪽: ""

이 자료에는 그 기다림을 대신해 주는 도우미가 있습니다. src/_ui/그려진뒤.js 입니다. 읽지 않아도 됩니다. "그 자리가 생기면 한 번 실행해 줘" 라는 뜻입니다.

```
그려진뒤("#정리자리", (자리) => {
  console.log("React 가 그린 뒤 안쪽 글자 수:", 자리.innerText.length > 0);
```

F12 콘솔 React 가 그린 뒤 안쪽 글자 수: true

```
새자리.remove();
});
```

정리하면 이렇습니다.

화면이 바뀌었는지는 콘솔로 읽지 말고 눈으로 보세요.

이 자료에서 화면 결과를 // 화면: 으로 따로 적는 이유가 이것입니다. 콘솔로 확인하려 들면 "아직 안 그려진" 상태를 보게 됩니다.

▸ 직접 해보기 4

위 그려진뒤 안의 "#정리자리" 를 "#없는자리" 로 바꿔 보세요. 콘솔에 무엇이 찍힐까요? 에러가 날까요?

화면 설명을 함수로 묶기

섹션 5

화면 설명이 길어지면 함수 하나로 묶습니다. 그 함수를 컴포넌트라고 부릅니다. 03단원에서 제대로 배웁니다.

```
function 인사() {
  return <h3>안녕하세요, React 입니다</h3>;
}
```

규칙이 두 개 있습니다. 지금 외우지 않아도 됩니다. 04단원까지 계속 나옵니다.

(1) 이름은 대문자나 한글로 시작합니다. 소문자로 시작하면 조용히 안 나옵니다. (2) 쓸 때는 <인사 /> 처럼 꺾쇠를 씁니다. 인사() 라고 부르지 않습니다.

이 파일이 화면에 내놓는 것은 맨 아래 export default 한 덩어리입니다. 왼쪽 목록에서 이 예제를 고르면 App.jsx 가 그것을 가져다 화면에 넣습니다.

```
export default function Concept02() { ... }
```

↑ "이 파일이 내놓는 것은 이겁니다" 라는 뜻입니다

export 와 import 는 08단원에서 제대로 배웁니다. 지금은 "파일마다 하나씩 내놓는다" 정도로 보세요.

```
console.log(typeof 인사);
```

F12 콘솔 function

⇒ 직접 해보기

5

인사 함수의 <h3> 를 <h1> 으로 바꾸고 저장해 보세요. 저장하는 순간 아래 ㉡ 상자의 글자가 커집니다.

저장하면 바로 바뀝니다

섹션 6

고친 뒤에 새로고침(F5)을 누를 필요가 없습니다. npm run dev 가 파일을 지켜보고 있다가, 바뀐 곳만 화면에 갈아 끼웁니다.

★ 오히려 F5 는 누르지 마세요. 왼쪽 목록에서 고른 예제가 풀려서 맨 처음 예제로 돌아갑니다.

화면이 안 바뀐다면 순서대로 보세요.

(1) 저장을 했나 (Ctrl+S) (2) 터미널에 빨간 글자가 떴나 ← 문법이 틀리면 여기 먼저 뜹니다 (3) F12 → Console 에 빨간 줄이 있나 (4) npm run dev 가 아직 돌고 있나 ← 터미널을 닫았으면 꺼진 것입니다

(2)를 잊지 마세요. 처음 배울 때 가장 많이 놓치는 곳입니다. 브라우저만 보고 있으면 "아무 일도 안 일어난다" 로 보이는데, 터미널에는 이미 무엇이 틀렸는지 적혀 있습니다.

⇒ 직접 해보기

6

npm run dev 를 켜 둔 터미널에서 Ctrl+C 로 서버를 끄고, 브라우저에서 F5 를 눌러 보세요. 무엇이 보입니까? 확인했으면 다시 npm run dev 로 켜세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

export default 를 안 붙인다

```
function Concept02() { ... }
```

← default 가 없습니다

왼쪽 목록에서 골라도 화면이 비고 빨간 상자가 뜹니다. App.jsx 는 "이 파일이 내놓는 것" 을 찾는데 내놓은 게 없기 때문입니다.

이런 실수를 합니다

한 파일에서 **export default** 를 두 번 쓴다

이런 실수를 합니다

터미널이 멈춥니다. Duplicate export of 'default' 내놓는 것은 파일마다 하나입니다.

이런 실수를 합니다

createRoot 를 내 파일에도 쓴다

```
const root = createRoot(document.querySelector("#root")); ← 하지 마세요
```

이런 경고가 납니다. You are calling ReactDOMClient.createRoot() on a container that has already been passed to createRoot() before. 그 자리는 이미 main.jsx 가 맡았습니다. 두 번 말을 수 없습니다. 여러분 파일은 export default 만 내놓으면 됩니다.

이런 실수를 합니다

컴포넌트 이름을 소문자로 시작한다

```
function 인사() {} → <인사 />      한글은 됩니다
function greet() {} → <greet />     소문자 영문은 안 됩니다
```

에러도 경고도 없이 화면만 빕니다. 개념03에서 정면으로 다룹니다.

정리

- 1 index.html 에는 빈 `<div id="root">` 하나만 있습니다. 화면에 보이는 글자는 전부 자바스크립트가 나중에 만들어 넣은 것입니다.
- 2 `createRoot(자리)` 는 자리를 맡는 줄, `{ " " } root.render(화면설명)` 은 그려 달라는 줄입니다.
- 3 그 두 줄은 **프로젝트 전체에 딱 한 번**, `{ " " } src/main.jsx` 에만 있습니다. 여러분 파일에는 쓰지 않습니다.
- 4 여러분 파일이 할 일은 `export default` 로 화면 설명 하나를 내놓는 것입니다.
- 5 `render` 는 부탁입니다. 부른 바로 다음 줄에서 그 자리를 읽으면 아직 비어 있습니다. 화면 결과는 눈으로 확인하세요.
- 6 고치고 저장하면 화면이 바로 바뀝니다. 안 바뀌면 ① 저장했나 ② 터미널에 빨간 글자가 있나 ③ 콘솔에 빨간 줄이 있나 순서로 보세요.

```

export default function Concept02() {
  return (
    <div>
      <h1>개념 02 – 첫 화면 띄우기</h1> <p className="guide">
        왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다.{ " "}
        <strong>F12 → Console</strong> 도 함께 보세요.
        <br /> <br /> <strong>이 파일은 직접 고쳐 보는 파일입니다.</strong> 고치고{"
      "
      <strong>저장(Ctrl+S)</strong> 하면 화면이 바로 바뀝니다. 새로고침(F5)은
      누르지 마세요 – 왼쪽에서 고른 예제가 풀립니다.
      <br /> <br /> <strong>이 파일이 말하는 파일 세 개를 같이 열어 두세요.
    </strong>{" "}
    <code>index.html</code> · <code>src/main.jsx</code> · {" "}
    <code>src/App.jsx</code> 입니다. 셋 다 고칠 일은 없습니다.
  </p>

  <div className="demo">
    <h3>① 화면이 여기까지 오는 길</h3>
    <div className="output">
      <p>
        <code>index.html</code> 의 <code>&lt;div id="root"&gt;</code>
      </p>
      <p>↓ 그 자리를 맡는다</p>
      <p>
        <code>src/main.jsx</code> 의 <code>createRoot(...).render(...)</code>
      </p>
      <p>↓ 무엇을 그릴지 넘긴다</p>
      <p>
        <code>src/App.jsx</code> – 왼쪽 목록을 그리고, 고른 예제를 옆에 놓는다
      </p>
      <p>↓ 고른 예제의 export default 를 가져다 놓는다</p>
      <p>
        <strong>지금 읽고 있는 이 파일</strong>
      </p>
    </div>
  </div>

  <div className="demo"> <h3>② 화면 설명을 함수로 묶은 것
</h3> <div className="output"> <인사 /> </div>
</div>

  <div id="정리자리"> </div>
</div>
);
}

```

직접 해보기 정답

1) 없습니다. 소스에는 `<div id="root"></div>` 만 있습니다. "개념 02" 라는 글자는 이 파일의 `<h1>` 이 만든 것이고, 그것은 브라우저가 자바스크립트를 실행한 뒤에 생깁니다. → 그래서 페이지 소스와 화면이 다릅니다. 개발자 도구의 Elements 탭을 보면

지금 화면 그대로가 보입니다. 그쪽은 '실행된 뒤' 를 보여 주기 때문입니다.

2) BODY 가 찍힙니다.

```
console.log(뿌리자리.parentElement.tagName);  
// 콘솔: BODY
```

→ index.html 에서 그 div 는 body 바로 아래에 있습니다.

3) main.jsx 에 1번, src 폴더 전체에도 1번입니다. 터미널에서 이렇게 세어 볼 수도 있습니다.

```
findstr /s /c:"createRoot" src\*.jsx      (윈도우)
```

→ 예제 파일 안에 나오는 것은 전부 주석 속 설명입니다. 실제 호출은 한 번뿐입니다.

4) 아무것도 안 찍히고, 에러도 안 납니다. 그려진 뒤 는 그 자리를 못 찾으면 3초쯤 기다려 보다가 조용히 그만둡니다. → "조용히 아무 일도 안 일어난다" 가 가장 찾기 어려운 고장입니다.

그래서 이 자료는 화면 결과를 // 화면: 으로 따로 적어 둡니다.

6) 브라우저에 "사이트에 연결할 수 없음" 이 나옵니다. → `npm run dev` 는 명령 한 번으로 끝나는 것이 아니라 서버를 켜 두는 것입니다.

그 터미널이 서버입니다. 닫으면 화면도 안 뜹니다.
"어제까지 되던 게 오늘 안 된다" 의 거의 전부가 이것입니다.

5) ㉠ 상자의 "안녕하세요, React 입니다" 가 큰 글씨가 됩니다. 저장하는 순간 바뀝니다. F5 는 누르지 않아도 됩니다. → 안 바뀌었다면 저장이 안 됐거나(제목 옆에 점이 있는지 보세요),

터미널에 빨간 글자가 떠 있습니다.

화면이 다시 그려진다

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

01

F12 → Console 도 함께 보세요.

개념02에서 화면을 한 번 띄웠습니다. 그런데 진짜 앱은 화면이 계속 바뀝니다. 숫자가 오르고 목록이 늘어납니다.

여기서 바로 의심이 듭니다. "바뀔 때마다 통째로 다시 그리면 느리지 않나? 입력칸에 치던 글자는 어떻게 되지?"

이 파일은 그 의심을 직접 확인하는 파일입니다. 똑같이 생긴 초시계를 두 개 놓고, 한쪽은 React 가, 다른 쪽은 우리가 그립니다. 그리고 두 초시계 옆에 입력칸을 하나씩 둡니다. 거기서 차이가 드러납니다.

★ 이 파일에는 앞으로 배울 것이 두 개 미리 나옵니다. `useState` — 04단원에서 배웁니다 `useEffect` — 09단원에서 배웁니다 지금은 "1초마다 숫자가 오르게 하는 장치" 정도로만 보고 넘어가세요. 이 파일에서 볼 것은 그 장치가 아니라 '입력칸이 살아남느냐' 입니다.

`useState` 는 04단원, `useEffect` 는 09단원에서 배웁니다. 지금은 결과만 보세요.

```
import { useEffect, useState } from "react";
import Summary from "../_ui/Summary.jsx";
import 그려진뒤 from "../_ui/그려진뒤.js";
```

React 가 1초마다 다시 그리는 초시계

섹션 1

아래가 ① 상자를 만드는 컴포넌트입니다. 1초마다 숫자를 하나 올리고, 올린 값으로 화면을 다시 그립니다.

```
function React초시계() {
  const [초, 초설정] = useState(0); // ← 04단원
```

1초마다 초설정 을 부릅니다. `useEffect` 는 09단원에서 배웁니다.

```
useEffect(() => {
  const 타이머 = setInterval(() => {
    초설정((지금) => 지금 + 1);
  }, 1000);
  return () => clearInterval(타이머); // 떠날 때 타이머를 끕니다
}, []);
```

`className` 은 02단원에서 배웁니다. 화면 모양을 정하는 이름입니다.

```
return (
  <div className="output"> <p>초시계: {초}초</p> <p>
    입력칸: <input /> </p> </div>
);
}
```

1초에 한 번, 화면 설명을 통째로 다시 만들어 넘겼습니다. 그러면 화면도 통째로 새로 만들어질까요?
아닙니다.

React 는 새 설명과 직전 설명을 비교합니다. 그리고 달라진 곳만 실제 화면에 반영합니다. 지금 달라지는
곳은 숫자 하나뿐이므로, 그 글자만 고칩니다. 옆의 입력칸은 손대지 않습니다.

▸ 직접 해보기 1

① 의 입력칸에 "김민준" 을 치고 3초쯤 기다려 보세요. 숫자는 계속 오르는데 친 글자는 그대로 남아
있습니다. 커서 위치도 그대로입니다.

같은 초시계를 React 없이 직접 그린 것

섹션 2

비교 대상을 만듭니다. ② 는 React 를 한 줄도 안 씁니다. JS자료 10단원 방식으로 1초마다 다시 그립니다.

```
let 손초 = 0;

function 손으로그리기() {
  const 상자 = document.querySelector("#손초시계");
  if (!상자) return; // 다른 예제로 옮겨 갔으면 그만둡니다
```

안을 통째로 새 HTML 로 갈아 끼웁니다

```
상자.innerHTML =
  '<div class="output"><p>초시계: ' +
  손초 +
  "초</p><p>입력칸: <input /></p></div>";
}

그려진뒤("#손초시계", () => {
  손초 = 0;
  손으로그리기(); // 첫 화면 const 타이머 = setInterval(() => {
    손초 = 손초 + 1;
    손으로그리기();
```

그 자리가 사라졌으면(다른 예제로 옮겨 갔으면) 타이머를 끕니다


```
if (!document.querySelector("#손초시계")) clearInterval(타이머);
  }, 1000);
});
```

화면만 보면 ① 과 ② 는 똑같습니다. 숫자가 1초마다 올라갑니다. 그런데 ② 의 입력칸에 글자를 쳐 놓고 1초만 기다려 보세요. 글자가 사라집니다.

이유는 innerHTML 에 있습니다. innerHTML 에 새 문자열을 넣으면 안에 있던 요소가 전부 버려지고 문자열대로 새 요소가 만들어집니다. 여러분이 글자를 친 그 입력칸은 사라지고, 빈 입력칸이 새로 생긴 것입니다.

직접 해보기

2

② 의 입력칸에 "김민준" 을 치고 1초만 기다려 보세요. ① 과 무엇이 다른지 확인하세요.

정말 같은 입력칸인지 확인하기

섹션 3

말로만 하면 믿기 어려우니 콘솔로 확인합니다. 화면에 있는 요소는 하나하나가 값입니다. === 로 같은 것인지 비교할 수 있습니다.

처음 그려진 입력칸을 기억해 뒀다가, 2초 뒤에 같은 것인지 봅니다.

```
그려진뒤("#React초시계", (상자) => {
  const 처음것 = 상자.querySelector("input");

  setTimeout(() => {
    const 지금것 = 상자.querySelector("input");
    console.log("[React] 2초 뒤에도 같은 입력칸인가?", 지금것 === 처음것);
  });
}, 2000);
});
```

F12 콘솔 [React] 2초 뒤에도 같은 입력칸인가? true

```
}, 2000);
});

그려진뒤("#손초시계", (상자) => {
  const 처음것 = 상자.querySelector("input");

  setTimeout(() => {
    const 지금것 = 상자.querySelector("input");
    console.log("[직접] 2초 뒤에도 같은 입력칸인가?", 지금것 === 처음것);
  });
}, 2000);
});
```

F12 콘솔 [직접] 2초 뒤에도 같은 입력칸인가? false

```
}, 2000);
});
```

React 쪽은 `true` 입니다. 그 사이에 화면 설명을 두 번 넘게 다시 넘겼는데도, 입력칸은 처음 만들어진 그 요소 그대로입니다. 새로 만들지 않았으니 안에 친 글자가 사라질 이유도 없습니다.

직접 그리는 쪽은 `false` 입니다. 같은 자리에 있지만 다른 요소입니다.

사라지는 것은 글자만이 아닙니다. 커서, 스크롤 위치, 체크 상태, 고르던 목록까지 같이 날아갑니다. JS자료 13단원 종합03에서 목록을 다시 그릴 때 겪었던 일이 이것입니다.

이것이 개념01에서 말한 “바뀐 부분만 찾아서 고친다”의 실체입니다.

▹ 직접 해보기 3

콘솔에서 `[React]` 와 `[직접]` 두 줄을 찾아 보세요. 한 줄은 `true`, 한 줄은 `false` 입니다.

직접 짜도 피할 수는 있습니다

섹션 4

오해하지 마세요. `innerHTML` 이 나쁜 것이 아닙니다. 바뀐 글자만 골라서 `textContent` 로 고치면 입력칸은 살아남습니다.

```
상자.querySelector("p").textContent = "초시계: " + 손초 + "초";
```

다만 화면이 커질수록 “무엇이 바뀌었는지”를 우리가 전부 관리해야 합니다. 글자 하나, 색 하나, 버튼 하나마다 “바뀌었으면 여기를 고쳐라”를 적어야 합니다. 한 곳만 빠뜨려도 화면이 조용히 어긋납니다.

React 는 그 관리를 대신해 주는 도구입니다. 여러분은 “지금은 이렇게 생겼다”만 넘기고, 비교는 React 가 합니다.

▹ 직접 해보기 4

위 손으로그리기 안의 `innerHTML` 줄을 지우고 아래 두 줄로 바꿔 보세요. ②의 입력칸이 살아남습니다.

```
const 줄 = 상자.querySelector("p");
if (줄) 줄.textContent = "초시계: " + 손초 + "초";
```

(확인이 끝나면 되돌려 놓으세요. 위 // 콘솔: 값과 어긋납니다)

아직 남은 문제 — 데이터만 바꾸면 화면은 그대로입니다

섹션 5

여기까지 보면 React 가 다 해 주는 것 같습니다. 아직 아닙니다.

React 는 “지금 다시 그려야 한다”는 것을 스스로 알지 못합니다. 그걸 알려 주는 장치가 따로 있어야 합니다.

아래 ③이 그 이야기입니다. 버튼을 누르면 장바구니 배열에는 담깁니다. 그런데 화면의 숫자는 0 그대로입니다. 에러도 경고도 없습니다.

```
const 장바구니 = [];

function 장바구니Demo() {
  return (
    <div className="output">
      <p>장바구니에 {장바구니.length}개 담겨 있습니다</p>
      { /* onClick 은 04단원 개념01에서 배웁니다 */ }
      <button
        onClick={() => {
          장바구니.push("아메리카노");
          console.log("담았습니다. 배열 길이:", 장바구니.length);

```

F12 콘솔 담았습니다. 배열 길이: 1

```
    })
  >
  담기
  </button>
</div>
);
}
```

몇 번을 눌러도 화면은 “0개” 입니다. 콘솔의 숫자만 올라갑니다. 데이터는 진짜로 바뀌었는데 화면만 안 따라온 것입니다.

이 문제를 푸는 것이 04단원의 useState 입니다. 이 파일에서는 “이런 문제가 있다” 까지만 알고 넘어갑니다.

▹ 직접 해보기 5

㉓ 의 담기 버튼을 다섯 번 누르고 콘솔을 보세요. 화면의 숫자와 콘솔의 숫자가 어떻게 다른가요?

- 1 화면 설명을 통째로 다시 넘겨도 화면이 통째로 새로 만들어지지 않습니다. React 가 달라진 곳만 고칩니다.
- 2 그래서 초시계가 갱신돼도 옆 입력칸의 글자와 커서가 그대로 남습니다.
- 3 `innerHTML` 로 직접 다시 그리면 안에 있던 요소가 전부 버려집니다. 글자·커서·스크롤·체크 상태가 같이 사라집니다.
- 4 `===` 로 비교해 보면 React 쪽은 `true`, 직접 그린 쪽은 `{""}` `false` 입니다. 같은 자리에 있어도 다른 요소입니다.
- 5 직접 짜도 피할 수는 있습니다. 다만 '무엇이 바뀌었는지' 를 전부 우리가 관리해야 합니다. React 는 그 관리를 대신해 줍니다.
- 6 다만 "지금 다시 그려라" 라고 알려 주는 일은 아직 남아 있습니다. `{""}` 그것이 04단원의 `useState` 입니다.

```
export default function Concept03() {
  return (
    <div> <h1>개념 03 - 화면이 다시 그려진다</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다.{" "}
      <strong>F12 → Console</strong> 도 함께 보세요.
    <br /> <br />
      이 파일에는 <strong>1초마다 스스로 움직이는 초시계</strong>가 두 개 있습니다.
    {" "}
    <strong>① 과 ② 의 입력칸에 글자를 쳐 놓고 기다려 보는 것</strong>이 이
    파일의
    핵심입니다.
    <br /> <br />
    ① 은 React 가 그립니다. ② 는 React 없이 <code>innerHTML</code> 로 그립니다.
    화면은 똑같이 생겼습니다.
    <p> <div className="demo"> <h3>① React 가 1초마다 다시 그리는 초시계
    </h3> <div id="React초시계"> <React초시계
    /> </div> </div> <div className="demo"> <h3>② 같은 초시계를 React 없이 직접 다시
    그린 것</h3> <div id="손초시계" /> </div> <div className="demo"> <h3>③ 데이터만
    바꾸면 화면은 그대로입니다</h3> <장바구니Demo /> </div>

    </div>
  );
}
```

직접 해보기 정답

1) 글자가 그대로 남아 있습니다. 커서 위치도 그대로입니다. → 숫자만 바뀌었으니 React 는 그 글자 하나만 고쳤습니다.

입력칸은 처음 만든 그대로 두었습니다.

2) 1초 만에 사라집니다. 커서도 빠집니다. → innerHTML 이 안을 통째로 갈아 끼웠기 때문입니다.

글자를 치던 그 입력칸은 이미 없습니다.

```
3) // 콘솔: [React] 2초 뒤에도 같은 입력칸인가? true
// 콘솔: [직접] 2초 뒤에도 같은 입력칸인가? false
```

→ 화면이 갈아 보여도 속은 다릅니다. React 쪽은 요소를 다시 쓰고,

직접 그린 쪽은 매번 새로 만듭니다.

```
4) const 줄 = 상자.querySelector("p");
if (줄) 줄.textContent = "초시계: " + 손초 + "초";
```

→ 이제 ㉓ 의 입력칸도 살아남습니다.

React 없이도 되지만, 고칠 곳을 우리가 하나하나 지목해야 합니다.
화면에 글자가 열 개면 열 줄을 적어야 하고, 하나를 빠뜨리면 그것만 어긋납니다.

5) 화면은 계속 "장바구니에 0개 담겨 있습니다" 입니다. 콘솔은 1 → 2 → 3 → 4 → 5 로 올라갑니다.

```
// 콘솔: 담았습니다. 배열 길이: 5
```

→ 데이터는 바뀌었는데 화면이 안 따라왔습니다.

React 에게 "다시 그려라" 라고 말하지 않았기 때문입니다.
그 말을 하는 방법이 04단원 useState 입니다.

에러와 경고 읽기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 을 반드시 함께 여세요. 이 파일은 콘솔을 보는 파일입니다.

앞으로 여러분은 에러를 아주 많이 만납니다. 그건 정상입니다. 문제는 에러가 났을 때 '어디를 보느냐' 입니다.

이 프로젝트에는 에러가 나올 수 있는 곳이 세 군데입니다.

(1) 터미널 — `npm run dev` 를 켜 둔 그 검은 창 (2) 브라우저 화면 — 빨간 상자가 뜹니다 (3) F12 → Console — 빨간 줄과 노란 줄

셋은 성격이 다릅니다. 어느 것이 났는지가 곧 '무엇이 틀렸는지' 입니다. 이 파일에서 그 셋을 구별하는 법을 익힙니다.

```
import Summary from "../_ui/Summary.jsx";
```

항상 나오는 줄들은 정상입니다

섹션 1

콘솔을 열면 우리가 찍지 않은 줄이 이미 몇 개 있습니다.

vite · connecting...

vite · connected.

Download the React DevTools for a better development experience: ...

세 줄 다 정상입니다. 고장이 아닙니다.

앞의 두 줄 — `npm run dev` 가 브라우저와 연결됐다는 뜻입니다.

이 연결 덕분에 파일을 저장하면 화면이 바로 바뀝니다.
이 두 줄이 안 보이면 dev 서버가 꺼진 것입니다.

세 번째 — React 개발자 도구(크롬 확장)를 깔면 편하다는 안내입니다.

안 깔아도 이 자료를 진행하는 데 아무 지장이 없습니다.

★ 회색이나 파란 글씨는 안내입니다. 빨간 글씨만 문제입니다. 처음에는 이 세 줄을 보고 "뭔가 잘못됐다" 하고 놀라는 경우가 많습니다.

```
console.log("이 줄은 우리가 찍은 것입니다");
```

F12 콘솔 이 줄은 우리가 찍은 것입니다

▸ 직접 해보기

1

콘솔에서 위 세 줄을 찾아 보세요. 우리가 찍은 줄보다 먼저 나와 있습니다.

터미널 에러 — 파일을 읽지도 못한 경우

섹션 2

가장 먼저 배울 것은 “터미널부터 본다” 입니다.

문법이 틀리면 브라우저까지 가지도 못합니다. npm run dev 가 파일을 읽다가 멈추고, 터미널에 빨간 글자를 뱉습니다. 이때 브라우저에는 아무 변화가 없거나 옛날 화면이 그대로 남아 있습니다.

예를 들어 JSX 를 .js 파일에 쓰면 이렇습니다.

```
src/01_React_시작하기/시험.js:2:10
```

```
Unexpected JSX expression
```

```
Help: JSX syntax is disabled and should be enabled via the parser options
```

읽는 법은 언제나 같습니다.

(1) 파일 이름과 줄 번호를 먼저 봅니다 ← 시험.js 의 2번째 줄 (2) 그 다음 무엇이 문제인지 봅니다 ← JSX 를 여기서 쓸 수 없다 (3) Help 줄이 있으면 거기에 답이 있습니다

이 경우 답은 “파일 이름을 .jsx 로 바꾸라” 입니다.

★ 브라우저만 보고 있으면 아무 일도 안 일어난 것처럼 보입니다. “저장했는데 화면이 안 바뀐다” 의 절반은 이 경우입니다. 터미널을 한 번 보세요.

▸ 직접 해보기

2

이 파일 아무 데나 닫는 중괄호 } 를 하나 더 넣고 저장해 보세요. 터미널과 브라우저에 각각 무엇이 뜨는지 보세요. 확인이 끝나면 반드시 다시 지우세요.

브라우저 에러 — 그리다가 터진 경우

섹션 3

문법은 맞는데 그리는 도중에 터지는 경우입니다. 이때는 터미널이 조용하고, 브라우저에 빨간 상자가 뜹니다.

이 자료는 예제마다 빨간 상자를 하나 준비해 뒀습니다(src/_ui/ErrorMessage.jsx). 한 예제가 터져도 왼쪽 목록과 다른 예제는 그대로 남습니다.

가장 흔한 것이 이것입니다.

```
Uncaught ReferenceError: React is not defined
```

JSX 만 쓸 때는 React 를 불러오지 않아도 됩니다. Vite 가 알아서 넣어 줍니다. 그런데 React 라는 이름을 손으로 적으면 그때는 불러와야 합니다.

```
import React from "react";
```

02단원에서 이 이야기를 다시 만납니다.

▸ 직접 해보기 3

아래 줄의 주석을 풀고 저장해 보세요. 터미널과 브라우저 중 어디에 무엇이 뜹니까? 확인이 끝나면 반드시 다시 // 를 붙이세요.

```
const 시험 = React.createElement("h1", null, "안녕");
```

조용한 고장 ① — 컴포넌트 이름이 소문자다

섹션 4

에러도 경고도 없이 화면만 이상해지는 경우가 있습니다. 이게 제일 어렵습니다. 첫 번째가 이름 문제입니다.

```
function menu() {
  return <p>아메리카노 4000원</p>;
}

function Menu() {
  return <p>아메리카노 4000원</p>;
}
```

두 함수는 내용이 똑같습니다. 이름의 첫 글자만 다릅니다. 그런데 화면은 전혀 다릅니다. 아래 ① 과 ② 를 비교해 보세요.

React 는 태그 이름의 첫 글자로 구별합니다.

<menu /> 소문자로 시작 → “브라우저가 아는 HTML 태그” 로 봅니다 <Menu /> 대문자로 시작 → “내가 만든 컴포넌트” 로 봅니다

menu 는 실제로 존재하는 HTML 태그입니다. 그래서 에러도 안 납니다. 우리가 만든 함수는 아예 불러 보지도 못하고, 빈 <menu> 태그만 그려집니다.

★ 한글 이름은 대문자 취급을 받습니다. `function 메뉴()` 로 만들고 `<메뉴 />` 라고 쓰면 잘 동작합니다. 이 자료가 한글 컴포넌트 이름을 자주 쓰는 이유입니다.

```
console.log(typeof menu, typeof Menu);
```

```
F12 콘솔    function function
```


▣ 직접 해보기 4

① 상자를 크롬 개발자 도구의 Elements 탭에서 찾아보세요. <menu> 태그가 비어 있는 것이 보입니다.

조용한 고장 ② — 화면에 넣는 것을 잊었다

섹션 5

컴포넌트를 잘 만들어 놓고 export default 안에 안 넣는 경우입니다. 만들기만 하고 쓰지 않았으니 아무 일도 일어나지 않습니다.

```
function 안쓰인메뉴() {
  return <p>이 글자는 화면에 안 나옵니다</p>;
}

console.log("안쓰인메뉴 는 만들어졌습니다:", typeof 안쓰인메뉴);
```

F12 콘솔 안쓰인메뉴 는 만들어졌습니다: function

만들어진 것은 맞습니다. 화면에 넣지 않았을 뿐입니다. 에러가 안 나는 게 당연합니다. 잘못된 게 없으니까요.

“분명히 만들었는데 안 보인다” 면 만든 곳이 아니라 쓰는 곳을 보세요.

▣ 직접 해보기 5

맨 아래 ③ 상자의 빈 흰 칸 안에 <안쓰인메뉴 /> 를 넣고 저장해 보세요. 이제 글자가 보입니다.

화면이 비었을 때 보는 순서

섹션 6

정리하면 이 순서입니다. 위에서부터 차례로 봅니다.

- 1 터미널에 빨간 글자가 있나 → 문법이 틀렸습니다. 파일·줄 번호를 봅니다
- 2 브라우저에 빨간 상자가 떴나 → 그리다가 터졌습니다. 상자 안 글을 읽습니다
- 3 F12 → Console 에 빨간 줄이 있나 → 위와 같은 내용이 더 자세히 있습니다
- 4 넷 다 조용한가 → ‘조용한 고장’ 입니다

- 컴포넌트 이름이 소문자인가
- export default 안에 넣었나
- 이름을 잘못 적었나

★ 1번을 건너뛰지 마세요. 초보자가 가장 많이 놓치는 곳입니다. 브라우저만 뚫어지게 보면서 “왜 안 되지” 하고 있을 때, 터미널에는 이미 몇 번째 줄이 틀렸는지 적혀 있습니다.

➤ 직접 해보기 6

위 네 단계를 순서대로 외워 보세요. 터미널 → 빨간 상자 → 콘솔 → 조용한 고장. 앞으로 화면이 안 나올 때마다 이 순서로 봅니다.

자주 하는 실수

섹션 7

이런 실수를 합니다

터미널 창을 닫는다

이런 실수를 합니다

화면이 아예 안 열리거나 “연결할 수 없음” 이 뜹니다. `npm run dev` 는 계속 켜 뒀야 합니다. 그 창이 서버입니다. 실수로 닫았으면 다시 `npm run dev` 하면 됩니다.

이런 실수를 합니다

콘솔의 회색 안내를 에러로 본다

이런 실수를 합니다

잘못한 게 없는데 코드를 뜯어고치게 됩니다. 빨간 글씨만 문제입니다. 회색·파란 글씨는 안내입니다.

이런 실수를 합니다

에러 메시지를 안 읽고 코드부터 고친다

이런 실수를 합니다

영뚱한 곳을 고치게 됩니다. 메시지에는 거의 항상 파일 이름과 줄 번호가 있습니다. 그 줄부터 보세요. 아니면 그 바로 위 줄입니다.

이런 실수를 합니다

예제를 고르지 않고 “화면이 비었다” 고 한다

이런 실수를 합니다

왼쪽 목록에서 아무것도 안 고르면 당연히 오른쪽이 비어 있습니다. 고장이 아닙니다.

정리

- 1 에러가 나올 곳은 세 군데입니다. 터미널 · 브라우저의 빨간 상자 · F12 콘솔.
- 2 문법이 틀리면 브라우저까지 가지 못하고 터미널에서 멈춥니다. 화면이 안 바뀌면 터미널부터 보세요.
- 3 콘솔의 회색·파란 글씨는 안내입니다. 빨간 글씨만 문제입니다.
- 4 에러 메시지는 **파일 이름과 줄 번호**부터 읽습니다. 그 줄이 아니면 바로 위 줄입니다.
- 5 컴포넌트 이름이 소문자로 시작하면 에러 없이 화면만 빕니다. 대문자나 한글로 시작하세요.
- 6 만들어 놓고 export default 안에 안 넣으면 아무 일도 안 일어납니다. 만든 곳이 아니라 쓰는 곳을 보세요.

```

export default function Concept04() {
  return (
    <div>
      <h1>개념 04 - 에러와 경고 읽기</h1>

      <p className="guide">
        왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다.{ " "}
        <strong>F12 → Console 을 반드시 함께 여세요.</strong> 이 파일은 콘솔을 보는
        파일입니다.
        <br />
        <br />
        아래 <strong>① 과 ③ 은 일부러 비워 둔 자리</strong>입니다. 고장이 아닙니다.
        왜
        비어 있는지 알아내는 것이 이 파일의 목표입니다.
        <br />
        <br />
        <strong>에러가 나올 수 있는 곳은 세 군데입니다.</strong> 터미널 · 브라우저의
        빨간
        상자 · F12 콘솔. 어느 것이 뚫는지가 곧 무엇이 틀렸는지입니다.
      </p>

      <div className="demo">
        <h3>① 이름을 소문자로 만든 것 (일부러 틀린 것)</h3>
        <div className="output">
          <menu />
        </div>
        <p>위 흰 칸이 비어 있습니다. 에러는 나지 않습니다.</p>
      </div>

      <div className="demo">
        <h3>② 같은 코드에서 이름만 대문자로 바꾼 것</h3>
        <div className="output">
          <Menu />
        </div>
      </div>

      <div className="demo">
        <h3>③ 만들어만 두고 화면에 안 넣은 컴포넌트</h3>

```

```

    <div className="output" />
    <p>
      위 흰 칸도 비어 있습니다. <code>안쓰인메뉴</code> 는 만들어졌지만 여기에
      넣지
      않았습니니다.
    </p>
  </div>

</div>
);
}

```

직접 해보기 정답

1) 세 줄 다 우리가 찍은 줄보다 위에 있습니다.

vite · connecting...

vite · connected.

Download the React DevTools ...

→ 페이지가 열리는 순간 도구들이 먼저 말을 겁니다.

우리 코드는 그 뒤에 실행됩니다.

2) 터미널에는 파일 이름·줄 번호와 함께 문법 오류가 뜹니다. 브라우저에는 아무 변화가 없거나 빨간 덮개가 씌워집니다. → 중요한 것은 '터미널이 먼저 안다' 는 것입니다.

브라우저는 바뀐 파일을 아예 못 받았습니다.
다시 지우고 저장하면 아무 일도 없었던 것처럼 돌아옵니다.

3) 터미널은 조용합니다. 브라우저에 빨간 상자가 뜨고, 콘솔에 이렇게 나옵니다.

Uncaught ReferenceError: React is not defined

→ 문법은 맞으니 파일은 잘 읽혔습니다. 실행하다가 없는 이름을 만난 것입니다.

해보기 2(중괄호)와 뜨는 곳이 다릅니다. 그 차이가 이 파일의 핵심입니다.

4) Elements 탭에서 ① 상자를 펼치면 이렇게 보입니다.

```
<div class="output"><menu></menu></div>
```

→ <menu> 는 진짜 HTML 태그라 브라우저가 군말 없이 만들어 줬습니다.

우리가 만든 menu 함수는 한 번도 불리지 않았습니다.
이름 첫 글자 하나가 "내 것" 과 "브라우저 것" 을 가릅니다.

5) ㉓ 상자에 "이 글자는 화면에 안 나옵니다" 가 보입니다. → 컴포넌트는 처음부터 멀쩡했습니다. 화면에 넣지 않았을 뿐입니다.

"만들었는데 안 보인다" 면 만든 곳이 아니라 쓰는 곳을 보세요.

6) 터미널 → 브라우저의 빨간 상자 → F12 콘솔 → 조용한 고장. → 앞의 셋은 무엇이 틀렸는지 알려 줍니다. 넷째는 아무 말도 없습니다.

그래서 앞의 셋을 먼저 확인하고, 다 조용할 때만 넷째를 의심합니다.

React 시작하기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

01

고치고 저장하면 화면이 바로 바뀝니다. F12 → Console 도 함께.

1~9는 기본, 10은 [응용], 11은 [도전], 12는 에러 확인입니다. TODO 자리를 채우고 저장하세요. 새로고침 (F5)은 누르지 마세요.

처음에는 아래 상자가 거의 다 비어 있습니다. 정상입니다. 여러분이 채우는 것입니다.

```
import Summary from "../_ui/Summary.jsx";
import 그려진뒤 from "../_ui/그려진뒤.js";
```

문제 1 (개념02)

아래 자리에 "안녕하세요" 를 큰 글씨(h3)로 보이게 하는 컴포넌트를 만드세요. 이름은 Q1 로 하세요. 맨 아래 export default 안의 문제 1 상자에 <Q1 /> 이 이미 넣어 있습니다.

기대 화면 문제 1 상자에 "안녕하세요" 가 굵고 큰 글씨로 보입니다.
안 보이면 이름을 Q1 으로 썼는지, return 을 적었는지 보세요.

여기에 코드를 쓰세요

문제 2 (개념02)

커피 이름과 값을 보여 주는 컴포넌트 Q2 를 만드세요. 화면에 "아메리카노 4000원" 이 p 태그로 보이면 됩니다.

기대 화면 문제 2 상자에 "아메리카노 4000원" 이 보입니다.

여기에 코드를 쓰세요

문제 3 (개념04)

아래 q3 함수는 이름이 소문자로 시작합니다. 이름을 대문자로 바꾸고, 맨 아래 상자에서 쓰는 곳도 함께 고치세요. (지금은 <q3 /> 라고 적혀 있어서 화면이 비어 있습니다)

기대 화면 문제 3 상자에 "케이크 6000원" 이 보입니다.
고치기 전에는 아무것도 안 보입니다.
기대 결과 (콘솔): 고치기 전에는 노란 경고가 한 줄 있습니다. 힌트입니다.
The tag <q3> is unrecognized in this browser.
If you meant to render a React component,
start its name with an uppercase letter.

```
function q3() {
  return <p>케이크 6000원</p>;
}
```

문제 4 (개념02)

컴포넌트 두 개를 만들어, 세 번째 컴포넌트 안에서 둘 다 쓰세요.

Q4가게이름 → <h3>동네카페</h3>
Q4오늘메뉴 → <p>오늘의 메뉴: 라떼</p>
Q4카드 → 위 둘을 위아래로 담은 것

기대 화면 문제 4 상자에 "동네카페" 와 "오늘의 메뉴: 라떼" 가 보입니다.

여기에 코드를 쓰세요

문제 5 (개념02 · 서술)

코드를 쓰는 문제가 아닙니다. 답을 주석으로 적으세요.

index.html 을 열어 보면 body 안에 이런 줄이 하나 있습니다.

```
<div id="root"></div>
```

이 빈 div 는 무엇을 하는 자리인가요?

답

문제 6 (개념02 · 서술)

src/main.jsx 를 열어 보세요. createRoot 가 몇 번 나오니까? 그리고 src 폴더 전체에서는 몇 번 나오니까? (주석 속 설명은 세지 마세요)

답: main.jsx 에 ____번 / src 전체에 ____번 그 이유는?

문제 7 (개념02)

아래 함수는 화면을 그린 직후에 그 자리를 읽으려고 합니다. 그런데 아직 안 그려져 있어서 빈 값이 나옵니다.

그려진뒤 를 써서 “그려진 다음에” 읽도록 고치세요.

쓰는 법: 그려진뒤("#선택자", (자리) => { ... });

기대 콘솔 문제 7 상자 안 글자: 여기는 문제 7 자리입니다
고치기 전에는 아무것도 안 찍히거나 빈 값이 찍힙니다.

아래 세 줄을 그려진뒤 로 감싸세요

```
const q7자리 = document.querySelector("#q7");
console.log("문제 7 상자 안 글자:", q7자리 ? q7자리.innerText : "(아직 없음)");
```

문제 8 (개념03)

코드를 쓰는 문제가 아닙니다. 눈으로 확인하는 문제입니다.

개념03 예제를 다시 열어, ① 과 ② 의 입력칸에 각각 "김민준" 을 치고 3초쯤 기다리세요.

① (React 가 그림) 글자가 _____ ② (innerHTML 로 그림) 글자가 _____

왜 그럴까요? _____

문제 9 (개념03)

아래 Q9 는 버튼을 누르면 배열에 담기지만 화면 숫자는 안 바뀝니다. 콘솔에는 담긴 개수를 찍도록 만드세요.

기대 화면 숫자가 계속 0 입니다. 정상입니다.
기대 결과 (콘솔): 담긴 개수: 1 → 2 → 3 ...

```
const q9목록 = [];

function Q9() {
  return (
    <div className="output">
      <p>담긴 개수: {q9목록.length}</p>
      { /* onClick 은 04단원에서 배웁니다 */ }
      <button
        onClick={() => {
          q9목록.push("아메리카노");

```

여기에서 콘솔에 담긴 개수를 찍으세요

```
    })
  >
    담기
  </button>
</div>
);
}
```

문제 10 [응용] (개념02)

컴포넌트를 두 번 쓰면 두 번 그려집니다. Q10카드 를 하나 만들고, 문제 10 상자 안에서 세 번 쓰세요.

(className 은 화면 모양을 정하는 이름입니다. 02단원에서 배웁니다) Q10카드 → `<p>아메리카노 4000원</p>` 을 흰 상자(className="output") 안에

기대 화면 같은 카드가 세 개 위아래로 보입니다.

여기에 코드를 쓰세요

문제 11 [도전] (개념03)

개념03 의 ㉔ 처럼, React 없이 innerHTML 로 화면을 그려 보는 문제입니다.

(1) 그려진뒤 로 #q11 자리를 잡으세요. (2) 그 안에 innerHTML 로 아래 모양을 넣으세요.

`<p>직접 그린 줄</p><p>입력칸: <input /></p>`

(3) 1초 뒤에 같은 내용으로 한 번 더 그리세요. (setTimeout) (4) 그 전후로 input 이 같은 요소인지 콘솔에 찍으세요.

기대 화면 입력칸에 글자를 쳐 두면 1초 뒤에 사라집니다.
기대 결과 (콘솔): 문제 11 같은 입력칸인가? false
true 가 나왔다면 다시 그리는 코드가 실제로 안 돌고 있는 것입니다.

여기에 코드를 쓰세요

문제 12 (에러 확인 - 맨 마지막)

⚠ 아래 (다)는 주석을 풀지 마세요. 눈으로만 보세요. 풀면 이 파일 전체가 안 읽혀서 화면이 통째로 빕니다.

(가) 아래 줄의 주석을 풀고 저장하세요. 어디에 무엇이 뜹니까?

다 보고 나면 다시 // 를 붙이세요.

```
console.log(없는이름);
```

답: 어디에 뒀나 (터미널 / 브라우저 빨간 상자 / 콘솔) _____

메시지 첫 줄 _____

(나) 아래 줄의 주석을 풀고 저장하세요. (가)와 뜨는 곳이 같습니까?

```
const 잘못 = React.createElement("h1", null, "안녕");
```

답: 어디에 뒀나 _____ 왜 그런가 _____

(다) 주석을 풀지 마세요. 이렇게 되면 어떤 일이 생길지만 적으세요.

```
function 망가짐( {
  return <p>안 달은 괄호</p>;
}
```

답

정리 _____

- 1 1~4는 컴포넌트를 만들고 쓰는 연습입니다. 이름 첫 글자와 export default 안에 넣었는지를 보세요.
- 2 5~6은 프로젝트가 화면을 띄우는 길을 확인하는 문제입니다. 파일을 직접 열어 보세요.
- 3 7은 render 가 부탁이라는 것을 확인하는 문제입니다.
- 4 8~9·11은 React 가 다시 그리는 방식과 innerHTML 의 차이를 보는 문제입니다.
- 5 12는 에러가 어디에 뜨는지 구별하는 문제입니다. 터미널과 콘솔은 뜨는 것이 다릅니다.

```

export default function Exercises() {
  return (
    <div>
      <h1>01단원 연습문제 – React 시작하기</h1>

      <p className="guide">
        이 파일의 <strong>TODO</strong> 자리에 코드를 쓰세요. 저장하면 화면이 바로
        바뀝니다. 새로고침(F5)은 누르지 마세요 – 왼쪽에서 고른 예제가 풀립니다.
      <br />
      <br />
        1~9는 기본, 10은 [응용], 11은 [도전], 12는 에러 확인입니다.{" "}
        <strong>F12 → Console</strong> 도 함께 여세요.
      <br />
      <br />
        처음에는 아래 상자가 거의 다 비어 있습니다. <strong>정상입니다.</strong>
        여러분이
        채우는 것입니다.
      </p>

      <div className="demo">
        <h3>문제 1</h3>
        <div className="output">{/* TODO: <Q1 /> */}</div>

        <h3>문제 2</h3>
        <div className="output">{/* TODO: <Q2 /> */}</div>

        <h3>문제 3</h3>
        <div className="output">
          <q3 />
        </div>

        <h3>문제 4</h3>
        <div className="output">{/* TODO: <Q4카드 /> */}</div>

        <h3>문제 7</h3>
        <div className="output" id="q7">
          여기는 문제 7 자리입니다
        </div>
      </div>
    </div>
  )
}

```

```

    </div>

    <h3>문제 9</h3> <Q9 /> <h3>문제 10 - 응용</h3> <div>{/* TODO: Q10카드 를 세
    번 */}</div> <h3>문제 11 - 도전</h3> <div className="output" id="q11" /> <h3>문제 12
    - 에러 확인</h3> <p>화면에 그릴 것이 없습니다. 터미널과 콘솔을 보세요.</p>
    </div>

    </div>
  );
}

```

01단원 마무리 React 시작하기

자주 넘어지는 자리

- 1 컴포넌트 이름을 **소문자**로 쓰면 — `<menu />` 처럼 HTML 태그와 겹치면 **경고조차 없이** 조용히 사라짐
| 에러가 안 나서 학생이 원인을 못 찾습니다
- 2 `createRoot` 만 하고 `render` 를 안 부르면 그 자리가 빈 채로 끝남 | “화면이 하예요” 의 원인 중 하나

자주 나오는 질문

Q “React 안 쓰고 그냥 JS로 하면 안 돼요?”

A 됩니다. 화면이 열 군데로 늘어나면 고칠 곳이 흩어져서 못 따라갑니다. 그때 이득이 생깁니다

JSX

OPEN 02_JSX

```
import React from "react";  
const title = <h1>오늘의 메뉴</h1>;  
console.log(typeof title);  
console.log(title.type);
```

01 JSX란 무엇인가

02 종괄호로 값 넣기

03 속성 넣기

04 태그 규칙

05 여러 줄로 쓰기

- 연습문제

JSX란 무엇인가

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

01단원에서 이런 코드를 봤습니다.

```
root.render(<h1>안녕</h1>);
```

자바스크립트 파일 안에 `<h1>` 이라는 태그가 그냥 들어가 있습니다. 따옴표도 없습니다. JS자료 어디에서도 이런 문법은 안 나왔습니다.

이 문법의 이름이 JSX 입니다. 이 파일에서는 딱 하나만 확실히 합니다.

"JSX 는 자바스크립트가 아니다. 자바스크립트로 '바꿔는' 것이다."

말로만 하지 않고, 바뀐 결과를 콘솔에 직접 찍어서 보겠습니다.

JSX 는 자바스크립트 안에 쓴 화면 설명이다

섹션 1

먼저 JSX 가 무엇이 아닌지부터 정리합니다.

· HTML 이 아닙니다. `.jsx` 파일의 자바스크립트 코드 안에 씁니다.

· 문자열이 아닙니다. 따옴표로 감싸지 않습니다.

○ · 자바스크립트 '식' 입니다. 값 하나입니다.

'식' 은 이 단원에서 계속 쓸 말이라 여기서 뜻을 정하고 갑니다.

식 = 계산하고 나면 값 하나가 남는 것

$1 + 2 \rightarrow 3$ 이 남습니다. 식입니다. `name \rightarrow "김민준"` 이 남습니다. 식입니다. `menu.length \rightarrow 3` 이 남습니다. 식입니다.

JS자료 05단원에서 본 '함수 표현식' 의 그 '표현식' 과 같은 말입니다. JSX 도 그렇습니다. 계산하면 값 하나가 남습니다.

값이 남으니까 변수에 담을 수 있습니다.


```
import React from "react";
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";

const title = <h1>오늘의 메뉴</h1>;
```

방금 <h1> 태그를 변수에 담았습니다. HTML 에서는 할 수 없는 일입니다. 이 변수 안에 무엇이 들어 있는지 봅시다.

```
console.log(typeof title);
```

F12 콘솔 object

태그처럼 생겼지만 실제로 담긴 것은 '객체' 입니다. (JS자료 07단원 객체) 그 객체가 어떻게 생겼는지 두 군데만 열어 봅니다.

```
console.log(title.type);
```

F12 콘솔 h1

```
console.log(title.props);
```

F12 콘솔 { children: '오늘의 메뉴' }

정리하면 이렇습니다.

type	- 무슨 태그인가	→ "h1"
props	- 그 안에 뭐가 들어가나	→ { children: "오늘의 메뉴" }

★ props 와 children 은 React 가 쓰는 '칸 이름' 입니다. props — 그 태그에 딸린 것들을 모아 둔 객체
children — 그중에서도 '태그 사이에 넣은 것' 이 담기는 칸 이 둘을 어떻게 쓰는지는 03단원에서 제대로 배웁니다. 이 파일에서는 "React 가 이런 이름으로 기억하는구나" 까지만 보면 됩니다.

즉 JSX 한 줄은 "h1 태그이고 안에 이런 글자가 들어간다" 는 정보를 담은 객체 하나로 바뀝니다. 이 객체를 React 엘리먼트라고 부릅니다.

이 객체는 아직 화면이 아닙니다. '화면 설계도' 입니다. 설계도를 실제 화면으로 만드는 것이 01단원에서 배운 render 입니다.

화면 데모 ① 에 "오늘의 메뉴" 가 큰 글씨로 보입니다.

▸ 직접 해보기

1

title 의 태그를 h1 에서 h2 로 바꾸고 저장해 보세요. 콘솔의 title.type 이 무엇으로 바뀌는지 확인하세요.

브라우저는 JSX 를 모릅니다. `<h1>` 을 코드 한가운데서 만나면 문법 오류로 봅니다. 그래서 브라우저에 보내기 전에 누군가 평범한 자바스크립트로 바꿔 놔야 합니다.

그 일을 Vite 가 합니다. 여러분이 파일을 저장하는 순간 이미 바뀌어서 갑니다. 지금 읽고 있는 이 파일도 브라우저에는 바뀐 모습으로 도착했습니다.

그러면 ‘무엇으로’ 바뀌었을까요. 그게 이 단원의 이야기입니다. 눈으로 보려고 같은 도구를 하나 더 가져왔습니다. 맨 위의 이 줄입니다.

```
import * as Babel from "@babel/standalone";
```

화면을 만드는 데 필요한 것이 아닙니다. 결과를 보여 주려고만 씁니다.

```
const result = Babel.transform("<h1>안녕</h1>", { presets: [["@react", { runtime: "classic" }]] });

console.log(result.code);
```

```
F12 콘솔    /*#__PURE__*/React.createElement("h1", null, "안녕");
```

★ 한 가지는 밝혀 두고 갑니다. 위에서 우리가 시킨 것은 runtime: “classic” 이라는 옛 방식입니다. Vite 가 실제로 만드는 것은 조금 다르게 생겼습니다.

React.createElement("h1", null, "안녕")	← 지금 우리가 본 것
_jsx("h1", { children: "안녕" })	← Vite 가 실제로 만드는 것

하는 일은 같습니다. “태그를 함수 호출 하나로 바꾼다” 입니다. 이 단원에서는 읽기 쉬운 옛 모양으로 봅니다. 이름만 다르다고 생각하세요.

나온 줄을 하나씩 뜯어봅니다.

`/*#__PURE__*/` — 도구가 붙이는 표시입니다. 무시해도 됩니다. `React.createElement(...)` — 우리가 아는 평범한 함수 호출입니다.

"h1"	– 첫 번째 인자: 무슨 태그인가
null	– 두 번째 인자: 속성(개념03에서 배웁니다). 없으면 null
"\uC548\uB155"	– 세 번째 인자: 태그 안에 들어갈 것

세 번째 인자가 이상하게 보일 것입니다. 한글이 사라진 것이 아닙니다. `\uC548\uB155` 는 “안녕” 을 컴퓨터가 적는 다른 방식일 뿐입니다. 글자 하나를 번호로 적은 것이고, 자바스크립트에서는 똑같은 문자열입니다. 믿기 어려우니 직접 확인합니다. 아래 두 줄은 진짜 코드입니다.

코드	▶ F12 콘솔
<code>console.log("\uC548\uB155");</code>	▶ 안녕
<code>console.log("\uC548\uB155" === "안녕");</code>	▶ true

같은 글자입니다. Babel 이 안전하게 적으려고 이 방식을 골랐을 뿐입니다.

앞으로 나올 변환 결과를 읽기 편하게 옵션을 두 개 넣어 두겠습니다. `jsescOption: { minimal: true }` → 한글을 번호 대신 글자 그대로 적습니다 `concise: true` → 결과를 한 줄로 붙여 줍니다 (generatorOpts 는 Babel 의 설정입니다. 외울 필요 없습니다. 보기 좋게만 바꾸는 것이고, 만들어지는 코드 자체는 위와 같습니다.)

```
function toJs(jsxCode) {
  return Babel.transform(jsxCode, {
    presets: [["@react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}
```

```
console.log(toJs("<h1>안녕</h1>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("h1", null, "안녕");
```

이제 읽을 만합니다. 몇 개 더 바꿔 봅시다.

```
console.log(toJs("<p>아메리카노 4000원</p>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("p", null, "아메리카노 4000원");
```

```
console.log(toJs("<li>라떼</li>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("li", null, "라떼");
```

태그 이름만 자리를 바꿔 들어갑니다. 규칙이 아주 단순합니다.

```
<태그>안쪽</태그> → React.createElement("태그", null, "안쪽")
```

태그 안에 태그가 들어가면 어떻게 될까요?

```
console.log(toJs("<div><h3>아메리카노</h3><p>4000원</p></div>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("div", null,
/*#__PURE__*/React.createElement("h3", null, "아메리카노"),
/*#__PURE__*/React.createElement("p", null, "4000원"));
```

세 번째 인자 위치에 createElement 가 또 들어갔습니다. 안쪽 태그가 여러 개면 네 번째, 다섯 번째 인자로 계속 붙습니다.

이렇게 “겉보기 문법을 편하게 바꿔 놓은 것” 을 문법 설탕이라고 부릅니다. 설탕을 걷어내면 남는 것은 여러분이 JS자료에서 계속 쓰던 함수 호출뿐입니다.

▫ 직접 해보기 2

toJs 에 “케이크” 를 넣어 콘솔에 찍어 보세요. 어떤 줄이 나올지 먼저 종이에 적고 확인하면 더 좋습니다.

React.createElement 를 직접 써 본다

섹션 3

섹션 2에서 본 그 함수를 우리가 직접 부르면 어떻게 될까요? Babel 을 거치지 않고 손으로 적어 봅니다.

```
const titleByHand = React.createElement("h1", null, "안녕");

console.log(titleByHand.type);
```

F12 콘솔 h1

```
console.log(titleByHand.props);
```

F12 콘솔 { children: '안녕' }

섹션 1의 title 과 똑같은 모양의 객체입니다. 이 객체도 render 에 넣으면 화면이 됩니다.

화면 데모 ② 에 "아메리카노" 와 "4000원" 이 보입니다.

같은 화면을 JSX 로 쓰면 이렇게 됩니다.

```
<div>
  <h3>아메리카노</h3> <p>4000원</p>
</div>
```

위의 createElement 여섯 줄과 이 네 줄은 완전히 같은 것입니다. 실제로 그런지 섹션 2의 도구로 확인해 봅시다.

```
console.log(toJs("<div><h3>아메리카노</h3><p>4000원</p></div>") === toJs("<div>\n<h3>\n아메리카노</h3>\n<p>4000원</p>\n</div>"));
```

F12 콘솔 true

줄바꿈만 다르게 써도 결과가 같습니다. 줄바꿈은 변환 결과에 영향을 주지 않습니다. (줄바꿈을 어디에 넣어야 하는지는 개념05에서 따로 봅니다)

▸ 직접 해보기 3

위 rootCe.render 의 "h3" 를 "h2" 로 바꿔 보세요. 데모 ② 의 글씨가 커지는지 확인하세요.

JSX 는 값이라서 자바스크립트처럼 다룰 수 있다

섹션 4

JSX 가 '값' 이라는 말을 다시 확인합니다. 값이면 이런 일이 됩니다.

(1) 변수에 담기 — 섹션 10에서 이미 했습니다.

```
const menuTitle = <h2>오늘의 추천</h2>;
console.log(menuTitle.type);
```

F12 콘솔 h2

(2) 함수가 돌려주기

```
function makeTitle() {
  return <h2>이 함수가 만든 제목</h2>;
}

console.log(makeTitle().type);
```

F12 콘솔 h2

```
console.log(makeTitle().props);
```

F12 콘솔 { children: '이 함수가 만든 제목' }

여기가 중요합니다. 01단원에서 만든 `function App()` 이 바로 이 모양이었습니다. "화면 설계도를 돌려주는 함수" 였던 것입니다.

(3) 배열에 담기

```
const three = [<li>아메리카노</li>, <li>라떼</li>, <li>케이크</li>];
console.log(three.length);
```

F12 콘솔 3

```
console.log(three[1].props);
```

F12 콘솔 { children: '라떼' }

배열에 담은 것을 화면에 그리는 방법은 05단원에서 배웁니다. 지금은 "담을 수 있다" 까지만 확인합니다.

(4) 조건에 따라 고르기 (JS자료 03단원 삼항 연산자)

```
const isMorning = true;
const greeting = isMorning ? <p>좋은 아침입니다</p> : <p>안녕하세요</p>;

console.log(greeting.props);
```

```
F12 콘솔 { children: '좋은 아침입니다' }
```

HTML 로는 이런 일을 할 수 없습니다. HTML 은 값이 아니라 문서니까요. JSX 는 값이라서 됩니다. 이것이 JSX 를 쓰는 가장 큰 이유입니다.

▹ 직접 해보기 4

isMorning 을 `false` 로 바꾸고 새로고침해서 `greeting.props` 가 무엇으로 바뀌는지 확인하세요.

그러면 왜 굳이 JSX 를 쓰나

섹션 5

"어차피 `createElement` 로 바뀔 거면 그냥 `createElement` 를 쓰면 되지 않나?" 화면이 조금만 깊어지면 답이 나옵니다. 아래 두 코드를 비교해 보세요.

— `createElement` 로 쓴 것

```
React.createElement(
  "div",
  null,
  React.createElement("h3", null, "김민준님의 장바구니"),
  React.createElement(
    "ul",
    null,
    React.createElement("li", null, "아메리카노"),
    React.createElement("li", null, "케이크")
  )
)
```

— JSX 로 쓴 것

```
<div>

  <h3>김민준님의 장바구니</h3>
  <ul> <li>아메리카노</li> <li>케이크</li>
  </ul>
```

```
</div>
```

둘은 같은 것입니다. 그런데 아래쪽만 화면 모양이 눈에 들어옵니다. 위쪽은 괄호 짝을 세느라 화면이 안 보입니다.

- 1 화면 구조가 코드 모양 그대로 보인다
- 2 태그 짝이 안 맞으면 Babel 이 바로 알려 준다 (개념04에서 봅니다)
- 3 자바스크립트 값을 그대로 끼워 넣을 수 있다 (개념02에서 배웁니다)

반대로 JSX 가 해 주지 않는 것도 분명합니다. - 화면을 실제로 만들지 않습니다. 만드는 것은 render 입니다. - HTML 이 되는 것이 아닙니다. 함수 호출이 됩니다.

길어도 재 볼까요. 같은 화면을 두 방식으로 쓴 글자 수입니다.

```
const jsxCode = "<div><h3>김민준님의 장바구니</h3><ul><li>아메리카노</li><li>케이크</li></ul></div>";

console.log(jsxCode.length);
```

F12 콘솔 65

```
console.log(toJs(jsxCode).length);
```

F12 콘솔 260

우리가 65글자를 쓰면 Babel 이 260글자를 만들어 줍니다. 손으로 안 써도 되는 195글자만큼이 JSX 가 덜어 주는 일입니다.

▢ 직접 해보기 5

jsxCode 안의 케이크 를 하나 더 늘려 두 길이가 각각 얼마나 늘어나는지 확인해 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다
JSX 를 따옴표로 감싼다

에러가 안 납니다. 그래서 가장 찾기 어렵습니다. 데모 ③ 을 보세요. 아래 코드가 만든 화면입니다.

화면 데모 ③ 에 태그가 글자 그대로 보입니다 - <h1>안녕</h1>

이런 실수를 합니다

다음표로 감싸면 그냥 문자열입니다. React 는 문자열을 글자로만 그립니다. "화면에 태그가 글자로 보인다"면 다음표부터 지우세요.

이런 실수를 합니다

JSX 를 .js 파일에 쓴다

이런 실수를 합니다

화면이 아니라 터미널이 먼저 멈춥니다. Unexpected JSX expression Help: JSX syntax is disabled and should be enabled via the parser options 파일 이름을 .jsx 로 바꾸면 됩니다. 확장자만 다른데 왜 그러냐면, .js 는 "JSX 가 없는 파일" 로 보고 아예 찾지도 않기 때문입니다.

이런 실수를 합니다

React 를 안 불러오고 React.createElement 를 손으로 쓴다

이런 실수를 합니다

터미널은 조용합니다. 빌드가 통과합니다. 그런데 그 예제를 화면에서 고르는 순간 터집니다. Uncaught ReferenceError: React is not defined 맨 위에 import React from "react"; 를 넣으면 됩니다.

- ★ 헛갈리는 곳입니다. JSX 만 쓸 때는 React 를 안 불러와도 됩니다. Vite 가 필요한 것을 알아서 팔려 넣기 때문입니다. React 라는 이름을 손으로 적을 때만 불러오면 됩니다.

이런 실수를 합니다

JSX 안에 자바스크립트를 그냥 쓴다

```
const name = "김민준";  
const bad = <p>name 님 안녕하세요</p>;
```

화면에 "김민준" 이 아니라 "name" 이라는 글자가 그대로 나옵니다. 태그 사이는 글자 자리라서 변수 이름도 그냥 글자로 봅니다. 값을 넣으려면 중괄호가 필요합니다. 개념02에서 바로 배웁니다.

직접 해보기 정답

```
1) const title = <h2>오늘의 메뉴</h2>;  
// 콘솔: h2
```

→ 화면의 글씨도 조금 작아집니다. type 은 태그 이름 그대로입니다.


```
2) console.log(toJs("<span>케이</span>"));
// 콘솔: /*#__PURE__*/React.createElement("span", null, "케이");
```

→ 태그 이름만 첫 번째 인자로, 안쪽 글자만 세 번째 인자로 갑니다.

3) `React.createElement("h2", null, "아메리카노")`

```
// 화면: 데모 ② 의 "아메리카노" 가 더 큰 글씨가 됩니다.
```

→ h3 가 h2 로 바뀌었을 뿐, 구조는 그대로입니다.

```
4) const isMorning = false;
// 콘솔: { children: '안녕하세요' }
```

→ 삼항 연산자가 고른 쪽의 JSX 가 greeting 에 담깁니다.

JSX 가 값이기 때문에 이런 식으로 고를 수 있습니다.

```
5) const jsxCode = "<div><h3>김민준님의 장바구니</h3><ul><li>아메리카노</li><li>
케이</li><li>케이</li></ul></div>";
// 콘솔: 77
// 콘솔: 313
```

→ 우리가 쓴 글자는 12 늘었는데 만들어진 코드는 53 늘었습니다.

화면이 커질수록 JSX 가 덜어 주는 양도 같이 커집니다.

정리

- 1 JSX 는 자바스크립트 식입니다. 값 하나가 남으므로 변수에 담고 함수로 돌려주고 배열에 넣을 수 있습니다.
- 2 Babel 이 JSX 를 `React.createElement("태그", 속성, 안쪽)` 함수 호출로 바꿔 줍니다. 브라우저는 JSX 를 모릅니다.
- 3 그 결과로 남는 것은 **화면 설계도 객체**입니다. `type` 과 `props` 를 가집니다. 아직 화면은 아닙니다.
- 4 설계도를 실제 화면으로 만드는 것은 `render` 입니다.
- 5 `createElement` 를 직접 써도 결과는 같습니다. JSX 는 그것을 읽기 쉽게 적는 방법입니다.
- 6 JSX 를 따옴표로 감싸면 그냥 문자열입니다. 화면에 태그가 글자로 보이면 이 실수입니다.

```

export default function Concept01() {
  return (
    <div>
      <h1>개념 01 - JSX란 무엇인가</h1>

      <p className="guide">
        왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
        Console</strong> 도 함께 보세요.
        <br />
        <br />
        이 파일은 <strong>콘솔이 주인공</strong>입니다. 화면보다 콘솔에 찍히는 글자를
        더 많이 봅니다.
      </p>

      <div className="demo">
        <h3>① JSX 로 만든 화면</h3>
        <div id="root">
          {title}
        </div>
      </div>

      <div className="demo">
        <h3>② React.createElement 로 만든 화면 - ① 과 같습니다</h3>
        <div id="rootCe">
          {
            React.createElement(
              "div",
              null,
              React.createElement("h3", null, "아메리카노"),
              React.createElement("p", null, "4000원")
            )
          }
        </div>
      </div>

      <div className="demo">
        <h3>③ JSX 를 따옴표로 감쌌을 때 (섹션 6에서 설명)</h3>
        <div id="rootQuoted">
          {"<h1>안녕</h1>"}
        </div>
      </div>

    </div>
  );
}

```

중괄호로 값 넣기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01 마지막에 이런 실수를 봤습니다.

```
const name = "김민준";
<p>name 님 안녕하세요</p>    → 화면: name 님 안녕하세요
```

태그와 태그 사이는 '글자 자리' 입니다. 변수 이름을 적어도 글자로 봅니다. 여기에 자바스크립트 값을 넣으려면 표시가 하나 필요합니다.

```
{ } ← 중괄호
```

중괄호는 "여기부터는 글자가 아니라 자바스크립트다" 라는 표시입니다. 이 파일에서는 중괄호 안에 무엇을 넣을 수 있고 무엇을 안 되는지를 정합니다.

중괄호로 값 넣기

섹션 1

```
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";

const name = "김민준";
const age = 20;
```

중괄호를 씌우면 글자가 아니라 값으로 읽습니다.

```
const hello = <p>{name}님 안녕하세요</p>;

console.log(hello.props);
```

```
F12 콘솔    { children: ['김민준', '님 안녕하세요'] }
```

children 이 배열이 되었습니다. React 가 이렇게 나눠서 기억합니다. "김민준" ← 중괄호에서 꺼낸 값 "님 안녕하세요" ← 그냥 글자

화면에서는 이어 붙어서 한 문장으로 보입니다.

중괄호는 한 태그 안에 여러 번 써도 됩니다.

```
const card = (
  <div className="output"> <h3>{name}님</h3> <p>나이: {age}살</p> </div>
);
```

className 은 개념03에서 배웁니다. 지금은 화면 색을 입히는 것이라고만 알면 됩니다. 소괄호로 감싸 여러 줄로 쓴 이유는 개념05에서 설명합니다.

화면 데모 ① 에 "김민준님" 과 "나이: 20살" 이 보입니다.

개념01의 도구로 중괄호가 무엇이 되는지 봅시다.

```
function toJs(jsxCODE) {
  return Babel.transform(jsxCODE, {
    presets: [["@react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}

console.log(toJs("<p>{name}님</p>"));
```

F12 콘솔 /*#__PURE__*/React.createElement("p", null, name, "님");

중괄호가 통째로 사라지고 name 이 그냥 변수로 들어갔습니다. 중괄호 자체는 코드가 아니라 "여기는 자바스크립트다" 라는 표시일 뿐입니다.

글자 자리와 비교하면 차이가 분명합니다.

```
console.log(toJs("<p>name님</p>"));
```

F12 콘솔 /*#__PURE__*/React.createElement("p", null, "name님");

중괄호가 없으면 따옴표가 붙어서 문자열이 됩니다. 그래서 글자로 보였던 것입니다.

▹ 직접 해보기

1

const menu = "아메리카노"; 를 만들고 <p>오늘은 {menu}</p> 의 props 를 콘솔에 찍어 보세요.

중괄호 안에는 '식' 이 들어간다

섹션 2

개념01에서 '식' 을 이렇게 정했습니다.

식 = 계산하고 나면 값 하나가 남는 것

중괄호 안에는 이 '식' 이 들어갑니다. 변수만 되는 것이 아닙니다. 값 하나가 남기만 하면 무엇이든 됩니다. 하나씩 확인해 봅시다.

```
const price = 4000;
const menu = ["아메리카노", "라떼", "케이크"];
const user = { name: "이서연", age: 22 };
```

(1) 계산

```
console.log(toJs("<p>{price * 2}원</p>"));
```

```
F12 콘솔    /#__PURE__*/React.createElement("p", null, price * 2, "원");
```

(2) 함수 호출 — 부르고 나면 돌려준 값 하나가 남습니다

```
console.log(price.toLocaleString());
```

```
F12 콘솔    4,000
```

(3) 메소드 이어 쓰기 (JS자료 06단원)

```
console.log(menu.join(" / "));
```

```
F12 콘솔    아메리카노 / 라떼 / 케이크
```

(4) 객체의 속성 꺼내기 (JS자료 07단원)

```
console.log(user.name);
```

```
F12 콘솔    이서연
```

(5) 삼항 연산자 (JS자료 03단원 개념05)

```
console.log(age >= 19 ? "성인" : "미성년");
```

```
F12 콘솔    성인
```

위 다섯 가지를 전부 한 화면에 넣어 봅시다.

```
const detail = (
  <div className="output"> <p>메뉴 수: {menu.length}개</p> <p>두 잔 값: {price * 2}원
</p> <p>가격 표기: {price.toLocaleString()}원</p> <p>메뉴: {menu.join(" / ")}</p> <p>
  {user.name}님은 {user.age >= 19 ? "성인" : "미성년"}입니다
</p> </div>
);
```

```
화면    데모 ② 에 다섯 줄이 보입니다.
        메뉴 수: 3개 / 두 잔 값: 8000원 / 가격 표기: 4,000원
        메뉴: 아메리카노 / 라떼 / 케이크 / 이서연님은 성인입니다
```

여기서 꼭 기억할 것이 있습니다. 중괄호 안의 코드는 '화면을 그리기 전에' 먼저 계산됩니다. {price * 2} 는 8000 이라는 값이 되고 나서 화면에 들어갑니다. 화면에 "price * 2" 라는 글자가 남는 일은 없습니다.

▫ 직접 해보기

2

<p>{price + 500}원</p> 을 만들어 props 를 찍어 보세요. children 배열에 무엇이 들어가는지 확인하세요.

if 문과 for 문은 왜 안 되나

섹션 3

여기가 이 파일에서 가장 중요한 곳입니다. 중괄호 안에 if 문을 넣으면 어떻게 될까요? 직접 시켜 봅시다.

진짜로 쓰면 파일이 멈추니까, 개념01의 Babel 에게 '문자열로' 물어봅니다. 이렇게 하면 에러 문구만 안전하게 받아 볼 수 있습니다.

```
function tryJsx(code) {
  try {
    Babel.transform(code, { presets: [["react", { runtime: "classic" }]] });
    return "문제 없음";
  } catch (e) {
    return e.message.split("\n")[0];
  }
}

console.log(tryJsx("const x = <div>{ if (age >= 19) { '성인' } }</div>;"));
```

F12 콘솔 unknown: Unexpected token (1:17)

```
console.log(tryJsx("const x = <div>{ for (const m of menu) {} }</div>;"));
```

F12 콘솔 unknown: Unexpected token (1:17)

```
console.log(tryJsx("const x = <div>{ const y = 1; }</div>;"));
```

F12 콘솔 unknown: Unexpected token (1:17)

셋 다 같은 에러입니다. (1:17) 은 17번째 글자에서 막혔다는 뜻이고, 그 자리가 바로 if / for / const 가 시작하는 자리입니다.

삼항 연산자는 어떨까요?

```
console.log(tryJsx("const x = <div>{ age >= 19 ? '성인' : '미성년' }</div>;"));
```

F12 콘솔 문제 없음

되는 것과 안 되는 것의 차이가 무엇일까요. 개념01에서 본 변환 결과를 다시 떠올리면 답이 나옵니다.

```
console.log(toJs("<div>{age >= 19 ? '성인' : '미성년'}</div>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("div", null, age >= 19 ? '성인' :
            '미성년');
```

중괄호 안의 것은 createElement 의 '인자 자리' 로 들어갑니다. 인자 자리에는 값이 들어가야 합니다. 그래서 값이 남는 것만 됩니다.

createElement("div", null, age >= 19 ? "성인" : "미성년") ← 값이 남는다. 된다.

```
createElement("div", null, if (age >= 19) { "성인" })    ← 값이 안 남는다. 안
된다.
```

두 번째 줄을 소리 내어 읽어 보면 이상합니다. 함수에 if 문을 인자로 넘긴 적이 없지 않습니까. 그래서 안 되는 것입니다.

if 문과 for 문은 '문' 입니다. 값을 남기지 않고 '일을 시키는' 것입니다.

```
if (a) { b }    → 실행은 되지만 남는 값이 없습니다.
for (...) {}    → 마찬가지입니다.
```

삼항 연산자는 '식' 입니다. 계산하면 값 하나가 남습니다. age >= 19 ? "성인" : "미성년" → "성인" 이 남습니다.

그래서 JSX 안에서 조건을 다룰 때는 삼항 연산자를 씁니다. 이것이 React 코드에 삼항 연산자가 유난히 많이 보이는 이유입니다.

```
const seat = <p>{user.age >= 19 ? "성인석" : "어린이석"}</p>;
console.log(seat.props);
```

```
F12 콘솔    { children: '성인석' }
```

if 문을 꼭 쓰고 싶으면 방법이 있습니다. 중괄호 '밖' 에서 쓰는 것입니다. 미리 계산해서 변수에 담아 두고, 중괄호에는 그 변수만 넣습니다.

```
let grade;
if (user.age >= 19) {
  grade = "성인";
} else {
  grade = "미성년";
}

console.log(grade);
```

```
F12 콘솔    성인
```

```
const seat2 = <p>{grade}</p>;
console.log(seat2.props);
```

```
F12 콘솔 { children: '성인' }
```

화면 모양이 복잡하면 이 방법이 더 읽기 좋습니다. 조건에 따라 화면을 그리는 방법은 05단원에서 더 자세히 배웁니다.

▹ 직접 해보기

3

tryJsx 에 "const x = <div>{ menu.length > 0 ? '있음' : '없음' }</div>;" 를 넣어 결과가 무엇인지 확인해 보세요.

중괄호에 넣어도 화면에 안 나오는 값들

섹션 4

중괄호에 넣은 값이 전부 글자로 보이는 것은 아닙니다. React 는 몇 가지 값을 일부러 '안 그립니다'. 직접 눈으로 확인합시다.

```
const nothing = (
  <div className="output"> <p>[{true}] ← true</p> <p>[{false}] ← false</p> <p>
  [{null}] ← null</p> <p>[{undefined}] ← undefined</p> <p>[{""}] ← 빈 문자열</p> <p>
  [{0}] ← 숫자 0</p> </div>
);
```

화면 데모 ③ 의 대괄호 안이 다섯 줄은 비어 있고, 마지막 줄만 [0] 입니다.

정리하면 이렇습니다. `true` / `false` / `null` / `undefined` → 아무것도 안 그립니다 숫자 0 → 0 이라고 그립니다 빈 문자열 "" → 아무것도 안 그립니다

0 만 그려지는 것이 05단원에서 함정이 됩니다. 그때 다시 다룹니다.

글자로 보고 싶으면 String 으로 바꿔서 넣으면 됩니다. (JS자료 02단원)

```
console.log(String(true));
```

```
F12 콘솔 true
```

```
const shown = <p>{String(user.age >= 19)}</p>;
console.log(shown.props);
```

```
F12 콘솔 { children: 'true' }
```

그리고 절대 못 넣는 값이 하나 있습니다. 바로 객체입니다.

```
const bad = <p>{user}</p>;
root.render(bad);
```


이렇게 하면 에러가 나면서 화면이 통째로 비어 버립니다. 콘솔에 나오는 문구는 이렇습니다.

Objects are not valid as a React child (found: object with keys {name, age}). If you meant to render a collection of children, use an array instead.

“객체는 React 의 자식이 될 수 없습니다” 라는 뜻입니다. 왜 그럴까요? 객체를 글자로 바꾸면 이렇게 되기 때문입니다.

```
console.log(String(user));
```

```
F12 콘솔 [object Object]
```

화면에 [object Object] 라고 나와도 아무 도움이 안 됩니다. 그래서 React 는 조용히 이상하게 그리는 대신 에러를 내서 알려 줍니다. 화면 어딘가에 [object Object] 가 보인다면 그건 여러분이 직접 String 을 씌운 경우입니다. React 가 그렇게 그려 준 것이 아닙니다.

고치는 방법은 둘 중 하나입니다.

(1) 필요한 속성만 꺼내서 넣는다 — 대부분 이쪽입니다

```
const good1 = (
  <p>
    {user.name} ({user.age}살)
  </p>
);
console.log(good1.props);
```

```
F12 콘솔 { children: ['이서연', '(', 22, '살')] }
```

(2) 통째로 보고 싶으면 `JSON.stringify` 로 글자를 만든다 (JS자료 12단원)

```
console.log(JSON.stringify(user));
```

```
F12 콘솔 {"name":"이서연","age":22}
```

배열은 어떨까요? 배열은 에러가 안 납니다. 하나씩 이어서 그림니다.

```
const numbers = <p>{[1, 2, 3]}</p>;
console.log(numbers.props);
```

```
F12 콘솔 { children: [1, 2, 3] }
```

화면에는 123 으로 붙어서 나옵니다. 사이에 아무것도 안 넣어 주기 때문입니다. 사이를 띄우려면 `join` 을 쓰면 됩니다.

```
console.log([1, 2, 3].join(", "));
```

```
F12 콘솔 1, 2, 3
```

배열 안에 태그를 담아 그리는 방법은 05단원에서 배웁니다.

▹ 직접 해보기 4

`<p>{menu}</p>` 를 만들어 화면에 어떻게 나올지 먼저 예상하고, props 를 콘솔에 찍어 확인해 보세요.

중괄호 안의 주석

섹션 5

JSX 한가운데에 주석을 달고 싶을 때가 있습니다. 그런데 태그 사이는 '글자 자리' 라서, `//` 를 쓰면 그대로 화면에 나옵니다.

```
const wrongComment = <p>// 이건 주석이 아닙니다</p>;
console.log(wrongComment.props);
```

```
F12 콘솔 { children: '// 이건 주석이 아닙니다' }
```

글자로 들어갔습니다. 화면에도 그대로 보일 것입니다.

그러면 어떻게 할까요? 답은 이미 배운 것 안에 있습니다. 중괄호 안은 자바스크립트 자리입니다. 자바스크립트 주석을 쓰면 됩니다.

```
{/* 이렇게 씁니다 */}
```

중괄호를 열고, 여러 줄 주석을 쓰고, 중괄호를 닫습니다. 실제로 어떻게 처리되는지 확인해 봅시다.

```
console.log(toJs("<p>{/* 안 보이는 메모 */}보이는 글자</p>"));
```

```
F12 콘솔 /##__PURE__*/React.createElement("p", null, "보이는 글자");
```

주석이 통째로 사라졌습니다. 화면에도 당연히 안 나옵니다.

```
const withComment = (
  <p className="output">
    {/* 아래 줄은 손님 이름입니다 */}
    {name}님
  </p>
);
console.log(withComment.props.children);
```

```
F12 콘솔 ['김민준', '님']
```

children 에 주석은 흔적도 없습니다.

참고로 태그 '밖' 에서는 평소처럼 `//` 를 쓰면 됩니다. 지금 여러분이 읽고 있는 이 줄이 그 경우입니다.

toJs 에 "`<p>{* 메모 *}아메리카노</p>`" 를 넣어 변환 결과에 주석이 남는지 확인해 보세요.

자주 하는 실수

섹션 6

⚠️ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

중괄호를 빼먹는다

에러가 안 납니다. 화면에 변수 이름이 그대로 나옵니다.

```
const forgot = <p>name 님 안녕하세요</p>;
console.log(forgot.props);
```

F12 콘솔 { children: 'name 님 안녕하세요' }

이런 실수를 합니다

children 이 통째로 문자열이면 종괄호를 빠뜨린 것입니다. 화면에 변수 이름이 보이면 이 실수부터 의심하세요.

이런 실수를 합니다

중괄호를 따옴표 안에 넣는다

이것도 예러가 안 납니다.

```
const inQuote = <p className="output">{"{name}"}</p>;
console.log(inQuote.props.children);
```

F12 콘솔 {name}

이런 실수를 합니다

따옴표 안의 중괄호는 그냥 글자입니다. 화면에 {name} 이 그대로 나옵니다.

이런 실수를 합니다

중괄호 안에 if 문을 쓴다 (섹션 3에서 봤습니다)

```
<div>{ if (age >= 19) { "성인" } }</div>
```

[SyntaxError] Unexpected token 이 납니다. 삼항 연산자를 쓰거나, 중괄호 밖에서 미리 변수에 담아 두고 그 변수를 넣으세요.

이런 실수를 합니다

객체를 그대로 넣는다 (섹션 4에서 봤습니다)

```
<p>{user}</p>
```

Objects are not valid as a React child 에러가 나고 화면이 빕니다. user.name 처럼 속성을 꺼내서 넣으세요.

이런 실수를 합니다

중괄호 안에 세미콜론을 찍는다

```
<p>{name;}</p>
```

[SyntaxError] 가 납니다. 중괄호 안은 '식' 자리라서 문장 끝 표시가 못 옵니다. 세미콜론을 지우면 됩니다.

```
console.log(tryJsx("const x = <p>{name;}</p>;"));
```

F12 콘솔 unknown: Unexpected token, expected "]" (1:18)

이런 실수를 합니다

중괄호를 두 개 겹쳐 쓴다

```
<p>{{name}}</p>
```

화면이 통째로 비고 섹션 4의 객체 에러가 납니다. 안쪽 중괄호가 객체로 읽히기 때문입니다.

자바스크립트에서 { name } 은 { name: name } 의 줄여 쓴 모양입니다. (키와 변수 이름이 같으면 한 번만 적어도 됩니다. 이 자료에서 처음 나왔습니다) 그래서 객체 하나를 통째로 넣은 셈이 됩니다. 중괄호 두 개를 제대로 쓰는 자리는 style 뿐입니다. 개념03에서 배웁니다.

직접 해보기 정답

```
1) const menu2 = "아메리카노";
console.log((<p>오늘은 {menu2}</p>).props);
// 콘솔: { children: ['오늘은 ', '아메리카노'] }
```

→ 글자 "오늘은 " 과 중괄호에서 꺼낸 값이 배열로 나뉘어 담깁니다.

뒤의 공백까지 글자에 포함된다는 점을 보세요.

```
2) console.log(<p>{price + 500}원</p>).props);
// 콘솔: { children: [4500, '원'] }
```

→ 4500 은 따옴표가 없습니다. 계산이 먼저 끝나서 숫자로 들어갔기 때문입니다.

```
3) console.log(tryJsx("const x = <div>{ menu.length > 0 ? '있음' : '없음' }
</div>;"));
// 콘솔: 문제 없음
```

→ 삼항 연산자는 값이 하나 남은 '식' 이라 인자 자리에 들어갈 수 있습니다.

```
4) console.log(<p>{menu}</p>).props);
// 콘솔: { children: ['아메리카노', '라떼', '케이크'] }
```

→ 화면에는 "아메리카노라떼케이크" 로 붙어서 나옵니다.

사이를 띄우려면 {menu.join(" / ")} 처럼 join 을 쓰세요.

```
5) console.log(toJs("<p>{/* 메모 */}아메리카노</p>"));
// 콘솔: /*#__PURE__*/React.createElement("p", null, "아메리카노");
```

→ 주석은 변환 결과에 남지 않습니다. 화면에도 안 나옵니다.

정리

- 1 태그 사이에 자바스크립트 값을 넣으려면 { } 로 감쌉니다. 안 감싸면 글자가 됩니다.
- 2 중괄호 안에는 식만 들어갑니다. 계산·함수 호출·속성 꺼내기·삼항 연산자가 됩니다.
- 3 if 문과 for 문은 값을 남기지 않아서 안 됩니다. 밖에서 변수에 담아 두고 그 변수를 넣으세요.
- 4 중괄호 안의 것은 createElement 의 인자 자리로 들어갑니다. 그래서 값만 됩니다.
- 5 true·false·null·undefined·빈 문자열은 화면에 안 나옵니다. 숫자 0 은 나옵니다.
- 6 객체를 그대로 넣으면 에러입니다. 속성을 꺼내서 넣으세요. 배열은 이어 붙여 씁니다.
- 7 JSX 안의 주석은 { /* ... */ } 로 씁니다.

```
export default function Concept02() {
  return (
    <div> <h1>개념 02 - 중괄호로 값 넣기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
    </p> <div className="demo"> <h3>① 중괄호로 값을 넣은 화면 (섹션 1)
```

```

</h3> <div id="root">
  {card}
  </div> </div> <div className="demo"> <h3>② 종괄호 안에 넣을 수 있는 것들
(섹션 2)</h3> <div id="rootDetail">
  {detail}
  </div> </div> <div className="demo"> <h3>③ 화면에 안 보이는 값들 (섹션 4)
</h3> <div id="rootFalsy">
  {nothing}
  </div> </div> </div>

);
}

```

속성 넣기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

지금까지는 태그 '사이'에 무엇을 넣을지만 봤습니다.

```
<p>여기에 들어가는 것</p>
```

이번에는 태그 '안쪽'에 붙이는 것을 봅시다.

```
<p className="output" title="설명">글자</p>
~~~~~
이 부분을 속성이라고 부릅니다
```

속성은 그 태그를 어떻게 다룰지 알려 주는 추가 정보입니다. 색을 입히거나, 그림 주소를 지정하거나, 버튼을 잠그는 데 씁니다.

JSX의 속성은 HTML의 속성과 아주 비슷하지만 몇 군데가 다릅니다. 그 '다른 몇 군데'가 이 파일의 내용 전부입니다.

```
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";

function toJs(jsxCODE) {
  return Babel.transform(jsxCODE, {
    presets: [["react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}
```

속성은 두 번째 인자로 들어간다

섹션 1

개념01에서 `createElement`의 두 번째 인자가 `null`이었던 것을 기억하세요.

```
React.createElement("h1", null, "안녕")
~~~~~
여기가 속성 자리였습니다
```

속성을 붙이면 그 자리에 무엇이 들어가는지 봅시다.

```
console.log(toJs('<p title="설명">글자</p>'));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("p", { title: "설명" }, "글자");
```

null 이던 자리에 객체가 들어갔습니다. (JS자료 07단원 객체) 속성 이름이 키가 되고, 속성 값이 값이 됩니다.

속성을 두 개 붙이면 키가 두 개인 객체가 됩니다.

```
console.log(toJs('<p title="설명" lang="ko">글자</p>'));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("p", { title: "설명", lang: "ko" },  
"글자");
```

만들어진 엘리먼트에서도 확인할 수 있습니다.

```
const withTitle = <p title="마우스를 올려 보세요">아메리카노</p>;
```

```
console.log(withTitle.props);
```

```
F12 콘솔    { title: '마우스를 올려 보세요', children: '아메리카노' }
```

props 안에 속성과 children 이 함께 들어 있습니다. 즉 React 에게 속성과 '태그 사이의 내용' 은 같은 종류의 정보입니다. 둘 다 props 라는 객체 하나에 담깁니다.

이 props 라는 이름은 03단원에서 다시 아주 중요하게 나옵니다.

▹ 직접 해보기

1

toJs 에 '`<p lang="ko">라떼</p>`' 를 넣어 두 번째 인자가 어떤 객체가 되는지 확인해 보세요.

class 가 아니라 className

섹션 2

HTML 에서 색이나 모양을 주려면 class 를 씁니다.

```
<p class="output">글자</p>    ← HTML
```

JSX 에서는 이렇게 씁니다.

```
<p className="output">글자</p>    ← JSX
```

왜 이름이 다를까요? 규칙이라서가 아닙니다. 이유가 있습니다.

개념01에서 본 것처럼 속성은 결국 자바스크립트 객체의 키가 됩니다. 그런데 class 는 자바스크립트가 이미 쓰고 있는 예약어입니다. (클래스를 만들 때 쓰는 단어입니다. 이 자료에서는 안 씁니다)

예약어를 키로 쓰면 옛날 브라우저에서 문제가 생겼습니다. 그래서 React 는 실제 DOM 이 쓰는 이름을 그대로 가져왔습니다. JS자료 10단원에서 클래스를 붙일 때 이렇게 썼던 것을 떠올려 보세요.

```
element.className = "output";
element.classList.add("on");
```

className 은 React 가 새로 만든 말이 아니라, 여러분이 이미 쓰던 말입니다.

```
const styled = (
  <div> <p className="output">className 을 붙인 줄입니다</p> <p className="output
on">클래스 두 개는 한 문자열에 띄어쓰기로 씁니다</p> </div>
);
```

화면 데모 ① 첫 줄은 흰 상자, 둘째 줄은 파란 바탕에 흰 글자입니다.

```
console.log(styled.props.children[1].props.className);
```

F12 콘솔 output on

클래스를 여러 개 줄 때 className 을 두 번 쓰지 않습니다. 한 문자열 안에 띄어쓰기로 이어 씁니다. HTML 과 같습니다.

▸ 직접 해보기

2

<p className="output"> 의 className 을 "output done" 으로 바꿔 보세요. 화면의 글자에 줄이 그어지는지 확인하세요.

속성에 값 넣기

섹션 3

속성 값에도 자바스크립트 값을 넣을 수 있습니다. 방법은 개념02와 같습니다. 따옴표 대신 중괄호를 씁니다.

```
title="설명"      ← 글자 그대로
title={message} ← 변수의 값
```

따옴표와 중괄호를 같이 쓰지 않습니다. 둘 중 하나만 씁니다.

```
const linkUrl = "https://example.com";

console.log(toJs("<a href={linkUrl}>이동</a>"));
```

F12 콘솔 /*#__PURE__*/React.createElement("a", { href: linkUrl }, "이동");

```
console.log(toJs('<a href="linkUrl">이동</a>'));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("a", { href: "linkUrl" }, "이동");
```

위는 변수의 값이 들어가고, 아래는 "linkUrl" 이라는 글자가 그대로 들어갑니다. 개념02에서 본 차이와 완전히 같습니다.

그림 주소를 변수로 넣어 봅시다. 아래 긴 글자는 그림 파일 하나를 글자로 적어 둔 것입니다. 인터넷 없이도 보이려고 이렇게 넣었습니다. 내용은 몰라도 됩니다.

```
const blueBox =
  "data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='60'
  height='60'><rect width='60' height='60' fill='%232d6cdf' /></svg>";

const attrDemo = (
  <div> <img src={blueBox} alt="파란 사각형" width={60} height={60} /> <p> <a href=
  {linkUrl}>이 링크의 주소는 변수에서 왔습니다
  </a> </p> <input type="text" placeholder="여기는 04단원부터 씁니다" /> <p> <button>
  누를 수 있는 버튼</button> <button disabled>못 누르는 버튼</button> </p> </div>
);
```

화면 데모 ② 에 파란 사각형, 링크, 입력칸, 버튼 두 개가 보입니다.
오른쪽 버튼은 회색이고 눌리지 않습니다.

숫자를 넣을 때를 특히 조심하세요.

```
width={60}     ← 숫자 60
width="60"     ← 글자 "60"
```

이 둘은 그림에서는 결과가 같아 보이지만 값의 종류가 다릅니다. 03단원에서 값을 넘겨줄 때 이 차이가 중요해집니다.

```
console.log(toJs("<img width={60} />"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("img", { width: 60 });
```

```
console.log(toJs('<img width="60" />'));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("img", { width: "60" });
```

```
console.log(typeof 60, typeof "60");
```

```
F12 콘솔    number string
```

▹ 직접 해보기 3

위 attrDemo 의 에 width={120} height={120} 을 주고 사각형이 커지는지 확인해 보세요.

style 은 중괄호가 두 개다

섹션 4

여기가 처음 볼 때 가장 헷갈리는 곳입니다.

```
<p style={{ color: "white" }}>글자</p>
      ~~~~~
      중괄호가 두 개입니다
```

두 개가 각각 무슨 뜻인지 떼어 놓고 보면 하나도 안 어렵습니다.

```
바깥 중괄호 { } ← 개념02에서 배운 것. "여기는 자바스크립트다"
안쪽 중괄호 { } ← JS자료 07단원의 객체. "이건 객체다"
```

즉 style 에는 '객체를 넣는다' 는 뜻입니다. 그래서 객체를 먼저 만들어 두면 중괄호가 하나로 줄어듭니다. 확인해 봅시다.

```
const boxStyle = { color: "white", backgroundColor: "#2d6cdf", padding: 10 };

console.log(typeof boxStyle);
```

F12 콘솔 object

```
console.log(boxStyle);
```

F12 콘솔 { color: 'white', backgroundColor: '#2d6cdf', padding: 10 }

이렇게 만들어 두면 중괄호 한 개로 넣습니다.

```
<p style={boxStyle}>글자</p>
```

두 방식이 완전히 같다는 것을 변환 결과로 확인합시다.

```
console.log(toJs("<p style={boxStyle}>글자</p>"));
```

F12 콘솔 /*#__PURE__*/React.createElement("p", { style: boxStyle }, "글자");

```
console.log(toJs('<p style={{ color: "white" }}>글자</p>'));
```

F12 콘솔 /*#__PURE__*/React.createElement("p", { style: { color: "white" } }, "글자");

아래쪽은 객체를 그 자리에서 바로 만들어 넣은 것뿐입니다. 중괄호가 두 개라서 특별한 문법인 것이 아닙니다.

이제 CSS 속성 이름을 봅시다. HTML 과 이름이 다릅니다.

background-color	← CSS 에서 쓰는 이름
backgroundColor	← JSX 에서 쓰는 이름

이것도 이유가 있습니다. 객체의 키로 쓰기 때문입니다. 자바스크립트에서 이렇게 쓰면 어떻게 될까요?

```
{ background-color: "yellow" }
```

자바스크립트는 이 줄을 “background 빼기 color” 라는 뽕셈으로 읽습니다. 키 이름에 빼기 기호를 그냥 쓸 수 없기 때문입니다. 그래서 대시를 없애고 뒷 글자를 대문자로 올립니다. 카멜케이스입니다.

background-color	→	backgroundColor
font-size	→	fontSize
text-align	→	textAlign
border-radius	→	borderRadius

이름 짓는 규칙 하나면 전부 해결됩니다. 외울 목록이 아닙니다.

```
const styleDemo = (
  <div> <p style={boxStyle}>미리 만든 객체를 넣었습니다</p> <p style={{ color:
"#c00", fontWeight: "bold" }}>그 자리에서 만든 객체입니다</p> <p style=
{{ fontSize: 24 }}>숫자만 쓰면 px 이 자동으로 붙습니다</p> <p style={{ fontSize:
"24pt" }}>px 이 아니면 단위를 글자로 적습니다</p> </div>
);
```

화면 데모 ③ 첫 줄은 파란 바탕에 흰 글자입니다.
 둘째 줄은 빨간 굵은 글자, 셋째·넷째 줄은 큰 글자입니다.

숫자 이야기를 한 번 더 봅시다.

fontSize: 24	→	화면에서는 24px
fontSize: "24pt"	→	화면에서는 24pt

숫자만 쓰면 React 가 px 을 붙여 줍니다. 다른 단위는 글자로 적어야 합니다. 실제로 붙는지 화면의 셋째 줄과 넷째 줄 크기를 비교해 보세요.

⇒ 직접 해보기

4

styleDemo 의 두 번째 줄 style 에 textAlign: "center" 를 더해 보세요. 글자가 가운데로 가는지 확인하세요.

이름이 바뀐 다른 속성들

섹션 5

className 말고도 이름이 바뀐 속성이 몇 개 있습니다. 전부 같은 이유입니다. 자바스크립트 예약어이거나, 대시가 들어 있어서입니다.

HTML	JSX	왜
class	→ className	class 는 예약어
for	→ htmlFor	for 는 예약어 (반복문)
tabindex	→ tabIndex	카멜케이스로
maxlength	→ maxLength	카멜케이스로
readonly	→ readOnly	카멜케이스로

htmlFor 는 입력칸에 이름표를 붙일 때 씁니다. 이름표를 눌러도 입력칸이 선택되게 해 주는 속성입니다.

```
const labelDemo = (
  <p> <label htmlFor="nameBox">이름:
</label> <input id="nameBox" type="text" placeholder="이름표를 눌러 보세요" /> </p>
);

console.log(labelDemo.props.children[0].props.htmlFor);
```

F12 콘솔 nameBox

화면 데모 ④ 에 "이름:" 이름표와 입력칸이 보입니다.
"이름:" 글자를 눌러도 입력칸에 커서가 갑니다.

반대로 안 바뀌는 것이 훨씬 많습니다. 대부분은 HTML 이름 그대로입니다. id · src · href · alt · title · type · placeholder · value · width · height

그래서 외울 것은 사실상 className 과 htmlFor 두 개뿐입니다. 나머지는 잘못 쓰면 React 가 콘솔에서 바른 이름을 알려 줍니다. 그 경고 문구는 섹션 7에서 그대로 보여 드립니다.

▸ 직접 해보기 5

labelDemo 의 input 에 maxLength={3} 을 넣고, 입력칸에 네 글자를 쳐 보세요. 세 글자에서 멈춥니다.

불리언 속성

섹션 6

값이 없이 이름만 적는 속성이 있습니다.

```
<button disabled>못 누름</button>
```

HTML 에서도 이렇게 씁니다. 값을 안 적으면 “켜짐” 이라는 뜻입니다. JSX 에서 이것이 무엇이 되는지 봅시다.

```
console.log(toJs("<button disabled>못 누름</button>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("button", { disabled: true }, "못  
누름");
```

`true` 가 자동으로 들어갔습니다. 그러면 “끄고 싶을 때” 는 어떻게 할까요? 이름을 지우거나, `false` 를 넣습니다.

```
console.log(toJs("<button disabled={false}>누를 수 있음</button>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("button", { disabled: false }, "누를 수  
있음");
```

여기서 아주 자주 하는 실수가 하나 나옵니다.

```
<button disabled="false"> ← 이러면 잠깁니다
```

“false” 는 글자입니다. 빈 문자열이 아닌 글자는 참으로 취급됩니다. (JS자료 03단원에서 본 truthy 이야기입니다) 그래서 잠금이 풀리지 않습니다. 반드시 중괄호로 값을 넣어야 합니다.

코드

```
console.log(Boolean("false"));
```

```
console.log(Boolean(false));
```

▶ F12 콘솔

▶ true

▶ false

조건에 따라 잠그려면 중괄호 안에 비교식을 넣습니다.

```
const stock = 0;
```

```
const buyButton = <button disabled={stock === 0}>담기</button>;
```

```
console.log(buyButton.props.disabled);
```

```
F12 콘솔    true
```

화면 데모 ⑤ 에 회색 “담기” 버튼이 보입니다. 재고가 0이라 잠겼습니다.

▫ 직접 해보기

6

stock 을 3 으로 바꾸고 저장해 보세요. “담기” 버튼의 잠금이 풀리는지 확인하세요.

자주 하는 실수

섹션 7

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

className 대신 class 를 쓴다

```
<p class="output">글자</p>
```

에러는 안 납니다. 화면도 어찌어찌 그려집니다. 하지만 콘솔에 빨간 경고가 나옵니다.

```
Invalid DOM property `class`. Did you mean `className`?
```

"class 라는 속성은 없습니다. className 을 말한 건가요?" 라는 뜻입니다.
콘솔에 Did you mean 이 보이면 그 이름으로 바꾸면 끝입니다.

이런 실수를 합니다

htmlFor 대신 for 를 쓴다

```
<label for="nameBox">이름</label>
```

같은 모양의 경고가 나옵니다.

```
Invalid DOM property `for`. Did you mean `htmlFor`?
```

이런 실수를 합니다

style 에 문자열을 넣는다

```
<p style="color: red">글자</p>
```

에러가 나면서 화면이 비어 버립니다. HTML 습관이 그대로 나오는 자리입니다. 콘솔 문구는 이렇습니다.

```
The `style` prop expects a mapping from style properties to values,
not a string.
```

"style 에는 문자열이 아니라 객체를 달라" 는 뜻입니다.
style={{ color: "red" }} 로 고치세요.

이런 실수를 합니다

style 객체에 CSS 이름을 그대로 쓴다

```
<p style={{ "background-color": "yellow" }}>글자</p>
```

에러는 안 나지만 색이 안 칠해집니다. 그리고 콘솔에 경고가 나옵니다.

```
Unsupported style property background-color.
Did you mean backgroundColor?
```

"안 칠해지는데 에러도 없다" 면 이름부터 카멜케이스로 고쳐 보세요.

이런 실수를 합니다

중괄호와 따옴표를 같이 쓴다

```

```

에러는 안 납니다. 그런데 그림이 안 보입니다. 따옴표 안이라서 "{blueBox}" 라는 글자가 주소로 들어갔기 때문입니다. 그런 주소는 없으니 그림도 없습니다. 따옴표를 지우세요.

```
const wrongSrc = ;
console.log(wrongSrc.props.src);
```

```
F12 콘솔    {blueBox}
```

이런 실수를 합니다

props.src 가 주소가 아니라 중괄호 글자 그대로면 이 실수입니다.

이런 실수를 합니다

속성 이름과 값 사이에 띄어쓰기를 넣는다

```
<p className = "output">글자</p>
```

이건 사실 동작합니다. 하지만 아무도 이렇게 쓰지 않습니다. 속성은 className="output" 처럼 붙여 씁니다.

이런 실수를 합니다

disabled="false" 로 끄려고 한다 (섹션 6에서 봤습니다)

이런 실수를 합니다

글자 "false" 는 참으로 취급되어 버튼이 계속 잠깁니다. `disabled={false}` 로 쓰거나 속성을 아예 지우세요.

직접 해보기 정답

```
1) console.log(toJs('<p lang="ko">라떼</p>'));
// 콘솔: /#__PURE__*/React.createElement("p", { lang: "ko" }, "라떼");
```

→ 속성 이름이 키, 속성 값이 값인 객체 하나가 두 번째 인자로 들어갑니다.

2) `<p className="output done">` className 을 붙인 줄입니다

```
// 화면: 글자에 취소선이 그어지고 회색이 됩니다.
```

→ done 은 전역 CSS(src/index.css)에 미리 정의해 둔 클래스입니다.

클래스 두 개는 한 문자열에 띄어쓰기로 이어 씁니다.

```
3) <img src={blueBox} alt="파란 사각형" width={120} height={120} />
// 화면: 파란 사각형이 가로세로 두 배가 됩니다.
```

→ 중괄호 안은 숫자입니다. 따옴표를 쓰면 글자가 됩니다.

```
4) <p style={{ color: "#c00", fontWeight: "bold", textAlign: "center" }}>
// 화면: 빨간 굵은 글자가 상자 가운데로 옮겨집니다.
```

→ text-align 이 아니라 textAlign 입니다. 대시를 빼고 뒷 글자를 대문자로.

```
5) <input id="nameBox" type="text" maxLength={3} placeholder="이름표를 눌러 보세요" />
// 화면: 네 글자를 쳐도 세 글자에서 더 들어가지 않습니다.
```

→ maxLength 가 아니라 maxLength 입니다. 소문자로 쓰면 경고가 납니다.

```
6) const stock = 3;
// 콘솔: false
// 화면: "담기" 버튼이 검은 글씨가 되고 눌립니다.
```

→ stock === 0 이 false 라서 disabled 가 꺼집니다.

누른 뒤에 무슨 일이 일어나게 하는 것은 04단원입니다.

정리

- 1 속성은 `createElement` 의 **두 번째 인자**, 즉 객체가 됩니다. 태그 사이 내용과 함께 `props` 에 담깁니다.
- 2 `class` 는 자바스크립트 예약어라서 `className` 을 씁니다. `for` 도 같은 이유로 `htmlFor` 입니다.
- 3 속성 값에 자바스크립트를 넣으려면 중괄호를 씁니다. 따옴표와 같이 쓰지 않습니다.
- 4 `style={{ }}` 의 바깥 중괄호는 "자바스크립트다", 안쪽 중괄호는 "객체다" 라는 뜻입니다.
- 5 CSS 속성 이름은 대시를 빼고 카멜케이스로 씁니다. `background-color` → `backgroundColor`.
- 6 숫자만 쓴 크기 값에는 `px` 이 자동으로 붙습니다. 다른 단위는 글자로 적습니다.
- 7 이름만 적은 속성은 `true` 입니다. 끌 때는 `{false}` 로 씁니다. "`false`" 는 글자라서 켜진 것으로 취급됩니다.

```

export default function Concept03() {
  return (
    <div>
      <h1>개념 03 - 속성 넣기</h1>

      <p className="guide">
        왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
        Console</strong> 도 함께 보세요.
      <br />
      <br />
      이 파일에는 <strong>버튼이 나옵니다. 눌러도 아무 일도 일어나지 않습니다.
      </strong> 버튼을 동작하게 만드는 것은 04단원입니다. 여기서는 모양만 봅니다.
      </p>

      <div className="demo">
        <h3>① className (섹션 2)</h3>
        <div id="rootClass">
          {styled}
        </div>
      </div>

      <div className="demo">
        <h3>② 속성에 값 넣기 (섹션 3)</h3>
        <div id="rootAttr">
          {attrDemo}
        </div>
      </div>

      <div className="demo">
        <h3>③ style (섹션 4)</h3>
        <div id="root">
          {styleDemo}
        </div>
      </div>

      <div className="demo">
        <h3>④ htmlFor 로 이름표 붙이기 (섹션 5)</h3>
        <div id="rootLabel">

```

```
        {labelDemo}  
      </div>  
    </div>  
  
    <div className="demo"> <h3>⑤ 불리언 속성 (섹션 6)</h3> <div id="rootBuy">  
      {buyButton}  
    </div> </div>  
  
  </div>  
);  
}
```

태그 규칙

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

JSX 는 HTML 과 비슷하게 생겼습니다. 그래서 HTML 습관대로 쓰다가 막힙니다. 막히는 자리는 딱 네 군데입니다.

- 1 태그를 나란히 두 개 쓸 수 없다
- 2 안 닫은 태그를 그냥 둘 수 없다
- 3 태그 이름의 대소문자가 뜻을 바꾼다
- 4 여는 태그와 닫는 태그의 이름이 정확히 같아야 한다

네 가지 다 “규칙이니 외우세요” 로 넘어갈 것이 없습니다. 개념01에서 본 변환 결과를 보면 전부 이유가 보입니다.

```
import React from "react";
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";
import 그려진뒤 from "../_ui/그려진뒤.js";

function toJs(jsxCODE) {
  return Babel.transform(jsxCODE, {
    presets: [["react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}

function tryJsx(code) {
  try {
    Babel.transform(code, { presets: [["react", { runtime: "classic" }]] });
    return "문제 없음";
  } catch (e) {
    return e.message.split("\n")[0];
  }
}
```

화면에 제목과 가격을 나란히 보여 주고 싶습니다. 01단원에서 배운 대로 App 함수가 화면을 돌려주게 만들어 봅시다.

```
function App() {
  return <h1>아메리카노</h1><p>4000원</p>;
}
```

이렇게 쓰면 안 됩니다. Babel 에게 물어봅시다.

```
console.log(tryJsx("function App() { return <h1>아메리카노</h1><p>4000원</p>; }"));
```

```
F12 콘솔    unknown: Adjacent JSX elements must be wrapped in an enclosing tag. Did
            you want a JSX fragment <>...</>? (1:38)
```

“나란히 있는 JSX 는 감싸는 태그 하나로 묶어야 합니다” 라는 뜻입니다. 친절하게도 뒤에 해결 방법까지 알려 줍니다. 그건 섹션 3에서 씁니다.

왜 이런 제한이 있을까요? JS자료 05단원에서 이미 답을 봤습니다.

```
function getTwo() {
  return 1, 2;
}
console.log(getTwo()); → 2
```

함수는 값을 하나만 돌려줍니다. 두 개를 적어도 하나만 나갑니다. 그래서 여러 값을 돌려주려면 배열이나 객체로 ‘묶어서’ 하나로 만들었습니다.

JSX 도 똑같습니다. 개념01에서 본 것처럼 JSX 는 값 하나입니다.

```
const one = <h1>아메리카노</h1>;
console.log(one.type);
```

```
F12 콘솔    h1
```

태그 하나 = 값 하나입니다. 그러면 태그 두 개는 값 두 개입니다. 값 두 개를 `return` 자리에 그냥 늘어놓을 수 없습니다.

실제로 값 두 개를 만들어 보면 이렇게 됩니다.

```
const two = [<h1>아메리카노</h1>, <p>4000원</p>];
console.log(two.length);
```

```
F12 콘솔    2
```

배열에 넣으면 두 개를 담을 수 있습니다. 담긴 것이 두 개라는 게 눈에 보입니다. `return` 뒤에 늘어놓은 것은 이 두 개를 묶지 않고 그냥 둔 것과 같습니다.

그래서 답은 하나입니다. 하나로 묶어야 합니다. 묶는 방법이 두 가지 있습니다. 섹션 2와 섹션 3에서 하나씩 봅니다.

▸ 직접 해보기 1

tryJsx 에 "`const x = <p>가</p><p>나</p>;`" 를 넣어 같은 에러가 나는지 확인해 보세요.

방법 1 — 태그로 감싸기

섹션 2

가장 쉬운 방법은 `div` 같은 태그로 감싸는 것입니다.

```
function App() {
  return (
    <div className="output"> <h1>아메리카노</h1> <p>4000원</p> </div>
  );
}

console.log(App().type);
```

F12 콘솔 div

```
console.log(App().props.children.length);
```

F12 콘솔 2

바깥은 `div` 하나입니다. 그래서 `return` 이 돌려주는 값은 하나입니다. 그 하나 안에 두 개가 들어 있습니다. 배열로 묶었던 것과 같은 방식입니다.

화면 데모 ① 에 "아메리카노" 와 "4000원" 이 흰 상자 안에 보입니다.

방금 `<App />` 이라고 대문자로 쓴 것이 눈에 띄었을 것입니다. 그 규칙은 섹션 5에서 설명합니다. 지금은 넘어가세요.

이 방법에는 대가가 하나 있습니다. 실제 화면에 `div` 가 하나 더 생깁니다. 눈으로는 안 보이니 코드로 확인합시다.

```
그려진뒤("#rootDiv", (자리) => {
  console.log(자리.innerHTML);
```

F12 콘솔 <div class="output"><h1>아메리카노</h1><p>4000원</p></div>

```
});
```

우리가 원하는 것은 h1 과 p 두 개인데 div 가 하나 더 붙었습니다. 대부분은 문제가 없습니다. 그런데 문제가 되는 자리가 있습니다.

- ul 안에는 li 만 들어가야 하는데 div 가 끼면 목록 모양이 깨집니다 - 표(table) 안에도 아무 태그나 넣을 수 없습니다 - 화면 모양을 잡는 규칙이 div 하나 때문에 어긋나기도 합니다

이럴 때 쓰라고 만든 것이 섹션 3의 Fragment 입니다.

▫ 직접 해보기 2

App 의 바깥 div 를 section 으로 바꿔 보세요. `App().type` 이 무엇으로 바뀌는지 확인하세요.

방법 2 — Fragment <>...</>

섹션 3

이름은 낯설지만 생김새는 아주 단순합니다.

```
<> ... </>
```

이름 없는 태그입니다. 감싸는 역할만 하고 화면에는 안 남습니다.

```
function AppFragment() {
  return (
    <> <h1>아메리카노</h1> <p>4000원</p> </>
  );
}
```

화면 데모 ② 는 데모 ① 과 거의 같아 보입니다. 흰 상자 테두리만 없습니다.

화면에 정말 안 남는지 확인합시다.

```
그려진뒤("#rootFrag", (자리) => {
  console.log(자리.innerHTML);
```

```
F12 콘솔      <h1>아메리카노</h1><p>4000원</p>
```

```
});
```

데모 ① 에는 있던 바깥 div 가 없습니다. 우리가 쓴 두 개만 남았습니다.

그러면 Fragment 는 무엇으로 바뀔까요?

```
console.log(toJs("<><h1>아메리카노</h1><p>4000원</p></>"));
```

```
F12 콘솔      /*__PURE__*/React.createElement(React.Fragment, null,
/*__PURE__*/React.createElement("h1", null, "아메리카노"),
/*__PURE__*/React.createElement("p", null, "4000원"));
```


첫 번째 인자가 "div" 같은 글자가 아니라 React.Fragment 입니다. React 가 미리 만들어 둔 특별한 표시입니다. "이건 화면에 만들지 말고 안쪽만 꺼내 놓아라" 라는 뜻입니다.

```
console.log(AppFragment().type === React.Fragment);
```

```
F12 콘솔 true
```

언제 무엇을 쓰면 될까요.

감싸는 태그에 className 이나 style 을 줄 것이다 → div 로 감싼다 그냥 묶기만 하면 된다 → Fragment 로 감싼다

헛갈리면 Fragment 를 먼저 쓰고, 꾸밀 일이 생기면 div 로 바꾸면 됩니다.

▸ 직접 해보기 3

AppFragment 의 `<> </>` 를 `<div> </div>` 로 바꾸고 데모 ② 의 innerHTML 이 어떻게 바뀌는지 확인하세요.

스스로 달는 태그

섹션 4

HTML 에서는 이렇게 써도 잘 돌아갔습니다.

```
<br> <hr>  <input type="text">
```

JSX 에서는 안 됩니다. 확인해 봅시다.

```
console.log(tryJsx("const x = <br>;"));
```

```
F12 콘솔 unknown: Unterminated JSX contents. (1:14)
```

"JSX 내용이 안 끝났습니다" 라는 뜻입니다. Babel 은 `<br>` 을 '여는 태그' 로 읽고 `</br>` 을 계속 찾다가 파일 끝에 닿았습니다.

왜 HTML 은 되고 JSX 는 안 될까요? HTML 은 브라우저가 알아서 봐 주는 문서입니다. 조금 틀려도 고쳐서 읽습니다. JSX 는 코드입니다. 코드는 짝이 안 맞으면 그 자리에서 멈춥니다.

해결은 끝에 슬래시를 붙이는 것입니다.

```
<br /> <hr />  <input type="text" />
```

"여기서 바로 닫는다" 는 표시입니다.

```
console.log(toJs("<br />"));
```

```
F12 콘솔 /*#__PURE__*/React.createElement("br", null);
```

```
const selfClosing = (
  <div> <p>
    첫째 줄<br />
    둘째 줄
  </p> <hr /> <input type="text" placeholder="입력칸도 스스로 닫습니다" /> </div>
);
```

화면 데모 ③ 에 두 줄짜리 글, 가로줄, 입력칸이 차례로 보입니다.

안이 비어 있는 태그는 무엇이든 이렇게 줄여 쓸 수 있습니다.

```
console.log(toJs("<div></div>") === toJs("<div />"));
```

F12 콘솔 true

<div></div> 와 <div /> 는 완전히 같습니다. 안에 넣을 것이 없으면 짧은 쪽을 쓰면 됩니다.

▸ 직접 해보기

4

selfClosing 의 <hr /> 을 하나 더 넣어 보세요. 데모 ③ 에 가로줄이 두 개가 되는지 확인하세요.

소문자는 HTML, 대문자는 여러분의 함수

섹션 5

섹션 2에서 <App /> 이라고 대문자로 썼습니다. 왜 대문자였을까요? 변환 결과를 나란히 놓으면 한눈에 보입니다.

```
console.log(toJs("<div />"));
```

F12 콘솔 /*#__PURE__*/React.createElement("div", null);

```
console.log(toJs("<App />"));
```

F12 콘솔 /*#__PURE__*/React.createElement(App, null);

자세히 보세요. 따옴표가 있고 없과의 차이입니다.

소문자	→ "div"	따옴표가 붙습니다. 문자열입니다.
대문자	→ App	따옴표가 없습니다. 변수입니다.

문자열이면 React 는 “브라우저야, div 태그를 만들어라” 라고 시킵니다. 변수면 React 는 “App 이라는 함수를 찾아서 불러라” 라고 합니다.

첫 글자 하나로 완전히 다른 일이 됩니다. 그래서 대소문자가 규칙이 아니라 뜻입니다.

```
console.log(toJs("<myBox />"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("myBox", null);
```

myBox 는 소문자로 시작하니 문자열이 되었습니다. 여러분이 myBox 라는 함수를 만들어 줘어도 React 는 그 함수를 안 찾습니다. 브라우저에게 "myBox 태그를 만들어라" 라고 시키고, 브라우저는 그런 태그를 모릅니다.

그래서 규칙은 이렇습니다.

여러분이 만든 화면 함수는 이름을 반드시 대문자로 시작한다

이 '화면을 돌려주는 함수' 를 컴포넌트라고 부릅니다. 03단원에서 제대로 배웁니다. 02단원에서는 App 하나만 쓰고, 대문자로 쓴다는 것만 기억하면 됩니다.

참고로 Fragment 도 같은 규칙을 따릅니다.

```
console.log(toJs("<React.Fragment><p>가</p></React.Fragment>") === toJs("<><p>가</p></>"));
```

```
F12 콘솔    true
```

<>...</> 는 <React.Fragment>...</React.Fragment> 의 짧은 표기입니다. React 가 대문자로 시작하니 변수로 읽힙니다. 규칙이 그대로 맞아떨어집니다.

▹ 직접 해보기 5

toJs 에 "<Card />" 와 "<card />" 를 각각 넣어 따옴표가 붙는 쪽과 안 붙는 쪽을 확인해 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

태그를 나란히 두 개 쓴다 (섹션 1에서 봤습니다)

```
return <h1>아메리카노</h1><p>4000원</p>;
```

[SyntaxError] Adjacent JSX elements must be wrapped in an enclosing tag. <> </> 로 감싸거나 <div> </div> 로 감싸세요.

이런 실수를 합니다

닫는 슬래시를 빠뜨린다 (섹션 4에서 봤습니다)

```

```

[SyntaxError] Unterminated JSX contents. 에러가 나는 자리가 실제로 틀린 자리보다 훨씬 아래로 잡힙니다. “이 줄은 멀쩡한데 왜 여기서 에러가 나지” 싶으면 위쪽에서 안 닫은 태그를 찾으세요.

```
console.log(tryJsx("const x = <img src='a.png'>;"));
```

F12 콘솔 unknown: Unterminated JSX contents. (1:27)

위 문자열은 27글자밖에 안 되는데 에러 자리가 (1:27), 즉 맨 끝입니다. 정작 틀린 곳은 11번째 글자 부근의 <img 입니다.

이런 실수를 합니다

여는 태그와 닫는 태그 이름이 다르다

```
<div>가</dv>
```

[SyntaxError] 가 납니다. 이 에러는 친절해서 어느 태그를 찾는지 알려 줍니다.

```
console.log(tryJsx("const x = <div>가</dv>;"));
```

F12 콘솔 unknown: Expected corresponding JSX closing tag for <div>. (1:16)

“div 에 맞는 닫는 태그를 기대했다” 는 뜻입니다. 오타 찾기가 쉬워집니다.

이런 실수를 합니다

태그가 서로 엇갈린다

```
<div><p>가</div></p>
```

안쪽 태그를 먼저 닫아야 합니다. 나중에 연 것을 먼저 닫는 순서입니다.

```
console.log(tryJsx("const x = <div><p>가</div></p>;"));
```

F12 콘솔 unknown: Expected corresponding JSX closing tag for <p>. (1:19)

이런 실수를 합니다

컴포넌트 이름을 소문자로 시작한다

```
function app() { return <h1>안녕</h1>; }
root.render(<app />);
```

에러가 안 납니다. 화면만 이상해집니다. 그래서 찾기 어렵습니다. React 는 app 을 브라우저 태그로 보고 그대로 만들어 버립니다. 콘솔에 이런 경고가 나옵니다.

The tag <app> is unrecognized in this browser.

If you meant to render a React component, start its name with an uppercase letter.

"React 컴포넌트를 그리려던 거면 이름을 대문자로 시작하세요" 라고 고치는 방법까지 알려 줍니다. 이 경고가 보이면 함수 이름을 대문자로 바꾸세요.

이런 실수를 합니다

Fragment 에 className 을 준다

<div className="output"> ... </div> ← 됩니다 <> ... </> ← 감싸기만 합니다. 속성을 못 줍니다

이런 실수를 합니다

<> </> 는 화면에 남지 않으니 꾸밀 대상도 없습니다. 속성을 주고 싶으면 div 같은 진짜 태그로 감싸야 합니다.

직접 해보기 정답

```
1) console.log(tryJsx("const x = <p>가</p><p>나</p>;"));
// 콘솔: unknown: Adjacent JSX elements must be wrapped in an enclosing tag. Did you want a JSX fragment <>...</>? (1:18)
```

→ 함수 안이 아니어도 똑같습니다. 나란히 둔 것 자체가 문제입니다.

뒤의 (1:18) 만 다릅니다. 문자열 길이가 달라서 막힌 자리도 달라진 것입니다.

```
2) function App() {
```

```
  return (
    <section className="output"> <h1>아메리카노</h1> <p>4000원</p> </section>
  );
```

```
}
// 콘솔: section
```

→ 감싸는 태그는 div 여야만 하는 것이 아닙니다. 하나면 무엇이든 됩니다.

```
3) function AppFragment() {

  return (
    <div> <h1>아메리카노</h1> <p>4000원</p> </div>
  );

}
// 콘솔: <div><h1>아메리카노</h1><p>4000원</p></div>
```

→ 감싼 div 가 화면에 그대로 남습니다. Fragment 었을 때는 없던 것입니다.

```
4) <hr />
<hr />
// 화면: 데모 ③ 의 가로줄이 두 개가 됩니다.
```

→ 스스로 닫는 태그는 안에 넣을 것이 없으니 그냥 여러 번 쓰면 됩니다.

```
5) console.log(toJs("<Card />"));
// 콘솔: /*#__PURE__*/React.createElement(Card, null);
console.log(toJs("<card />"));
// 콘솔: /*#__PURE__*/React.createElement("card", null);
```

→ 대문자는 따옴표 없이 변수로, 소문자는 따옴표가 붙어 문자열로 들어갑니다.

Card 라는 변수가 없으면 실행할 때 **ReferenceError** 가 납니다.

정리

- 1 JSX 는 값 하나입니다. 함수는 값을 하나만 돌려주므로 태그를 나란히 둘 수 없습니다.
- 2 묶는 방법은 두 가지입니다. `<div>` 로 감싸면 화면에 div 가 남고, `<>...</>` 로 감싸면 안 남습니다.
- 3 꾸밀 일이 있으면 `div`, 묶기만 하면 `<>...</>`(Fragment) 를 씁니다.
- 4 안이 빈 태그는 반드시 스스로 닫습니다. `<br />`, `<hr />`, `<img ... />`.
- 5 소문자 태그는 문자열이 되어 브라우저 태그가 되고, 대문자 태그는 변수가 되어 여러분의 함수를 찾습니다.
- 6 여는 태그와 닫는 태그의 이름은 정확히 같아야 하고, 나중에 연 것을 먼저 닫습니다.

```
export default function Concept04() {
  return (
    <div> <h1>개념 04 - 태그 규칙</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
```

Console 도 함께 보세요.

```

    </p> <div className="demo"> <h3>① div 로 감싼 화면
</h3> <div id="rootDiv"> <App /> </div> </div> <div className="demo"> <h3>② Fragment
로 감싼 화면 - 눈으로는 ① 과 같습니다</h3> <div id="rootFrag"> <AppFragment
/> </div> </div> <div className="demo"> <h3>③ 스스로 닫는 태그
</h3> <div id="rootSelf">
    {selfClosing}
</div> </div> </div>

);
}

```

여러 줄로 쓰기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

앞의 파일들에서 이런 모양을 계속 봤습니다.

```
return (
  <div>
    ...
  </div>
);
```

`return` 뒤에 소괄호가 붙어 있습니다. 저것이 왜 있을까요? 없으면 무슨 일이 생기는지부터 직접 보겠습니다. 이 파일은 문법 이야기가 아니라 ‘안 그러면 조용히 망가지는’ 이야기입니다.

```
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";
import 그려진뒤 from "../_ui/그려진뒤.js";

function toJs(jsxCODE) {
  return Babel.transform(jsxCODE, {
    presets: [["react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}

function tryJsx(code) {
  try {
    Babel.transform(code, { presets: [["react", { runtime: "classic" }]] });
    return "문제 없음";
  } catch (e) {
    return e.message.split("\n")[0];
  }
}
```

return 뒤에서 줄을 바꾸면 `undefined` 가 된다

섹션 1

JS자료 05단원 개념03의 [실수 2] 를 기억하나요? 이런 코드였습니다.


```
function brokenReturn() {
  return
  1 + 2;
}
console.log(brokenReturn()); → undefined
```

자바스크립트가 `return` 뒤에 세미콜론을 자동으로 넣어 버려서 아래 줄이 통째로 버려졌습니다. 같은 일이 JSX 에서도 그대로 일어납니다. 진짜로 해 봅시다.

```
function makeTitleBad() {
  return <h1>오늘의 메뉴</h1>;
}

console.log(makeTitleBad());
```

F12 콘솔 undefined

에러가 하나도 안 났습니다. 그냥 `undefined` 가 나왔습니다. 이런 것이 가장 찾기 어렵습니다. 화면만 비고 콘솔은 조용합니다.

왜 그런지는 개념01의 도구로 확인하면 끝납니다.

```
console.log(toJs("function App() {\n  return\n  <h1>안녕</h1>;\n}"));
```

F12 콘솔 function App() { return; /*#__PURE__*/React.createElement("h1", null, "안녕"); }

`return` 뒤에 세미콜론이 붙어 있습니다. 우리가 안 찍었는데 생겼습니다. 그 뒤의 `createElement` 는 이미 끝난 함수 뒤에 남은, 아무도 안 보는 줄입니다.

1 자바스크립트가 `return` 뒤에 세미콜론을 자동으로 넣는다

2 함수는 거기서 끝난다. 돌려주는 값이 없으니 `undefined`

3 아래 줄의 JSX 는 만들어지기는 하지만 아무 데도 안 간다

이것을 그대로 `render` 에 넣으면 어떻게 될까요? `undefined` 를 `render` 에 넣는 것이라 화면에 아무것도 안 그려집니다.

같은 줄에 쓰면 아무 문제가 없습니다.

```
function makeTitleSameLine() {
  return <h1>오늘의 메뉴</h1>;
}

console.log(makeTitleSameLine().type);
```

직접 해보기 1

makeTitleBad 의 `return` 과 `<h1>` 을 한 줄로 붙여 보세요. 콘솔의 `undefined` 가 무엇으로 바뀌는지 확인하세요.

소괄호가 하는 일

섹션 2

그런데 화면이 열 줄, 스무 줄이 되면 한 줄로 쓸 수가 없습니다. 그래서 소괄호를 씁니다.

```
function makeTitleGood() {
  return (
    <h1>오늘의 메뉴</h1>
  );
}

console.log(makeTitleGood().type);
```

소괄호가 무슨 일을 한 걸까요? 두 가지입니다.

1. `return` 과 소괄호가 '같은 줄' 에 있습니다.

자바스크립트는 `return` 바로 뒤에 무언가 있으면 세미콜론을 안 넣습니다.

2. 소괄호는 닫힐 때까지 하나의 값으로 묶입니다.

그래서 중간에서 줄을 몇 번 바꿔도 끊기지 않습니다.

변환 결과를 보면 세미콜론이 사라진 것이 보입니다.

```
console.log(toJs("function App() {\n  return (\n    <h1>안녕</h1>\n  );\n}"));
```

```
F12 콘솔    function App() { return /*#__PURE__*/React.createElement("h1", null,
  "안녕"); }
```

섹션 1의 결과와 나란히 놓고 비교해 보세요.

```
return; /*#__PURE__*/React.createElement(...) ← 소괄호 없음. 끊겼습니다.
return /*#__PURE__*/React.createElement(...) ← 소괄호 있음. 이어졌습니다.
```

세미콜론 하나 차이입니다. 그 하나 때문에 화면이 비었던 것입니다.

소괄호는 JSX 만을 위한 문법이 아닙니다. 그냥 자바스크립트의 소괄호입니다. JS자료 05단원에서도 “값이 길면 괄호로 감싸세요” 라고 했습니다. 같은 이야기입니다.

소괄호를 어디에 쓰나 정리하면 이렇습니다.

필요함 · `return` 뒤에 여러 줄 JSX 를 쓸 때

필요함 · 변수에 여러 줄 JSX 를 담을 때 (아래 card 처럼)

필요 없음 · JSX 가 한 줄이면 안 써도 됩니다

변수에 담을 때도 똑같이 씁니다.

```
const card = (
  <div className="output"> <h3>아메리카노</h3> <p>4000원</p> </div>
);

console.log(card.type);
```

F12 콘솔 div

사실 변수에 담을 때는 소괄호가 없어도 동작합니다. `const card = <div ...` 로 시작하면 `return` 이 아니라서 세미콜론이 안 끼어듭니다. 그래도 모두가 소괄호를 씁니다. 시작과 끝이 눈에 잘 보이기 때문입니다.

▸ 직접 해보기

2

`makeTitleGood` 의 소괄호를 지우고, `return` 뒤에서 줄을 바꿔 보세요. 콘솔이 어떻게 되는지 확인하고 다시 되돌리세요.

여러 줄로 나누는 방법

섹션 3

규칙은 세 개뿐입니다.

- 1 `return` (를 같은 줄에 쓰고, 그 줄에서는 더 안 씁니다.
- 2 태그 하나가 안으로 들어갈 때마다 두 칸씩 더 들여씁니다.
- 3 닫는 소괄호와 세미콜론 `);` 은 `return` 과 같은 칸에 둡니다.

눈으로 보면 이렇습니다.

```
function App() {
  return (
    <div>                ← 두 칸 들어감
      <h3>제목</h3>      ← 또 두 칸
      <p>내용</p>        ← 형제니까 같은 칸
    </div>
  );                    ← return 과 같은 칸
}
```

형제 태그는 반드시 같은 칸에 둡니다. 칸이 어긋나면 누가 누구 안에 있는지 코드만 보고는 알 수 없습니다. JSX 는 화면 구조를 눈으로 보라고 쓰는 문법인데, 들여쓰기가 어긋나면 그 장점이 사라집니다.

들여쓰기는 화면에 아무 영향이 없습니다. 사람이 읽으라고 하는 것입니다. 개념01에서 확인했던 것을 다시 확인해 봅시다.

```
console.log(toJs("<div><h3>제목</h3></div>") === toJs("<div>\n  <h3>제목\n</h3>\n</div>"));
```

F12 콘솔 true

실제로 여러 줄로 쓴 화면을 하나 만들어 봅시다.

```
const name = "김민준";
const price = 4000;

function App() {
  return (
    <div className="output"> <h3>{name}님의 주문</h3> <p>아메리카노
{price.toLocaleString()}원</p> <p>
  라떼 {(4500).toLocaleString()}원
  <br />
  케이크 {(6000).toLocaleString()}원
  </p> <p>합계 {(price + 4500 + 6000).toLocaleString()}원</p> </div>
  );
}
```

화면 데모 ① 에 주문서 한 장이 보입니다.
 김민준님의 주문 / 아메리카노 4,000원 / 라떼 4,500원 / 케이크 6,000원 /
 합계 14,500원

```
console.log((price + 4500 + 6000).toLocaleString());
```

F12 콘솔 14,500

▣ 직접 해보기 3

App 안에 `<p>삼각김밥 1,200원</p>` 을 한 줄 더 넣어 보세요. 들여쓰기를 형제 태그와 같은 칸에 맞추세요.

줄을 바꾸면 공백은 어떻게 되나

섹션 4

여러 줄로 쓰다 보면 뜻하지 않은 일이 생깁니다. 세 가지 경우를 만들어 봅시다.

```
const spaceTest = (
  <div className="output"> <p>
    아메리카노는
    4000원입니다
  </p> <p> <span>김민준</span> <span>이서연</span> </p> <p> <span>김민준</span>{"
"}
  <span>이서연</span> </p> </div>
);
```

화면 데모 ② 에 세 줄이 보입니다. 둘째 줄만 두 이름이 붙어 있습니다.

```
그려진뒤("#rootSpace", (자리) => {
  const lines = 자리.querySelectorAll("p");

  console.log(lines[0].textContent);
```

F12 콘솔 아메리카노는 4000원입니다

코드	▶ F12 콘솔
<code>console.log(lines[1].textContent);</code>	▶ 김민준이서연
<code>console.log(lines[2].textContent);</code>	▶ 김민준 이서연
<code>});</code>	

결과를 보면 규칙이 보입니다.

글자와 글자 사이에서 줄을 바꾸면 → 공백 한 칸으로 이어집니다 태그와 태그 사이에서 줄을 바꾸면 → 아무것도 안 남습니다. 붙습니다

첫째 줄은 글자 두 줄이라 “아메리카노는 4000원입니다” 로 한 칸 띄워졌습니다. 둘째 줄은 span 두 개라 “김민준이서연” 으로 붙어 버렸습니다.

붙는 게 싫으면 셋째 줄처럼 `{ " " }` 를 넣습니다. 개념02에서 배운 대로 중괄호 안에 공백 한 칸짜리 문자열을 넣은 것입니다. 특별한 문법이 아닙니다.

```
console.log(toJs('<div><span>가</span>{" "><span>나</span></div>'));
```

```
F12 콘솔    /##__PURE__*/React.createElement("div", null,
          /##__PURE__*/React.createElement("span", null, "가"), " ",
          /##__PURE__*/React.createElement("span", null, "나"));
```

인자 사이에 " " 가 하나 들어갔습니다. 그것뿐입니다.

참고로 태그 두 개를 '같은 줄'에 띄어쓰기로 두면 그 공백은 남습니다.

```
console.log(toJs("<div><span>가</span> <span>나</span></div>"));
```

```
F12 콘솔    /##__PURE__*/React.createElement("div", null,
          /##__PURE__*/React.createElement("span", null, "가"), " ",
          /##__PURE__*/React.createElement("span", null, "나"));
```

위 {" "} 버전과 결과가 완전히 같습니다. 그래서 줄만 안 바꾸면 {" "}도 필요 없습니다. 줄을 바꿀 때만 필요합니다.

▹ 직접 해보기 4

spaceTest의 둘째 p 안에 있는 두 span을 한 줄로 붙여 쓰고 사이에 띄어쓰기를 하나 넣어 보세요. 화면이 어떻게 되나요?

여러 개를 배열에 담아 두기

섹션 5

JSX는 값입니다. 값이니까 배열에도 담깁니다. (개념01 섹션 4) 여러 줄짜리 화면을 만들다 보면 이 성질이 쓸모 있어집니다.

```
const menu = ["아메리카노", "라떼", "케이크"];

const items = menu.map((item) => <li>{item}</li>);

console.log(items.length);
```

```
F12 콘솔    3
```

```
console.log(items[0].type);
```

```
F12 콘솔    li
```

```
console.log(items[0].props);
```

```
F12 콘솔    { children: '아메리카노' }
```

JS자료 08단원의 map 그대로입니다. 글자 배열이 JSX 배열이 되었습니다.

이 배열을 화면에 그리는 것이 목록 만들기입니다. 다만 그냥 그리면 React 가 “key 를 넣으세요” 라고 경고합니다. key 가 무엇이고 왜 필요한지는 05단원에서 제대로 배웁니다. 여기서는 “배열까지는 만들 수 있다” 까지만 확인하고 넘어갑니다.

지금 당장 화면에 뿌리고 싶다면 글자로 이어 붙이면 됩니다. (개념02 섹션 2)

```
console.log(menu.join(", "));
```

F12 콘솔 아메리카노, 라떼, 케이크

```
const menuLine = <p className="output">오늘의 메뉴: {menu.join(", ")}</p>;
```

화면 데모 ③ 에 "오늘의 메뉴: 아메리카노, 라떼, 케이크" 가 보입니다.

▸ 직접 해보기 5

menu 를 map 으로 `<p>{item}</p>` 배열로 만들고 `items2.length` 와 `items2[2].props` 를 콘솔에 찍어 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

`return` 뒤에서 줄을 바꾼다 (섹션 1에서 봤습니다)

이런 실수를 합니다

에러가 안 납니다. 화면만 비고 콘솔은 조용합니다. “화면이 안 나오는데 에러도 없다” 면 `return` 줄을 가장 먼저 보세요. `return` 바로 뒤에 (가 있지만 확인하면 됩니다.

이런 실수를 합니다

소괄호를 열고 닫는 것을 잊는다

```
return (
```

```
<div>...</div>
```

```
;
```

[SyntaxError] 가 납니다. 소괄호를 열었으면 반드시 닫아야 합니다. 마지막 줄은 `);` 로 끝납니다. `)` 와 `;` 의 순서를 바꾸면 안 됩니다.

이런 실수를 합니다

소괄호 안을 비워 둔다

```
return (
);
```

[SyntaxError] 가 납니다. 소괄호 안에는 값이 하나 있어야 합니다. 아무것도 안 그리고 싶으면 `return null;` 이라고 씁니다.

```
console.log(tryJsx("function App() {\n  return (\n  );\n}"));
```

F12 콘솔 unknown: Unexpected token (3:2)

(3:2) 는 세 번째 줄에서 막혔다는 뜻입니다. 닫는 소괄호를 만난 자리입니다.

이런 실수를 합니다

소괄호 대신 중괄호를 쓴다

```
return {
```

```
<div>...</div>
```

```
};
```

[SyntaxError] 가 납니다. 중괄호는 '객체' 이거나 '코드 묶음' 입니다. JSX 를 감싸는 것은 소괄호입니다. 모양이 비슷해서 자주 틀립니다.

이런 실수를 합니다

태그와 태그 사이가 붙어 버린다 (섹션 4에서 봤습니다)

이런 실수를 합니다

에러가 안 납니다. 글자만 붙어서 나옵니다. "김민준이서연" 처럼 붙었으면 사이에 { " } 를 넣으세요.

이런 실수를 합니다

여러 줄 텍스트를 그대로 두면 줄이 안 나뉜다

코드에서 줄을 바꿨다고 화면에서도 줄이 바뀌지 않습니다. 섹션 4의 첫째 줄이 "아메리카노는 4000원입니다" 한 줄로 나온 것이 그 예입니다. 화면에서 줄을 나누려면 개념04의 `<br />` 을 넣어야 합니다. 위 App 함수의 라떼와 케이크 사이에 `<br />` 이 들어간 이유가 그것입니다.

직접 해보기 정답

```
1) function makeTitleBad() {

  return <h1>오늘의 메뉴</h1>;

}
console.log(makeTitleBad().type);
// 콘솔: h1
```

→ `undefined` 가 사라졌습니다. 함수가 값을 돌려주기 시작한 것입니다.

`makeTitleBad()` 를 그대로 찍으면 `React` 엘리먼트 객체가 통째로 나옵니다. 길고 읽기 어려워서 `.type` 만 찍었습니다.

```
2) function makeTitleGood() {

  return <h1>오늘의 메뉴</h1>;

}
// 콘솔: Uncaught TypeError: Cannot read properties of undefined (reading 'type')
```

→ 여기서 중요한 것은 이 에러가 “문법 에러가 아니다” 라는 점입니다.

소괄호를 지워도 문법은 멀쩡합니다. `Babel` 은 아무 말도 안 합니다. `makeTitleGood()` 이 `undefined` 가 되었을 뿐이고, `undefined` 에서 `.type` 을 꺼내려다가 그제서야 에러가 난 것입니다. `.type` 을 안 찍었다면 에러도 없이 화면만 비었을 것입니다.

```
3) <p>아메리카노 {price.toLocaleString()}원</p>
<p>삼각김밥 {(1200).toLocaleString()}원</p>
// 화면: 데모 ① 에 "삼각김밥 1,200원" 줄이 하나 더 생깁니다.
```

→ 형제 태그와 같은 칸에 두면 됩니다. 합계 줄은 그대로라 값이 안 맞으니,

합계도 고치고 싶으면 `+ 1200` 을 더하세요.

4) `<span>김민준</span> <span>이서연</span>`

```
// 화면: 두 이름 사이가 한 칸 띄워집니다. {" "} 를 쓴 셋째 줄과 같아집니다.
```

→ 같은 줄에 둔 띄어쓰기는 살아남습니다. 줄을 바꿀 때만 사라집니다.

```
5) const items2 = menu.map((item) => <p>{item}</p>);
console.log(items2.length);
// 콘솔: 3
console.log(items2[2].props);
// 콘솔: { children: '케이크' }
```

→ li 가 p 로 바뀌었을 뿐 구조는 같습니다.

화면에 그리는 것은 05단원입니다. 지금 그리면 key 경고가 납니다.

정리

- 1 `return` 뒤에서 줄을 바꾸면 자바스크립트가 세미콜론을 자동으로 넣어 `undefined` 가 됩니다. 에러는 안 납니다.
- 2 그래서 여러 줄 JSX 는 `return (` (처럼 소괄호를 같은 줄에 열고 `);` 로 닫습니다.
- 3 소괄호는 JSX 전용 문법이 아닙니다. 값이 길 때 감싸는 평범한 소괄호입니다.
- 4 안으로 들어갈 때마다 두 칸씩 들여쓰고, 형제 태그는 같은 칸에 둡니다. 들여쓰기는 화면에 영향이 없습니다.
- 5 글자 사이에서 줄을 바꾸면 공백 한 칸이 남고, 태그 사이에서 줄을 바꾸면 아무것도 안 남습니다. 띄우려면 { " " } 를 넣습니다.
- 6 코드에서 줄을 바꿔도 화면에서는 안 나옵니다. 화면에서 줄을 나누려면 `<br />` 을 씁니다.
- 7 JSX 를 배열에 담을 수 있습니다. 화면에 목록으로 그리는 것은 05단원입니다.

```
export default function Concept05() {
  return (
    <div> <h1>개념 05 - 여러 줄로 쓰기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
    </p> <div className="demo"> <h3>① 여러 줄로 쓴 화면 (섹션 3)
    </h3> <div id="root"> <App /> </div> </div> <div className="demo"> <h3>② 줄바꿈과
    공백 실험 (섹션 4)</h3> <div id="rootSpace">
      {spaceTest}
    </div> </div> <div className="demo"> <h3>③ 배열을 글자로 이어 붙인 화면
    (섹션 5)</h3> <div id="rootMenu">
      {menuLine}
    </div> </div> </div>
  );
}
```

JSX

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

고치고 저장하면 화면이 바로 바뀝니다.

F12 → Console 도 함께 보세요.

준비물

```
import * as Babel from "@babel/standalone";
const user = { name: "김민준", age: 20 };
const menu = ["아메리카노", "라떼", "케이크"];
const price = 4000;
const isOpen = true;
const stock = 0;
const homepage = "https://example.com";
```

개념01에서 만든 도구입니다. 문제 2에서 씁니다.

```
function toJs(jsxCODE) {
  return Babel.transform(jsxCODE, {
    presets: [["react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}
```

문제 1 (개념01)

`<h2>오늘의 메뉴</h2>` 를 `title` 이라는 변수에 담고, `title.type` 과 `title.props` 를 콘솔에 찍으세요.

기대 콘솔 `h2`
 `{ children: '오늘의 메뉴' }`
 화면에 `<h2>오늘의 메뉴</h2>` 가 글자로 보이면 따옴표로 감싼 것입니다.

여기에 코드를 쓰세요

문제 2 (개념01)

toJs 를 써서 "<p>라떼 4500원</p>" 가 무엇으로 바뀌는지 콘솔에 찍으세요.

기대 콘솔 `/*#__PURE__*/React.createElement("p", null, "라떼 4500원");`
아무것도 안 찍히면 `console.log` 로 감싸지 않은 것입니다.

여기에 코드를 쓰세요

문제 3 (개념02)

answer3 이 "김민준님 안녕하세요" 를 보여 주게 고치세요. 이름은 반드시 `user` 에서 꺼내 쓰세요. 직접 적으면 안 됩니다.

기대 화면 김민준님 안녕하세요
화면에 `user.name` 이라는 글자가 그대로 보이면 중괄호를 안 쓴 것입니다.

아래 줄을 고치세요

```
const answer3 = <p>아직 안 풀었습니다</p>;
```

문제 4 (개념02)

answer4 가 아메리카노 두 잔 값을 보여 주게 고치세요. 8000 이라고 직접 적지 말고 `price` 로 계산하세요.

기대 화면 아메리카노 두 잔은 8000원입니다
화면에 `price * 2` 라는 글자가 보이면 중괄호를 안 쓴 것입니다.

아래 줄을 고치세요

```
const answer4 = <p>아직 안 풀었습니다</p>;
```

문제 5 (개념02)

answer5 가 나이에 따라 다른 글자를 보여 주게 고치세요. 19살 이상이면 "성인", 아니면 "미성년" 입니다. 중괄호 안에는 if 문을 쓸 수 없습니다. 삼항 연산자를 쓰세요.

기대 화면 김민준님은 성인입니다
화면이 통째로 비면 중괄호 안에 if 문을 쓴 것입니다.

아래 줄을 고치세요

```
const answer5 = <p>아직 안 풀었습니다</p>;
```

문제 6 (개념02)

answer6 이 메뉴 개수와 메뉴 이름을 함께 보여 주게 고치세요. 3 이라고 직접 적지 말고 menu 에서 개수를 구하세요.

기대 화면 메뉴 3개: 아메리카노, 라떼, 케이크
"아메리카노라떼케이크" 로 붙어 나오면 join 을 안 쓴 것입니다.

아래 줄을 고치세요

```
const answer6 = <p>아직 안 풀었습니다</p>;
```

문제 7 (개념03)

answer7 에 클래스 두 개(output 과 on)를 붙이세요. 두 클래스는 전역 CSS(src/index.css)에 이미 만들어져 있습니다.

기대 화면 "파란 상자" 가 파란 바탕에 흰 글씨로 보입니다.
색이 안 변하고 콘솔에 Invalid DOM property 경고가 나오면
className 이 아니라 class 라고 쓴 것입니다.

아래 줄을 고치세요

```
const answer7 = <p>파란 상자</p>;
```

문제 8 (개념03)

answer8 에 style 을 붙이세요. 글자색은 "#c00", 배경색은 "#fff8dc" 입니다. 중괄호가 몇 개 필요한지 생각해 보세요.

기대 화면 "노란 바탕에 빨간 글자" 가 정말 그렇게 보입니다.
색이 안 칠해지고 Unsupported style property 경고가 나오면
배경색 이름을 카멜케이스로 안 쓴 것입니다.

아래 줄을 고치세요

```
const answer8 = <p>노란 바탕에 빨간 글자</p>;
```

문제 9 (개념03)

answer9 의 링크가 homepage 변수의 주소로 가게 하세요. 주소를 직접 적지 말고 변수를 쓰세요.

기대 화면 "홈페이지로 가기" 링크가 보입니다.
마우스를 올리면 브라우저 왼쪽 아래에 example.com 이 보입니다.
거기에 homepage 라는 글자가 보이면 따옴표를 같이 쓴 것입니다.

아래 줄을 고치세요

```
const answer9 = <a>홈페이지로 가기</a>;
```

문제 10 (개념03)

answer10 의 버튼이 재고가 0일 때만 잠기게 하세요. stock 값을 써서 잠금 여부를 정하세요. true 나 false 를 직접 적지 마세요.

기대 화면 stock 이 0 이므로 "담기" 버튼이 회색이고 눌리지 않습니다.
준비물의 stock 을 3 으로 바꾸면 잠금이 풀려야 합니다.
(확인한 뒤에는 0 으로 되돌리세요)

아래 줄을 고치세요

```
const answer10 = <button>담기</button>;
```

문제 11 (개념04)

answer11 이 <h4>아메리카노</h4> 와 <p>4000원</p> 두 줄을 함께 담게 하세요. 단, 화면에 감싸는 태그가 남으면 안 됩니다.

기대 화면 "아메리카노" 와 "4000원" 두 줄이 나옵니다.
화면이 통째로 비면 두 태그를 감싸지 않고 나란히 둔 것입니다.

아래 줄을 고치세요

```
const answer11 = <p>아직 안 풀었습니다</p>;
```

문제 12 [응용] (개념02 · 개념03 · 개념05)

answer12 에 주문서 한 장을 만드세요. 여러 줄로 써야 하니 소괄호를 쓰세요.

- 바깥은 className 이 "output" 인 div - 그 안에 <h4> 로 "김민준님의 주문" (이름은 user 에서 꺼낼 것) - <p> 로 "아메리카노 4,000원" ← 심표가 들어간 표기입니다 - <p> 로 "세 잔 합계 12,000원" ← 12000 을 직접 적지 말고 계산할 것

기대 화면 흰 상자 안에 세 줄이 보입니다.
김민준님의 주문 / 아메리카노 4,000원 / 세 잔 합계 12,000원
4000 처럼 심표가 없으면 toLocaleString 을 안 쓴 것입니다.
(개념02 섹션 2에서 썼습니다)

아래 줄을 고치세요

```
const answer12 = <p>아직 안 풀었습니다</p>;
```

문제 13 [도전] (개념02 · 개념03 · 개념04 · 개념05)

answer13 에 가게 상태를 보여 주는 화면을 만드세요. 감싸는 태그가 화면에 남으면 안 됩니다.

- 첫 줄 <p> : isOpen 이 true 면 "영업 중", false 면 "준비 중"

글자색도 같이 바꿉니다. true 면 "green", false 면 "gray"

- 둘째 줄 <p> : "3종류를 팝니다" (3 은 menu 에서 구할 것)

기대 화면 초록 글씨로 "영업 중", 그 아래 "3종류를 팝니다"
 준비물의 isOpen 을 false 로 바꾸면
 회색 글씨로 "준비 중" 이 되어야 합니다.
 (확인한 뒤에는 true 로 되돌리세요)
 글자만 바뀌고 색이 그대로면 style 안에는 삼항을 안 넣은 것입니다.

아래 줄을 고치세요

```
const answer13 = <p>아직 안 풀었습니다</p>;
```

문제 14 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 저장해 보세요. 화면이 통째로 비고 콘솔에 빨간 글씨가 나옵니다. 그 첫 줄을 읽고, 왜 그런지 답해 보세요.

```
const badLine = <p>{user}</p>;  
ReactDOM.createRoot(document.querySelector("#root")).render(badLine);
```

콘솔의 빨간 글씨 첫 줄

왜 이렇게 되나

고치는 방법

★ 확인한 뒤에는 반드시 다시 // 를 붙여 주석으로 되돌리세요. 안 그러면 아래 화면이 계속 비어 있습니다.

여기부터는 고치지 마세요

위에서 만든 답들을 화면에 넣어놓는 코드입니다.

```
function App() {  
  return (  
    <div>  
      <div className="output">  
        <h4>문제 3</h4>  
        {answer3}  
      </div>  
      <div className="output">  
        <h4>문제 4</h4>  
        {answer4}  
      </div>  
      <div className="output">  
        <h4>문제 5</h4>  
        {answer5}  
      </div>  
      <div className="output">  
        <h4>문제 6</h4>  
        {answer6}  
      </div>  
      <div className="output">  
        <h4>문제 7</h4>  
        {answer7}  
      </div>  
      <div className="output">  
        <h4>문제 8</h4>  
        {answer8}  
      </div>  
      <div className="output">  
        <h4>문제 9</h4>  
        {answer9}  
      </div>  
      <div className="output">  
        <h4>문제 10</h4>  
        {answer10}  
      </div>  
      <div className="output">
```

```

    <h4>문제 11</h4>
    {answer11}
  </div>
  <div className="output"> <h4>문제 12 [응용]</h4>
    {answer12}
  </div> <div className="output"> <h4>문제 13 [도전]</h4>
    {answer13}
  </div>
</div>
);
}

export default function Exercises() {
  return (
    <div> <h1>02단원 연습문제 - JSX</h1> <p className="guide">
      이 파일의 TODO 자리를 고치고 저장하면 화면이 바로 바뀝니다.<br /> 1~11은
      기본, 12는 [응용], 13은 [도전], 14는 예러 확인입니다.
      <br /> <br /> <strong>화면이 통째로 비면</strong> 문법이 깨진 것입니다.
      놀라지 마세요. <strong>F12 → Console</strong> 의 빨간 글씨 <strong>첫 줄</strong>을
      읽고, 방금 고친 곳을 되돌리면 화면이 돌아옵니다.
      </p> <div className="demo"> <h3>내가 만든 화면</h3> <div id="root"> <App
    /> </div> </div> </div>
  );
}

```

02단원 마무리 JSX

자주 넘어지는 자리

- 1 `return` 뒤에서 줄바꿈하면 `undefined` (JS자료 05단원 ASI) | 눈에 안 보이는 예러

컴포넌트와 props

OPEN 03_컴포넌트와_props

```
function double(n) {  
  return n * 2;  
  console.log(double(4));  
  function Hello() {
```

01 컴포넌트 만들기

02 props 전달하기

03 props 구조분해와 기본값

04 children

05 컴포넌트 쪼개기

- 연습문제

컴포넌트 만들기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

01단원에서 이런 함수를 만들었습니다.

```
function App() {
  return <h1>안녕하세요</h1>;
}
```

특별해 보이는 이름이지만, 그냥 함수입니다. 그리고 이런 함수를 부르는 이름이 있습니다. '컴포넌트'입니다.

지금까지는 컴포넌트를 하나만 만들었습니다. 화면 전체를 App 혼자 그렸습니다. 이 단원에서는 컴포넌트를 여러 개 만들어 화면을 나눠 그립니다.

- 1 컴포넌트는 화면을 돌려주는 '함수' 다
- 2 이름은 반드시 대문자로 시작한다 — 안 지키면 조용히 망가진다
- 3 컴포넌트끼리 조립한다

이 단원에서는 화면이 바뀌지 않습니다. 버튼을 눌러 값을 바꾸는 것은 04단원입니다.

컴포넌트는 화면을 돌려주는 함수입니다

섹션 1

JS자료 05단원 개념03 에서 함수는 값을 '돌려준다' 고 배웠습니다.

```
import Summary from "../_ui/Summary.jsx";

function double(n) {
  return n * 2;
}

console.log(double(4));
```

F12 콘솔 8

숫자를 돌려주는 함수를 만들 수 있으니, 문자열을 돌려주는 함수도 만들 수 있고, 배열이나 객체를 돌려주는 함수도 만들 수 있었습니다.

그렇다면 '화면' 을 돌려주는 함수도 만들 수 있습니다. 02단원에서 배운 JSX 를 `return` 하면 됩니다.

```
function Hello() {
  return <p>안녕하세요, 저는 컴포넌트입니다.</p>;
}
```

이것이 컴포넌트입니다. 새 문법은 한 글자도 없습니다. 그냥 함수이고, 돌려주는 값이 JSX 일 뿐입니다.

```
console.log(typeof Hello);
```

```
F12 콘솔    function
```

← 컴포넌트라고 불려도 정체는 함수입니다.

화면에 그릴 때는 `double(4)` 처럼 괄호를 붙여 부르지 않습니다. 태그처럼 `<Hello />` 라고 씁니다.

```
root.render(<Hello />);
```

이 파일은 맨 아래에서 한 번에 그립니다. 화면 ① 을 보세요. 왜 괄호로 부르지 않는지는 섹션 6 [실수 4] 에서 확인합니다.

▸ 직접 해보기

1

`<p>또 만나요</p>` 를 돌려주는 Bye 컴포넌트를 만드세요. (화면에 넣는 것은 직접 해보기 5에서 합니다)

이름은 대문자로 시작합니다

섹션 2

컴포넌트 이름은 반드시 대문자로 시작합니다. "규칙이니 외우세요" 가 아닙니다. React 가 그 글자로 구분하기 때문입니다.

02단원에서 JSX 가 `React.createElement` 로 바뀐다고 배웠습니다. 바뀐 결과 안을 들여다보면 이유가 바로 보입니다.

```
const divElement = <div />;

console.log(divElement.type);
```

```
F12 콘솔    div
```

```
console.log(typeof divElement.type);
```

```
F12 콘솔    string
```

← 'div' 라는 문자열이 들어 있습니다. React 는 이 문자열을 보고 브라우저에게 `<div>` 태그를 만들라고 시킵니다.

```
const helloElement = <Hello />;

console.log(typeof helloElement.type);
```

F12 콘솔 function

```
console.log(helloElement.type === Hello);
```

F12 콘솔 true

← 문자열이 아니라 위에서 만든 Hello 함수 그 자체가 들어 있습니다. React 는 이 함수를 대신 불러서, 돌려받은 JSX 를 그립니다.

그럼 React 는 둘을 어떻게 구분할까요? 태그 이름의 첫 글자만 봅니다.

소문자로 시작 → 'div' 처럼 문자열로 넣는다 → HTML 태그 대문자로 시작 → 함수 그 자체를 넣는다 → 내가 만든 컴포넌트

HTML 태그 이름은 전부 소문자입니다(div, p, ul, h1 ...). 그래서 "대문자로 시작하면 내 것" 이라는 표시가 겹치지 않고 통합니다.

▫ 직접 해보기

2

`const pElement = <p />;` 를 만들고 `pElement.type` 을 콘솔에 찍어 보세요. 무엇이 나올까요?

소문자로 쓰면 조용히 이상해집니다

섹션 3

★ 이 단원에서 가장 조심해야 할 부분입니다.

컴포넌트 이름을 소문자로 시작하게 만들면 어떻게 될까요? "에러가 나겠지" 싶지만, 에러가 나지 않습니다. 그래서 더 위험합니다.

```
function menu() {
  return <p>아메리카노 4000원</p>;
}

const menuElement = <menu />;

console.log(menuElement.type);
```

F12 콘솔 menu

```
console.log(menuElement.type === menu);
```

F12 콘솔 false

← 위에서 만든 menu 함수가 아니라 'menu' 라는 문자열이 들어갔습니다. React 는 "menu 라는 HTML 태그를 만들어라" 로 읽었습니다. 그래서 menu 함수는 한 번도 불리지 않습니다.

화면 ㉓ 을 보세요. 아메리카노 4000원이 나와야 할 자리가 비어 있습니다. F12 → Elements 에서 그 자리를 보면 <menu></menu> 라는 빈 태그가 들어 있습니다.

menu 는 실제로 존재하는 HTML 태그입니다(목록을 담는 태그). 그래서 브라우저는 아무 불평 없이 빈 태그를 하나 만들고 끝냅니다.

그럼 HTML 에 없는 이름이면 어떻게 될까요? 예를 들어 myBox 라면요. 그때는 React 가 콘솔에 빨간 경고를 두 줄 띄워 줍니다.

```
<myBox /> is using incorrect casing.
```

```
Use PascalCase for React components, or lowercase for HTML elements.
```

The tag <myBox> is unrecognized in this browser.

```
If you meant to render a React component, start its name with
an uppercase letter.
```

정리하면 이렇습니다. 이름이 HTML 에 없는 태그 → 빨간 경고가 뜬다 (그나마 다행) 이름이 HTML 에 있는 태그 → 경고도 없이 조용히 사라진다 (menu, header, section ...)

화면에서 무언가가 통째로 사라졌는데 에러가 없다면, 컴포넌트 이름의 첫 글자부터 확인하세요.

```
function myBox() {
  return <p>나는 myBox 입니다</p>;
}
```

▹ 직접 해보기 3

아래 LowercaseDemo 안의 `/* <myBox /> */` 에서 주석을 풀고 저장한 뒤 F12 → Console 을 보세요. 위에 적힌 빨간 경고 두 줄이 실제로 나옵니다. 확인했으면 다시 `/* */` 로 감싸 두세요.

```
function LowercaseDemo() {
  return (
    <div className="demo"> <h3>㉓ 소문자로 만든 컴포넌트 - 아무것도 안 보입니다
    </h3> <div className="output"> <menu /> </div>
    /* <myBox /> */
    <p>위 흰 칸이 비어 있습니다. 에러는 한 줄도 나지 않았습니다.</p> </div>
  );
}
```

이름만 대문자로 고치면 바로 됩니다. 다른 곳은 손대지 않았습니다.

```
function Menu() {
  return <p>아메리카노 4000원</p>;
}
```

한 파일에 컴포넌트를 여러 개

섹션 4

컴포넌트는 함수이므로, 함수를 여러 개 만들듯이 얼마든지 만들 수 있습니다. 파일을 나누는 방법은 08단원에서 배웁니다. 07단원까지는 한 파일 안에 다 씁니다.

이름은 그 조각이 화면에서 무엇인지 알 수 있게 짓습니다. 함수 이름을 짓던 방식과 같습니다(JS자료 05단원 개념01).

```
function ShopTitle() {
  return <h4>동네 카페</h4>;
}

function TodayMenu() {
  return <p>오늘의 메뉴 - 아메리카노 4000원 / 라떼 4500원</p>;
}

function ClosingTime() {
  return <p>영업 종료 22:00</p>;
}
```

셋 다 화면 조각 하나씩을 돌려줍니다. 아직 서로 아무 관계가 없습니다.

▣ 직접 해보기 4

<p>오늘 케이크 6000원 할인</p> 를 돌려주는 Notice 컴포넌트를 만드세요.

컴포넌트 안에 컴포넌트 넣기

섹션 5

만든 컴포넌트는 다른 컴포넌트의 JSX 안에서 태그처럼 쓸 수 있습니다. 이렇게 조각을 모아 큰 화면을 만드는 것을 '조립' 이라고 부르겠습니다.

```
function ShopCard() {
  return (
    <div className="output"> <ShopTitle /> <TodayMenu /> <ClosingTime /> </div>
  );
}
```

화면 흰 칸 안에 "동네 카페" 제목과 메뉴 줄, 영업 종료 줄이 차례로 나옵니다.

순서는 적은 순서 그대로입니다. ShopTitle 을 아래로 내리면 화면에서도 내려갑니다.

같은 컴포넌트를 여러 번 써도 됩니다. 쓴 만큼 나옵니다. 지금은 세 번 쓰면 세 개가 똑같이 나옵니다. 내용이 다 같아서 쓸모가 없어 보이지만, 개념02에서 값을 넘기기 시작하면 이게 가장 많이 쓰는 방법이 됩니다.

목록을 자동으로 여러 개 그리는 방법(map)은 05단원에서 배웁니다. 그때까지는 필요한 만큼 손으로 씁니다.

▫ 직접 해보기 5

직접 해보기 1에서 만든 Bye 를 아래 App 의 '㉠ 컴포넌트 하나' 상자 안에 넣어 화면에 띄워 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

컴포넌트 이름을 소문자로 시작함

에러가 나지 않습니다. HTML 태그로 취급되어 화면에서 조용히 사라집니다. 섹션 3에서 확인했습니다. 이 단원에서 가장 자주 나는 사고입니다.

이런 실수를 합니다

return 을 빼먹음

이것도 에러가 나지 않습니다. 화면에서 그 컴포넌트만 사라집니다. 아래 NoReturn 은 JSX 를 만들어서 변수에 담기만 하고 돌려주지 않습니다.

```
function NoReturn() {
  const message = <p>이 줄은 화면에 안 나옵니다</p>;
}
```

화면 NoReturn 자리에는 아무것도 안 나옵니다. 콘솔도 조용합니다.

화살표 함수로 쓸 때 중괄호를 붙이면 이 실수가 특히 자주 납니다.

```
const A = () => <p>나옴</p>;      ← 돌려줍니다
const B = () => { <p>안 나옴</p>; }; ← 돌려주지 않습니다 (JS자료 08단원 map 과 같은 함정)
```

이런 실수를 합니다

여는 태그만 쓰고 닫지 않음

```
<Hello>
```

[SyntaxError] 입니다. 화면이 통째로 빡니다. 내용이 없는 컴포넌트는 <Hello /> 처럼 끝에 슬래시를 붙여 스스로 닫습니다. <Hello></Hello> 라고 써도 같습니다. (02단원 개념04)

이런 실수를 합니다

태그 대신 함수처럼 부름

에러는 안 납니다. 화면도 똑같이 나옵니다. 그래서 알아채기 어렵습니다.

```
console.log(Hello().type);
```

F12 콘솔 p

← `Hello()` 를 부르면 결과인 `<p>` 만 남고 'Hello' 라는 이름은 사라집니다.

```
console.log((<Hello />).type === Hello);
```

F12 콘솔 true

← 태그로 쓰면 `Hello` 함수가 그대로 들어 있어서, `React` 가 이것을 컴포넌트로 압니다.

지금 화면이 같아 보이지만, `React` 는 `{Hello()}` 를 '컴포넌트' 로 세지 않습니다. 04단원에서 컴포넌트마다 값을 기억시키기 시작하면 여기서 문제가 생깁니다. 처음부터 `<Hello />` 로 쓰세요.

이런 실수를 합니다

두 덩어리를 나란히 돌려줌

```
function Two(){  
  return <p>하나</p><p>둘</p>;  
}
```

[SyntaxError] 입니다. 돌려주는 것은 항상 하나여야 합니다. `<div>` 로 감싸거나, 02단원에서 배운 `<>` `</>` 로 감싸세요.

화면에 그리기 — 위에서 만든 컴포넌트를 한 번에 조립합니다

정리

- 1 컴포넌트는 **화면(JSX)**을 돌려주는 함수입니다. 새 문법이 아닙니다.
- 2 이름은 반드시 **대문자로** 시작합니다. `React` 는 첫 글자로 `HTML` 태그와 내 컴포넌트를 구분합니다.
- 3 소문자로 쓰면 **에러 없이** `HTML` 태그로 취급되어 화면에서 조용히 사라집니다. 가장 자주 나는 사고입니다.
- 4 화면에 그릴 때는 `Hello()` 가 아니라 `<Hello />` 로 씁니다.
- 5 한 파일에 컴포넌트를 여러 개 만들고, 다른 컴포넌트 안에 넣어 조립합니다.
- 6 같은 컴포넌트를 여러 번 써도 됩니다. 쓴 만큼 화면에 나옵니다.
- 7 `return` 을 빼먹어도 에러가 안 납니다. 그 컴포넌트만 사라집니다.

```

export default function Concept01() {
  return (
    <div> <h1>개념 01 – 컴포넌트 만들기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      아래 회색 상자 ① ~ ⑥ 은 <strong>전부 React 가 그린 것</strong>입니다.
      코드를 읽으면서 번호가 같은 상자를 함께 보세요.
      <p> <> <div className="demo"> <h3>① 컴포넌트 하나
    </h3> <div className="output"> <Hello /> </div> </div> <div className="demo"> <h3>②
    같은 컴포넌트를 두 번 쓰기</h3> <div className="output"> <Hello /> <Hello
    /> </div> </div> <LowercaseDemo /> <div className="demo"> <h3>④ 이름만 대문자로 고친
    것</h3> <div className="output"> <Menu /> </div> </div>

    <div className="demo">
      <h3>⑤ 조각을 조립한 것 – 같은 카드를 두 번</h3> <ShopCard /> <ShopCard />
    </div>

    <div className="demo"> <h3>⑥ return 을 빼먹은 컴포넌트
  </h3> <div className="output"> <NoReturn /> </div> <p>위 흰 칸도 비어 있습니다.
  에러는 나지 않습니다.</p> </div>
  </>

  </div>
);
}

```

직접 해보기 정답

```
1) function Bye() {
```

```
  return <p>또 만나요</p>;
```

```
}
```

→ 이름은 대문자 B 로 시작해야 합니다. bye 로 만들면 화면에서 사라집니다.

```
2) const pElement = <p />;
   console.log(pElement.type);
   // 콘솔: p
```

→ 소문자로 시작하니 문자열 'p' 가 들어 있습니다. HTML 태그라는 뜻입니다.

3) 콘솔에 빨간 글씨로 두 줄이 나옵니다.

```
<myBox /> is using incorrect casing. ...  
The tag <myBox> is unrecognized in this browser. ...
```

화면(㉠ 상자)에는 여전히 아무것도 안 나옵니다. myBox 는 HTML 에 없는 이름이라 React 가 알려 준 것입니다. menu 처럼 HTML 에 있는 이름이면 이 경고조차 나오지 않습니다.

```
4) function Notice() {
```

```
  return <p>오늘 케이크 6000원 할인</p>;
```

```
}
```

5) App 의 ㉠ 상자 안에 <Bye /> 를 한 줄 넣습니다.

```
<div className="output">  
  <Hello /> <Bye />  
</div>
```

화면 "안녕하세요, 저는 컴포넌트입니다." 아래에 "또 만나요" 가 나옵니다.

→ 아무 변화가 없다면 컴포넌트 이름이 소문자로 시작하지 않는지 확인하세요.

props 전달하기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념01에서 같은 컴포넌트를 여러 번 썼습니다. 그런데 내용이 전부 같았습니다.

```
<ShopCard />
<ShopCard />    ← 글자 하나까지 똑같은 카드 두 개
```

쓸모가 있으려면 “모양은 같고 내용은 다른” 것을 만들 수 있어야 합니다. 메뉴판이라면 카드 모양은 같고 이름과 가격만 달라야 합니다.

함수에서는 이미 해 본 일입니다. 인자를 다르게 넘기면 결과가 달라졌습니다. 컴포넌트에 값을 넘기는 것을 props 라고 부릅니다. properties(속성)를 줄인 말입니다.

- 1 컴포넌트에 값을 넘기고 받는 법
- 2 props 는 객체 하나로 온다
- 3 언제 중괄호가 필요한가 — 이것 때문에 조용히 틀리는 일이 자주 생깁니다

값을 넘기면 결과가 달라집니다

섹션 1

먼저 JS자료 05단원 개념02(매개변수와 인자)를 떠올려 봅시다.

```
import Summary from "../_ui/Summary.jsx";

function greet(name) {
  return `${name}님 환영합니다`;
}

console.log(greet("김민준"));
```

F12 콘솔 김민준님 환영합니다

```
console.log(greet("이서연"));
```

F12 콘솔 이서연님 환영합니다

함수는 하나인데 넘긴 값이 달라서 결과가 달라졌습니다. 컴포넌트도 똑같이 합니다. 넘기는 '모양' 만 다릅니다.

```
function Greet(props) {
  return <p>{props.name}님 환영합니다</p>;
}
```

쓸 때는 HTML 속성처럼 씁니다.

```
<Greet name="김민준" />
<Greet name="이서연" />
```

나란히 놓고 보면 이렇습니다.

```
함수      greet("김민준")
컴포넌트  <Greet name="김민준" />
```

함수는 '순서' 로 넘겼습니다. 첫 번째 자리에 있으니 name 이었습니다. 컴포넌트는 '이름' 을 붙여 넘깁니다. 그래서 순서를 바꿔도 상관없습니다. JS자료 09단원 개념02 마지막에서 본 order2({ menu, size, ice }) 와 같은 방식입니다.

화면 ① 을 보세요. 같은 Greet 인데 두 줄의 이름이 다릅니다.

➤ 직접 해보기 1

<Greet name="박지훈" /> 을 화면 ① 상자에 한 줄 더 넣어 보세요.

props 는 객체 하나로 옵니다

섹션 2

넘긴 값들이 각각 매개변수로 오는 것이 아닙니다. 전부 하나의 객체에 담겨서 '첫 번째 매개변수' 하나로 옵니다.

```
function ShowProps(props) {
  console.log(props);
}
```

```
F12 콘솔    { name: '이서연', age: 22 }
```

```
return (
  <p>
    {props.name} / {props.age}세
  </p>
);
}
```

위 컴포넌트를 <ShowProps name="이서연" age={22} /> 로 썼습니다. 넘긴 두 개가 { name: '이서연', age: 22 } 라는 객체 하나가 되었습니다.

그래서 꺼낼 때는 JS자료 07단원에서 배운 점 표기법을 그대로 씁니다.

```
props.name
props.age
```

매개변수 이름은 마음대로 지어도 동작합니다. p 라고 써도 됩니다. 하지만 React 를 쓰는 사람들은 거의 전부 props 라고 씁니다. 남의 코드를 읽을 때 헛갈리지 않도록 여러분도 props 라고 쓰세요.

몇 개를 넘기든 매개변수는 항상 하나입니다. 열 개를 넘겨도 객체 하나에 다 담깁니다.

◀ 직접 해보기 2

ShowProps 에 city="부산" 을 하나 더 넘기고 콘솔에 찍히는 객체가 어떻게 바뀌는지 보세요.

03

문자열은 따옴표, 나머지는 중괄호

섹션 3

값을 넘기는 방법은 두 가지입니다.

```
title="아메리카노"    ← 문자열
price={4000}          ← 그 밖의 값 (숫자·불리언·배열·객체·변수)
```

왜 나뉘어 있을까요? 02단원 개념02에서 배운 것과 같은 규칙 때문입니다. JSX 의 속성 자리는 기본이 '글자' 입니다. 따옴표 안에 쓴 것은 글자 그대로 갑니다. 자바스크립트 값을 넣고 싶으면 "여기부터는 자바스크립트다" 라고 표시해야 하고, 그 표시가 중괄호입니다.

실제로 어떤 자료형으로 들어가는지 확인해 봅시다.

```
function TypeCheck(props) {
  console.log(typeof props.price, typeof props.priceText);
```

```
F12 콘솔    number string
```

```
console.log(typeof props.hot, Array.isArray(props.tags));
```

```
F12 콘솔    boolean true
```

```
return (
  <p>
    {props.name} {props.price}원 (문자열로 넘긴 것: {props.priceText})
  </p>
);
}
```

위 컴포넌트에 이렇게 넘겼습니다.

```

<TypeCheck
  name="아메리카노"      ← 문자열
  price={4000}           ← 숫자
  priceText="4000"       ← 따옴표로 넘긴 숫자 → 문자열이 됩니다
  hot={true}             ← 불리언
  tags={["인기", "따뜻함"]} ← 배열
/>

```

priceText 를 눈여겨보세요. 화면에는 4000 이라고 똑같이 나옵니다. 그런데 자료형은 문자열입니다. 계산에 쓰는 순간 조용히 틀립니다. 섹션 6 [실수 1] 에서 실제로 확인합니다.

몇 가지 더 알아 두면 좋은 것

변수 넘기기 name={menuName} ← 중괄호 안에 변수 이름

객체 넘기기 user={{ name: "김민준" }}

중괄호가 두 개입니다. 바깥은 "여기부터 자바스크립트",
안쪽은 객체를 만드는 중괄호입니다. (02단원 style={{ }} 와 같습니다)

불리언 축약 hot ← hot={true} 와 같습니다 문자열도 중괄호로 title={"아메리카노"} ← 되기는 하지만 굳이 이렇게 쓰지 않습니다

```

const menuName = "라떼";
console.log(menuName);

```

F12 콘솔 라떼

⇒ 직접 해보기 3

TypeCheck 에 count={2} 를 넘기고, 컴포넌트 안에서 `typeof props.count` 를 콘솔에 찍어 보세요.

여러 개 넘기고 여러 번 쓰기

섹션 4

이제 개념01에서 못 했던 것을 할 수 있습니다. 모양은 한 번만 적고, 내용은 쓸 때마다 다르게 넣습니다.

```

function MenuItem(props) {
  return (
    <div className="output"> <strong>{props.name}</strong> – {props.price}원
    <br />
    세 개 사면 {props.price * 3}원
  </div>
  );
}

```

props 는 그냥 값이라서 계산에 써도 됩니다. {props.price * 3} 처럼 중괄호 안에서 식을 쓸 수 있습니다. (02단원 개념02)

화면 ㉔ 를 보세요. 카드 세 개가 같은 모양, 다른 내용으로 나옵니다. 이 세 줄이 화면 ㉔ 를 만듭니다.

```
<MenuItem name="아메리카노" price={4000} />
<MenuItem name="라떼" price={4500} />
<MenuItem name="케이크" price={6000} />
```

지금은 손으로 세 번 썼습니다. 배열을 받아 자동으로 여러 개 그리는 방법(map)은 05단원에서 배웁니다.

▸ 직접 해보기

4

화면 ㉔ 에 삼각김밥 1200원 카드를 한 줄 더 추가해 보세요.

props 는 위에서 아래로만 갑니다

섹션 5

props 는 '쓰는 쪽' 에서 '만든 쪽' 으로 갑니다. 즉 부모 컴포넌트가 자식 컴포넌트에게 주는 것입니다. 반대 방향은 안 됩니다.

```
App — name="김민준" → Greet
```

자식이 부모에게 무언가 알려 주고 싶을 때는 다른 방법을 씁니다. 07단원에서 배웁니다.

그럼 부모가 아무것도 안 넘기면 어떻게 될까요?

```
function NoProps(props) {
  console.log("안 넘겼을 때:", String(props.name));
}
```

```
F12 콘솔  안 넘겼을 때: undefined
```

```
console.log(Object.keys(props));
```

```
F12 콘솔  []
```

```
return <p>{props.name}님 환영합니다</p>;
}
```

화면 ㉕ 를 보세요. "님 환영합니다" 만 나옵니다. props.name 이 undefined 인데도 에러가 나지 않습니다. JSX 안의 undefined 는 '아무것도 안 그림' 으로 처리되기 때문입니다.

이름이 안 나오는데 콘솔은 조용합니다. 그래서 원인을 찾기 어렵습니다. 화면에 값 하나가 비어 있으면 "넘겼는지" 와 "이름이 같은지" 부터 확인하세요.

안 넘겼을 때 쓸 기본값을 정해 두는 방법은 개념03에서 배웁니다.

화면 ㉔ 의 <NoProps /> 에 name="박지훈" 을 넘겨 콘솔의 undefined 가 사라지는지 확인하세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

숫자를 따옴표로 넘김 ★ 가장 자주 납니다

에러가 나지 않습니다. 화면도 멀쩡해 보입니다. 계산할 때만 틀립니다.

```
function BadPrice(props) {
  console.log(props.price + 500);
```

F12 콘솔 4000500

```
console.log(typeof (props.price + 500));
```

F12 콘솔 string

```
return <p>4000원에 500원을 더하면 {props.price + 500}원</p>;
}
```

<BadPrice price="4000" /> 처럼 따옴표로 넘겼습니다. 문자열 + 숫자는 이어 붙이기가 됩니다. JS자료 02단원 개념01에서 본 그대로입니다. 고치는 법: price={4000} 처럼 중괄호로 넘깁니다.

이런 실수를 합니다

중괄호를 빼먹고 그냥 씬

글자가 그대로 화면에 나옵니다. 이것도 에러가 안 납니다.

```
function NoBrace(props) {
  return <p>props.name</p>;
}
```

화면 props.name 이라는 글자가 그대로 나옵니다.

고치는 법: <p>{props.name}</p>

이런 실수를 합니다

속성 이름의 대소문자를 틀림

넘기는 쪽은 Name, 받는 쪽은 name 이면 서로 다른 이름입니다.

```
function TypoProps(props) {
  console.log("실수 3 키 목록:", Object.keys(props));
```

F12 콘솔 실수 3 키 목록: ['Name']

```
return <p>[{props.name}] 이름이 비어 있습니다</p>;
}
```

<TypoProps Name="김민준" /> 로 넘겼습니다. props.Name 에는 값이 있지만 props.name 은 undefined 입니다. 경고 한 줄 없습니다. 이름은 넘기는 쪽과 받는 쪽이 글자까지 같아야 합니다.

이런 실수를 합니다

받는 쪽에 매개변수를 안 적음

```
function Bad() {
return <p>{props.name}</p>;
}
```

ReferenceError: props is not defined

가 납니다. 이건 빨간 에러가 나므로 오히려 찾기 쉽습니다. 화면은 그 지점부터 안 그려집니다.

이런 실수를 합니다

숫자에 중괄호를 안 붙임

<Greet name="김민준" age=20 /> [SyntaxError] 입니다. 화면이 통째로 빡니다. JSX 속성값 자리에는 따옴표 아니면 중괄호만 올 수 있습니다. age={20} 이라고 써야 합니다.

화면에 그리기

정리

- 1 컴포넌트에 값을 넘기는 것을 **props** 라고 합니다. 함수의 인자와 같은 역할입니다.
- 2 넘길 때는 `<Greet name="김민준" />` 처럼 속성으로 씁니다.
- 3 받을 때는 매개변수 하나로 받습니다. 넘긴 것이 전부 **객체 하나**에 담겨 옵니다. `props.name` 으로 꺼냅니다.
- 4 문자열은 `title="글자"`, 그 밖의 값은 `price={4000}` 처럼 **중괄호**가 필요합니다. 중괄호는 "여기부터 자바스크립트" 표시입니다.
- 5 숫자를 따옴표로 넘기면 문자열이 됩니다. 에러 없이 계산만 틀립니다.
- 6 props 는 부모 → 자식 한 방향입니다. 반대 방향은 07단원에서 배웁니다.
- 7 안 넘긴 props 는 `undefined` 이고, 화면에는 아무것도 안 나옵니다.

```

export default function Concept02() {
  return (
    <div>
      <h1>개념 02 – props 전달하기</h1>

      <p className="guide">
        왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
        Console</strong> 도 함께 보세요.
      <br />
      <br />
        아래 회색 상자 ① ~ ⑦ 은 전부 React 가 그린 것입니다. 번호가 같은 코드와
        함께 보세요.
      </p>

      <>
        <div className="demo">
          <h3>① 같은 컴포넌트, 다른 값</h3>
          <div className="output">
            <Greet name="김민준" />
            <Greet name="이서연" />
          </div>
        </div>

        <div className="demo">
          <h3>② props 는 객체 하나 (콘솔도 함께 보세요)</h3>
          <div className="output">
            <ShowProps name="이서연" age={22} />
          </div>
        </div>

        <div className="demo">
          <h3>③ 중괄호가 필요한 값들</h3>
          <div className="output">
            <TypeCheck
              name="아메리카노"
              price={4000}
              priceText="4000"
              hot={true}
            />
          </div>
        </div>
      </>
    </div>
  )
}

```

```

        tags={["인기", "따뜻함"]}
      />
    </div>
  </div>

  <div className="demo"> <h3>④ 모양은 하나, 내용은 셋
</h3> <MenuItem name="아메리카노" price={4000} /> <MenuItem name="라떼" price={4500}
/> <MenuItem name="케이크" price={6000} /> </div> <div className="demo"> <h3>⑤
아무것도 안 넘겼을 때</h3> <div className="output"> <NoProps
/> </div> </div> <div className="demo"> <h3>⑥ 숫자를 따옴표로 넘긴 결과
</h3> <div className="output"> <BadPrice price="4000"
/> </div> </div> <div className="demo"> <h3>⑦ 중괄호를 빼먹은 결과 / 이름을 틀린
결과</h3> <div className="output"> <NoBrace name="김민준" /> <TypoProps Name="김민준"
/> </div> </div>
  </>

</div>
);
}

```

직접 해보기 정답

1) App 의 ① 상자 안에 한 줄 넣습니다.

```
<Greet name="박지훈" />
```

화면 김민준님 환영합니다 / 이서연님 환영합니다 / 박지훈님 환영합니다

→ "님 환영합니다" 만 나오면 name 을 안 넘겼거나 이름을 틀린 것입니다.

```
2) <ShowProps name="이서연" age={22} city="부산" />
// 콘솔: { name: '이서연', age: 22, city: '부산' }
```

→ 넘긴 것이 늘어난 만큼 객체의 속성도 늘어납니다.

화면은 그대로입니다. ShowProps 가 city 를 안 쓰기 때문입니다.

3) 넘기는 쪽: <TypeCheck ... count={2} />

```
받는 쪽:   console.log(typeof props.count);
// 콘솔: number
```

→ count="2" 로 넘기면 string 이 나옵니다. 중괄호 하나 차이입니다.

4) App 의 ④ 상자 안에 한 줄 넣습니다.


```
<MenuItem name="삼각김밥" price={1200} />
```

화면 삼각김밥 - 1200원 / 세 개 사면 3600원

→ 카드가 아예 안 늘어나면 <MenuItem /> 을 App 의 ㉔ 상자 밖에 쓴 것입니다.

"원" 앞이 비어 있으면 price 를 안 넘긴 것입니다.

참고로 price="1200" 처럼 따옴표로 넘겨도 곱하기는 3600 으로 잘 나옵니다.

곱하기는 문자열을 숫자로 바꿔서 계산하기 때문입니다. 더하기만 틀립니다.

5) <NoProps name="박지훈" />

```
// 콘솔: 안 넘겼을 때: 박지훈
// 콘솔: ['name']
// 화면: 박지훈님 환영합니다
```

→ 넘기지 않았을 때는 props 가 빈 객체 {} 라서 키 목록도 빈 배열이었습니다.

props 구조분해와 기본값

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념02에서 props 를 이렇게 꺼냈습니다.

```
function MenuItem(props) {
  return <p>{props.name} {props.price}원</p>;
}
```

props 가 늘어나면 props. 를 계속 붙여야 합니다. 다섯 개면 다섯 번입니다. 이미 이 문제를 겪었고 해결책도 배웠습니다. JS자료 09단원 개념02입니다.

```
function introduce({ name, age, city }) { ... }
```

React 에서는 이 문법을 거의 모든 컴포넌트에서 씁니다. 09단원에서 "React 를 배우면 매 줄 나옵니다" 라고 적었던 것이 바로 이것입니다.

- 1 props 를 매개변수 자리에서 바로 꺼내기
- 2 안 넘어왔을 때 쓸 기본값 정하기
- 3 객체를 통째로 넘기고 안에서 꺼내기

props. 를 반복하지 않기

섹션 1

먼저 순수한 자바스크립트로 09단원을 한 번 복습합니다.

```
import Summary from "../_ui/Summary.jsx";

const user = { name: "김민준", age: 20, city: "부산" };
```

구조분해 없이 user. 를 계속 붙입니다

```
function introduce1(person) {
  return `${person.name} / ${person.age}세 / ${person.city}`;
}

console.log(introduce1(user));
```

F12 콘솔 김민준 / 20세 / 부산

— 매개변수 자리에서 구조분해

```
function introduce2({ name, age, city }) {
  return `${name} / ${age}세 / ${city}`;
}

console.log(introduce2(user));
```

F12 콘솔 김민준 / 20세 / 부산

결과가 같습니다. 함수 안이 짧아졌을 뿐입니다.

컴포넌트도 함수라고 했습니다. props 도 객체 하나라고 했습니다. 그러니 똑같이 하면 됩니다.

```
function MenuItemA(props) {
  return (
    <p>
      {props.name} - {props.price}원
    </p>
  );
}

function MenuItemB({ name, price }) {
  return (
    <p>
      {name} - {price}원
    </p>
  );
}
```

두 컴포넌트는 완전히 같은 일을 합니다. 쓰는 쪽도 똑같습니다.

```
<MenuItemA name="아메리카노" price={4000} />
<MenuItemB name="아메리카노" price={4000} />
```

화면 ① 에서 두 줄이 똑같이 나오는 것을 확인하세요.

넘기는 쪽은 하나도 바뀌지 않습니다. 받는 쪽 괄호 안만 바뀝니다. 그리고 매개변수만 봐도 “이 컴포넌트가 무엇을 받는지” 가 보입니다. 남이 만든 컴포넌트를 읽을 때 이 한 줄이 설명서 역할을 합니다.

▸ 직접 해보기 1

MenuItemB 를 고쳐서 “아메리카노 — 4000원 (뜨거움)” 처럼 hot 이라는 props 도 함께 받아 화면에 찍어 보세요. (넘길 때는 hot=“뜨거움” 으로 넘기면 됩니다)

구조분해는 매개변수 자리 말고 함수 안에서 해도 됩니다. JS자료 09단원 개념02 섹션1에서 하던 방식 그대로입니다.

```
function MenuItemC(props) {
  const { name, price } = props; // ← 함수 안에서 꺼내기 return (
    <p>
      {name} - {price}원
    </p>
  );
}
```

세 가지 방법(A·B·C)이 전부 같은 결과를 냅니다.

A: props.name 을 그때그때 — props 가 한두 개일 때 편합니다 B: 매개변수 자리에서 { name } — 가장 많이 씁니다 C: 함수 안에서 `const { name }` — props 전체도 함께 써야 할 때 편합니다

무엇을 써도 틀리지 않습니다. 이 자료는 앞으로 B 를 주로 씁니다.

▸ 직접 해보기 2

MenuItemC 안에서 `console.log(props)` 를 찍어 구조분해를 해도 props 객체는 그대로 있는지 확인하세요.

기본값 — 안 넘어왔을 때 쓸 값

개념02 섹션5에서 props 를 안 넘기면 `undefined` 라고 했습니다. 화면에는 아무것도 안 나왔습니다.

이럴 때 쓸 값을 미리 정해 둘 수 있습니다. 이것도 09단원 개념02 섹션3에서 배운 문법 그대로입니다.

```
function pickName({ name = "손님" }) {
  return name;
}

console.log(pickName({ name: "이서연" }));
```

F12 콘솔 이서연

```
console.log(pickName({}));
```

F12 콘솔 손님

값이 있으면 그 값을, 없으면 기본값을 씁니다.

주의 · 기본값이 쓰이는 것은 `undefined` 일 때뿐입니다. 09단원에서 본 그대로입니다.

코드	▶ F12 콘솔
<code>console.log(pickName({ name: null }));</code>	▶ null
<code>console.log(pickName({ name: "" }) === "");</code>	▶ true

`null` 이나 빈 문자열을 넘기면 그 값이 그대로 쓰입니다. 빈 문자열이면 화면에 아무것도 안 나옵니다. 기본값이 안 먹는 것처럼 보입니다.

컴포넌트에 그대로 옮기면 이렇게 됩니다.

```
function Greeting({ name = "손님" }) {
  return <p>{name}님, 어서 오세요.</p>;
}
```

화면 ㉓ 에서 확인하세요.

<code><Greeting name="이서연" /></code>	→ 이서연님, 어서 오세요.
<code><Greeting /></code>	→ 손님님, 어서 오세요.

"손님님" 이 어색하지만, 기본값이 실제로 쓰였다는 것은 확실히 보입니다.

기본값은 이럴 때 씁니다. - 대부분 같은 값이고 가끔만 다를 때 (버튼 색, 크기 같은 것) - 안 넘겨도 화면이 깨지지 않게 하고 싶을 때

▫ 직접 해보기 3

MenuItemB 에 `price = 0` 기본값을 주고 `<MenuItemB name="물" />` 을 화면 ㉓ 에 넣어 보세요.

이름을 바꿔 받기

섹션 4

09단원 개념02 섹션2에서 배운 것입니다. 왼쪽이 원래 이름, 오른쪽이 내가 쓸 이름입니다.

```
function ItemLine({ name: itemName, price: itemPrice = 0 }) {
  return (
    <p>
      {itemName} / {itemPrice}원
    </p>
  );
}
```

넘기는 쪽은 여전히 `name`, `price` 입니다. 받는 쪽 안에서만 이름이 바뀝니다.

```
<ItemLine name="케이크" price={6000} />
```

1 같은 이름의 변수가 이미 있을 때 (09단원 개념02 섹션2의 productName 예)

2 name 처럼 너무 흔한 이름을 더 분명하게 바꾸고 싶을 때

이름 바꾸기와 기본값은 함께 쓸 수 있습니다. 위 price 가 그렇습니다.

```
price: itemPrice = 0
```

원래	내가 쓸	기본값
이름	이름	

▹ 직접 해보기 4

ItemLine 을 <ItemLine name="삼각김밥" /> 으로 써 보고 가격 자리에 무엇이 나오는지 확인하세요.

객체를 통째로 넘기기

섹션 5

props 를 하나하나 넘기다 보면 이런 코드가 나옵니다.

```
<UserCard name="이서연" age={22} city="서울" />
```

그런데 원래 데이터가 객체 하나였다면, 통째로 넘기는 편이 짧습니다.

```
const seoyeon = { name: "이서연", age: 22, city: "서울" };

function UserCard({ user }) {
  const { name, age, city } = user; // 받은 객체에서 다시 꺼냅니다 return (
    <div className="output"> <strong>{name}</strong> ({age}세)
    <br />
    사는 곳: {city}
  </div>
);
}
```

쓸 때는 이렇게 씁니다.

```
<UserCard user={seoyeon} />
```

중괄호가 필요합니다. 객체는 문자열이 아니기 때문입니다. (개념02 섹션3)

매개변수 자리에서 한 번에 꺼낼 수도 있습니다. 09단원 개념02 섹션7의 중첩 구조분해입니다.

```
function UserCard({ user: { name, age, city } }) { ... }
```

되기는 하지만 읽기 어렵습니다. 위처럼 두 줄로 나누는 편이 낫습니다.

하나씩 넘기기와 통째로 넘기기 중 무엇이 나올까요? - 컴포넌트가 그 객체의 대부분을 쓴다 → 통째로 - 두세 개만 쓴다 → 필요한 것만 통째로 넘기면 그 컴포넌트는 “그 모양의 객체”에만 쓸 수 있게 됩니다.

▹ 직접 해보기 5

`const jihun = { name: "박지훈", age: 28, city: "대구" };` 를 만들고 화면 ⑤ 에 카드를 하나 더 그려 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

구조분해해 놓고 **props** 를 계속 씀

```
function Bad1({ name }) {  
  return <p>{props.name}</p>;  
}
```

ReferenceError: props is not defined

입니다. 매개변수 자리에 { name } 이라고 쓰면 props 라는 이름은 만들어지지 않습니다. 구조분해로 꺼낸 name 을 그대로 쓰세요.

이런 실수를 합니다

속성 이름을 틀림 ★ 조용히 틀립니다

09단원 개념02 [실수 1] 과 똑같습니다. 객체 구조분해는 '이름' 으로 찾습니다.

```
function TypoItem({ nmae }) {  
  console.log("실수 2 -", String(nmae));
```

F12 콘솔 실수 2 - undefined

```
return <p>[{nmae}] 여기가 비어 있습니다</p>;  
}
```

`<TypoItem name="아메리카노" />` 로 제대로 넘겨도 nmae 는 `undefined` 입니다. 에러도 경고도 없습니다. 화면에 빈칸만 남습니다. 화면에 값 하나가 안 보이면 넘기는 쪽과 받는 쪽의 철자를 나란히 놓고 비교하세요.

이런 실수를 합니다

중괄호를 빼먹음 ★ 자주 납니다

```
function NoBraceItem(name, price) {  
  console.log("실수 3 - 첫 번째 매개변수:", typeof name, Object.keys(name));
```

F12 콘솔 실수 3 - 첫 번째 매개변수: object ['name', 'price']

```
console.log("실수 3 - price 가 4000 인가?", price === 4000);
```

F12 콘솔 실수 3 - price 가 4000 인가? false

```
return <p>콘솔을 보세요</p>;  
}
```

<NoBraceItem name="아메리카노" price={4000} /> 로 넘겼습니다.

중괄호를 빼면 '구조분해'가 아니라 그냥 매개변수 두 개가 됩니다. 컴포넌트는 언제나 props 객체 하나만 받으므로

name → props 객체 전체가 통째로 들어옵니다

price → 우리가 넘긴 4000 이 아닙니다. React 가 내부에서 쓰는 값이 옵니다

이 됩니다.

이 상태에서 화면에 {name} 을 그리면 이런 에러가 납니다.

Error: Objects are not valid as a React child

"객체는 화면에 그릴 수 없다" 는 뜻입니다. 이 에러를 보면 중괄호부터 확인하세요.

이런 실수를 합니다

기본값을 등호가 아니라 콜론으로 씀

```
function Bad4({ name: "손님" }) {  
  return <p>{name}</p>;  
}
```

[SyntaxError] 입니다. 화면이 통째로 빕니다. 콜론은 '이름 바꾸기' 이고 등호가 '기본값' 입니다. (09단원 개념02) { name = "손님" } 이라고 써야 합니다.

이런 실수를 합니다

기본값이 안 먹는다고 생각함

섹션 3에서 본 것처럼 기본값은 `undefined` 일 때만 쓰입니다. 빈 문자열("")이나 `null` 을 넘기면 그 값이 그대로 화면에 갑니다. 빈 문자열은 화면에 아무것도 안 그려서 “안 넘긴 것” 처럼 보입니다.

화면에 그리기

정리

- 1 `function Item({ name, price })` — 매개변수 자리에서 props 를 꺼냅니다. JS자료 09단원 객체 구조분해 그대로입니다.
- 2 넘기는 쪽은 바뀌지 않습니다. 받는 쪽 괄호 안만 바꿉니다.
- 3 매개변수 한 줄만 봐도 그 컴포넌트가 무엇을 받는지 보입니다. 설명서 역할을 합니다.
- 4 `{ name = “손님” }` — 안 넘어왔을 때 쓸 **기본값**을 정합니다.
- 5 기본값은 **undefined 일 때만** 쓰입니다. `null` 이나 빈 문자열은 그대로 쓰입니다.
- 6 `{ name: itemName }` — 왼쪽이 원래 이름, 오른쪽이 내가 쓸 이름입니다.
- 7 객체를 통째로 넘기고(`user={seoyeon}`) 컴포넌트 안에서 다시 꺼내도 됩니다.

```
export default function Concept03() {
  return (
    <div> <h1>개념 03 – props 구조분해와 기본값</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일은 새 React 문법이 거의 없습니다. <strong>JS자료 09단원 개념02(객체
      구조분해)를 그대로 씁니다.</strong> 기억이 흐리면 그 파일을 먼저 한 번 열어 보세요.
      </p> <> <div className="demo"> <h3>① 같은 결과 – props. 방식과 구조분해 방식
      </h3> <div className="output"> <MenuItemA name="아메리카노" price={4000}
      /> <MenuItemB name="아메리카노" price={4000}
      /> </div> </div> <div className="demo"> <h3>② 함수 안에서 꺼낸 것
      </h3> <div className="output"> <MenuItemC name="라떼" price={4500}
      /> </div> </div> <div className="demo"> <h3>③ 기본값 – 넘긴 것과 안 넘긴 것
      </h3> <div className="output"> <Greeting name="이서연" /> <Greeting /> </div> </div>
```

```

        <div className="demo">
            <h3>④ 이름을 바꿔 받기
        </h3> <div className="output"> <ItemLine name="케이크" price={6000} /> </div>
        </div>

        <div className="demo"> <h3>⑤ 객체를 통째로 넘기기</h3> <UserCard user=
        {seoyeon} /> </div> <div className="demo"> <h3>⑥ 실수 2 · 3 (콘솔도 함께 보세요)
        </h3> <div className="output"> <TypoItem name="아메리카노"
        /> <NoBraceItem name="아메리카노" price={4000} /> </div> </div>
        </>

    </div>
    );
}

```

직접 해보기 정답

```
1) function MenuItemB({ name, price, hot }) {
```

```

    return (
        <p>
            {name} - {price}원 ({hot})
        </p>
    );

```

```
}
```

쓸 때: <MenuItemB name="아메리카노" price={4000} hot="뜨거움" />

```
// 화면: 아메리카노 - 4000원 (뜨거움)
```

→ 괄호만 남고 안이 비면 hot 을 안 넘겼거나 철자를 틀린 것입니다.

```
2) function MenuItemC(props) {
```

```

    const { name, price } = props;
    console.log(props);
    // 콘솔: { name: '라떼', price: 4500 }
    ...

```

```
}
```

→ 구조분해는 '꺼내서 복사' 하는 것이라 원본 props 는 그대로 남습니다.

09단원 개념02 섹션4에서 원본이 안 바뀐 것과 같습니다.

```
3) function MenuItemB({ name, price = 0 }) { ... }
```

쓸 때: <MenuItemB name="물" />

```
// 화면: 물 - 0원
```

→ "물 — 원" 이 나오면 기본값을 안 준 것입니다. `undefined` 는 화면에 안 그려집니다.

4) <ItemLine name="삼각김밥" />

```
// 화면: 삼각김밥 / 0원
```

→ price 를 안 넘겼으므로 기본값 0 이 쓰였습니다.

기본값이 없었다면 "삼각김밥 / 원" 이 됐을 것입니다.

```
5) const jihun = { name: "박지훈", age: 28, city: "대구" };
```

App 의 ㉔ 상자 안에 <UserCard user={jihun} /> 를 한 줄 넣습니다.

```
// 화면: 박지훈 (28세) / 사는 곳: 대구
```

→ user={jihun} 에서 중괄호를 빼고 user="jihun" 이라고 쓰면

문자열이 넘어가서 `const { name } = "jihun"` 이 되고 name 은 `undefined` 가 됩니다.

children

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

지금까지 값을 넘길 때는 전부 속성 자리를 썼습니다.

```
<MenuItem name="아메리카노" price={4000} />
```

그런데 HTML 을 떠올려 보면 이런 모양이 더 익숙합니다.

```
<div>안녕하세요</div>
<li>아메리카노</li>
```

태그 '사이' 에 내용을 넣는 방식입니다. 내 컴포넌트도 이렇게 쓸 수 있습니다.

```
<Box>안녕하세요</Box>
```

이때 태그 사이에 넣은 것은 children 이라는 props 로 들어옵니다. children 은 '자식들' 이라는 뜻입니다.

- 1 태그 사이에 넣은 것이 children 으로 온다
- 2 children 을 화면 어디에 놓을지는 내가 정한다
- 3 테두리·제목 같은 겉모습을 담당하는 '상자 컴포넌트' 만들기

태그 사이에 넣은 것이 children 입니다

섹션 1

```
import Summary from "../_ui/Summary.jsx";
```

```
function Box(props) {
  console.log(props.children);
}
```

F12 콘솔 안녕하세요

코드

```
console.log(typeof props.children);
```

```
console.log(Object.keys(props));
```

▶ F12 콘솔

▶ string

▶ ['children']

```
return <div className="output">{props.children}</div>;
}
```

이렇게 씁니다.

```
<Box>안녕하세요</Box>
```

속성은 하나도 안 넘겼는데 props 에 children 이 들어 있습니다. 태그 사이에 쓴 글자가 children 이라는 이름으로 자동으로 담긴 것입니다.

즉 children 은 특별한 문법이 아니라 props 의 한 속성입니다. 이름을 우리가 짓지 않고 React 가 정해 뒀을 뿐입니다.

화면 ㉠ 을 보세요. 흰 칸 안에 "안녕하세요" 가 들어 있습니다.

▣ 직접 해보기 1

화면 ㉠ 의 <Box>안녕하세요</Box> 안 글자를 "오늘의 메뉴" 로 바꾸고 콘솔이 어떻게 바뀌는지 보세요.

children 을 어디에 놓을지는 내가 정합니다

섹션 2

받은 children 은 그냥 값입니다. JSX 안 아무 데나 넣을 수 있습니다. 개념03에서 배운 구조분해로 받으면 더 짧습니다.

```
function Card({ title, children }) {
  return (
    <div className="output"> <h4>{title}</h4>
    {children}
    <p>- 이 줄은 Card 가 항상 붙입니다 -</p> </div>
  );
}
```

이렇게 씁니다.

```
<Card title="오늘의 커피">
  <p>아메리카노 4000원</p>
</Card>
```

제목은 위, children 은 가운데, 고정 문구는 아래입니다. 순서를 바꾸면 화면 순서도 바뀝니다. 정하는 사람은 Card 를 만든 나입니다.

그리고 중요한 점이 하나 있습니다. {children} 을 안 쓰면 넘긴 내용이 화면에 나오지 않습니다. 에러도 경고도 없습니다. 화면 ㉡ 에서 직접 확인합니다.

▣ 직접 해보기 2

Card 안에서 {children} 과 <h4>{title}</h4> 의 순서를 바꿔 보고 화면 ㉔ 가 어떻게 달라지는지 확인하세요.

태그도 넣을 수 있고 여러 개도 됩니다

섹션 3

태그 사이에는 글자만이 아니라 태그도 넣을 수 있습니다. 몇 개를 넣든 상관없습니다.

```
function ListBox({ children }) {  
  console.log("ListBox children 이 배열인가?", Array.isArray(children));
```

F12 콘솔 ListBox children 이 배열인가? true

```
  return <div className="output">{children}</div>;  
}
```

```
function OneBox({ children }) {  
  console.log("OneBox children 이 배열인가?", Array.isArray(children));
```

F12 콘솔 OneBox children 이 배열인가? false

```
  return <div className="output">{children}</div>;  
}
```

넣은 것이 하나면 그 값 하나가 오고, 둘 이상이면 배열이 옵니다.

```
<OneBox><p>아메리카노 4000원</p></OneBox>                    → 배열이 아님  
<ListBox>  
  <p>아메리카노 4000원</p>  
  <p>라떼 4500원</p>  
</ListBox>                                                        → 배열
```

그런데 이걸 구분해서 쓸 일은 거의 없습니다. {children} 이라고만 쓰면 하나든 여럿이든 React 가 알아서 다 그려 줍니다.

다만 '몇 개인지 세려고' 할 때는 조심해야 합니다. 섹션 6 [실수 3] 을 보세요.

▣ 직접 해보기 3

화면 ㉔ 의 ListBox 안에 <p>케이크 6000원</p> 을 한 줄 더 넣어 보세요.

속성으로 넘길까, children 으로 넘길까

섹션 4

둘 다 결국 props 입니다. 아래처럼 나눠 쓰면 읽기 좋습니다.

짧은 값 하나 (제목·가격·이름) → 속성 title="오늘의 커피" 화면 덩어리 (여러 줄, 태그 포함) → children
태그 사이에 넣기

실제로 children 은 속성으로도 넘길 수 있습니다.

```
<Box children="안녕하세요" />
```

동작은 똑같습니다. children 이 그냥 props 라는 증거이기도 합니다. 하지만 아무도 이렇게 쓰지 않습니다. 읽기 어렵기 때문입니다. "태그 사이에 넣는다" 는 모양 자체가 이미 설명이 되어 주기 때문에 그 방식을 씁니다.

```
function Panel(props) {
  console.log("Panel 이 받은 props 이름들:", Object.keys(props));
```

F12 콘솔 Panel 이 받은 props 이름들: ['title', 'children']

```
const { title, children } = props;

return (
  <div className="output"> <h4>{title}</h4>
    {children}
  </div>
);
}
```

▹ 직접 해보기 4

<Panel title="공지"> ... </Panel> 를 화면 ㉔ 에 하나 더 만들고 안에 <p>내일은 쉽니다</p> 를 넣어 보세요.

상자 컴포넌트 만들기

섹션 5

children 이 가장 빛나는 곳은 '겉모습만 담당하는 컴포넌트' 입니다. 안에 무엇이 들어올지는 모르지만, 테두리와 여백은 항상 같게 하고 싶을 때 씁니다.

예를 들어 알림 상자를 만들어 봅시다. 겉모습은 한 곳(Notice)에만 적혀 있고, 안의 내용은 쓰는 쪽이 정합니다.

```
function Notice({ children }) {
  return (
    <div className="output"> <strong>[알림]</strong> {children}
    </div>
  );
}
```

상자는 겹쳐 쓸 수도 있습니다. Notice 안에 또 Card 를 넣어도 됩니다. 화면 ㉔ 의 두 번째 흰 칸이 그렇게 만든 것입니다.

이렇게 하면 좋은 점이 있습니다. 나중에 알림 상자 모양을 바꾸고 싶으면 Notice 한 곳만 고치면 됩니다. 쓰는 쪽 열 군데를 전부 찾아다닐 필요가 없습니다.

이것이 컴포넌트를 나누는 이유이기도 합니다. 개념05에서 더 다룹니다.

▣ 직접 해보기 5

Notice 의 [알림] 을 [공지] 로 바꿔 보세요. 쓰는 쪽은 한 글자도 안 고쳐도 화면 ㉔ 가 전부 바뀝니다.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

{children} 을 안 씀 ★ 가장 자주 납니다

에러가 없습니다. 넘긴 내용이 통째로 사라집니다.

```
function ForgetChildren({ title, children }) {
  return (
    <div className="output"> <h4>{title}</h4> <p>여기에 children 이 나와야 하는데
    없습니다</p> </div>
  );
}
```

화면 ㉔ 의 첫 번째 칸입니다. 넘긴 <p>아메리카노 4000원</p> 이 안 보입니다. “분명히 넣었는데 화면에 없다” 면 받는 컴포넌트에 {children} 이 있는지 보세요.

이런 실수를 합니다

스스로 닫는 태그로 씀

<Card title="제목" /> 처럼 쓰면 태그 사이가 없으므로 children 도 없습니다.


```
function EmptyBox({ children }) {
  console.log("실수 2 - children:", String(children));
```

```
F12 콘솔    실수 2 - children: undefined
```

```
return <div className="output">[{children}]/>
```

화면 ㉔ 의 두 번째 칸은 대괄호 두 개만 나옵니다. `undefined` 는 화면에 아무것도 안 그린다고 개념02에서 배웠습니다.

이런 실수를 합니다

children.length 를 '개수' 로 생각함 ★ 조용히 틀립니다

```
function CountBox({ children }) {
  console.log("실수 3 - children.length:", children.length);
```

```
F12 콘솔    실수 3 - children.length: 5
```

```
return <div className="output">개수라고 생각한 값: {children.length}/>
```

`<CountBox>안녕하세요</CountBox>` 로 썼습니다. 넣은 것은 한 덩어리인데 5가 나옵니다. `children` 이 문자열이라 '글자 수' 가 나온 것입니다. (JS자료 01단원 `length`) 태그를 두 개 넣으면 배열이 되어 2가 나옵니다. 같은 코드인데 뜻이 달라집니다. 그래서 `children` 의 개수를 세는 코드는 되도록 쓰지 않습니다.

이런 실수를 합니다

children 을 함수처럼 부름

```
function Bad4({ children }) {
  return <div>{children()}</div>;
}
```

```
TypeError: children is not a function
```

이 납니다. `children` 은 화면 조각이지 함수가 아닙니다. 괄호 없이 `{children}` 으로 쓰세요.

이런 실수를 합니다

여는 태그와 닫는 태그 이름이 다름

```
<Card title="제목">
  <p>내용</p>
</Panel>
```

[SyntaxError] 입니다. 화면이 통째로 빡니다. 컴포넌트 이름을 바꿀 때 닫는 태그를 안 고쳐서 자주 납니다.

정리

- 1 `<Box>안녕</Box>` 처럼 **태그 사이에 넣은 것은** `props.children` 으로 옵니다.
- 2 `children` 은 특별한 문법이 아니라 **props 의 한 속성**입니다.
- 3 받은 `children` 을 화면 어디에 놓을지는 **받는 컴포넌트가** 정합니다.
- 4 `{children}` 을 안 쓰면 넘긴 내용이 **에러 없이** 사라집니다.
- 5 짧은 값은 속성으로, 여러 줄짜리 화면 덩어리는 `children` 으로 넘기면 읽기 좋습니다.
- 6 테두리·제목 같은 겉모습만 담당하는 **상자 컴포넌트**를 만들면 모양을 한 곳에서 고칠 수 있습니다.
- 7 `children.length` 는 개수가 아닐 수 있습니다. 글자 수일 수도 있습니다.

```
export default function Concept04() {
  return (
    <div> <h1>개념 04 - children</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      아래 회색 상자 ① ~ ⑥ 이 이 파일의 화면입니다. 흰 칸이 '상자 컴포넌트' 가
      그린 부분입니다.
      </p> <> <div className="demo"> <h3>① 태그 사이에 넣은 글자</h3> <Box>
      안녕하세요</Box> </div> <div className="demo"> <h3>② 제목은 속성, 본문은
      children</h3> <Card title="오늘의 커피"> <p>아메리카노 4000원
      </p> </Card> </div> <div className="demo"> <h3>③ 하나 넣기 / 여러 개 넣기
      </h3> <OneBox> <p>아메리카노 4000원</p> </OneBox> <ListBox> <p>아메리카노 4000원
      </p> <p>라떼 4500원</p> </ListBox> </div>
```

```

    <div className="demo">
      <h3>④ 속성과 children 을 함께</h3> <Panel title="영업 안내"> <p>오전
8시부터 오후 10시까지 문을 엽니다.</p> </Panel>
    </div>

    <div className="demo"> <h3>⑤ 상자 컴포넌트 - 겹모습은 한 곳에서
</h3> <Notice>케이크가 6000원입니다.</Notice> <Notice> <Card title="이번 주
신메뉴"> <p>삼각김밥 1200원
</p> </Card> </Notice> </div> <div className="demo"> <h3>⑥ 실수 1 · 2 ·
3</h3> <ForgetChildren title="children 을 안 그린 상자"> <p>아메리카노 4000원
</p> </ForgetChildren> <EmptyBox /> <CountBox>안녕하세요</CountBox> </div>
    </>

  </div>
);
}

```

직접 해보기 정답

1) <Box>오늘의 메뉴</Box>

```

// 콘솔: 오늘의 메뉴
// 콘솔: string
// 콘솔: ['children']
// 화면: 흰 칸 안에 "오늘의 메뉴"

```

→ 자료형과 키 목록은 그대로입니다. 값만 바꿉니다.

2) `function Card({ title, children }) {`

```

  return (
    <div className="output">
      {children}
      <h4>{title}</h4> <p>- 이 줄은 Card 가 항상 붙입니다 -</p> </div>
    );

```

```

  }
  // 화면: "아메리카노 4000원" 이 먼저 나오고 그 아래에 제목 "오늘의 커피" 가 나옵니다.

```

→ 넘기는 쪽은 한 글자도 안 고쳤습니다. 자리를 정하는 것은 Card 입니다.

3) <ListBox>

```

<p>아메리카노 4000원</p>
<p>라떼 4500원</p>
<p>케이크 6000원</p>

```

```

</ListBox>
// 화면: 흰 칸 안에 세 줄이 나옵니다.
// 콘솔: ListBox children 이 배열인가? true

```

→ 세 개여도 배열이라는 사실은 그대로입니다.

4) App 의 ④ 상자 안에 아래를 넣습니다.

```

<Panel title="공지">
  <p>내일은 쉽니다</p>
</Panel>

```

화면 "공지" 제목 아래에 "내일은 쉽니다" 가 나옵니다.

```

// 콘솔: Panel 이 받은 props 이름들: ['title', 'children'] (두 번 찍힙니다)

```

→ Panel 을 두 번 썼으니 콘솔에도 두 번 찍힙니다.

```

5) function Notice({ children }) {

```

```

  return (
    <div className="output"> <strong>[공지]</strong> {children}
    </div>
  );

```

```

}
// 화면: ⑤ 의 두 칸이 모두 [알림] 에서 [공지] 로 바뀝니다.

```

→ 쓰는 쪽 두 군데를 고치지 않았는데 둘 다 바뀌었습니다.

이것이 겉모습을 컴포넌트 하나에 모아 두는 이유입니다.

컴포넌트 쪼개기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

여기까지 컴포넌트를 만들고, props 를 넘기고, children 을 받아 봤습니다. 이제 남은 질문은 하나입니다.

"그래서 화면을 어디서 잘라야 하나요?"

정답은 없습니다. 다만 판단 기준은 있습니다. 그리고 쪼갤 때 꼭 지켜야 하는 규칙이 하나 있습니다. props 는 읽기 전용이라는 것입니다.

- 1 언제 쪼개나 — 기준 세 가지
- 2 반복되는 화면을 컴포넌트로 옮기기
- 3 props 는 읽기 전용이다 — 바꾸면 어떻게 되는지 직접 확인합니다
- 4 너무 잘게 쪼개면 오히려 나빠진다

안 쪼갠 코드부터 봅시다

섹션 1

아래는 메뉴판 화면 전체를 컴포넌트 하나에 다 넣은 것입니다. 돌아가기는 잘 돌아갑니다.

```
import Summary from "../_ui/Summary.jsx";

function BigMenuBoard() {
  return (
    <div className="output"> <h4>동네 카페 메뉴판</h4> <div> <strong>아메리카노</strong> - 4000원
      <br /> <small>가장 많이 팔립니다</small> </div> <div> <strong>라떼</strong> - 4500원
      <br /> <small>가장 많이 팔립니다</small> </div> <div> <strong>케이크</strong> - 6000원
      <br /> <small>가장 많이 팔립니다</small> </div> </div>
  );
}
```

이 코드의 문제는 '지금' 이 아니라 '나중' 에 나타납니다.

- 같은 모양이 세 번 반복됩니다. 메뉴가 열 개면 열 번이 됩니다. - <small> 문구를 바꾸려면 세 군데를 전부 고쳐야 합니다.

한 군데를 빠뜨리면 그 줄만 다른 문구가 됩니다. 실제로 아주 자주 일어납니다.

- 함수 하나가 길어져서 "이 화면이 무엇으로 이루어져 있는지" 가 안 보입니다.

쪼갤지 말지 정하는 기준 세 가지입니다.

기준 1 · 같은 모양이 두 번 이상 반복되는가

기준 2 · 그 덩어리에 이름을 붙일 수 있는가 ("메뉴 한 줄", "알림 상자")

기준 3 · 한 화면에 안 들어와서 위아래로 스크롤해야 읽히는가

하나라도 그렇다면 쪼갤 때가 된 것입니다. 세 기준 모두 JS자료 05단원 개념01에서 "왜 함수인가" 를 이야기할 때 나온 것과 같습니다.

⇒ 직접 해보기 1

BigMenuBoard 의 <small> 문구 세 개를 전부 "오늘의 추천" 으로 바꿔 보세요. 몇 군데를 고쳤나요?

반복되는 덩어리를 컴포넌트로

섹션 2

위에서 세 번 반복된 덩어리에 이름을 붙입니다. "메뉴 한 줄" 이니 MenuRow 로 하겠습니다. 달라지는 값 (이름·가격)은 props 로 받습니다.

```
function MenuRow({ name, price }) {
  return (
    <div> <strong>{name}</strong> - {price}원
    <br /> <small>가장 많이 팔립니다</small> </div>
  );
}

function MenuBoard() {
  return (
    <div className="output"> <h4>동네 카페 메뉴판
    </h4> <MenuRow name="아메리카노" price={4000} /> <MenuRow name="라떼" price={4500}
    /> <MenuRow name="케이크" price={6000} /> </div>
  );
}
```

화면 ① 과 ② 를 비교해 보세요. 똑같습니다. 달라진 것은 코드입니다.

- <small> 문구는 이제 한 곳에만 있습니다. 고칠 곳이 한 곳입니다. - MenuBoard 만 읽어도 “제목 하나에 메뉴 세 줄”이라는 구조가 보입니다. - 메뉴를 하나 더 넣으려면 한 줄만 추가하면 됩니다.

- 1 반복되는 덩어리를 찾는다
- 2 그 덩어리를 새 함수로 옮긴다
- 3 줄마다 달라지는 값을 props 로 뺀다
- 4 원래 자리에 <MenuRow ... /> 를 넣는다

▹ 직접 해보기

2

MenuBoard 에 삼각김밥 1200원 줄을 하나 더 넣어 보세요.

props 로 값만 바꿔 여러 개 만들기

섹션 3

같은 컴포넌트에 다른 값을 넣어 여러 개를 만드는 것이 컴포넌트의 핵심 쓸모입니다. 여기에 개념03의 기본값을 더하면 더 편해집니다.

```
function MenuCard({ name, price, note = "설명 없음" }) {
  return (
    <div className="output"> <strong>{name}</strong> - {price}원
    <br /> <small>{note}</small> </div>
  );
}
```

note 를 넘긴 카드와 안 넘긴 카드를 화면 ㉓ 에서 비교하세요.

지금은 카드를 손으로 네 번 썼습니다. 배열 하나로 여러 개를 자동으로 그리는 방법(map)은 05단원에서 배웁니다. 그때가 되면 이 네 줄이 한 줄로 줄어듭니다. 컴포넌트 쪽은 하나도 안 고칩니다.

▹ 직접 해보기

3

화면 ㉓ 에 <MenuCard name="물" price={0} /> 를 넣어 note 자리에 무엇이 나오는지 확인하세요.

props 는 읽기 전용입니다

섹션 4

★ 이 섹션이 개념05에서 가장 중요합니다.

컴포넌트를 쪼개면 값이 부모에서 자식으로 내려갑니다. 이때 자식은 받은 props 를 ‘읽기만’ 해야 합니다. 고치면 안 됩니다.

말로만 하면 와닿지 않으니 직접 해 봅시다.

```
function TryChangeProps(props) {
  let result = "";
```

JS자료 12단원 개념05에서 배운 try / catch 입니다. 에러가 나도 파일 전체가 멈추지 않게 감싼 것입니다.

```
try {
  props.name = "바꿈";
  result = "조용히 바뀌었습니다";
} catch (error) {
  result = error.message;
}

console.log("props 를 바꾸려 하면:", result);
```

F12 콘솔 props 를 바꾸려 하면: Cannot assign to read only property 'name' of object '#<Object>'

```
return (
  <div className="output">
    받은 이름: {props.name}
    <br /> <span className="error">{result}</span> </div>
  );
}
```

에러 메시지를 우리말로 옮기면 “읽기 전용 속성에는 대입할 수 없습니다” 입니다. React 는 개발 중에 props 객체를 아예 못 고치게 잠가 둡니다.

왜 이렇게까지 막아 둘까요? 이유는 두 가지입니다.

1. 부모는 자기가 준 값이 그대로 있을 것이라고 믿습니다.

자식이 몰래 바꾸면 화면에 보이는 값과 부모가 아는 값이 달라집니다.
그리고 어느 자식이 바꿨는지 찾을 방법이 없습니다.

2. 바뀌 봤자 화면은 다시 그려지지 않습니다.

값을 바꿔서 화면을 바꾸는 방법은 따로 있습니다. 04단원의 state 입니다.

그럼 값을 가공해서 쓰고 싶을 때는 어떻게 할까요? ‘읽는’ 것은 얼마든지 자유입니다. 읽어서 ‘새 값’ 을 만들면 됩니다.


```
function OrderLine({ name, price, count = 1 }) {
  const total = price * count; // 읽어서 새 변수를 만드는 것은 괜찮습니다 const label
  = `${name} ${count}개`;

  return (
    <div className="output">
      {label} - 합계 {total}원
    </div>
  );
}
```

props 를 고친 것이 아니라, props 를 읽어서 total 과 label 을 새로 만들었습니다. 이건 아무 문제가 없습니다. 앞으로 이 방식을 계속 씁니다.

⇒ 직접 해보기

4

OrderLine 에 count={3} 을 넘겨 합계가 어떻게 바뀌는지 보세요.

너무 잘게 쪼개지 마세요

섹션 5

쪼개는 게 좋다고 해서 무조건 잘게 나누면 오히려 읽기 어려워집니다. 아래는 지나치게 쪼갠 예입니다.

```
function NameText({ value }) {
  return <strong>{value}</strong>;
}

function PriceText({ value }) {
  return <span>{value}원</span>;
}

function TooSmallRow({ name, price }) {
  return (
    <div className="output"> <NameText value={name} /> - <PriceText value={price}
  /> </div>
  );
}
```

화면 ㉔ 는 앞의 메뉴 줄과 똑같이 나옵니다. 그런데 한 줄을 이해하려고 세 곳을 왔다 갔다 해야 합니다.

`<strong>{name}</strong>` 은 그 자리에 그냥 두는 편이 훨씬 잘 읽힙니다.

판단이 어려울 때 쓸 만한 기준입니다.

쪼갬다 — 두 번 이상 쓰인다 / 이름이 자연스럽게 붙는다 / 혼자서도 뜻이 통한다 안 쪼갬다 — 한 번만 쓰인다 / 이름이 “그냥 글자” 처럼 어색하다 / 한 줄짜리다

처음부터 완벽하게 나눌 필요는 없습니다. 한 덩어리로 쓰다가 두 번째로 같은 모양이 나올 때 쪼개도 늦지 않습니다.

참고로 지금은 컴포넌트를 전부 한 파일에 쓰고 있습니다. 파일을 나누는 것은 08단원에서 배웁니다.

▣ 직접 해보기 5

TooSmallRow 를 NameText·PriceText 없이 한 컴포넌트로 다시 쓰고 화면이 같은지 확인하세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

props 를 직접 고침

섹션 4에서 확인했습니다. TypeError 가 납니다. try / catch 로 감싸지 않으면 그 지점부터 화면이 안 그려집니다.

이런 실수를 합니다

props 로 받은 '객체 안의 값' 을 고침 ★ 이걸 에러도 안 납니다

```
const sharedUser = { name: "이서연", age: 22 };

function UserBadge({ user }) {
  return (
    <div className="output">
      {user.name} - {user.age}세
    </div>
  );
}

function SecretChanger({ user }) {
  user.age = 99; // props 객체는 잠겨 있지만, 그 '안' 의 객체는 안 잠겨
  있습니다 console.log("원본 객체의 나이:", sharedUser.age);
```

F12 콘솔 원본 객체의 나이: 99

```
return <div className="output">여기서 몰래 user.age 를 99로 바꿨습니다</div>;
}
```

화면 ㉔ 을 보세요. 같은 sharedUser 를 넘겼는데 위 배지는 22세, 아래 배지는 99세로 나옵니다. SecretChanger 가 그 사이에서 원본 객체를 고쳤기 때문입니다.

에러도 경고도 없습니다. 화면만 조용히 어긋납니다. 그리고 `sharedUser` 를 쓰는 다른 화면도 전부 함께 바뀝니다. `props` 로 받은 것은 안쪽까지 손대지 않는다고 생각하세요. 객체를 안 건드리고 바꾸는 방법은 07단원에서 제대로 배웁니다.

이런 실수를 합니다

쪼개 놓고 `props` 를 안 넘김

```
function ForgotProps({ name, price }) {
  return (
    <div className="output"> <strong>{name}</strong> - {price}원
    </div>
  );
}
```

화면 ㉠ 의 첫 줄입니다. `<ForgotProps />` 로만 썼습니다. “— 원” 만 남습니다. 에러는 없습니다. (개념02 섹션5) 쪼개 직후에 화면이 비면 이것부터 확인하세요.

이런 실수를 합니다

`return` 다음 줄에서 `JSX` 를 시작함 ★ 조용히 사라집니다

```
function ReturnNewLine() {
  return;
  <div className="output">이 줄은 화면에 안 나옵니다</div>;
}
```

화면 ㉠ 의 두 번째 자리입니다. 아무것도 안 나옵니다. 에러도 없습니다. `return` 뒤에서 줄을 바꾸면 자바스크립트가 거기서 문장을 끝내 버립니다. 02단원 개념05에서 배운 ASI 입니다. 그래서 여러 줄 `JSX` 는 소괄호로 감쌉니다.

```
return (
  <div>...</div>
);
```

이런 실수를 합니다

컴포넌트 이름에 하이픈을 씀

```
function Menu-Row() {
  return <p>메뉴</p>;
}
```

[SyntaxError] 입니다. 화면이 통째로 빡니다. HTML 태그 이름에는 하이픈을 쓰지만 함수 이름에는 못 씁니다. `MenuRow` 처럼 붙여 쓰고 단어마다 대문자로 시작하세요.

화면에 그리기

정리

- 1 같은 모양이 반복되거나, 이름을 붙일 수 있거나, 너무 길면 컴포넌트로 쪼갭니다.
- 2 쪼개면 **고칠 곳이 한 곳**이 됩니다. 이것이 쪼개는 가장 큰 이유입니다.
- 3 달라지는 값만 props 로 빼고, 같은 부분은 컴포넌트 안에 둡니다.
- 4 **props 는 읽기 전용입니다.** 자식에서 고치려고 하면 `TypeError: Cannot assign to read only property` 가 납니다.
- 5 props 로 받은 **객체 안의 값**은 에러 없이 바뀝니다. 원본까지 함께 바뀌므로 더 위험합니다. 손대지 마세요.
- 6 읽어서 새 변수를 만드는 것은 자유입니다. 값을 바꿔 화면을 바꾸는 것은 04단원입니다.
- 7 너무 잘게 쪼개면 오히려 읽기 어려워집니다. 두 번째로 반복될 때 쪼개도 늦지 않습니다.

```
export default function Concept05() {
  return (
    <div> <h1>개념 05 - 컴포넌트 쪼개기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일에는 <strong>일부러 잘못 만든 컴포넌트</strong>가 들어 있습니다. 화면
      ⑤ ~ ⑦ 이 그렇습니다. 무엇이 잘못됐는지 코드와 함께 확인하세요.
      </p> <> <div className="demo"> <h3>① 안 쪼개 메뉴판</h3> <BigMenuBoard
      /> </div> <div className="demo"> <h3>② 쪼개 메뉴판 - 화면은 ① 과 똑같습니다
      </h3> <MenuBoard /> </div> <div className="demo"> <h3>③ props 로 값만 바꿔 여러 개
      </h3> <MenuCard name="아메리카노" price={4000} note="가장 많이 팔립니다"
      /> <MenuCard name="라떼" price={4500} note="우유가 들어갑니다"
      /> <MenuCard name="케이크" price={6000} /> <MenuCard name="삼각김밥" price={1200}
      /> </div> <div className="demo"> <h3>④ 읽어서 새 값 만들기 / 너무 잘게 쪼개 예
      </h3> <OrderLine name="아메리카노" price={4000} /> <TooSmallRow name="라떼" price=
      {4500} /> </div>
```

```

    <div className="demo">
      <h3>⑤ props 를 바꾸려고 하면 (실수 1)</h3> <TryChangeProps name="김민준"
    />
    </div>

    <div className="demo"> <h3>⑥ props 안의 객체를 바꾸면 (실수 2)
  </h3> <UserBadge user={sharedUser} /> <SecretChanger user={sharedUser}
  /> <UserBadge user={sharedUser} /> </div> <div className="demo"> <h3>⑦ props 를 안
  넘김 / return 다음 줄 (실수 3 · 4)</h3> <ForgotProps /> <ReturnNewLine /> </div>
  </>

  </div>
);
}

```

직접 해보기 정답

1) 세 군데를 고쳐야 합니다. BigMenuBoard 안의 <small> 이 세 개이기 때문입니다.

```
// 화면: ① 의 세 줄이 모두 "오늘의 추천" 으로 바뀝니다.
```

→ 두 군데만 고치면 한 줄만 옛날 문구로 남습니다. 이런 사고가 실제로 자주 납니다.

```
② 의 MenuBoard 는 MenuRow 한 곳만 고치면 세 줄이 다 바뀝니다.
```

2) MenuBoard 안에 한 줄 넣습니다.

```
<MenuRow name="삼각김밥" price={1200} />
```

화면 메뉴판에 "삼각김밥 - 1200원 / 가장 많이 팔립니다" 가 늘어납니다.

→ <small> 문구는 MenuRow 가 가지고 있으므로 자동으로 따라옵니다.

```
3) <MenuCard name="물" price={0} />
// 화면: 물 - 0원 / 설명 없음
```

→ note 를 안 넘겼으므로 기본값이 쓰였습니다. (개념03 섹션3)

```
4) <OrderLine name="아메리카노" price={4000} count={3} />
// 화면: 아메리카노 3개 - 합계 12000원
```

→ props 를 고친 것이 아니라 읽어서 total 을 새로 만든 것입니다.

```
count 를 안 넘기면 기본값 1이 쓰여 "아메리카노 1개 - 합계 4000원" 이 됩니다.
```

```
5) function SmallRow({ name, price }) {
```

```
  return (  
    <div className="output"> <strong>{name}</strong> - <span>{price}원</span> </div>  
  );
```

```
}  
// 화면: 라떼 - 4500원 (TooSmallRow 와 완전히 같습니다)
```

→ 화면이 같다면 굳이 컴포넌트 두 개를 더 만들 이유가 없습니다.

컴포넌트와 props

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

고치고 저장하면 화면이 바로 바뀝니다. F12 → Console 도 함께.

문제 1 (개념01 컴포넌트 만들기)

<p>안녕하세요, 김민준입니다</p> 를 돌려주는 Hello 컴포넌트를 만드세요. 만든 다음 아래 App 의 '문제 1' 흰 칸에 <Hello /> 를 넣으세요.

기대 화면 문제 1 칸에 "안녕하세요, 김민준입니다" 가 나옵니다.
아무것도 안 나오는데 에러도 없다면 컴포넌트 이름이 소문자로 시작하는지 확인하세요.

여기에 코드를 쓰세요

문제 2 (개념01 대문자 규칙)

아래 header 컴포넌트는 App 의 '문제 2' 칸에 이미 들어 있습니다. 그런데 화면에 아무것도 안 나옵니다. 이유를 생각하고 고치세요. 만드는 쪽과 쓰는 쪽을 둘 다 고쳐야 합니다.

기대 화면 문제 2 칸에 "동네 카페" 라는 제목이 나옵니다.
이름만 고치고 App 의 태그를 안 고치면 여전히 안 나옵니다.

```
function header() {
  return <h4>동네 카페</h4>;
}
```

문제 3 (개념01 컴포넌트 조립)

아래 Coffee 와 Cake 를 둘 다 담은 MenuList 컴포넌트를 만드세요. App 의 '문제 3' 칸에는 <MenuList /> 하나만 넣습니다.

기대 화면 문제 3 칸에 "아메리카노" 와 "케이크" 두 줄이 차례로 나옵니다.
한 줄만 나오면 MenuList 안에 하나만 넣은 것입니다.
화면이 통째로 비면 두 덩어리를 하나로 안 감싼 것입니다.

```
function Coffee() {  
  return <p>아메리카노</p>;  
}  
  
function Cake() {  
  return <p>케이크</p>;  
}
```

여기에 MenuList 를 만드세요

문제 4 (개념02 props 받기)

name 을 props 로 받아 "OOO님 환영합니다" 를 보여 주는 Welcome 컴포넌트를 만드세요. App 의 '문제 4' 칸에 <Welcome name="이서연" /> 을 넣으세요.

기대 화면 이서연님 환영합니다
"님 환영합니다" 만 나오면 넘기는 이름과 받는 이름이 다른 것입니다.

여기에 코드를 쓰세요

문제 5 (개념02 숫자 props)

price 를 props 로 받아 두 줄을 보여 주는 PriceTag 컴포넌트를 만드세요. 첫 줄: 4000원 둘째 줄: 500원 더하면 4500원 App 의 '문제 5' 칸에 <PriceTag price={4000} /> 을 넣으세요.

기대 화면 4000원
500원 더하면 4500원
둘째 줄이 "4000500원" 이면 price 를 따옴표로 넘긴 것입니다.

여기에 코드를 쓰세요

문제 6 (개념02 같은 컴포넌트 여러 번)

name 과 price 를 받아 “아메리카노 — 4000원” 처럼 보여 주는 MenuItem 을 만드세요. App 의 ‘문제 6’ 칸에 아래 세 줄을 손으로 넣으세요. 아메리카노 4000 / 라떼 4500 / 케이크 6000

기대 화면 아메리카노 - 4000원
 라떼 - 4500원
 케이크 - 6000원
 세 줄이 전부 같은 내용이면 props 를 안 넘긴 것입니다.

여기에 코드를 쓰세요

문제 7 (개념03 구조분해)

문제 6에서 만든 MenuItem 을 매개변수 자리에서 구조분해하는 방식으로 고치세요. props. 라는 글자가 한 번도 안 나와야 합니다.

기대 화면 문제 6과 화면이 하나도 안 달라져야 합니다.
 ReferenceError: props is not defined 가 나면
 구조분해해 놓고 아래에서 props. 를 계속 쓴 것입니다.

문제 6의 MenuItem 을 직접 고치세요

문제 8 (개념03 기본값)

name 을 받아 "OOO님, 어서 오세요." 를 보여 주는 Greeting 컴포넌트를 만드세요. name 을 안 넘기면 "손님" 이 쓰이게 기본값을 주세요. App 의 '문제 8' 칸에 <Greeting name="박지훈" /> 과 <Greeting /> 을 둘 다 넣으세요.

기대 화면 박지훈님, 어서 오세요.
 손님님, 어서 오세요.
 둘째 줄이 "님, 어서 오세요." 면 기본값을 안 준 것입니다.

여기에 코드를 쓰세요

문제 9 (개념04 children)

태그 사이에 넣은 것을 흰 칸에 그려 주는 Box 컴포넌트를 만드세요. 흰 칸은 <div className="output"> 입니다. App 의 '문제 9' 칸에 <Box>오늘도 좋은 하루</Box> 를 넣으세요.

기대 화면 테두리가 있는 흰 칸 안에 "오늘도 좋은 하루" 가 나옵니다.
 빈 칸만 나오면 Box 안에서 {children} 을 안 그린 것입니다.

여기에 코드를 쓰세요

문제 10 (개념04 children + 속성)

title 은 속성으로, 본문은 children 으로 받는 TitleBox 컴포넌트를 만드세요. 제목은 <h4> 로, 본문은 그 아래에 그림니다. App 의 '문제 10' 칸에 아래처럼 넣으세요.

```
<TitleBox title="오늘의 메뉴">
  <p>아메리카노 4000원</p>
</TitleBox>
```

기대 화면 "오늘의 메뉴" 제목 아래에 "아메리카노 4000원" 이 나옵니다.
제목만 나오면 {children} 을 안 그린 것입니다.

여기에 코드를 쓰세요

문제 11 (개념05 쪼개기)

아래 BigNotice 는 같은 모양이 세 번 반복됩니다. 반복되는 한 줄을 NoticeLine 컴포넌트로 쪼개고, BigNotice 안을 <NoticeLine ... /> 세 줄로 바꾸세요. 달라지는 값은 글자 하나뿐입니다. 그것만 props 로 빼세요.

기대 화면 화면이 하나도 안 바뀌어야 합니다.

알림 · 표시가 사라졌다면 NoticeLine 안에 넣지 않은 것입니다.

```
function BigNotice() {
  return (
    <div className="output"> <div> <strong>[알림]</strong> 오전 8시에 문을 엽니다
    </div> <div> <strong>[알림]</strong> 오후 10시에 문을 닫습니다
    </div> <div> <strong>[알림]</strong> 케이크는 6000원입니다
    </div> </div>
  );
}
```

NoticeLine 을 만들고 BigNotice 를 고치세요

문제 12 [응용] (개념03 · 05)

주문 한 줄을 그리는 OrderCard 컴포넌트를 만드세요. - item 이라는 props 로 { name, price } 모양의 객체를 통째로 받습니다 - count 도 받습니다. 안 넘기면 1 이 되게 기본값을 주세요 - 화면에는 "아메리카노 3개 — 합계 12000원" 처럼 보여 줍니다 아래 두 객체를 그대로 쓰세요.

기대 화면 아메리카노 3개 - 합계 12000원
 라떼 1개 - 합계 4500원
 합계가 4003원이면 곱하기가 아니라 더하기를 쓴 것입니다.
 count 를 안 넘긴 줄이 "라떼 개 - 합계 원" 이면 기본값을 안 준 것입니다.

```
const americano = { name: "아메리카노", price: 4000 };
const latte = { name: "라떼", price: 4500 };
```

여기에 코드를 쓰세요

App 의 '문제 12' 칸에 아래 두 줄을 넣으세요.

```
<OrderCard item={americano} count={3} />
<OrderCard item={latte} />
```

문제 13 [도전] (개념04 · 05)

영수증 상자 Receipt 컴포넌트를 만드세요. - title 을 속성으로 받아 <h4> 로 그립니다 - 주문 줄들은 children 으로 받아 제목 아래에 그립니다 - total 을 속성으로 받아 맨 아래에 "합계 16500원" 을 그립니다 - total 을 안 넘기면 0 이 되게 기본값을 주세요 그리고 App 의 '문제 13' 칸에서 Receipt 안에 문제 12의 OrderCard 두 개를 넣으세요. 합계 금액은 직접 계산해서 숫자로 넘기면 됩니다.

기대 화면 "영수증" 제목
 아메리카노 3개 - 합계 12000원
 라떼 1개 - 합계 4500원
 합계 16500원
 주문 두 줄이 안 보이면 Receipt 안에서 {children} 을 안 그린 것입니다.
 합계가 0원이면 total 을 안 넘긴 것입니다.

여기에 코드를 쓰세요

문제 14 (에러 확인 - 맨 마지막)

아래 두 줄의 주석을 풀고 저장한 뒤 화면과 F12 → Console 을 보세요. (App 의 '문제 14' 칸에 있는 `{/* <ChangeProps name="김민준" /> */}` 도 함께 푸세요)

```
function ChangeProps(props) {
  props.name = "바꿈";
  return <p>{props.name}</p>;
}
```

기대 출력 이 문제는 먼저 예상해 보고 나서 확인하는 문제입니다.
 "문제 14 칸만 비겠지" 라고 예상했다면, 실제로 어떻게 되는지 보세요.
 힌트 - 개념05 섹션4에서 본 것과 같은 일이 일어납니다.

화면이 어떻게 되나요? 콘솔에는 무슨 에러가 찍히나요?

■ 답

확인했으면 다시 주석으로 돌려놓으세요.

화면에 그리기 — 문제에서 만든 컴포넌트를 여기에 넣으세요

```

export default function Exercises() {
  return (
    <div>
      <h1>03단원 연습문제 - 컴포넌트와 props</h1>

      <p className="guide">
        이 파일의 TODO 자리에 코드를 쓰고 저장하면 화면이 바로 바뀝니다. 브라우저를
        <strong>새로고침</strong>해서 확인하세요. <strong>F12 → Console</strong> 도 함께
        보세요.<br /> 1~11은 기본, 12는 [응용], 13은 [도전], 14는 예러 확인입니다.
        <br />
        <strong>푸는 순서를 지키세요.</strong> 컴포넌트를 만들기 전에 아래 App 안에
        먼저 <code>&lt;Hello /&gt;</code> 를 써 넣으면 화면이 통째로 비어 버립니다. 없는
        이름을 쓴 것이기 때문입니다. <strong>먼저 만들고, 그다음에 App 에 넣으세요.</strong>
        <br />
        <br />
        이 단원에서는 <strong>화면이 바뀌지 않습니다.</strong> 버튼을 눌러 값을
        바꾸는 것은 04단원입니다.
      </p>

      <>
        <div className="demo">
          <h3>문제 1</h3>
          <div className="output">{/* TODO 문제 1: <Hello /> */}</div>
        </div>

        <div className="demo">
          <h3>문제 2</h3>
          <div className="output">
            <header />
          </div>
        </div>

        <div className="demo">
          <h3>문제 3</h3>
          <div className="output">{/* TODO 문제 3: <MenuList /> */}</div>
        </div>

        <div className="demo">
          <h3>문제 4</h3>
          <div className="output">{/* TODO 문제 4: <Welcome name="이서연" /> */}
        </div>
      </>
    </div>
  )
}

```

```

    </div>

    <div className="demo"> <h3>문제 5</h3> <div className="output">{/* TODO 문제
5: <PriceTag price={4000} /> */</div> </div> <div className="demo"> <h3>문제 6 .
7</h3> <div className="output">{/* TODO 문제 6: MenuItem 세 줄 */</div>
</div> </div> <div className="demo"> <h3>문제 8</h3> <div className="output">{/* TODO
문제 8: Greeting 두 줄 */</div> </div> <div className="demo"> <h3>문제 9</h3>
    { /* TODO 문제 9: <Box>오늘도 좋은 하루</Box> */</div>
    </div> <div className="demo"> <h3>문제 10</h3>
    { /* TODO 문제 10: TitleBox */</div>
    </div> <div className="demo"> <h3>문제 11</h3> <BigNotice
/> </div> <div className="demo"> <h3>문제 12 [응용]</h3>
    { /* TODO 문제 12: OrderCard 두 줄 */</div>
    </div>

    <div className="demo">
      <h3>문제 13 [도전]</h3>
      { /* TODO 문제 13: Receipt 와 그 안의 OrderCard 두 개 */</div>
    </div>

    <div className="demo"> <h3>문제 14 (에러 확인)</h3> <div className="output">
    { /* <ChangeProps name="김민준" /> */</div> </div>
    </>
  </div>
);
}

```

03단원 마무리 컴포넌트와 props

자주 넘어지는 자리

- 1 props.name = "x" → **TypeError** 로 화면 전체가 빔 / 그런데 props.user.age = 99 는 에러 없이
원본까지 바뀜 | 둘의 차이가 핵심
- 2 return 을 빼먹으면 에러 없이 그 컴포넌트만 조용히 사라짐 |

자주 나오는 질문

Q "컴포넌트를 언제 쪼개요?"

A 같은 화면이 두 번 나올 때, 또는 한 함수가 화면 두 가지 일을 할 때

state와 이벤트

OPEN 04_state와_이벤트

04

```
function Greeting() {  
  function handleClick() {  
    console.log("안녕하세요, 김민준님!");  
  }  
  return (  

```

01 이벤트 붙이기

02 useState 시작

03 state가 바뀌면 다시 그린다

04 여러 state 다루기

05 이전 값으로 갱신하기

06 state 규칙

- 연습문제

이벤트 붙이기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

01~03단원의 화면은 전부 '고정' 이었습니다. props 로 값을 바꿔 넣을 수는 있었지만, 화면을 열고 난 뒤에는 사용자가 무엇을 해도 아무 일이 없었습니다.

이제 사용자의 행동에 반응하는 화면을 만듭니다. 그 첫 단계가 '이벤트를 붙이는 일' 입니다.

JS자료 11단원에서는 이렇게 했습니다.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", handleClick);
```

① 요소를 찾아오고 ② 거기에 이벤트를 붙였습니다.

React 에서는 ①이 사라집니다. 요소를 찾아올 필요가 없습니다. 화면을 그리는 그 자리에서 바로 붙입니다.

```
<button onClick={handleClick}>눌러 보세요</button>
```

달라진 것은 '적는 위치' 와 '이름' 뿐입니다. 넘기는 것이 함수라는 점, 괄호를 붙이면 안 된다는 점은 똑같습니다. 그 '괄호' 이야기가 이 파일의 절반을 차지합니다. 그만큼 자주 틀립니다.

```
import Summary from "../_ui/Summary.jsx";

console.log("페이지를 열었습니다. 버튼을 눌러야 나머지가 나옵니다.");
```

F12 콘솔 페이지를 열었습니다. 버튼을 눌러야 나머지가 나옵니다.

React 에서 이벤트 붙이기

섹션 1

규칙은 세 가지입니다.

- (1) 이벤트를 붙일 태그 안에 속성처럼 적는다 (2) 이름은 on 으로 시작하고, 뒤 단어는 대문자로 (onClick)
- (3) 값은 중괄호 { } 안에 '함수' 를 넣는다
- (3)의 중괄호는 02단원에서 배운 그 중괄호입니다. JSX 안에서 자바스크립트 값을 쓰려면 { } 로 감싸야 했습니다. 함수도 값이므로 똑같이 { } 안에 넣습니다.

```
function Greeting() {
```

이벤트가 일어났을 때 실행할 함수를 컴포넌트 ‘안’에 만듭니다. 이름은 보통 handle 로 시작하게 짓습니다. handleClick, handleSave ... 규칙은 아니지만 대부분 이렇게 써서, 읽는 사람이 바로 알아봅니다.

```
function handleClick() {
  console.log("안녕하세요, 김민준님!");
```

F12 콘솔 안녕하세요, 김민준님!

```
}

return (
  <div className="demo"> <h3>① 가장 기본 - onClick={"{ }handleClick{"{ }"}
</h3> <button id="btnGreet" onClick={handleClick}>
  인사하기
</button> </div>
);
}
```

위 h3 에 있는 {"{ }"} 는 신경 쓰지 마세요. 화면에 중괄호 글자 자체를 보여 주려고 쓴 것입니다. (02단원 개념02)

이 handleClick 은 ‘지금’ 실행되지 않습니다. “누르면 이걸 해라” 라고 React 에게 맡겨 둔 것입니다. JS자료 11단원에서 addEventListener 에 콜백을 넘긴 것과 같은 뜻입니다.

함수를 컴포넌트 안에 만드는 이유가 있습니다. 곧 배울 state 값을 이 함수 안에서 써야 하기 때문입니다. 지금은 “핸들러는 컴포넌트 안에 만든다” 로 기억해 두세요.

▫ 직접 해보기 1

handleClick 이 찍는 이름을 “이서연” 으로 바꿔 보세요. 저장한 뒤 버튼을 눌러 확인합니다.

괄호를 붙이면 안 됩니다

섹션 2

여기가 이 파일에서 가장 중요한 곳입니다. JS자료 11단원 개념01 섹션 2에서 배운 그 실수가 React 에서 또 나옵니다.

```
handleClick    ← 함수 그 자체.            "이 종이를 받아 두세요"
handleClick() ← 함수를 실행한 결과.    "지금 요리해서 접시를 주세요"
```

비유는 여기까지입니다. 실제로 무슨 일이 일어나는지 봅시다. 괄호를 붙이면 React 가 화면을 그리는 그 순간에 함수가 실행됩니다. 그리고 함수가 돌려준 값(여기서는 undefined)이 onClick 에 들어갑니다.

undefined 는 함수가 아니므로, 정작 눌렀을 때는 아무 일도 없습니다.

```
function shout() {
  console.log("[실수] shout 가 지금 실행되었습니다. 아무도 안 눌렀는데요!");
}
```

F12 콘솔 [실수] shout 가 지금 실행되었습니다. 아무도 안 눌렀는데요!

```
}

function BadButton() {
```

↓ 괄호를 붙였습니다. 일부러 틀리게 쓴 것입니다.

```
return (
  <div className="demo"> <h3>② 괄호를 붙인 버튼 - 눌러도 아무 일이 없습니다
</h3> <button id="btnBad" onClick={shout()}>
  괄호 붙인 버튼
</button> </div>
);
}
```

위 컴포넌트가 화면에 그려지는 순간 콘솔에 한 줄이 이미 찍혔습니다. 아직 버튼을 누르기 전인데도 말입니다. 왼쪽에서 다른 예제를 골랐다 돌아오면 처음부터 다시 그립니다.

그리고 버튼을 눌러 보세요. 아무 일도 안 일어납니다. 에러도 안 납니다. 빨간 글씨도 없습니다. 그래서 더 찾기 어렵습니다. “버튼이 안 눌러요” 라고 할 때 옆에 아홉은 이 실수입니다.

```
function GoodButton() {
  return (
    <div className="demo"> <h3>③ 괄호를 댄 버튼 - 이게 맞습니다
</h3> <button id="btnGood" onClick={shout}>
    괄호 없는 버튼
</button> </div>
  );
}
```

이쪽은 누를 때마다 콘솔에 찍힙니다. 같은 shout 함수인데 괄호 하나로 동작이 완전히 달라집니다.

정리하면 이렇습니다.

onClick={shout}	그릴 때: 아무 일 없음 / 누르면: 실행됨	← 맞음
onClick={shout()}	그릴 때: 실행됨 / 누르면: 아무 일 없음	← 틀림

왜 이런지 한 번 더 짚습니다. React 는 onClick 에 들어온 값을 ‘나중에 부를 것’ 으로 보관합니다. 그러려면 값이 함수여야 합니다. shout() 는 함수가 아니라 실행 결과입니다.

▣ 직접 해보기 2

GoodButton 의 onClick 에도 괄호를 붙였다 때 보세요. 붙인 채로 왼쪽에서 다른 예제를 골랐다 돌아오면 콘솔에 언제 찍히는지 보세요. (확인했으면 반드시 다시 떼세요)

인자를 넘기려면 화살표로 감싼다

섹션 3

그럼 함수에 값을 넘기고 싶을 때는 어떻게 할까요? greetPerson("김민준") 처럼 쓰려면 괄호가 꼭 필요합니다. 그런데 괄호를 쓰면 바로 실행되어 버립니다. 막힌 것 같습니다.

해결은 JS자료 11단원과 똑같습니다. 화살표 함수로 한 겹 감쌉니다.

```
onClick={() => greetPerson("김민준")}
```

^^^^^^ 이 화살표 함수가 onClick 에 들어갑니다

onClick 이 받는 것은 화살표 함수입니다. 이건 함수가 맞습니다. greetPerson("김민준") 은 그 화살표 함수 '안' 에 적혀 있으므로, 눌렀을 때 화살표 함수가 실행되면서 그때 같이 실행됩니다.

```
function greetPerson(name) {
  console.log(`${name}님 안녕하세요`);
}
```

F12 콘솔 김민준님 안녕하세요
 이서연님 안녕하세요

```
}

function ArgButtons() {
  return (
    <div className="demo"> <h3>④ 인자 넘기기 - 화살표로 감싸기
  </h3> <button id="btnMinjun" onClick={() => greetPerson("김민준")}>
    김민준에게 인사
  </button> <button id="btnSeoyeon" onClick={() => greetPerson("이서연")}>
    이서연에게 인사
  </button> </div>
  );
}
```

버튼은 두 개인데 함수는 하나입니다. 넘기는 값만 다릅니다. JS자료 11단원에서는 이럴 때 data-price 같은 속성에 값을 숨겨 두고 e.target.dataset 으로 꺼냈습니다. React 에서는 그럴 필요가 없습니다. 화면을 그리는 자리에서 값을 그대로 적어 넘기면 됩니다.

다만 화살표를 빼먹으면 안 됩니다.

```
onClick={greetPerson("김민준")}     ← 그럴 때 실행되고 끝. 틀림
onClick={() => greetPerson("김민준")} ← 맞음
```

“박지훈에게 인사” 버튼을 하나 더 만들어 보세요. ArgButtons 안에 button 을 한 줄 더 적으면 됩니다.

이벤트 객체 e 받기

섹션 4

JS자료 11단원 개념02에서 콜백의 첫 매개변수로 이벤트 객체를 받았습니다. React 도 똑같이 줍니다. 이름은 보통 e 나 event 로 짓습니다.

```
function handleClick(e) { ... }
```

e 안에는 “무엇이 눌렸는지” 같은 정보가 들어 있습니다. 자주 쓰는 것은 이 정도입니다.

```
e.target      이벤트가 일어난 요소 (누른 그 버튼)
e.type        이벤트 이름 ("click")
e.preventDefault() 기본 동작 막기 (06단원 폼에서 씁니다)
```

```
function EventObjectButton() {
  function handleClick(e) {
    console.log("이벤트 이름:", e.type);
  }
}
```

```
F12 콘솔    이벤트 이름: click
```

```
console.log("눌린 요소의 글자:", e.target.textContent);
```

```
F12 콘솔    눌린 요소의 글자: 나를 눌러 보세요
```

```
console.log("눌린 요소의 태그:", e.target.tagName);
```

```
F12 콘솔    눌린 요소의 태그: BUTTON
```

```
  }

  return (
    <div className="demo"> <h3>⑤ 이벤트 객체 – 무엇이 눌렸는지 알려 줍니다
    </h3> <button id="btnEvent" onClick={handleClick}>
      나를 눌러 보세요
    </button> </div>
  );
}
```

tagName 이 대문자 BUTTON 으로 나오는 것도 JS자료 11단원과 같습니다.

화살표로 감싼 경우에도 e 를 받을 수 있습니다. 이렇게 적습니다.

```
onClick={e => handleClick(e, "김민준")}
```

다만 04단원에서는 e 를 거의 쓰지 않습니다. 값을 화면에 보여 주는 일은 곧 배울 state 가 대신해 주기 때문입니다. e 가 본격적으로 필요해지는 곳은 입력칸을 다루는 06단원입니다.

▫ 직접 해보기 4

handleClick 안에서 e.target.id 도 찍어 보세요. 버튼에 적어 둔 id 가 그대로 나옵니다.

한 버튼에 여러 가지 일 시키기

섹션 5

JS자료 11단원에서는 addEventListener 를 여러 번 붙일 수 있었습니다. 붙인 순서대로 전부 실행됐습니다.

React 의 onClick 은 그렇지 않습니다. '속성' 이라서 하나만 들어갑니다. 같은 태그에 onClick 을 두 번 적으면 뒤에 적은 것만 남습니다. 예전 방식인 btn.onclick = 함수 와 같은 성질입니다.

```
<button onClick={jobA} onClick={jobB}> ← jobA 는 사라집니다
```

그래서 두 가지 일을 시키려면 함수 하나 안에서 둘 다 부릅니다.

```
function TwoJobsButton() {
  function takeOrder() {
    console.log("[1] 주문을 받았습니다");
  }
}
```

F12 콘솔 [1] 주문을 받았습니다

```

}

function printReceipt() {
  console.log("[2] 영수증을 찍었습니다");
}
```

F12 콘솔 [2] 영수증을 찍었습니다

```

}

function handleClick() {
  takeOrder();
  printReceipt();
}

return (
  <div className="demo"> <h3>⑥ 한 번 누르면 두 가지 일
</h3> <button id="btnTwoJobs" onClick={handleClick}>
```

```

        주문 + 영수증
      </button> </div>
    );
  }

```

이렇게 하면 순서도 눈에 보이고, 나중에 하나만 빼기도 쉽습니다.

▹ 직접 해보기

5

handleClick 안의 두 줄 순서를 바꿔 보세요. 콘솔에 찍히는 순서도 따라 바뀝니다.

React 가 알아듣는 이벤트 이름은 아주 많습니다. 자주 쓰는 것만 봅시다.

onClick 눌렀을 때 ← 이 파일에서 씁니다 onDoubleClick 두 번 눌렀을 때

onMouseOver	마우스를 올렸을 때	
onMouseOut	마우스가 벗어났을 때	
onChange	입력칸의 값이 바뀔 때	(06단원)
onSubmit	폼을 제출할 때	(06단원)
onKeyDown	키를 눌렀을 때	

JS자료에서 쓴 문자열 이름에 on 을 붙이고 첫 글자를 대문자로 바꾼 모양입니다.

```

addEventListener("click", ...) → onClick={...}
addEventListener("mouseover", ...) → onMouseOver={...}

```

onChange 와 onSubmit 은 이름만 알아 두세요. 입력칸과 폼이 필요해서 06단원에서 제대로 다룹니다.

```

function HoverButton() {
  function handleClick() {
    console.log("[onClick] 눌렀습니다");
  }

```

F12 콘솔 [onClick] 눌렀습니다

```

  }

  function handleMouseOver() {
    console.log("[onMouseOver] 마우스가 올라왔습니다");
  }

```

↑ 이 줄은 '마우스를 올렸을 때' 만 나옵니다. 누르는 것과 다릅니다. 그래서 여기에는 // 콘솔: 을 적지 않았습니다.


```

    }

    return (
      <div className="demo"> <h3>㉗ 이벤트 이름만 바꾸면 됩니다
</h3> <button id="btnHover" onClick={handleClick} onMouseOver={handleMouseOver}>
      누르거나 마우스를 올려 보세요
    </button> </div>
    );
  }
}

```

이름이 다르면 서로 다른 속성이므로 한 태그에 함께 붙일 수 있습니다. 위 버튼은 onClick 과 onMouseOver 를 둘 다 갖고 있습니다.

▸ 직접 해보기

6

HoverButton 에 onMouseOut 을 붙여 마우스가 벗어날 때도 콘솔에 찍히게 해 보세요.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

함수에 괄호를 붙였다 (섹션 2)

```
<button onClick={handleClick()}>
```

에러가 안 납니다. 그럴 때 한 번 실행되고, 눌러도 반응이 없습니다. "버튼이 안 먹어요" 의 1순위 원인입니다. 위 데모 ㉔ 에서 직접 봤습니다.

이런 실수를 합니다

화살표 안에서 괄호를 빠뜨렸다

```
<button onClick={() => greetPerson}>
```

이것도 에러가 안 납니다. 그런데 눌러도 아무 일이 없습니다. 화살표 함수가 greetPerson 을 '돌려주기만' 하고 부르지는 않기 때문입니다. 아래 데모 ㉕ 에서 실제로 확인할 수 있습니다.

```

function MistakeDemo() {
  return (
    <div className="demo"> <h3>㉕ [실수 2] 눌러도 아무 일이 없는 버튼
</h3> <button id="btnMistake" onClick={() => greetPerson}>
      눌러도 조용합니다
    </button> <button id="btnMistakeFixed" onClick={() => greetPerson("박지훈")}>

```

```

이렇게 고칩니다
    </button> </div>
  );
}

```

두 버튼을 번갈아 눌러 보세요. 왼쪽은 조용하고 오른쪽만 짹힙니다. 오른쪽 버튼은 이렇게 짹힙니다.
박지훈님 안녕하세요

이런 실수를 합니다

이름을 소문자로 썼다

```
<button-onclick={handleClick}>
```

React 가 콘솔에 노란 경고를 냅니다. Invalid event handler property `onclick`. Did you mean `onClick`? 경고만 나오고 이벤트는 불지 않습니다. 눌러도 반응이 없습니다. JSX 의 이벤트 이름은 반드시 `onClick` 처럼 중간이 대문자입니다.

이런 실수를 합니다

중괄호 대신 따옴표로 감쌌다

```
<button-onClick="handleClick">
```

이것도 노란 경고가 납니다. Expected `onClick` listener to be a `function`, instead got a value of `string` type. `onClick` 에는 함수가 들어가야 하는데 "handleClick" 은 그냥 글자입니다. 중괄호 { } 를 빼먹은 경우입니다.

이런 실수를 합니다

중괄호를 안 닫았다

```
<button-onClick={handleClick}>눌러 보세요</button>
```

[SyntaxError] 입니다. 파일 전체가 안 돌아가고 화면이 통째로 빡니다. 괄호·중괄호가 짝이 맞는지부터 세어 보세요.

이런 실수를 합니다

React 안에서 `addEventListener` 를 쓰려고 한다

```
document.querySelector("#btnGreet").addEventListener("click", ...)
```

지금은 동작할 수도 있습니다. 하지만 React 가 화면을 다시 그리면서 그 요소를 바꿔 버리면 조용히 사라집니다. React 안에서는 `onClick` 을 씁니다. 요소를 직접 만져야 하는 드문 경우는 10단원 `useRef` 에서 다룹니다.

마지막: 위에서 만든 컴포넌트를 한 화면에 모으기

03단원에서 배운 대로, 컴포넌트 안에 컴포넌트를 넣어 조립합니다.

정리

- 1 React에서는 요소를 찾아올 필요 없이 태그에 바로 **onClick={함수}**를 적습니다.
- 2 이벤트 이름은 **on + 대문자**입니다. onclick 이 아니라 onClick.
- 3 **괄호를 붙이지 않습니다.** 붙이면 그럴 때 실행되고, 눌렀을 때는 아무 일도 안 일어납니다. 에러도 안 납니다.
- 4 값을 넘겨야 하면 **() ⇒ 함수(값)** 처럼 화살표 함수로 감쌉니다.
- 5 핸들러의 첫 매개변수로 이벤트 객체가 옵니다. e.target 으로 눌린 요소를 봅니다.
- 6 한 태그에 같은 이벤트를 두 번 붙일 수 없습니다. 두 가지 일은 함수 하나 안에서 차례로 부릅니다.
- 7 아직 화면은 바뀌지 않습니다. 콘솔만 찍힙니다. 화면을 바꾸는 방법은 개념02에서 배웁니다.

```
export default function Concept01() {
  return (
    <div> <h1>개념 01 - 이벤트 붙이기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br /> <strong>이번 단원부터는 버튼을 직접 눌러야 결과가 나옵니다.
      </strong> 페이지를 여는 것만으로는 대부분 아무 일도 일어나지 않습니다. JS자료
      11단원과 같습니다.
      </p> <div> <Greeting /> <BadButton /> <GoodButton /> <ArgButtons
      /> <EventObjectButton /> <TwoJobsButton /> <HoverButton /> <MistakeDemo
      /> </div> </div>
  );
}
```

직접 해보기 정답

```
1) function handleClick() {
  console.log("안녕하세요, 이서연님!");
}
// 콘솔: 안녕하세요, 이서연님!
```

→ 버튼을 눌렀을 때만 나옵니다. 페이지를 열 때는 안 나옵니다.

```
2) <button id="btnGood" onClick={shout()}>
```

→ 이 예제를 고르는 순간, 즉 아직 누르지 않았을 때 콘솔에 찍힙니다.

```
// 콘솔: [실수] shout 가 지금 실행되었습니다. 아무도 안 눌렀는데요!
```

→ 그리고 버튼을 눌러도 더 이상 찍히지 않습니다.

onClick 에 undefined 가 들어가 이벤트가 아예 안 붙었기 때문입니다.

```
3) <button onClick={() => greetPerson("박지훈")}>박지훈에게 인사</button>  
// 콘솔: 박지훈님 안녕하세요
```

→ 화살표를 빼고 onClick={greetPerson("박지훈")} 로 쓰면

페이지를 열 때 한 번 찍히고 눌러도 조용해집니다.

```
4) function handleClick(e) {
```

```
  console.log("눌린 요소의 id:", e.target.id);
```

```
  }  
  // 콘솔: 눌린 요소의 id: btnEvent
```

→ id 를 안 적어 둔 요소라면 빈 문자열이 나옵니다.

```
5) function handleClick() {
```

```
  printReceipt();  
  takeOrder();
```

```
  }  
  // 콘솔: [2] 영수증을 찍었습니다  
  // 콘솔: [1] 주문을 받았습니다
```

→ 적은 순서대로 실행됩니다. 번호 순서가 아닙니다.

```
6) function handleMouseOut() {
```

```
  console.log("[onMouseOut] 마우스가 벗어났습니다");
```

```
  }  
  <button onClick={handleClick} onMouseOver={handleMouseOver} onMouseOut=  
    {handleMouseOut}>
```

→ 마우스를 올렸다 내리면 두 줄이 차례로 나옵니다.

이벤트 이름만 바꾸면 되고, 나머지는 onClick 과 똑같습니다.

useState 시작

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

개념01에서 버튼에 이벤트를 붙였습니다. 누르면 콘솔에는 잘 찍혔습니다. 그런데 화면은 한 글자도 안 바뀌었습니다.

이 파일에서 화면을 바꿉니다. 다만 곧바로 답을 알려 주지 않겠습니다. 먼저 '보통 변수'로 해 보고, 왜 안 되는지 눈으로 확인합니다. 그 뒤에 useState가 무엇을 대신해 주는지 보면 훨씬 잘 납니다.

React가 주는 도구는 파일 맨 위에서 import로 꺼내 씁니다. 중괄호 안에 필요한 이름을 적습니다. 이 문법 자체는 08단원에서 자세히 다룹니다. 지금은 "useState를 쓰려면 이 줄이 필요하다"까지만 알면 됩니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

console.log("useState를 꺼냈습니다. 정체는:", typeof useState);
```

F12 콘솔 useState를 꺼냈습니다. 정체는: function

useState는 '함수'입니다. 특별한 문법이 아니라 그냥 함수입니다.

보통 변수로 해 보면 어떻게 되나

섹션 1

JS자료 11단원에서는 이렇게 썼습니다. 잘 됐습니다.

```
let count = 0;
btn.addEventListener("click", () => {
  count++;
  out.textContent = count; // ← 화면 고치기를 '내가' 했습니다
});
```

React에서는 마지막 줄을 쓰지 않습니다. 화면 고치기는 React의 일입니다. 그럼 변수만 올리면 React가 알아서 화면을 고쳐 줄까요? 해 봅시다.

```
let outsideCount = 0; // 컴포넌트 '밖' 에 만든 보통 변수
function BrokenCounter({
  다시그리기 }) {
  let insideCount = 0; // 컴포넌트 '안' 에 만든 보통 변수
  function handleClick() {
    outsideCount = outsideCount + 1;
    insideCount = insideCount + 1;

    console.log("밖의 변수:", outsideCount, "/ 안의 변수:", insideCount);
  }
}
```

F12 콘솔 밖의 변수: 1 / 안의 변수: 1

```
}

return (
  <div className="demo"> <h3>① 보통 변수로 세어 보기 - 화면이 안 바뀝니다
</h3> <div className="output" id="brokenOut">
  화면에 보이는 값 - 밖: {outsideCount} / 안: {insideCount}
</div> <button id="btnBrokenUp" onClick={handleClick}>
  +1
</button> <button id="btnRedraw" onClick={다시그리기}>
  화면 다시 그리기
</button> </div>
);
}
```

데모 ①의 +1을 세 번 눌러 보세요. 콘솔의 숫자는 1, 2, 3으로 잘 올라갑니다. 값은 분명히 바뀌었습니다. 그런데 화면은 계속 0입니다. 화면(누르면): 화면에 보이는 값 — 밖: 0 / 안: 0

이유는 간단합니다. BrokenCounter 함수는 화면을 그릴 때 한 번 실행되고 끝났습니다. 그때 outsideCount가 0이었으니 화면에는 0이 박혔습니다. 그 뒤로 변수를 아무리 바꿔도, 함수를 다시 실행하지 않으면 화면은 그대로입니다.

React는 변수를 지켜보고 있지 않습니다. “값이 바뀌었으니 다시 그려라”라고 알려 주는 장치가 따로 필요합니다.

정말 값이 올라간 게 맞는지 확인해 봅시다. ①의 ‘화면 다시 그리기’ 버튼을 누르면 이 상자를 처음부터 다시 만듭니다.

그 버튼이 어떻게 그렇게 하는지는 이 파일 맨 아래에 있습니다. 미리 말하면 — 지금부터 배울 useState를 썼습니다. 여기서는 “누르면 이 상자가 처음부터 다시 그려진다”까지만 보면 됩니다.

+1을 세 번 누른 뒤 ‘화면 다시 그리기’를 눌러 보세요. 두 가지가 보입니다.

밖: 3 ← 값은 진짜로 올라가 있었습니다. 화면만 안 따라온 것입니다. 안: 0 ← 다시 실행되면서 let insideCount = 0이 처음부터 다시 실행됐습니다.

그래서 보통 변수로는 어느 쪽에 뒤도 안 됩니다. 밖에 두면 → 값은 남지만 화면이 안 따라옵니다. 안에 두면 → 다시 그릴 때마다 처음 값으로 되돌아갑니다.

필요한 것은 두 가지를 동시에 해 주는 물건입니다. (1) 다시 그려도 값을 기억하고 (2) 값이 바뀌면 다시 그리라고 React 에게 알려 주는 것

그게 useState 입니다.

▣ 직접 해보기 1

데모 ① 의 +1 을 다섯 번 누른 뒤 '화면 다시 그리기' 를 누르면 밖의 값이 몇으로 나올까요? 먼저 예상하고 눌러서 확인하세요.

useState 로 고치기

섹션 2

쓰는 모양은 한 줄입니다.

```
const [count, setCount] = useState(0);
      ^^^^^  ^^^^^^^^^      ^
      현재 값 바꾸는 함수   시작값
```

그리고 값을 바꿀 때는 대입이 아니라 함수를 부릅니다.

```
setCount(count + 1);
```

이 한 줄이 두 가지 일을 합니다. ① 다음 값을 React 에게 맡깁니다 ② "이 컴포넌트를 다시 그려라" 라고 알립니다

②가 바로 섹션 1에서 없던 그 장치입니다.

```
function GoodCounter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log("setCount 를 불렀습니다. 곧 화면이 따라옵니다.");
  }
}
```

F12 콘솔 setCount 를 불렀습니다. 곧 화면이 따라옵니다.

```
}

return (
  <div className="demo"> <h3>② useState 로 세기 - 화면이 따라옵니다
</h3> <div className="output" id="goodOut">
  담긴 아메리카노: {count}잔
</div> <button id="btnGoodUp" onClick={handleClick}>
  +1
</button> </div>
```

```
);  
}
```

화면(누르면): 담긴 아메리카노: 1잔 세 번 누르면 3잔이 됩니다. 이번에는 화면이 따라옵니다.

코드에서 화면을 고치는 줄을 한 줄도 안 썼다는 점을 보세요. `textContent` 도 없고 `innerHTML` 도 없습니다. `setCount` 를 부르면 React 가 `GoodCounter` 를 다시 실행하고, 그때 나온 새 화면 설명을 실제 화면에 반영합니다.

이런 값을 `state`(상태)라고 부릅니다. 우리말로는 '상태'지만 이 자료에서는 코드에 나오는 그대로 `state`라고 쓰겠습니다.

`state` 는 "바뀌면 화면도 같이 바뀌어야 하는 값" 입니다. 바뀌어도 화면과 상관없는 값이라면 `state` 로 만들 필요가 없습니다.

▫ 직접 해보기

2

시작값을 0 대신 5 로 바꿔 보세요. 왼쪽에서 다른 예제를 골랐다 돌아오면 화면이 몇으로 시작할까요?

대괄호는 왜 붙나 — 배열 구조분해

섹션 3

`const [count, setCount]` 의 대괄호가 낫설 수 있습니다. 새 문법이 아닙니다. JS자료 09단원 개념01에서 배운 배열 구조분해입니다.

```
const coffee = ["아메리카노", 4000];  
const [coffeeName, coffeePrice] = coffee;  
  
console.log(coffeeName, coffeePrice);
```

F12 콘솔 아메리카노 4000

왼쪽 대괄호는 '배열을 만드는 것' 이 아니라 '순서대로 꺼내는 틀' 입니다.

`useState(0)` 은 두 칸짜리 배열을 돌려줍니다.

— 0번 칸, 1번 칸

현재 값 그 값을 바꾸는 함수

그래서 이렇게 써도 똑같이 동작합니다. 구조분해를 안 쓴 모양입니다.

```
const state = useState(0);  
const count = state[0];  
const setCount = state[1];
```

세 줄을 한 줄로 줄인 것이 `const [count, setCount] = useState(0)` 입니다.

이름은 마음대로 지어도 됩니다. 위치만 맞으면 됩니다. 다만 거의 모든 React 코드가 [x, setX] 로 씁니다. 남이 읽기 쉬우니 이 관례를 따르세요.

```
const [count, setCount] = useState(0);
const [isOpen, setIsOpen] = useState(false);
const [userName, setUserName] = useState("김민준");
```

정말 두 칸짜리 배열인지 컴포넌트 안에서 확인해 봅시다.

```
function PairCheck() {
  const [count, setCount] = useState(0);

  console.log("0번 칸:", count, "/ 1번 칸의 정체:", typeof setCount);
```

F12 콘솔 0번 칸: 0 / 1번 칸의 정체: function

```
return (
  <div className="demo"> <h3>③ 두 칸의 정체 확인 – 콘솔을 보세요
</h3> <div className="output" id="pairOut">
    지금 값: {count}
  </div> <button id="btnPairUp" onClick={() => setCount(count + 1)}>
    +1 (누를 때마다 콘솔에 한 줄 더 찍힙니다)
  </button> </div>
);
}
```

console.log 를 컴포넌트 함수 안, return 위에 적었습니다. 그래서 이 줄은 '다시 그릴 때마다' 실행됩니다. 버튼을 누르면 콘솔에 0번 칸이 1, 2, 3 으로 바뀌며 다시 찍힙니다. 왜 다시 실행되는지는 개념03에서 자세히 봅니다.

위 버튼처럼 onClick 안에 화살표 함수를 바로 적어도 됩니다. 짧은 일 하나뿐이면 이렇게 쓰는 편이 읽기 좋습니다.

▸ 직접 해보기 3

PairCheck 의 두 이름을 [num, setNum] 으로 바꿔 보세요. (return 안에서 쓰는 곳도 같이 바뀌어야 합니다)

시작값 정하기

섹션 4

useState 의 괄호 안에 적은 값이 시작값입니다. 무엇이든 넣을 수 있습니다.

<code>useState(0)</code>	숫자
<code>useState("")</code>	빈 문자열
<code>useState(false)</code>	불리언
<code>useState([])</code>	빈 배열
<code>useState({})</code>	빈 객체

중요한 규칙이 하나 있습니다. 시작값은 '맨 처음 그릴 때 딱 한 번' 만 쓰입니다. 두 번째 렌더링부터는 React가 기억해 둔 값을 줍니다. 그래서 `useState(10)` 이라고 적혀 있어도 화면에는 12가 보일 수 있습니다.

```
function StartAtTen() {
  const [count, setCount] = useState(10);

  return (
    <div className="demo"> <h3>④ 시작값 10 - 되돌리려면 직접 넣어야 합니다
    </h3> <div className="output" id="tenOut">
      남은 케이크: {count}조각
    </div> <button id="btnTenDown" onClick={() => setCount(count - 1)}>
      한 조각 먹기
    </button> <button id="btnTenReset" onClick={() => setCount(10)}>
      10으로 되돌리기
    </button> </div>
  );
}
```

화면(누르면): 남은 케이크: 9조각

'한 조각 먹기'를 눌러 9가 된 다음, 코드의 `useState(10)` 은 그대로인데도 화면이 10으로 돌아가지 않는다는 점을 확인하세요. 되돌리려면 `setCount(10)` 처럼 직접 넣어 줘야 합니다.

왼쪽에서 다른 예제를 골랐다가 돌아오면 처음부터 다시 그리므로 다시 10이 됩니다. state는 브라우저 메모리에만 있습니다. 새로고침하면 사라집니다.

▢ 직접 해보기 4

'한 조각 더 굽기' 버튼을 만들어 1씩 늘려 보세요.

state는 컴포넌트마다 따로 있다

섹션 5

섹션 1의 `outsideCount`는 컴포넌트 밖에 있었습니다. 그런 변수는 컴포넌트를 두 개 만들어도 '하나'를 같이 씁니다.

state는 다릅니다. 컴포넌트가 화면에 놓인 개수만큼 따로 생깁니다. 03단원에서 같은 컴포넌트를 여러 번 썼던 것을 떠올려 보세요. 아래는 같은 컴포넌트를 두 번 쓴 것입니다. 값은 각자 따로 셉니다.

```
function CupCounter({ owner, btnId }) {
  const [cups, setCups] = useState(0);

  function handleClick() {
    setCups(cups + 1);
    console.log(owner, "님이 한 잔 더 마셨습니다");
  }
}
```

F12 콘솔 김민준 님이 한 잔 더 마셨습니다
이서연 님이 한 잔 더 마셨습니다

```

  }

  return (
    <div className="output">
      {owner}: {cups}잔{" "}
      <button id={btnId} onClick={handleClick}>
        한 잔 더
      </button> </div>
    );
  }
}
```

매개변수 자리의 { owner, btnId } 는 03단원 개념03의 props 구조분해입니다. btnId 는 버튼마다 다른 id 를 붙이려고 넘긴 값입니다. 한 화면에 같은 id 가 두 개 있으면 안 되기 때문입니다.

```
function TwinCounters() {
  return (
    <div className="demo"> <h3>⑤ 같은 컴포넌트 두 개 - 값은 각자 따로
  </h3> <CupCounter owner="김민준" btnId="btnCupA"
  /> <CupCounter owner="이서연" btnId="btnCupB" /> </div>
  );
}
```

김민준 쪽을 세 번 눌러도 이서연 쪽은 0 그대로입니다. "state 는 컴포넌트 안에 들어 있다" 는 뜻이 이것입니다. 두 값을 같이 움직이게 하고 싶다면 방법이 따로 있습니다. 07단원에서 배웁니다.

▢ 직접 해보기

5

TwinCounters 안에 박지훈 몫을 하나 더 넣어 보세요. btnId 는 다른 값으로 지어야 합니다.

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

대괄호 대신 중괄호로 꺼냈다

```
const { count, setCount } = useState(0);
```

에러가 안 납니다. 그런데 두 값 모두 `undefined` 가 됩니다. `useState` 가 돌려주는 것은 배열이라 '순서' 로 꺼내야 하는데, 중괄호는 '이름' 으로 꺼내는 문법이기 때문입니다. 화면에는 아무 숫자도 안 보이고, 버튼을 누르면 그제서야 `TypeError: setCount is not a function` 이 뜹니다.

실제로 어떻게 되는지 배열로 흉내 내 봅시다.

```
const wrongPair = ["아메리카노", 4000];  
const { itemName, itemPrice } = wrongPair;  
  
console.log("중괄호로 꺼내면:", itemName, itemPrice);
```

F12 콘솔 중괄호로 꺼내면: undefined undefined

배열에는 `itemName` 이라는 이름의 칸이 없습니다. 그래서 `undefined` 입니다.

이런 실수를 합니다

state 에 직접 대입했다

```
count = count + 1;
```

버튼을 누르는 순간 이런 에러가 납니다. `TypeError: Assignment to constant variable.` `const` 로 받았으니 대입이 막힙니다. 값을 바꾸려면 `setCount` 를 부릅니다. 그리고 `const` 가 아니었다 해도, 대입만으로는 화면이 안 바뀝니다. 섹션 1에서 본 그대로입니다.

이런 실수를 합니다

useState 를 꺼내지 않았다

```
const [count, setCount] = useState(0); // 파일 맨 위에 import 줄이 없을 때
```

ReferenceError: useState is not defined

화면이 통째로 빡니다. 파일 맨 위에 `import { useState } from "react";` 가 꼭 있어야 합니다. 이 줄이 없으면 `useState` 라는 이름 자체가 없습니다.

이런 실수를 합니다

set 함수를 컴포넌트 밖에서 부르려고 했다

```
setCount(1); // 컴포넌트 밖, 그냥 script 한복판에서
```

ReferenceError: setCount is not defined setCount

는 컴포넌트 함수 '안' 에서 만들어집니다. 밖에서는 이름 자체가 없습니다.

이런 실수를 합니다

set 을 부른 바로 다음 줄에서 값을 읽었다

```
setCount(count + 1);
console.log(count); // 아직 옛날 값입니다
```

에러는 없는데 값이 하나 뒤쳐져 보입니다. 아주 헷갈리는 실수입니다. 왜 그런지는 개념05에서 제대로 다룹니다.

이런 실수를 합니다

두 이름 사이에 쉼표를 빠뜨렸다

```
const [count, setCount] = useState(0);
```

[SyntaxError] 입니다. 파일 전체가 안 돌아가고 화면이 통째로 빡니다.

PARSE_ERROR · Expected `,` or `]` but found `Identifier`

메시지 아래에 문제의 줄과 위치가 그림으로 같이 나옵니다.
구조분해의 대괄호 안은 쉼표로 나눕니다. [count, setCount] 가 맞습니다.

마지막: 위에서 만든 컴포넌트를 한 화면에 모으기

정리

- 1 보통 변수는 바뀌어도 화면이 안 바뀝니다. 컴포넌트 함수를 다시 실행하지 않기 때문입니다.
- 2 컴포넌트 안의 보통 변수는 다시 그릴 때마다 처음 값으로 되돌아갑니다.
- 3 **const [값, set값] = useState(시작값)** 한 줄이 그 두 문제를 함께 해결합니다.
- 4 값을 바꿀 때는 대입이 아니라 **set 함수**를 부릅니다. 그러면 React 가 다시 그립니다.
- 5 대괄호는 JS자료 09단원의 배열 구조분해입니다. 새 문법이 아닙니다.
- 6 시작값은 맨 처음 그릴 때만 씁니다. 되돌리려면 set 으로 직접 넣습니다.
- 7 state 는 컴포넌트마다 따로 생깁니다. 같은 컴포넌트를 둘 놓으면 값도 둘입니다.

```
export default function Concept02() {
```

★ 위 섹션 1에서 쓴 '화면 다시 그리기' 버튼의 정체입니다. 숫자를 하나 바꾸면 React 가 key 가 달라진 것을 보고 그 안을 통째로 버리고 처음부터 다시 만듭니다. 더블클릭으로 열던 시절에는 root.render 를 손으로 다시 불렀습니다. Vite 프로젝트에서는 root 가 main.jsx 에 하나뿐이라 이렇게 합니다.

```
const [다시그린횟수, 다시그리기] = useState(0);

return (
  <div> <h1>개념 02 - useState 시작</h1> <p className="guide">
    왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
    Console</strong> 도 함께 보세요.
    <br /> <br />
    이 파일의 앞부분은 <strong>일부러 안 되는 코드</strong>입니다. 화면이 안
    바뀌는 것이 정상입니다. 왜 안 되는지 본 다음에 고칩니다.
    </p> <div> <div key={다시그린횟수}> <BrokenCounter 다시그리기={() => 다시그리기
    (다시그린횟수 + 1)} />
    </div> <GoodCounter /> <PairCheck /> <StartAtTen /> <TwinCounters
  /> </div> </div>
);
}
```

직접 해보기 정답

1) 밖: 5 / 안: 0 이 나옵니다.

```
// 화면: 화면에 보이는 값 - 밖: 5 / 안: 0
```

→ 밖의 변수는 값을 기억하고 있었고, 화면만 안 따라온 것입니다.

안의 변수는 컴포넌트 함수가 다시 실행되면서 0으로 새로 만들어졌습니다.

```
2) const [count, setCount] = useState(5);
// 화면: 담긴 아메리카노: 5잔
```

→ 시작값은 맨 처음 그릴 때만 쓰입니다.

이미 눌러서 값이 올라간 상태라면, 왼쪽에서 다른 예제를 골랐다 돌아와서 눌러야 5부터 다시 시작합니다.

```
3) const [num, setNum] = useState(0);
console.log("0번 칸:", num, "/ 1번 칸의 정체:", typeof setNum);
```

... <div className="output" id="pairOut">지금 값: {num}</div> ... <button id="btnPairUp" onClick={() => setNum(num + 1)}> → 동작은 똑같습니다. 이름은 위치만 맞으면 무엇이든 됩니다.

다만 setNum 을 바꾸는 것을 잊으면
ReferenceError: setCount is not defined 가 납니다.

```
4) <button onClick={() => setCount(count + 1)}>한 조각 더 굽기</button>  
// 화면(누르면): 남은 케이크: 11조각
```

→ 시작값 10에서 한 번 누르면 11입니다.

5) <CupCounter owner="박지훈" btnId="btnCupC" />

```
// 화면: 박지훈: 0잔 [한 잔 더]
```

→ 세 사람의 값이 전부 따로 셉니다.

```
btnId 를 btnCupA 로 똑같이 적으면 화면에 같은 id 가 둘이 생겨  
나중에 그 버튼을 찾을 때 앞의 것만 잡힙니다.
```

state가 바뀌면 다시 그린다

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

개념02에서 `setCount` 를 부르니 화면이 따라왔습니다. 그 사이에 정확히 무슨 일이 일어난 걸까요?

답은 한 문장입니다.

`set` 함수를 부르면, 그 컴포넌트 함수가 처음부터 다시 실행된다.

‘처음부터 다시’ 라는 말이 핵심입니다. 함수 안의 첫 줄부터 `return` 까지 전부 다시 지나갑니다. 이 일을 렌더링(rendering)이라고 부릅니다. 우리말로는 ‘그리기’ 입니다.

말로만 들으면 잘 안 믿깁니다. 그래서 이 파일에서는 세어 봅니다.

다시 실행되는 것을 눈으로 세어 보기

섹션 1

컴포넌트 함수가 몇 번 실행됐는지 셀 변수를 하나 둡니다. 컴포넌트 ‘밖’ 에 뒤야 다시 실행돼도 값이 남습니다. (개념02 섹션 1)

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

let renderCount = 0;

function RenderCounter() {
  const [count, setCount] = useState(0);

  renderCount = renderCount + 1;

  console.log("그리는 중 -", renderCount, "번째 실행 / count =", count);
}
```

```
F12 콘솔    그리는 중 - 1 번째 실행 / count = 0
            그리는 중 - 2 번째 실행 / count = 0
```

↑ 개발 중에는 React 가 함수를 일부러 두 번 실행합니다(09단원 개념02). 그래서 한 번 그릴 때마다 두 줄씩 찍히고, 숫자도 둘씩 올라갑니다. “다시 실행된다” 는 것만 보면 되니 숫자 자체는 신경 쓰지 마세요.


```

return (
  <div className="demo"> <h3>① 누를 때마다 함수가 다시 실행됩니다
</h3> <div className="output" id="renderOut">
    count: {count} / 이 컴포넌트가 실행된 횟수: {renderCount}
  </div> <button id="btnRenderUp" onClick={() => setCount(count + 1)}>
    +1
  </button> </div>
);
}

```

페이지를 열면 콘솔에 “1 번째 실행” “2 번째 실행” 두 줄이 찍힙니다. 버튼을 한 번 누르면 “3 번째” “4 번째” 가 또 찍힙니다. 한 번 그릴 때마다 두 줄인 것은 위에 적은 개발 중 설정 때문입니다. 중요한 것은 숫자가 아니라 “버튼을 누를 때마다 또 실행된다” 는 사실입니다.

`console.log` 를 `return` 위, 함수 몸통 한복판에 적었다는 점을 보세요. 그 줄이 매번 다시 실행된다는 뜻입니다.

순서를 정리하면 이렇습니다.

① 버튼을 누른다 ② `onClick`에 맡겨 둔 함수가 실행된다 → `setCount(1)`이 불린다 ③ React가 “이 컴포넌트 다시 그려야겠다” 라고 표시해 둔다 ④ `RenderCounter` 함수가 처음부터 다시 실행된다

이때 `useState(0)`은 0을 주지 않습니다. 기억해 둔 1을 줍니다.

⑤ 새로 돌려받은 화면 설명을 예전 것과 비교해 바뀐 곳만 실제 화면에 반영한다

④가 이 파일의 주제입니다. `useState(0)`의 0이 두 번째부터는 무시된다는 점을 다시 확인하세요. 개념02 섹션 4에서 본 “시작값은 맨 처음 한 번만”이 이 이야기입니다.

▹ 직접 해보기 1

①의 +1을 네 번 누르면 콘솔의 마지막 줄은 몇 번째 실행이라고 나올까요? 예상하고 확인하세요. (한 번 그릴 때 두 줄씩 찍힌다는 것을 계산에 넣으세요)

다시 실행되면 함수 안의 변수도 새로 만들어진다

섹션 2

함수가 처음부터 다시 실행된다는 말에는 이런 뜻도 들어 있습니다. 함수 안에서 만든 보통 변수는 매번 새로 태어납니다.

state와 나란히 놓고 비교해 봅시다.

```
function CompareVars() {
  const [saved, setSaved] = useState(0);

  let normal = 0; // ← 이 줄이 렌더링마다 다시 실행됩니다
  function handleClick() {
    normal = normal + 1; // 보통 변수를 올려 봅시다
    setSaved(saved + 1); // state 는 React 에게 맡깁니다
    console.log("버튼 안에서 본 normal:", normal);
  }
}
```

F12 콘솔 버튼 안에서 본 normal: 1

```

}

console.log("새로 그린 화면의 normal:", normal, "/ saved:", saved);

```

F12 콘솔 새로 그린 화면의 normal: 0 / saved: 0
 새로 그린 화면의 normal: 0 / saved: 1

```
return (
  <div className="demo"> <h3>② 보통 변수는 사라지고 state 만 남습니다
</h3> <div className="output" id="compareOut">
  normal: {normal} / saved: {saved}
</div> <button id="btnCompareUp" onClick={handleClick}>
  둘 다 +1 해 보기
</button> </div>
);
}
```

버튼을 눌렀을 때 콘솔에 두 줄이 나옵니다. 순서와 값을 잘 보세요.

버튼 안에서 본 normal: 1 ← 올라간 게 맞습니다 새로 그린 화면의 normal: 0 / saved: 1

normal 은 분명히 1이 됐는데, 다시 그리고 나니 0으로 돌아왔습니다.

```
let normal = 0; 이 다시 실행됐기 때문입니다. 올려 둔 1은 버려졌습니다.
```

화면(누르면): normal: 0 / saved: 1

saved 는 남았습니다. React 가 컴포넌트 밖 어딘가에 따로 보관하고 있다가 다시 실행될 때 useState 를 통해 돌려주기 때문입니다.

그래서 이렇게 나눠 생각하면 됩니다. 화면에 보여야 하고 다음에도 기억해야 하는 값 → state 이번에 그릴 때만 잠깐 쓰는 값 → 보통 변수

예를 들어 “가격에 개수를 곱한 합계” 같은 값은 보통 변수면 충분합니다. 매번 다시 계산하면 되니까요. 그 이야기는 개념04에서 다시 나옵니다.

▣ 직접 해보기 2

let normal = 0 을 let normal = 100 으로 바꾸고 버튼을 눌러 보세요. 화면의 normal 은 무엇이 될까요?

값이 그대로면 다시 그리지 않는다

섹션 3

set 함수를 부르지만 하면 무조건 다시 그리는 걸까요? 아닙니다. React 는 새로 들어온 값이 지금 값과 같은지 먼저 비교합니다. 같으면 다시 그릴 이유가 없으니 그냥 넘어갑니다.

```
function SameValue() {
  const [count, setCount] = useState(0);

  console.log("[값 비교 데모] 다시 그렸습니다. count =", count);
```

```
F12 콘솔    [값 비교 데모] 다시 그렸습니다. count = 0
            [값 비교 데모] 다시 그렸습니다. count = 1
```

```
return (
  <div className="demo"> <h3>③ 같은 값을 넣으면 콘솔에 아무것도 안 늘어납니다
</h3> <div className="output" id="sameOut">
    count: {count}
  </div> <button id="btnSame" onClick={() => setCount(count)}>
    같은 값 넣기
  </button> <button id="btnSameUp" onClick={() => setCount(count + 1)}>
    +1
  </button> </div>
);
}
```

‘같은 값 넣기’를 열 번 눌러 보세요. 콘솔에 한 줄도 안 늘어납니다. ‘+1’을 누르면 그때 한 줄이 늘어납니다.

이 성질 덕분에 마음 놓고 set 을 불러도 됩니다. 값이 안 바뀌었다면 React 가 알아서 일을 안 합니다.

다만 여기에 함정이 하나 있습니다. 배열이나 객체를 state 로 쓸 때, 안의 내용을 고치고 같은 배열을 다시 넣으면 React 는 “같은 값”으로 보고 다시 그리지 않습니다. “화면이 안 바뀌어요”의 큰 원인입니다.

개념06에서 실제로 재현해 봅니다.

▣ 직접 해보기 3

‘같은 값 넣기’버튼의 setCount(count) 를 setCount(count + 0) 으로 바꿔 보세요. 결과가 달라질까요?

다시 그려지는 범위는 그 컴포넌트 하나가 아닙니다. 그 안에 들어 있는 컴포넌트들도 같이 다시 실행됩니다. 자식에게 state 가 없어도 그렇습니다.

```
function Child() {
  console.log("  L Child 도 다시 실행되었습니다");
}
```

F12 콘솔 L Child 도 다시 실행되었습니다

```
return <div className="output">저는 자식입니다. state 가 없습니다.</div>;
}
```

```
function Parent() {
  const [n, setN] = useState(0);

  console.log("Parent 실행 - n =", n);
}
```

F12 콘솔 Parent 실행 - n = 0
Parent 실행 - n = 1

```
return (
  <div className="demo"> <h3>④ 부모를 누르면 자식도 다시 실행됩니다
</h3> <div className="output" id="parentOut">
  부모의 n: {n}
</div> <Child /> <button id="btnParentUp" onClick={() => setN(n + 1)}>
  부모의 n 올리기
</button> </div>
);
}
```

버튼을 한 번 누르면 콘솔에 두 줄이 나옵니다. 부모 한 줄, 자식 한 줄입니다.

“자식은 바뀐 게 없는데 왜 또 실행하지?” 라는 생각이 들 수 있습니다. React 는 자식이 바뀌었는지 미리 알 방법이 없어서 일단 전부 다시 실행합니다. 다시 실행한 결과가 예전과 같으면, 실제 화면은 건드리지 않습니다.

함수를 다시 실행하는 일은 대개 아주 빠릅니다. 그래서 보통은 신경 쓸 필요가 없습니다. 정말 느려지는 경우에 막는 방법이 있는데, 그건 10단원에서 배웁니다. 지금 미리 막으려 들면 코드만 복잡해집니다.

▫ 직접 해보기 4

Child 안에 자기만의 state 를 하나 넣어 보세요. (`const [x, setX] = useState(0);` 를 맨 위에 적으면 됩니다) 부모를 눌러도 x 값은 그대로일까요?

다시 그린다고 화면을 새로 만들지는 않는다

섹션 5

‘다시 그린다’는 말을 들으면 화면을 전부 지우고 새로 만드는 그림이 떠오릅니다. 실제로는 그렇지 않습니다.

React는 새로 나온 화면 설명과 예전 설명을 비교해서 달라진 곳만 실제 화면에 반영합니다. 01단원 개념03에서 본 그 이야기입니다.

정말 그런지 확인해 봅시다. 화면의 요소 하나를 기억해 뒀다가, 다시 그린 뒤에도 같은 요소인지 비교해 보겠습니다.

```
let savedNode = null;

function DomCheck() {
  const [count, setCount] = useState(0);

  function remember() {
    savedNode = document.querySelector("#domBox");
    console.log("지금 화면의 요소를 기억해 뒀습니다");
  }
}
```

F12 콘솔 지금 화면의 요소를 기억해 뒀습니다

```
}

function check() {
  const nodeNow = document.querySelector("#domBox");

  console.log("아까 그 요소와 같은 것인가?", savedNode === nodeNow);
}
```

F12 콘솔 아까 그 요소와 같은 것인가? true

```
console.log("안의 글자는:", nodeNow.textContent);
```

F12 콘솔 안의 글자는: 눌린 횟수: 1

```
}

return (
  <div className="demo"> <h3>⑤ 같은 요소의 글자만 바꿉니다 - ①②③ 순서로 눌러 보세요</h3> <div className="output" id="domBox">
    눌린 횟수: {count}
  </div> <button id="btnRemember" onClick={remember}>
    ① 요소 기억하기
  </button> <button id="btnDomUp" onClick={() => setCount(count + 1)}>
    ② +1
  </div>
);
```

여러 state 다루기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

지금까지 state 는 한 컴포넌트에 하나뿐이었습니다. 실제 화면에는 바뀌는 값이 여러 개 있습니다. 담은 개수, 고른 메뉴, 포장할지 말지, 로그인했는지 ...

이 파일에서 두 가지를 배웁니다. ① state 를 여러 개 두는 법과, 숫자 말고 어떤 값을 담을 수 있는지 ② 값을 언제 나누고 언제 하나로 묶는지

state 를 여러 개 두기

섹션 1

방법은 아주 단순합니다. `useState` 를 필요한 만큼 줄줄이 적으면 됩니다. 서로 아무 상관이 없습니다. 각자 자기 값을 따로 기억합니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function TwoCounters() {
  const [americano, setAmericano] = useState(0);
  const [latte, setLatte] = useState(0);

  function addAmericano() {
    setAmericano(americano + 1);
    console.log("아메리카노를 담았습니다. 지금까지:", americano + 1, "잔");
  }
}
```

F12 콘솔 아메리카노를 담았습니다. 지금까지: 1 잔

```
  }

  function addLatte() {
    setLatte(latte + 1);
    console.log("라떼를 담았습니다. 지금까지:", latte + 1, "잔");
  }
}
```

F12 콘솔 라떼를 담았습니다. 지금까지: 1 잔

```
  }

  return (
    <div className="demo"> <h3>① state 두 개 - 따로 셉니다
```

```

</h3> <div className="output" id="twoOut">
  아메리카노 {americano}잔 / 라떼 {latte}잔
</div> <button id="btnAmericano" onClick={addAmericano}>
  아메리카노 +1
</button> <button id="btnLatte" onClick={addLatte}>
  라떼 +1
</button> </div>
);
}

```

화면(누르면): 아메리카노 1잔 / 라떼 0잔

아메리카노 쪽을 눌러도 라떼는 그대로입니다. 서로 남남입니다.

다만 둘 중 하나만 바뀌어도 컴포넌트 전체가 다시 실행됩니다. 개념03에서 본 그대로입니다. `useState` 두 줄이 모두 다시 지나가고, 그때 React 는 각각 기억해 둔 값을 순서대로 돌려줍니다.

‘순서대로’ 라는 말을 기억해 두세요. React 는 이름으로 구분하지 않습니다. 몇 번째로 부른 `useState` 인지로 구분합니다. 그래서 `useState` 를 부르는 순서가 렌더링마다 달라지면 안 됩니다. 이 규칙은 개념06에서 다시 나옵니다.

직접 해보기

1

케이크 개수를 세는 state 를 하나 더 넣고 ‘케이크 +1’ 버튼을 만들어 보세요.

숫자 말고 다른 값도 state 가 됩니다

섹션 2

state 에는 자바스크립트 값이면 무엇이든 담깁니다. 이번에는 문자열을 담아 봅시다. 오늘 고른 메뉴 이름입니다.

```

function MenuPicker() {
  const [menu, setMenu] = useState("아직 안 골랐습니다");

  function pick(name) {
    setMenu(name);
    console.log("고른 메뉴:", name);
  }
}

```

```

F12 콘솔   고른 메뉴: 아메리카노
           고른 메뉴: 라떼
           고른 메뉴: 케이크

```

```

}

return (
  <div className="demo"> <h3>② 문자열 state - 고른 메뉴
</h3> <div className="output" id="menuOut">
  오늘의 메뉴: {menu}

```

이전 값으로 갱신하기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

여기까지 `setCount(count + 1)` 로 잘 세어 왔습니다. 그런데 이 방식이 통하지 않는 자리가 있습니다.

```
setCount(count + 1);
setCount(count + 1); // 두 번 불렀으니 2가 오를까요?
```

안 오릅니다. 1만 오릅니다. 왜 그런지, 그리고 어떻게 고치는지가 이 파일의 전부입니다.

이 내용은 React 를 배울 때 가장 많이 걸려 넘어지는 곳입니다. 에러가 안 나기 때문입니다. 그냥 값이 하나 모자랄 뿐입니다.

두 번 불렀는데 1만 오릅니다

섹션 1

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function TwiceBroken() {
  const [count, setCount] = useState(0);

  function addTwice() {
    setCount(count + 1);
    setCount(count + 1);

    console.log("setCount 를 두 번 불렀습니다. 이번 렌더의 count:", count);
  }
}
```

F12 콘솔 setCount 를 두 번 불렀습니다. 이번 렌더의 count: 0

```
}

return (
  <div className="demo"> <h3>① setCount 를 두 번 불러 봅니다
</h3> <div className="output" id="twiceOut">
  개수: {count}
</div> <button id="btnTwice" onClick={addTwice}>
  +1 을 두 번 부르기
</button> </div>
```



```
);  
}
```

화면(누르면): 개수: 1

한 번 더 누르면 2가 됩니다. 세 번 누르면 3입니다. 누를 때마다 2씩 오를 것 같은데 1씩 오릅니다. 절반이 사라진 셈입니다.

코드를 다시 보면 이상한 곳이 없습니다. 오타도 없습니다. 그러니 코드를 노려보지 말고, 값이 실제로 어떻게 흘러가는지 봅시다.

▸ 직접 해보기 1

setCount(count + 1) 을 세 줄로 나눠 보세요. 한 번 누르면 몇이 될까요? 예상하고 확인하세요.

04

set 을 부른 직후의 값을 찍어 보기

섹션 2

살마리는 "set 을 부른 뒤에 count 가 얼마인가" 입니다. 직접 찍어 봅시다.

```
function ReadAfterSet() {  
  const [count, setCount] = useState(0);  
  
  function handleClick() {  
    console.log("① set 부르기 전:", count);
```

F12 콘솔 ① set 부르기 전: 0

```
    setCount(count + 1);  
  
    console.log("② set 부른 직후:", count);
```

F12 콘솔 ② set 부른 직후: 0

```
  }  
  
  console.log("③ 다시 그린 뒤:", count);
```

F12 콘솔 ③ 다시 그린 뒤: 0
③ 다시 그린 뒤: 1

```

return (
  <div className="demo"> <h3>② set 직후에는 아직 옛날 값입니다
</h3> <div className="output" id="readOut">
    개수: {count}
  </div> <button id="btnRead" onClick={handleClick}>
    눌러서 ①②③ 을 확인
  </button> </div>
);
}

```

버튼을 한 번 누르면 콘솔에 이렇게 나옵니다.

① set 부르기 전: 0 ② set 부른 직후: 0 ← 1이 아닙니다! ③ 다시 그린 뒤: 1

②가 핵심입니다. setCount(1) 을 불렀는데도 바로 다음 줄의 count 는 0입니다.

set 함수는 값을 '지금 당장' 바꾸지 않습니다. "다음 화면은 1로 그려 주세요" 라고 React 에게 부탁만 하고 돌아옵니다. 실제로 값이 바뀐 모습은 ③, 즉 다음 렌더링에서 보입니다.

이 성질을 한 줄로 적으면 이렇습니다.

state 는 즉시 바뀌지 않는다. 다음 렌더링에서 바뀐다.

▹ 직접 해보기

2

② 자리에서 count 대신 count + 1 을 찍어 보세요. 이 값은 화면에 보이는 다음 값과 같을까요?

왜 두 번이 한 번이 되는가

섹션 3

이제 섹션 1의 코드를 다시 봅시다. 버튼을 처음 눌렀을 때, 이번 렌더의 count 는 0입니다. count 는 `const` 로 받은 값이라 이 함수가 도는 동안 절대 안 바뀝니다.

```

setCount(count + 1); → setCount(0 + 1); → setCount(1);
setCount(count + 1); → setCount(0 + 1); → setCount(1);

```

두 줄 다 "1로 만들어 주세요" 입니다. 그래서 결과는 1입니다. 두 번째 줄은 첫 번째 줄이 부탁한 1을 알지 못합니다.

비유하면 count 는 이번 렌더링 때 찍어 둔 '사진' 입니다. 사진 속 숫자는 나중에 실제 값이 바뀌어도 그대로입니다.

비유를 건어내고 실제로 무슨 일이 일어나는지 보면 이렇습니다. 컴포넌트 함수가 실행될 때 count 라는 변수에 0이 담겼고, 그 안에서 만들어진 addTwice 함수는 그 0을 계속 보고 있습니다. JS자료 05단원에서 배운 스코프 이야기 그대로입니다. 함수는 자기가 만들어진 자리의 변수를 기억합니다.

눈으로 확인해 봅시다.

```
function WhyOnlyOne() {
  const [count, setCount] = useState(0);

  function show() {
    console.log("이번 렌더에서 count + 1 은:", count + 1);
  }
}
```

F12 콘솔 이번 렌더에서 count + 1 은: 1

```
console.log("한 줄 아래에서 또 계산해도:", count + 1);
```

F12 콘솔 한 줄 아래에서 또 계산해도: 1

```
setCount(count + 1);
}

return (
  <div className="demo"> <h3>③ 같은 렌더 안에서는 몇 번을 계산해도 같은 값
</h3> <div className="output" id="whyOut">
  개수: {count}
</div> <button id="btnWhy" onClick={show}>
  계산해 보기
</button> </div>
);
}
```

두 줄 다 1이 나옵니다. 사이에서 아무리 set 을 불러도 마찬가지입니다. 필요한 것은 “지금 이 순간의 진짜 값” 을 아는 방법입니다.

▸ 직접 해보기 3

③ 을 세 번 누른 뒤 다시 한 번 누르면 콘솔에 어떤 숫자가 나올까요? 예상하고 확인하세요.

함수형 갱신 — 이전 값을 React 에게 받아 쓴다

섹션 4

set 함수에는 값 대신 '함수' 를 넣을 수도 있습니다.

```
setCount((prev) => prev + 1);
      ^^^^      ^^^^^^^
      직전 값   다음 값
```

이렇게 넣으면 React 가 그 함수를 나중에 부르면서 '그 시점의 진짜 값' 을 prev 자리에 넣어 줍니다. 내가 사진 속 숫자를 쓰는 게 아니라, React 가 현재 값을 건네주는 것입니다.

prev 라는 이름은 규칙이 아닙니다. previous(이전)의 줄임말로 많이 쓸 뿐입니다. p, prevCount, c 무엇이든 됩니다.

정말 다른 값이 들어오는지 찍어 봅시다.

```
function FunctionalUpdate() {
  const [count, setCount] = useState(0);

  function addTwice() {
    setCount((prev) => {
      console.log("첫 번째 갱신 함수가 받은 prev:", prev);
```

F12 콘솔 첫 번째 갱신 함수가 받은 prev: 0

```
    return prev + 1;
  });

  setCount((prev) => {
    console.log("두 번째 갱신 함수가 받은 prev:", prev);
```

F12 콘솔 두 번째 갱신 함수가 받은 prev: 1

```
    return prev + 1;
  });
}

return (
  <div className="demo"> <h3>④ 함수형 갱신 - 두 번 부르면 2가 옵니다
</h3> <div className="output" id="fnOut">
  개수: {count}
</div> <button id="btnFnTwice" onClick={addTwice}>
  함수형으로 +1 두 번
</button> </div>
);
}
```

화면(누르면): 개수: 2

첫 번째 함수는 0을 받아 1을 돌려주고, 두 번째 함수는 그 1을 받아 2를 돌려줍니다. 앞의 결과가 뒤로 이어집니다. 그래서 두 번 부르면 2가 옵니다.

React 는 set 을 부를 때마다 그 요청을 줄 세워 두고, 이벤트 처리가 끝난 뒤 줄 선 순서대로 하나씩 적용합니다. 값을 넣으면 → “무조건 이 값으로” 라는 요청 함수를 넣으면 → “앞의 결과를 받아 이렇게 바뀌라” 라는 요청

갱신 함수 안에서는 계산만 하고 값을 돌려주세요. return 을 빠뜨리면 undefined 가 새 값이 되어 화면이 빡니다.

짧게 쓰면 이렇게 한 줄입니다. 위 코드는 prev 를 찍어 보려고 길게 쓴 것입니다.

```
setCount((prev) => prev + 1);
```

▣ 직접 해보기 4

섹션 1의 TwiceBroken 을 함수형으로 고쳐 보세요. 한 번 누르면 2씩 오르면 성공입니다.

어느 쪽을 언제 쓰나

섹션 5

규칙은 간단합니다.

지금 값과 상관없는 값을 넣을 때 → `setMenu("라떼")` 그냥 값
지금 값을 바탕으로 다음 값을 만들 때 → `setCount(p => p + 1)` 함수형

한 이벤트에서 set 을 한 번만 부른다면 둘 다 같은 결과가 나옵니다. 그래서 `setCount(count + 1)` 도 대부분은 잘 됩니다. 다만 이런 자리에서는 함수형만 안전합니다.

① 한 이벤트에서 같은 state 를 두 번 이상 바꿀 때 (섹션 1) ② 나중에 실행될 코드 안에서 바꿀 때

예: 타이머 안, 서버 응답이 도착한 뒤. 09단원에서 다시 만납니다.

함수형은 숫자에만 쓰는 게 아닙니다. 이전 값이 필요한 곳이면 어디든 됩니다.

```
function PrevOthers() {
  const [isOn, setIsOn] = useState(false);
  const [value, setValue] = useState(1);

  function toggleTwice() {
    setIsOn((prev) => !prev);
    setIsOn((prev) => !prev);

    console.log("두 번 뒤집었으니 원래대로 돌아옵니다");
  }
}
```

F12 콘솔 두 번 뒤집었으니 원래대로 돌아옵니다

```
}

function doubleTwice() {
  setValue((prev) => prev * 2);
  setValue((prev) => prev * 2);

  console.log("두 번 곱했으니 네 배가 됩니다");
}
```

F12 콘솔 두 번 곱했으니 네 배가 됩니다

```

    }

    const onText = isOn ? "켜짐" : "꺼짐";

    return (
      <div className="demo"> <h3>⑤ 숫자 말고도 이전 값이 필요합니다
    </h3> <div className="output" id="prevOut">
      불: {onText} / 값: {value}
    </div> <button id="btnPrevToggle" onClick={toggleTwice}>
      두 번 뒤집기
    </button> <button id="btnDouble" onClick={doubleTwice}>
      두 번 곱하기
    </button> </div>
    );
  }
}

```

화면(누르면): 불: 꺼짐 / 값: 4

‘두 번 뒤집기’는 눌러도 불이 그대로입니다. 그게 맞는 결과입니다. `false` → `true` → `false` 로 두 번 뒤집혔기 때문입니다. 만약 `setIsOn(!isOn)` 을 두 번 썼다면 둘 다 “true 로 해 달라” 가 되어 한 번 뒤집힌 것처럼 보였을 것입니다. 이것도 조용히 틀리는 경우입니다.

‘두 번 곱하기’는 `1` → `2` → `4` 가 됩니다. 한 번 더 누르면 16입니다.

▢ 직접 해보기

5

‘두 번 곱하기’를 세 번 부르도록 고치면 한 번 눌렀을 때 값이 얼마가 될까요?

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

함수형으로 쓰긴 했는데 `prev` 를 안 썼다

```
setCount((prev) => count + 1);
```

에러가 안 납니다. 모양은 함수형인데 안에서 옛날 `count` 를 쓰고 있습니다. 그래서 두 번 불러도 1만 오릅니다. 고친 것 같은데 그대로인 상황입니다. 아래 데모 ⑥ 에서 직접 확인하세요.

```
function WrongFunctional() {
  const [count, setCount] = useState(0);

  function addTwice() {
    setCount((prev) => count + 1); // prev 를 안 쓰고 count 를 씀 setCount((prev) =>
    count + 1);

    console.log("함수형처럼 보이지만 안에서 옛날 값을 씁니다");
  }
}
```

F12 콘솔 함수형처럼 보이지만 안에서 옛날 값을 씁니다

```
}

return (
  <div className="demo"> <h3>⑥ [실수 1] 모양만 함수형
</h3> <div className="output" id="wrongFnOut">
  개수: {count}
</div> <button id="btnWrongFn" onClick={addTwice}>
  눌러 보세요 (1만 오릅니다)
</button> </div>
);
}
```

화면(누르면): 개수: 1

이런 실수를 합니다

set 다음 줄에서 값을 읽고 그 값으로 무언가를 했다

```
setCount(count + 1);
console.log("지금 개수는", count); // ← 옛날 값
if (count >= 3) { ... } // ← 판단도 옛날 값으로
```

에러가 안 납니다. 값이 한 박자씩 늦게 반응합니다. "3개부터 경고"를 만들었는데 4개가 돼서야 경고가 뜨는 식입니다. 다음 값이 필요하면 미리 변수에 담아 두고 그걸 쓰세요. `const next = count + 1;` `setCount(next); if (next >= 3) { ... }`

이런 실수를 합니다

갱신 함수에서 ++ 를 썼다

```
setCount((prev) => prev++);
```

화면이 아예 안 바뀝니다. `prev++` 는 '올리기 전의 값'을 돌려주기 때문입니다. 돌려준 값이 지금 값과 같아서 React 는 다시 그리지 않습니다. (개념03 섹션 3) `prev + 1` 이라고 써야 합니다.

이런 실수를 합니다

갱신 함수에서 `return` 을 빠뜨렸다

```
setCount((prev) => {  
  prev + 1;  
});
```

새 값이 `undefined` 가 됩니다. 화면의 숫자 자리가 빈칸이 됩니다. 중괄호를 열었으면 `return` 을 적어야 합니다. (JS자로 05단원 화살표 함수) 중괄호 없이 `(prev) => prev + 1` 로 쓰면 `return` 이 필요 없습니다.

이런 실수를 합니다

갱신 함수 안에서 다른 일까지 했다

```
setCount((prev) => {  
  console.log("올립니다");  
  return prev + 1;  
});
```

지금은 잘 됩니다. 하지만 이 함수는 React 가 언제 몇 번 부를지 모릅니다. 값을 계산해서 돌려주는 일만 하세요. 화면을 바꾸거나 다른 `set` 을 부르면 안 됩니다.

이런 실수를 합니다

괄호를 하나 덜 닫았다

```
setCount((prev) => prev + 1;
```

[SyntaxError] 입니다. 파일 전체가 안 돌아가고 화면이 통째로 빡니다.

PARSE_ERROR · Expected , or) but found ;

메시지 아래 그림이 짚어 주는 줄부터 보면 됩니다.

함수형 갱신은 괄호가 겹쳐서 짝을 놓치기 쉽습니다.

`setCount( ( prev ) => prev + 1 )` 처럼 여는 것과 닫는 것을 세어 보세요.

마지막: 위에서 만든 컴포넌트를 한 화면에 모으기

정리

- 1 set 함수는 값을 즉시 바꾸지 않습니다. 다음 렌더링에서 바뀝니다.
- 2 그래서 set 을 부른 바로 다음 줄에서 읽으면 아직 옛날 값입니다.
- 3 한 렌더 안의 count 는 고정된 숫자입니다. setCount(count + 1) 을 두 번 써도 둘 다 같은 값을 넣습니다.
- 4 **setCount((prev) ⇒ prev + 1)** 로 쓰면 React 가 그 시점의 값을 prev 로 건네줍니다.
- 5 함수형은 앞의 결과가 뒤로 이어집니다. 두 번 부르면 2가 옵니다.
- 6 지금 값을 바탕으로 다음 값을 만들 때는 함수형을 쓰세요. 상관없는 값을 넣을 때는 그냥 값으로 충분합니다.
- 7 갱신 함수는 계산해서 **값을 돌려주는 일만** 합니다.

```
export default function Concept05() {
  return (
    <div> <h1>개념 05 - 이전 값으로 갱신하기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일은 <strong>"분명히 맞게 썼는데 값이 이상한"</strong> 경우를 다룹니다.
      앞부분의 예제는 일부러 틀린 코드입니다.
      </p> <div> <TwiceBroken /> <ReadAfterSet /> <WhyOnlyOne /> <FunctionalUpdate
      /> <PrevOthers /> <WrongFunctional /> </div> </div>
    );
  }
}
```

직접 해보기 정답

```
1) function addTwice() {

  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);

}
// 화면(누르면): 개수: 1
```

→ 세 줄 모두 setCount(0 + 1) 입니다. 몇 줄을 적어도 결과는 1입니다.

```
2) console.log("② set 부른 직후:", count + 1);  
// 콘솔: ② set 부른 직후: 1
```

→ 화면에 보이는 다음 값과 같습니다.

set 에 넣은 값이 count + 1 이었으니 당연합니다.
다음 값이 필요하면 이렇게 미리 계산해서 쓰면 됩니다.

3) 4가 나옵니다.

```
// 콘솔: 이번 렌더에서 count + 1 은: 4  
// 콘솔: 한 줄 아래에서 또 계산해도: 4
```

→ 세 번 눌러 count 가 3이 된 상태에서 다시 누른 것이므로 3 + 1 입니다.

두 줄 모두 같은 값이라는 점이 핵심입니다.

```
4) function addTwice() {
```

```
  setCount((prev) => prev + 1);  
  setCount((prev) => prev + 1);
```

```
}  
// 화면(누르면): 개수: 2
```

→ 한 번 누를 때마다 2씩 오릅니다. 두 번 누르면 4입니다.

5) 8이 됩니다.

```
function doubleTwice() {
```

```
  setValue((prev) => prev * 2);  
  setValue((prev) => prev * 2);  
  setValue((prev) => prev * 2);
```

```
}  
// 화면(누르면): 불: 꺼짐 / 값: 8
```

→ 1 → 2 → 4 → 8 입니다.

값을 넣는 방식(setValue(value * 2))으로 세 줄을 썼다면 2에서 멈춥니다.

state 규칙

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

버튼을 눌러 보세요. F12 → Console 도 함께.

개념02~05에서 다룬 state 는 숫자·문자열·참거짓이었습니다. 이런 값은 바꿀 방법이 set 함수밖에 없습니다. 그래서 실수할 여지가 적었습니다.

배열과 객체는 다릅니다. JS자료에서 배운 방법이 많습니다. push, pop, splice, sort, reverse, `user.name = "..."` ...

그 방법들을 state 에 그대로 쓰면 어떻게 될까요? 에러가 나면 차라리 좋겠는데, 에러가 안 납니다. 콘솔은 조용하고 화면만 그대로입니다. 이 파일에서 그 장면을 직접 봅니다.

배열 state 에 push 하면 어떻게 되나

섹션 1

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function PushBroken() {
  const [items, setItems] = useState(["아메리카노"]);

  function addWrong() {
    items.push("라떼"); // ← state 를 직접 고쳤습니다 console.log("배열은 이렇게 바뀌었습니다:", items);
  }
}
```

F12 콘솔 배열은 이렇게 바뀌었습니다: ['아메리카노', '라떼']

```
console.log("길이도 늘었습니다:", items.length);
```

F12 콘솔 길이도 늘었습니다: 2

```
setItems(items); // set 도 불러 봅니다 console.log("setItems 까지 불렀는데 화면은 그대로입니다");
```

F12 콘솔 setItems 까지 불렀는데 화면은 그대로입니다

```
}

return (
  <div className="demo"> <h3>① push 로 담기 - 콘솔과 화면이 어긋납니다
```

```

</h3> <div className="output" id="pushOut">
  담긴 것: {items.join(", ")} (모두 {items.length}개)
</div> <button id="btnPushWrong" onClick={addWrong}>
  라떼 담기 (잘못된 방법)
</button> </div>
);
}

```

화면(누르면): 담긴 것: 아메리카노 (모두 1개)

콘솔은 2개라고 하고 화면은 1개라고 합니다. 둘이 어긋났습니다. 빨간 글씨도 없고 노란 경고도 없습니다. 이게 이 실수의 무서운 점입니다.

join 은 JS자료 06단원에서 배운 것입니다. 배열을 글자로 이어 붙입니다. 목록을 여러 줄로 그리는 방법은 05단원에서 배웁니다.

▣ 직접 해보기

1

'라떼 담기' 를 세 번 눌러 보세요. 콘솔의 길이는 몇이 되고, 화면의 개수는 몇으로 남을까요?

객체 state 도 마찬가지로입니다

섹션 2

```

function MutateObject() {
  const [user, setUser] = useState({ name: "김민준", age: 20 });

  function birthdayWrong() {
    user.age = user.age + 1; // ← 속성에 직접 대입했습니다 console.log("객체는 이렇게 바뀌었습니다:", user);
  }

```

F12 콘솔 객체는 이렇게 바뀌었습니다: { name: '김민준', age: 21 }

```

setUser(user);

console.log("setUser 도 불렀지만 화면의 나이는 그대로입니다");

```

F12 콘솔 setUser 도 불렀지만 화면의 나이는 그대로입니다

```

}

return (
  <div className="demo"> <h3>② 속성에 직접 대입 - 역시 화면이 안 바뀝니다
</h3> <div className="output" id="mutateOut">
  {user.name} ({user.age}세)
</div> <button id="btnMutateWrong" onClick={birthdayWrong}>
  생일 축하 (잘못된 방법)
</button> </div>

```

```
);
}
```

화면(누르면): 김민준 (20세)

개념04 섹션 4에서 { ...user, age: ... } 로 새 객체를 만들었던 이유가 이것입니다. 그때는 “그렇게 한다” 고만 했는데, 이제 안 그러면 어떻게 되는지 봤습니다.

▹ 직접 해보기 2

이름을 직접 바꾸는 버튼을 만들어 보세요. user.name = “이서연”; setUser(user); 두 줄이면 됩니다. 화면이 바뀔까요?

왜 화면이 안 바뀌나

섹션 3

개념03 섹션 3에서 배웠습니다. React 는 새 값이 지금 값과 같으면 다시 그리지 않습니다.

그럼 push 로 내용을 바꿨는데 왜 ‘같은 값’ 일까요? JS자료 06단원 개념01의 이야기가 여기서 그대로 쓰입니다. 배열과 객체를 담은 변수 안에는 값이 아니라 ‘주소’가 들어 있습니다. 그리고 === 는 그 주소를 비교합니다.

```
const list1 = ["아메리카노"];
const list2 = list1; // 주소를 복사했을 뿐, 배열은 하나입니다

list2.push("라떼");

console.log("list1 도 같이 바뀔니다:", list1);
```

F12 콘솔 list1 도 같이 바뀔니다: ['아메리카노', '라떼']

```
console.log("주소가 같으니 === 는:", list1 === list2);
```

F12 콘솔 주소가 같으니 === 는: true

이번에는 스프레드로 새 배열을 만들어 봅시다. (JS자료 09단원 개념03)

```
const list3 = [...list1];

console.log("내용은 똑같습니다:", list3);
```

F12 콘솔 내용은 똑같습니다: ['아메리카노', '라떼']

```
console.log("그래도 주소가 다르니 === 는:", list1 === list3);
```

F12 콘솔 그래도 주소가 다르니 === 는: false

여기까지 오면 이유가 분명해집니다.

```
items.push("라떼") → 안의 내용은 바뀌지만 주소는 그대로
setItems(items) → React 가 예전 주소와 비교 → 같음 → "안 바뀌었네"
→ 다시 그리지 않음 → 화면 그대로
```

React 는 배열 안을 하나하나 들여다보지 않습니다. 주소만 비교합니다. 그래야 빠르기 때문입니다. 값이 천 개인 배열을 매번 전부 비교하면 화면이 느려집니다.

그래서 우리 쪽에서 규칙을 지켜야 합니다.

state 를 바꿀 때는 '새 배열 · 새 객체' 를 만들어 넣는다.

이 성질을 불변성(immutability)이라고 부릅니다. 이름은 어려워 보이지만 뜻은 "원본을 건드리지 않는다"입니다. JS자료에서 sort 와 reverse 를 쓸 때 이미 조심하던 그 이야기입니다.

▹ 직접 해보기

3

list3 에 "케이크" 를 push 한 뒤 list1 을 찍어 보세요. list1 에도 케이크가 들어갈까요?

고치는 방법 (맛보기)

섹션 4

배열에 추가할 때는 스프레드로 새 배열을 만듭니다.

```
setItems([...items, "라떼"]);
```

객체를 고칠 때도 스프레드로 새 객체를 만듭니다.

```
setUser({ ...user, age: user.age + 1 });
```

둘 다 JS자료 09단원에서 배운 문법입니다. 새로 배울 것이 없습니다.

```
function FixedVersion() {
  const [items, setItems] = useState(["아메리카노"]);
  const [user, setUser] = useState({ name: "김민준", age: 20 });

  function addRight() {
    const nextItems = [...items, "라떼"]; // 새 배열 console.log("새 배열을 만들었습니다:", nextItems);
```

F12 콘솔 새 배열을 만들었습니다: ['아메리카노', '라떼']

```
console.log("원본과 같은 배열인가?:", nextItems === items);
```

F12 콘솔 원본과 같은 배열인가?: false

false니까 '다른 배열' 입니다. 그래서 React 가 다시 그립니다.

```
setItems(nextItems);
}

function birthdayRight() {
  const nextUser = { ...user, age: user.age + 1 }; // 새 객체 console.log("새
객체를 만들었습니다:", nextUser);
```

F12 콘솔 새 객체를 만들었습니다: { name: '김민준', age: 21 }

```
setUser(nextUser);
}

return (
  <div className="demo"> <h3>③ 새로 만들어 넣기 - 이번에는 화면이 따라옵니다
</h3> <div className="output" id="fixedOut">
    담긴 것: {items.join(", ")} (모두 {items.length}개) / 손님: {user.name} (
    {user.age}세)
  </div> <button id="btnAddRight" onClick={addRight}>
    라떼 담기 (올바른 방법)
  </button> <button id="btnBirthdayRight" onClick={birthdayRight}>
    생일 축하 (올바른 방법)
  </button> </div>
);
}
```

화면(누르면): 담긴 것: 아메리카노, 라떼 (모두 2개) / 손님: 김민준 (20세)

짧게 쓰면 변수를 안 만들고 바로 넣어도 됩니다.

```
setItems([...items, "라떼"]);
```

위 코드에서 변수에 담은 것은 콘솔에 찍어 보려던 것뿐입니다.

여기서는 '추가' 만 봤습니다. 목록에서 빼기, 특정 항목만 고치기, 객체 안의 객체 고치기 같은 것은 07단원에서 제대로 다룹니다. 그때 filter 와 map 을 씁니다. 지금은 "새로 만들어 넣는다" 는 원칙만 챙기면 충분합니다.

◁ 직접 해보기 4

'케이크 담기' 버튼을 올바른 방법으로 만들어 보세요.

useState 처럼 use 로 시작하는 React 기능을 훅(Hook)이라고 부릅니다. 앞으로 useEffect(09단원), useRef(10단원) 같은 것을 더 배웁니다.

훅에는 지켜야 할 자리 규칙이 있습니다.

(1) 컴포넌트 함수 '안' 에서 부른다. 밖에서 부르면 안 된다. (2) 컴포넌트의 '맨 위' 에서 부른다.

if 문 안, for 문 안, 다른 함수 안에서 부르면 안 된다.

왜 그럴까요? 개념04 섹션 1에서 한 줄 짚었습니다. React 는 useState 를 '이름' 으로 구분하지 않습니다. 몇 번째로 불렀는지, 그 순서로 어느 값인지 알아냅니다.

첫 번째 useState → 0번 서랍
두 번째 useState → 1번 서랍

조건문 안에 훅이 있으면 어떤 날은 두 번, 어떤 날은 한 번 불립니다. 그러면 서랍 번호가 밀려서 다른 값이 튀어나옵니다.

실제로 이렇게 쓰면 React 가 알아채고 에러를 냅니다.

```
function Bad({ isMember }) {
  if (isMember) {
    const [point, setPoint] = useState(0); // ← 조건문 안
  }
  const [name, setName] = useState("김민준");
  ...
}
```

Error: Rendered fewer hooks than expected.

This may be caused by an accidental early **return** statement.

화면이 통째로 비어 버립니다. 그래서 이 파일에서는 재현하지 않습니다. 훅 순서가 왜 이렇게 동작하는지, 안쪽에서 무슨 일이 일어나는지는 10단원 개념04에서 자세히 다룹니다.

그럼 조건에 따라 다르게 하고 싶을 때는 어떻게 할까요? 훅은 위에서 그냥 부르고, '값' 만 조건으로 정하면 됩니다.

```
function MemberPrice() {
```

훅은 조건과 상관없이 항상 맨 위에서 부릅니다.


```
const [isMember, setIsMember] = useState(false);
const [count, setCount] = useState(1);
```

조건은 혹은 아니라 값에만 씁니다. 이건 보통 변수입니다.

```
const price = isMember ? 3600 : 4000;
const total = price * count;
const memberText = isMember ? "회원" : "비회원";

function toggleMember() {
  setIsMember((prev) => !prev);
  console.log("회원 여부를 뒤집었습니다");
}
```

F12 콘솔 회원 여부를 뒤집었습니다

```
}

function addOne() {
  setCount((prev) => prev + 1);
  console.log("한 잔 더 담았습니다");
}
```

F12 콘솔 한 잔 더 담았습니다

```
}

return (
  <div className="demo"> <h3>④ 조건은 혹은 아니라 값에 씁니다
</h3> <div className="output" id="memberOut">
  {memberText} · 아메리카노 {price}원 × {count}잔 = {total}원
</div> <button id="btnMember" onClick={toggleMember}>
  회원/비회원 바꾸기
</button> <button id="btnMemberAdd" onClick={addOne}>
  한 잔 더
</button> </div>
);
}
```

화면(누르면): 회원 · 아메리카노 3600원 × 1잔 = 3600원

useState 두 줄은 언제나 같은 순서로 불립니다. 바뀌는 것은 price 와 total 뿐인데, 둘은 혹은 아니라 보통 변수입니다. 개념04 섹션 5에서 본 “계산되는 값은 state 로 두지 않는다” 와 같은 이야기입니다.

갱신에 (prev) => 를 쓴 것은 개념05에서 배운 함수형 갱신입니다. 지금 값을 바탕으로 다음 값을 만들 때 안전한 방법입니다.

회원 가격을 3600원 대신 10% 할인으로 계산해 보세요. (4000 * 0.9 를 쓰면 됩니다)

자주 하는 실수

섹션 6

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이런 실수를 합니다

배열 state 를 원본째 바꾸는 메소드로 고쳤다 (섹션 1)

```
items.push(...) / items.pop() / items.splice(...) / items.sort() / items.reverse()
```

에러가 안 납니다. 화면만 안 바뀝니다. 특히 sort 와 reverse 는 "정렬해서 보여 주려던 것뿐" 인데 원본을 망칩니다. 원본을 안 건드리는 방법은 07단원에서 배웁니다.

이런 실수를 합니다

객체 state 의 속성에 직접 대입했다 (섹션 2)

```
user.name = "이서연";
```

역시 조용합니다. { ...user, name: "이서연" } 으로 새 객체를 만드세요.

이런 실수를 합니다

새로 만들긴 했는데 안쪽은 그대로다

```
const next = { ...user };
next.address.city = "부산"; // ← 안쪽 객체는 복사되지 않았습니다
```

스프레드는 한 겹만 복사합니다. 안쪽 객체는 주소를 그대로 나눠 씁니다. JS자료 09단원 개념04에서 본 그 이야기입니다. 중첩된 객체를 다루는 방법은 07단원 개념02에서 배웁니다.

이런 실수를 합니다

혹을 조건문·반복문 안에서 불렀다 (섹션 5)

```
if (isMember) { const [point, setPoint] = useState(0); }
```

Rendered fewer hooks than expected 에러가 나고 화면이 빕니다. 조건은 혹이 아니라 값에만 씁니다.

이런 실수를 합니다

컴포넌트가 아닌 곳에서 훅을 불렀다

```
function handleClick() {
  const [x, setX] = useState(0); // ← 이벤트 핸들러 안
}
```

버튼을 누르는 순간 이런 에러가 납니다. Error: Invalid hook call. Hooks can only be called inside of the body of a `function` component. 훅은 컴포넌트 함수의 몸통, 또는 `use` 로 시작하는 커스텀 훅 안에서만 부릅니다. 커스텀 훅은 10단원에서 배웁니다.

이런 실수를 합니다

state 를 고쳐 놓고 다른 state 를 바꿔서 화면을 맞췄다

```
items.push("라떼");
setCount(count + 1); // ← 다른 값을 바꿔 역자로 다시 그리자
```

화면이 맞는 것처럼 보여서 더 위험합니다. 다시 그리는 시점에 따라 어떤 때는 반영되고 어떤 때는 안 됩니다. 새 배열을 만들어 넣는 방법으로 고치세요.

이런 실수를 합니다

대괄호를 안 닫았다

```
setItems([...items, "라떼"]);
```

[SyntaxError] 입니다. 파일 전체가 안 돌아가고 화면이 통째로 빕니다.

PARSE_ERROR · Expected , or] but found)

메시지 아래 그림이 짚어 주는 줄부터 보면 됩니다.

로 열었으면 · 로 닫아야 합니다. 스프레드를 쓰면 괄호가 겹쳐서 자주 틀립니다.

마지막: 위에서 만든 컴포넌트를 한 화면에 모으기

정리

- 1 배열·객체 state 를 **직접 고치면** 에러 없이 화면만 안 바뀝니다. 가장 찾기 어려운 실수입니다.
- 2 React 는 예전 값과 새 값의 **주소**를 비교합니다. push 는 주소를 바꾸지 않습니다.
- 3 바꿀 때는 **새 배열·새 객체**를 만들어 넣습니다. [...items, 새것] / { ...user, 바꿀것 }
- 4 이 원칙을 불변성이라고 부릅니다. 뜻은 “원본을 건드리지 않는다” 입니다.
- 5 빼기·고치기·중첩 객체는 07단원에서 제대로 다룹니다.
- 6 혹은 **컴포넌트 안, 맨 위에서만** 부릅니다. if-for·다른 함수 안에서 부르지 않습니다.
- 7 조건에 따라 달라져야 하면 **혹**이 아니라 **값**을 조건으로 정합니다.

```
export default function Concept06() {
  return (
    <div> <h1>개념 06 - state 규칙</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      <br /> <br />
      이 파일에서 다루는 실수는 <strong>에러가 나지 않습니다.</strong> 콘솔이
      조용한데 화면만 안 바뀝니다. 그래서 가장 찾기 어렵습니다.
      </p> <div> <PushBroken /> <MutateObject /> <FixedVersion /> <MemberPrice
    /> </div> </div>
  );
}
```

직접 해보기 정답

- 1) 콘솔의 길이는 4가 되고, 화면은 계속 1개입니다.

```
// 콘솔: 길이도 늘었습니다: 4
// 화면: 담긴 것: 아메리카노 (모두 1개)
```

→ 시작값이 한 개였으니 세 번 누르면 배열은 4개입니다.

화면은 첫 렌더링 그대로라 1개에서 멈춰 있습니다.

```
2) function renameWrong() {
```

```
  user.name = "이서연";
  setUser(user);
```

```
}

```

... <button onClick={renameWrong}>이름 바꾸기 (잘못된 방법)</button>

```
// 화면(누르면): 김민준 (20세)

```

→ 안 바뀝니다. 주소가 그대로라 React 가 같은 값으로 보기 때문입니다.

3) 안 들어갑니다.

```
list3.push("케이크");
console.log(list1);
// 콘솔: ['아메리카노', '라떼']

```

→ list3 는 스프레드로 만든 '다른' 배열입니다.

```
list2.push 때와 결과가 다른 이유가 바로 이것입니다.

```

4) `function addCake() {`

```
  setItems([...items, "케이크"]);

```

```
}

```

... <button onClick={addCake}>케이크 담기</button>

```
// 화면(누르면): 담긴 것: 아메리카노, 케이크 (모두 2개) / 손님: 김민준 (20세)

```

→ `items.push("케이크")` 로 쓰면 섹션 1처럼 조용히 실패합니다.

```
5) const price = isMember ? 4000 * 0.9 : 4000;
// 화면(누르면): 회원 · 아메리카노 3600원 × 1잔 = 3600원

```

→ $4000 * 0.9$ 는 3600 입니다. 결과가 같습니다.

다만 할인율이 바뀔 때 고칠 곳이 한 군데로 줄어듭니다.

state와 이벤트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 눌러 보세요. F12 → Console 도 함께.

```
import { useState } from "react";

console.log("연습문제 파일을 열었습니다. TODO 를 채워 보세요.");
```

F12 콘솔 연습문제 파일을 열었습니다. TODO 를 채워 보세요.

문제 1 (개념01)

버튼을 누르면 콘솔에 “안녕하세요, 김민준님!” 이 찍히게 하세요. handleClick 함수는 이미 만들어져 있습니다. onClick 만 붙이면 됩니다.

기대 콘솔 버튼을 누를 때마다 → 안녕하세요, 김민준님!
페이지를 열자마자 한 번 찍히고 눌러도 조용하면
괄호를 붙인 것입니다.

```
function Q1() {
  function handleClick() {
    console.log("안녕하세요, 김민준님!");
  }

  return (
    <div className="demo"> <h3>문제 1 – 버튼에 이벤트 붙이기</h3>
    { /* TODO: 아래 button 에 onClick 을 붙이세요 */ }
    <button id="q1Btn">인사하기</button> </div>
  );
}
```

문제 2 (개념01)

버튼 두 개가 greet 함수 하나를 같이 쓰게 하세요. 김민준 버튼은 “김민준” 을, 이서연 버튼은 “이서연” 을 넘깁니다.

기대 콘솔 김민준 버튼 → 김민준님 반갑습니다
 이서연 버튼 → 이서연님 반갑습니다
 페이지를 열 때 두 줄이 이미 찍혔다면
 화살표 함수로 감싸지 않은 것입니다.

```
function Q2() {
  function greet(name) {
    console.log(`${name}님 반갑습니다`);
  }

  return (
    <div className="demo"> <h3>문제 2 - 인자 넘기기</h3>
    { /* TODO: 두 버튼에 각각 onClick 을 붙이세요 */ }
    <button id="q2BtnA">김민준</button> <button id="q2BtnB">이서연</button> </div>
  );
}
```

문제 3 (개념02)

useState 로 count 를 만들고, 버튼을 누를 때마다 1씩 올려 화면에 보여 주세요.

기대 화면 처음 "답은 개수: 0", 한 번 누르면 1, 세 번 누르면 3
 콘솔의 숫자만 올라가고 화면이 0 그대로면
 보통 변수로 만든 것입니다.

```
function Q3() {
```

여기에 useState 를 쓰세요

```
return (
  <div className="demo"> <h3>문제 3 - 카운터 만들기
</h3> <div className="output" id="q3Out">
  답은 개수: (여기에 count 를 넣으세요)
</div>
  { /* TODO: 이 버튼에 onClick 을 붙이세요 */ }
  <button id="q3Btn">+1</button> </div>
);
}
```

문제 4 (개념02)

케이크가 6조각 있습니다. '한 조각 먹기'는 1씩 줄이고, '새로 굽기'는 다시 6으로 되돌리세요.

기대 화면 처음 "남은 케이크: 6조각", 두 번 먹으면 4조각,
'새로 굽기'를 누르면 다시 6조각
'새로 굽기'가 안 먹히면 시작값을 다시 넣지 않은 것입니다.
useState(6)이 적혀 있다고 저절로 6으로 돌아가지 않습니다.

```
function Q4() {
```

시작값이 6인 state를 만드세요

```
  return (  
    <div className="demo"> <h3>문제 4 - 시작값과 되돌리기  
  </h3> <div className="output" id="q4Out">  
    남은 케이크: (여기에 개수)조각  
  </div>  
    { /* TODO: 두 버튼에 onClick을 붙이세요 */ }  
    <button id="q4BtnEat">한 조각 먹기</button> <button id="q4BtnBake">새로 굽기  
  </button> </div>  
  );  
}
```

문제 5 (개념03)

컴포넌트 함수의 맨 위(return보다 위)에 console.log("Q5그리는 중")을 넣으세요. 그리고 버튼을 눌러 콘솔 줄이 몇 개씩 늘어나는지 확인하세요.

기대 콘솔 페이지를 열 때 1줄, 버튼을 누를 때마다 1줄씩 늘어납니다.
안 늘어나면 return 아래에 적었거나
버튼이 state를 바꾸고 있지 않은 것입니다.

```
function Q5() {  
  const [count, setCount] = useState(0);
```


여기에 `console.log` 를 한 줄 쓰세요

```
return (
  <div className="demo"> <h3>문제 5 - 다시 그려지는 횡수 세기
</h3> <div className="output" id="q5Out">
  누른 횡수: {count}
</div> <button id="q5Btn" onClick={() => setCount(count + 1)}>
    +1
  </button> </div>
);
}
```

04

문제 6 (개념 04)

state 를 두 개 두어 아메리카노와 라떼를 따로 세게 하세요.

기대 화면 아메리카노만 한 번 누르면
 "아메리카노 1잔 / 라떼 0잔"
 한쪽을 누를 때 다른 쪽 숫자도 움직이면
 state 하나를 같이 쓰고 있는 것입니다.

```
function Q6() {
```

state 두 개를 만드세요

```

return (
  <div className="demo"> <h3>문제 6 – state 두 개
</h3> <div className="output" id="q6Out">
    아메리카노 (여기)잔 / 라떼 (여기)잔
  </div>
  { /* TODO: 두 버튼에 onClick 을 붙이세요 */}
  <button id="q6BtnA">아메리카노 +1</button> <button id="q6BtnB">라떼
+1</button> </div>
);
}

```

문제 7 (개념 04)

문자열 state 를 만들어, 누른 버튼의 메뉴 이름이 화면에 나오게 하세요. 버튼은 세 개지만 함수는 하나만 만드세요.

기대 화면 케이크를 누르면 "오늘의 메뉴: 케이크"
 버튼마다 state 를 따로 만들었다면 다시 생각해 보세요.
 한 번에 하나만 고르는 값입니다.

```
function Q7() {
```

문자열 state 를 만드세요

```

return (
  <div className="demo"> <h3>문제 7 – 문자열
state</h3> <div className="output" id="q7Out">
    오늘의 메뉴: (여기)
  </div>
  { /* TODO: 세 버튼에 onClick 을 붙이세요 */}
  <button id="q7BtnA">아메리카노</button> <button id="q7BtnB">라떼
</button> <button id="q7BtnC">케이크</button> </div>
);
}

```

문제 8 (개념 04)

참/거짓 state 로 불을 켜고 끄세요. 화면 글자와 버튼 글자가 함께 바뀌어야 합니다.

기대 화면 처음 "불이 꺼져 있습니다" / 버튼은 "켜기"
 누르면 "불이 켜졌습니다" / 버튼은 "끄기"
 또 누르면 처음으로 돌아옵니다.
 한 번 켜지고 안 꺼지면 set 에 true 를 넣은 것입니다.

```
function Q8() {
```

참/거짓 state 를 만들고, 화면에 쓸 글자도 만드세요

```
    return (
      <div className="demo"> <h3>문제 8 - 켜고 끄기
    </h3> <div className="output" id="q8Out">
      (여기에 상태 글자)
    </div>
    { /* TODO: 버튼에 onClick 을 붙이고 글자도 바뀌게 하세요 */ }
    <button id="q8Btn">켜기</button> </div>
  );
}
```

문제 9 (개념 04)

아메리카노(4000원)와 케이크(6000원)를 담습니다. 합계 금액을 화면에 보여 주세요. 단, 합계는 state 로 만들지 마세요.

기대 화면 아메리카노를 두 번, 케이크를 한 번 누르면
 "아메리카노 2개 + 케이크 1개 = 14000원"
 합계를 state 로 두면 언젠가 개수와 어긋납니다.

```
function Q9() {
  const [americano, setAmericano] = useState(0);
  const [cake, setCake] = useState(0);
```

합계를 계산하는 보통 변수를 만드세요

```
return (
  <div className="demo"> <h3>문제 9 - 합계는 계산해서
</h3> <div className="output" id="q9Out">
  아메리카노 {americano}개 + 케이크 {cake}개 = (여기에 합계)원
</div> <button id="q9BtnA" onClick={() => setAmericano(americano + 1)}>
  아메리카노 담기
</button> <button id="q9BtnC" onClick={() => setCake(cake + 1)}>
  케이크 담기
</button> </div>
);
}
```

문제 10 (개념 05)

버튼을 한 번 누르면 2가 오르게 하세요. 단, `setCount(count + 2)` 는 쓰지 말고 `set` 함수를 두 번 부르세요.

기대 화면 한 번 누르면 "값: 2", 두 번 누르면 "값: 4"
1씩만 오르면 함수형 갱신을 쓰지 않은 것입니다.

```
function Q10() {
  const [count, setCount] = useState(0);

  function handleClick() {
```

`set` 함수를 두 번 불러 2가 오르게 하세요

```
}

return (
  <div className="demo"> <h3>문제 10 - 한 번에 2 올리기
</h3> <div className="output" id="q10Out">
```

```

        값: {count}
      </div> <button id="q10Btn" onClick={handleClick}>
        +2
      </button> </div>
    );
  }

```

문제 11 (개념05)

setCount 를 부른 바로 다음 줄에 `console.log("set 직후:", count)` 를 넣으세요. 화면의 숫자와 콘솔의 숫자를 비교해 보세요.

기대 콘솔 처음 눌렀을 때 → set 직후: 0
 기대 결과 (화면): 같은 순간 화면은 "값: 1"
 콘솔에도 1이 나왔다면 count 가 아니라
 count + 1 을 찍은 것입니다.

```

function Q11() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

```

여기에 `console.log` 를 한 줄 쓰세요

```

  }

  return (
    <div className="demo"> <h3>문제 11 – set 직후의 값
  </h3> <div className="output" id="q11Out">
    값: {count}
  </div> <button id="q11Btn" onClick={handleClick}>
    +1
  </button> </div>
  );
}

```

문제 12 (개념06)

객체 state 의 age 만 1 올리세요. name 은 그대로 남아야 합니다.

기대 화면 한 번 누르면 "김민준 (21세)"
이름 자리가 비면 스프레드(...user)를 빠뜨린 것입니다.
나이가 안 바뀌면 user.age = ... 처럼 직접 고친 것입니다.

```
function Q12() {  
  const [user, setUser] = useState({ name: "김민준", age: 20 });  
  
  function birthday() {
```

새 객체를 만들어 setUser 에 넣으세요

```
  }  
  
  return (  
    <div className="demo"> <h3>문제 12 – 객체 state 올바르게 고치기  
</h3> <div className="output" id="q12Out">  
      {user.name} ({user.age}세)  
</div> <button id="q12Btn" onClick={birthday}>  
      생일 축하  
</button> </div>  
  );  
}
```

문제 13 [응용] (개념04·05)

커피 주문 화면을 만드세요. state 세 개가 필요합니다. ① 고른 메뉴 (문자열) — 아메리카노 4000 / 라떼 4500 ② 잔 수 (숫자) — '한 잔 더' 로 늘립니다 ③ 포장 여부 (참/거짓) — 포장이면 500원을 더합니다
합계는 state 로 두지 말고 계산해서 보여 주세요.

기대 화면 라떼를 고르고 '한 잔 더' 를 한 번 누르고 포장을 켜면
 "라떼 · 2잔 · 포장 · 9500원"
 (4500 × 2 + 500)
 포장을 껴줄 때 500원이 안 빠지면
 포장료를 state 에 더해 둔 것입니다.

```
function Q13() {
```

state 세 개를 만드세요

가격과 합계를 계산하는 보통 변수를 만드세요

```
return (
  <div className="demo"> <h3>문제 13 [응용] – 커피 주문
</h3> <div className="output" id="q13Out">
  (메뉴) · (잔 수)잔 · (포장 여부) · (합계)원
</div>
  { /* TODO: 각 버튼에 onClick 을 붙이세요 */ }
  <button id="q13BtnA">아메리카노</button> <button id="q13BtnL">라떼
</button> <button id="q13BtnPlus">한 잔 더</button> <button id="q13BtnPack">포장
바꾸기</button> </div>
);
}
```

문제 14 [도전] (개념05·06)

주문 기록을 배열 state 에 쌓으세요. '한 잔 담기' → 배열에 "아메리카노" 를 하나 추가 '두 잔 담기' → 한 번 눌러서 두 개가 추가되어야 합니다 '비우기' → 빈 배열로 되돌립니다 화면에는 join(", ") 으로 이어 붙인 목록과 개수를 보여 주세요.

힌트: 배열을 새로 만들어야 화면이 바뀝니다. (개념06)

한 번에 두 개를 넣으려면 함수형 갱신이 필요합니다. (개념05)

기대 화면 '한 잔 담기' 한 번 → "아메리카노 (모두 1개)"
이어서 '두 잔 담기' 한 번 → 모두 3개
'두 잔 담기' 에서 1개만 늘면 함수형 갱신을 안 쓴 것입니다.
아무것도 안 늘면 push 로 직접 고친 것입니다.

```
function Q14() {  
  const [orders, setOrders] = useState([]);  
  
  function addOne() {
```

“아메리카노” 를 하나 추가하세요

```
}
```

```
function addTwo() {
```

한 번에 두 개가 추가되게 하세요

```
}
```

```
function clear() {
```

빈 배열로 되돌리세요

```
}
```

```
return (  
  <div className="demo"> <h3>문제 14 [도전] – 주문 기록 쌓기  
</h3> <div className="output" id="q14Out">  
    {orders.join(", ")} (모두 {orders.length}개)
```


조건부 렌더링과 리스트

OPEN 05_조건부_렌더링과_리스트

05

```
const isLoggedInDemo = false;
function LoginGreeting() {
  return (
    <div className="demo">
```

-
- 01 조건부 렌더링
 - 02 리스트 그리기
 - 03 key
 - 04 걸러서 그리기
 - 05 비었을 때와 falsy 함정
 - 연습문제

조건부 렌더링

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

04단원에서 state 로 '값' 을 바꿨습니다. 숫자가 0 에서 1 로 바뀌고, 화면의 그 자리 글자도 따라 바뀌었습니다.

이번에는 값이 아니라 '화면 조각 자체' 를 바꿉니다.

로그인 전 → "로그인해 주세요"
로그인 후 → "김민준님 안녕하세요"

글자만 다른 게 아니라, 상황에 따라 아예 다른 태그가 나와야 할 때도 있고 어떤 조각은 아예 안 보여야 할 때도 있습니다. 이렇게 조건에 따라 다른 화면을 그리는 것을 '조건부 렌더링' 이라고 합니다.

새로 배우는 문법은 거의 없습니다. JS자료 03단원에서 배운 삼항 연산자와 && 를 그대로 씁니다. 달라지는 것은 '그 식이 돌려주는 값이 화면 조각' 이라는 점뿐입니다.

JSX 안에서는 if 를 못 쓴다 — 삼항 연산자

섹션 1

02단원에서 중괄호 `{}` 안에는 '값' 이 들어간다고 배웠습니다. 변수도 값이고, 계산식도 값이고, 함수를 부른 결과도 값입니다.

그런데 if 는 값이 아닙니다. if 는 '문장' 입니다. 문장은 실행만 될 뿐 남는 값이 없어서 중괄호 안에 넣을 수 없습니다.

대신 삼항 연산자를 씁니다. 삼항은 '값' 을 돌려주기 때문입니다.

조건 ? 참일_때_값 : 거짓일_때_값

JS자료 03단원에서 글자를 고를 때 썼던 그 문법입니다. 먼저 콘솔로 확인해 봅시다. 여기까지는 React 와 아무 상관 없는 자바스크립트입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

const isLoggedInDemo = false;

console.log(isLoggedInDemo ? "김민준님 안녕하세요" : "로그인해 주세요");
```

F12 콘솔 로그인해 주세요

이제 돌려주는 값을 글자 대신 '화면 조각'으로 바꾸기만 하면 됩니다.

```
{isLoggedIn ? <p>김민준님 안녕하세요</p> : <p>로그인해 주세요</p>}
```

삼항이 하는 일은 똑같습니다. 둘 중 하나를 골라 돌려줍니다. 고른 것이 글자가 아니라 태그일 뿐입니다.

```
function LoginGreeting() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <div className="demo"> <h3>섹션 1 - 삼항 연산자: 둘 중 하나를 고른다
    </h3> <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
      {isLoggedIn ? "로그아웃" : "로그인"}
    </button> <div className="output">
      {isLoggedIn ? <p>김민준님 안녕하세요</p> : <p>로그인해 주세요</p>}
    </div> </div>
  );
}
```

화면 버튼 [로그인] · 아래 상자에 "로그인해 주세요"

화면(누르면): 버튼 [로그아웃] · 아래 상자에 "김민준님 안녕하세요"

버튼 글자에도 삼항을 썼습니다. 삼항은 태그 안 어디에나 쓸 수 있습니다. 태그 자리든, 글자 자리든, 나중에 배울 속성 자리든 상관없습니다. "값이 들어갈 수 있는 자리면 삼항도 들어갈 수 있다"고 기억하세요.

▹ 직접 해보기

1

위 LoginGreeting의 삼항에서 참·거짓 자리를 서로 바꿔 보세요. 저장하면 처음 화면이 어떻게 달라질까요?

&& — 한쪽만 보여 줄 때

섹션 2

삼항은 "이거 아니면 저거"입니다. 그런데 "있으면 보여 주고, 없으면 그냥 없다"인 경우가 훨씬 많습니다. 장바구니 배지, 새 알림 표시, 예러 문구 같은 것들입니다.

이때 삼항을 쓰면 안 보여 줄 쪽에 적을 게 마땅치 않습니다.

```
{count > 0 ? <p>담긴 상품 {count}개</p> : null}
```

틀린 코드는 아닙니다. 다만 : null이 매번 붙어서 눈에 거슬립니다. 이럴 때 &&를 씁니다.

```
{count > 0 && <p>담긴 상품 {count}개</p>}
```

&&가 왜 이렇게 동작하는지는 JS자료 03단원에서 배웠습니다. 다시 확인해 봅시다.

코드	▶ F12 콘솔
<code>console.log(true && "보인다");</code>	▶ 보인다
<code>console.log(false && "보인다");</code>	▶ false

&& 는 왼쪽이 참이면 '오른쪽 값' 을 돌려주고, 왼쪽이 거짓이면 '왼쪽 값' 을 그대로 돌려줍니다.

그래서 {조건 && <p>...</p>} 는 조건이 참 → <p>...</p> 가 나온다 → 화면에 그려진다 조건이 거짓 → false 가 나온다 → false 는 화면에 안 그려진다

마지막 줄이 핵심입니다. React 는 false 를 받으면 아무것도 그리지 않습니다. 섹션 4에서 어떤 값들이 안 그려지는지 한 번에 정리합니다.

```
const cartCountDemo = 3;

console.log(cartCountDemo > 0 && `담긴 상품 ${cartCountDemo}개`);
```

F12 콘솔 담긴 상품 3개

```
function CartBadge() {
  const [count, setCount] = useState(0);

  return (
    <div className="demo"> <h3>섹션 2 - &&: 있을 때만 보인다</h3> <button onClick={() => setCount(count + 1)}>아메리카노 담기</button> <button onClick={() => setCount(0)}>비우기</button> <div className="output"> <p>장바구니</p>
      {count > 0 && <p>담긴 상품 {count}개</p>}
    </div> </div>
  );
}
```

화면 "장바구니" 한 줄만 보입니다

화면(누르면): "장바구니" 아래에 "담긴 상품 1개" 가 생깁니다 화면(비우기를 누르면): 다시 "장바구니" 한 줄만 남습니다

★ 왼쪽에 count 가 아니라 count > 0 이라고 쓴 것을 눈여겨보세요. count 만 쓰면 목록이 비었을 때 화면에 0 이 찍히는 함정이 있습니다. 섹션 6의 [실수 3] 에서 실제로 보여 주고, 개념05에서 자세히 다룹니다.

◦ 직접 해보기 2

CartBadge 안에 "3개가 넘으면 '많이 담았습니다' 가 보이는" 줄을 && 로 한 줄 더 넣어 보세요.

변수에 담아 두기

섹션 3

조건이 두 갈래면 삼항으로 충분합니다. 그런데 세 갈래, 네 갈래가 되면 삼항을 겹쳐 쓰게 되고 금방 읽기 어려워집니다.

```
{point >= 100 ? <p>VIP</p> : point >= 50 ? <p>일반</p> : <p>신규</p>}
```

이건 돌아가긴 합니다. 하지만 한 줄 안에 물음표가 두 개라 눈이 아픕니다.

이럴 때는 `return` 문 '밖'에서 미리 변수에 담아 두면 됩니다. `return` 밖은 그냥 평범한 함수 안에서라서 `if` 문을 마음껏 쓸 수 있습니다. `if` 를 못 쓰는 곳은 JSX 의 종괄호 안뿐입니다.

```
function GradeBox() {
  const [point, setPoint] = useState(0);
```

여기는 `return` 밖이라 `if` 를 써도 됩니다.

```
let badge; // 아직 아무것도 안 담긴 변수입니다
if (point >= 100) {
  badge = <p>VIP 회원입니다</p>;
} else if (point >= 50) {
  badge = <p>일반 회원입니다</p>;
} else {
  badge = <p>신규 회원입니다</p>;
}
```

화면 조각도 그냥 값입니다. 변수에 담아 뒀다가 나중에 꺼내 쓸 수 있습니다.

```
return (
  <div className="demo"> <h3>섹션 3 - 변수에 담아 두기: 갈래가 셋 이상일 때
</h3> <button onClick={() => setPoint(point + 30)}>30점 적립
</button> <button onClick={() => setPoint(0)}>점수 초기화
</button> <div className="output"> <p>이서연님 · {point}점</p>
    {badge}
  </div> </div>
);
}
```

화면 "이서연님 · 0점" 과 "신규 회원입니다"

화면(30점 적립을 두 번 누르면): "이서연님 · 60점" 과 "일반 회원입니다"

`return` 안이 아주 단순해졌습니다. `{badge}` 한 줄뿐입니다.

"무엇을 보여 줄지 고르는 일" 과 "어디에 놓을지 정하는 일" 이 나뉘었습니다. 조건이 복잡할수록 이 방법이 편합니다.

GradeBox 에 “200점 이상이면 VVIP 회원입니다” 갈래를 if 로 하나 더 넣어 보세요. (순서에 주의하세요)

아무것도 안 그리기

섹션 4

React 는 어떤 값을 받으면 화면에 아무것도 그리지 않습니다. 무엇이 그려지고 무엇이 안 그려지는지 한 번 확인하고 갑시다.

```
function NothingDemo() {
  return (
    <div className="demo"> <h3>섹션 4 - 아무것도 안 그려지는 값들
  </h3> <div className="output"> <p>
    대괄호 사이에 값을 넣었습니다 → [{null}] [{undefined}] [{false}]
    [{true}]
  </p> <p>
    이걸 그려줍니다 → [{0}] [{"글자"}]
  </p> </div> </div>
);
}
```

화면 대괄호 사이에 값을 넣었습니다 → [] [] [] []
 이걸 그려줍니다 → [0] [글자]

정리하면 이렇습니다.

`null · undefined · true · false` → 아무것도 안 그려진다 숫자 0 · 문자열 → 그대로 그려진다

앞 줄 네 개가 안 그려지는 덕분에 && 가 동작합니다. 뒷 줄의 0 때문에 개념05의 함정이 생깁니다. 지금은 “0 은 그려진다” 만 기억하세요.

여기서 가장 많이 쓰는 것이 `null` 입니다. “이 자리에 아무것도 놓지 않겠다” 를 분명하게 적는 방법입니다.

```
function Notice() {
  const [closed, setClosed] = useState(false);

  return (
    <div className="demo"> <h3>섹션 4 - null: 자리 자체를 비우기
  </h3> <button onClick={() => setClosed(!closed)}>
    {closed ? "공지 다시 보기" : "공지 닫기"}
  </button> <div className="output">
    {closed ? null : <p>오늘 케이크 20% 할인</p>}
    <p>메뉴판</p> </div> </div>
  );
}
```

화면 "오늘 케이크 20% 할인" 과 "메뉴판" 두 줄

화면(공지 닫기를 누르면): "메뉴판" 한 줄만 남습니다

{closed ? null : <p>...</p>} 대신 {!closed && <p>...</p>} 라고 써도 같습니다. 어느 쪽이든 됩니다. 읽기 좋은 쪽을 고르세요. 갈래가 둘 다 화면 조각이면 삼항, 한쪽이 '없음' 이면 && 가 보통 더 짧습니다.

▹ 직접 해보기

4

Notice 의 {closed ? null : <p>...</p>} 를 && 를 쓴 형태로 바꿔 보세요. 동작은 같아야 합니다.

일찍 return 하기

섹션 5

지금까지는 return 안에서 골랐습니다. 그런데 상황에 따라 화면 '전체' 가 통째로 달라져야 할 때도 있습니다.

회원 정보가 아직 없다 → "불러온 회원이 없습니다" 한 줄만 회원 정보가 있다 → 이름·나이·등급이 들어간 카드 전체

이 둘을 삼항 하나에 담으면 return 이 아주 길어집니다. 이럴 때는 함수 맨 위에서 조건을 확인하고 그냥 먼저 return 해 버립니다.

컴포넌트도 결국 함수입니다(03단원). 함수는 return 을 만나면 거기서 끝납니다. 그래서 아래 코드에서 user 가 없으면 두 번째 return 은 아예 실행되지 않습니다.

```
function ProfileCard({ user }) {
```

여기서 걸리면 아래는 실행되지 않습니다.

```
  if (!user) {
    return <p>불러온 회원이 없습니다</p>;
  }
```

여기까지 왔다는 것은 user 가 있다는 뜻입니다. 그래서 아래에서는 user 가 있는지 다시 확인할 필요가 없습니다.

```
  return (
    <div> <p>
      {user.name} ({user.age}세)
    </p> <p>{user.age} >= 19 ? "성인 회원" : "청소년 회원"</p> </div>
  );
}

function ProfileDemo() {
  const [user, setUser] = useState(null);
```

리스트 그리기

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

03단원에서 컴포넌트를 손으로 여러 번 썼습니다.

```
<MenuItem name="아메리카노" />
<MenuItem name="라떼" />
<MenuItem name="케이크" />
```

메뉴가 세 개일 때는 괜찮습니다. 그런데 서른 개면 어떨까요? 메뉴가 늘거나 줄 때마다 손으로 줄을 고쳐야 합니다.

데이터는 이미 배열에 들어 있습니다. 배열을 화면 조각으로 바꾸면 됩니다. 배열을 다른 배열로 바꾸는 도구를 여러분은 이미 압니다. map 입니다.

★ 이 파일에서 새로 배우는 것은 사실 하나뿐입니다. “map 이 돌려주는 값을 글자가 아니라 화면 조각으로 만든다.” map 자체는 JS자료 08단원 개념03에서 배운 그대로입니다.

map 은 이미 배웠습니다

섹션 1

먼저 아는 것부터 확인합시다. 여기까지는 React 와 아무 상관 없습니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

const menuNames = ["아메리카노", "라떼", "케이크"];

console.log(menuNames.map((name) => `${name} 있습니다`));
```

F12 콘솔 ['아메리카노 있습니다', '라떼 있습니다', '케이크 있습니다']

개수는 그대로입니다. 3개를 넣으면 3개가 나옵니다. (08단원에서 배운 그대로)

```
console.log(menuNames.length);
```

F12 콘솔 3

이제 돌려주는 값을 문자열 대신 화면 조각으로 바꿉니다. 화살표 함수가 ... 를 돌려주게만 하면 됩니다.

여기서 걸리는 부분이 하나 있습니다. “태그를 어떻게 돌려주지? 태그가 값인가?” 하는 것입니다.

값 맞습니다. 02단원 개념01에서 확인했습니다. JSX 는 결국 함수 호출 하나로 바뀝니다(Vite 가 `jsx(...)` 로 바꿔 줍니다). 함수를 부르면 결과가 나옵니다. 그 결과가 화면 조각이라는 값입니다.

그러니까 이렇게 봐도 됩니다.

```
(name) => `${name} 있습니다`    ← 문자열이라는 값을 돌려준다
(name) => <li>{name}</li>        ← 화면 조각이라는 값을 돌려준다
```

`map` 입장에서는 둘이 똑같습니다. 무엇을 돌려주든 그걸 모아 새 배열을 만듭니다. 여러분이 08단원에서 숫자를 문자열로, 객체를 값으로 바꿔 본 것과 같은 일입니다.

```
const menuRows = menuNames.map((name) => <li key={name}>{name}</li>);
```

결과도 여전히 ‘배열’ 입니다. 안에 든 것이 글자에서 화면 조각으로 바뀌었을 뿐입니다.

코드	▶ F12 콘솔
<code>console.log(Array.isArray(menuRows));</code>	▶ true
<code>console.log(menuRows.length);</code>	▶ 3

React 는 화면 조각이 든 배열을 받으면 하나씩 순서대로 그려 줍니다. 그래서 중괄호 안에 배열을 그냥 넣으면 됩니다.

```
function FirstList() {
  return (
    <div className="demo"> <h3>섹션 1 - 배열을 화면에 그리기
  </h3> <div className="output"> <ul>{menuRows}</ul> </div> </div>
  );
}
```

화면 아메리카노 / 라떼 / 케이크 세 줄

위처럼 변수에 담아 뒀다가 넣어도 되고, 아래처럼 중괄호 안에서 바로 `map` 을 불러도 됩니다. 결과는 똑같습니다. 실무에서는 아래쪽을 훨씬 많이 씁니다. 데이터 바로 옆에 화면 모양이 보이니까요.

```
function FirstListInline() {
  return (
    <div className="demo"> <h3>섹션 1 - 중괄호 안에서 바로
  map</h3> <div className="output"> <ul>
      {menuNames.map((name) => (
        <li key={name}>{name}</li>
      ))}
    </ul> </div> </div>
```

```
);  
}
```

화면 아메리카노 / 라떼 / 케이크 세 줄 (위와 같습니다)

★ 화살표 함수 뒤가 소괄호 (로 시작하는 것을 눈여겨보세요. 여러 줄에 걸쳐 화면 조각을 돌려줄 때는 소괄호로 감쌉니다. 02단원 개념05에서 `return` 뒤에 소괄호를 붙인 것과 같은 이유입니다. 중괄호 { 로 시작하면 '함수 몸통' 이 되어 버려서 `return` 을 꼭 써야 합니다. 이걸 빠뜨리는 실수는 섹션 5에서 실제로 보여 줍니다.

▫ 직접 해보기 1

menuNames 배열에 "삼각김밥" 을 하나 더 넣고 저장해 보세요. map 코드를 안 고쳐도 화면에 몇 줄이 나오나요?

배열 안의 객체 그리기

섹션 2

실제 데이터는 대부분 객체 배열입니다. JS자료 08단원에서 계속 다룬 모양입니다.

```
const menu = [  
  { id: "americano", name: "아메리카노", price: 4000 },  
  { id: "latte", name: "라떼", price: 4500 },  
  { id: "cake", name: "케이크", price: 6000 },  
  { id: "gimbap", name: "삼각김밥", price: 1200 },  
];
```

콘솔에서 이름만 뽑는 것은 이미 해 봤습니다.

```
console.log(menu.map((item) => item.name));
```

F12 콘솔 ['아메리카노', '라떼', '케이크', '삼각김밥']

화면에서는 한 항목에서 값을 여러 개 꺼내 쓸 수 있습니다. 콜백 안에서 `item.name`, `item.price` 를 각각 꺼내 놓으면 됩니다.

```
function MenuList() {  
  return (  
    <div className="demo"> <h3>섹션 2 - 객체 배열 그리기  
  </h3> <div className="output"> <ul>  
    {menu.map((item) => (  
      <li key={item.id}>  
        {item.name} - {item.price}원  
      </li>  
    ))}  
    </ul> </div> </div>
```

```
);
}
```

```
화면      아메리카노 - 4000원
          라떼 - 4500원
          케이크 - 6000원
          삼각김밥 - 1200원
```

key 자리에 item.id 를 넣었습니다. 데이터에 id 가 있으면 그걸 쓴다고만 기억하세요. 이유는 개념03입니다.

데이터만 바꾸면 화면이 따라온다는 것을 눈으로 봅시다. 아래 데모는 배열 두 개를 준비해 두고 버튼으로 갈아 끼웁니다. 화면을 그리는 map 코드는 한 벌뿐인데, 두 배열 모두 잘 그려집니다.

```
const todayMenu = [
  { id: "americano", name: "아메리카노", price: 4000 },
  { id: "latte", name: "라떼", price: 4500 },
];

const tomorrowMenu = [
  { id: "cake", name: "케이크", price: 6000 },
  { id: "gimbap", name: "삼각김밥", price: 1200 },
  { id: "americano", name: "아메리카노", price: 4000 },
];

function SwitchMenu() {
  const [list, setList] = useState(todayMenu);

  return (
    <div className="demo"> <h3>섹션 2 - 데이터만 갈아 끼우기</h3> <button onClick={() => setList(todayMenu)}>오늘 메뉴</button> <button onClick={() => setList(tomorrowMenu)}>내일 메뉴</button> <div className="output"> <ul>
      {list.map((item) => (
        <li key={item.id}>
          {item.name} - {item.price}원
        </li>
      ))}
    </ul> </div> </div>
  );
}
```

```
화면      아메리카노 - 4000원 / 라떼 - 4500원 (두 줄)
```

화면(내일 메뉴를 누르면): 케이크 / 삼각김밥 / 아메리카노 (세 줄)

줄 수가 2개에서 3개로 늘었는데 화면을 고치는 코드는 한 줄도 없습니다. 01단원에서 말한 '선언형' 이 바로 이것입니다.

JS자료 10단원에서 목록을 그릴 때를 떠올려 보세요. 목록을 다시 그리려면 먼저 있던 li들을 지우고 (innerHTML = "" 같은 것) 새 li를 하나씩 만들어 붙여야 했습니다. 지우는 것을 깜빡하면 줄이 쌓였습니다.

여기서는 그 두 가지가 다 없습니다. 우리가 적은 것은 "list가 이러면 화면은 이렇게 생겼다" 뿐입니다. 지울지 만들지 고칠지는 React가 판단합니다. 그 판단을 도우려고 붙이는 것이 key입니다. 개념03에서 바로 이어집니다.

▣ 직접 해보기

2

MenuList의 각 줄에 "(4자)"처럼 이름 글자 수도 같이 보이게 해 보세요. (item.name.length를 씁니다)

컴포넌트에 넘기기

섹션 3

한 줄이 길어지면 map안이 답답해집니다. 03단원에서 배운 대로 한 줄을 컴포넌트로 떼어 내면 됩니다.

언제 떼어 낼까요? 정해진 기준은 없지만 대체로 이럴 때입니다. · map안의 화면 조각이 대여섯 줄을 넘어갈 때 · 한 줄 안에 조건이 두 개 이상 들어갈 때 · 같은 모양의 줄을 다른 목록에서도 써야 할 때

반대로 {name}처럼 짧으면 굳이 떼어 내지 않습니다. 컴포넌트를 만들면 그만큼 이름을 하나 더 외워야 하니까요.

props를 매개변수 자리에서 구조분해로 받았습니다. (03단원 개념03)

```
function MenuItem({ name, price }) {
  return (
    <li>
      {name} - {price}원 {price >= 4500 ? "(비쌘)" : ""}
    </li>
  );
}

function MenuListWithComponent() {
  return (
    <div className="demo"> <h3>섹션 3 - 한 줄을 컴포넌트로
</h3> <div className="output"> <ul>
      {menu.map((item) => (
        <MenuItem key={item.id} name={item.name} price={item.price} />
      ))}
    </ul> </div> </div>
  );
}
```

화면	아메리카노 - 4000원
	라떼 - 4500원 (비쌘)
	케이크 - 6000원 (비쌘)
	삼각김밥 - 1200원

★ key 를 MenuItem 에 붙였습니다. MenuItem 안의 에 붙이는 게 아닙니다. key 는 “map 이 직접 돌려주는 것” 에 붙입니다. 이 규칙도 개념03에서 다룹니다.

객체를 통째로 넘기는 방법도 있습니다. 넘길 값이 많을 때 편합니다.

```
<MenuItem key={item.id} item={item} />
```

받는 쪽은 `function MenuItem({ item })` 으로 받고 `item.name` 처럼 씁니다. 어느 쪽이든 됩니다. 값이 두세 개면 따로 넘기는 쪽이 읽기 좋습니다.

▸ 직접 해보기 3

MenuItem 을 고쳐서 1200원 이하면 “(싸다)” 가 붙게 해 보세요. (삼항 연산자를 하나 더 씁니다)

목록 안에서 조건부 렌더링

섹션 4

개념01에서 배운 삼항과 && 를 map 안에서 그대로 쓸 수 있습니다. 목록의 각 줄이 서로 다르게 보여야 할 때 이 조합을 계속 씁니다.

```
const todos = [
  { id: 1, text: "우유 사기", done: true },
  { id: 2, text: "책 반납", done: false },
  { id: 3, text: "청소하기", done: false },
];

function TodoList() {
  return (
    <div className="demo"> <h3>섹션 4 – 줄마다 다르게 보이기
  </h3> <div className="output"> <ul>
    {todos.map((todo) => (
      <li key={todo.id} className={todo.done ? "done" : ""}>
        {todo.text} {todo.done && "(완료)}"
      </li>
    ))}
  </ul> </div> </div>
  );
}
```

화면 우유 사기 (완료) ← 가운데 줄이 그어져 있습니다
 책 반납
 청소하기

두 가지가 한꺼번에 일어났습니다.

```
className={todo.done ? "done" : ""} → 완료된 줄에만 done 스타일(줄긋기)
```

{todo.done && "(완료)"} → 완료된 줄에만 글자 추가

className 에 삼항을 쓰는 것은 02단원 개념03에서 배운 속성 자리 종괄호입니다. 조건에 따라 스타일을 바꾸는 가장 흔한 방법입니다.

★ 여기서 한 가지 짚고 갑시다. map 안이라고 해서 특별한 문법이 있는 게 아닙니다. 개념01에서 배운 것을 그대로 쓴 것뿐입니다. 달라진 점은 조건을 보는 대상이 '하나' 가 아니라 '줄마다' 라는 것입니다. 콜백 안의 todo 는 그 줄의 데이터입니다. 그래서 줄마다 다르게 판단됩니다.

JS자료 10단원에서 목록을 그릴 때 이런 코드를 썼습니다.

```
if (todo.done) li.classList.add("done");
```

그때는 "이 li 에 클래스를 붙여라" 라고 시켰습니다(명령형). 지금은 "완료면 이 줄의 클래스는 done 이다" 라고 적습니다(선언형). 붙이고 떼는 일은 React 가 합니다.

⇒ 직접 해보기

4

ToDoList 를 고쳐서 아직 안 한 일에만 "(할 일)" 이 붙게 해 보세요.

자주 하는 실수

섹션 5

⚠ [SyntaxError] 라고 적힌 것은 주석을 풀지 말고 눈으로만 보세요. 풀면 파일 전체가 멈춰서 화면이 통째로 비어 버립니다. 다시 // 를 붙이면 돌아옵니다.

이 섹션의 [실수 1] 과 [실수 2] 는 주석이 아니라 '실제로 도는 코드' 입니다. 에러가 하나도 안 나면서 목록만 사라집니다. 그래서 가장 찾기 어렵습니다.

```
function MistakeDemo() {
  return (
    <div className="demo"> <h3>섹션 5 – 조용히 비어 버리는 목록
  </h3> <div className="output"> <p>[실수 1] 종괄호를 쓰고 return 을 빠뜨림</p> <ul>
    {menuNames.map((name) => {
      <li key={name}>{name}</li>
    })}
  </ul> <p>[실수 2] map 이 아니라 forEach</p> <ul>
    {menuNames.forEach((name) => (
      <li key={name}>{name}</li>
    ))}
  </ul> <p>[정상] 소괄호로 감싼 map</p> <ul>
    {menuNames.map((name) => (
      <li key={name}>{name}</li>
    ))}
  </ul> </div> </div>
);
}
```

화면 [실수 1] 아래는 아무 줄도 없습니다
 [실수 2] 아래도 아무 줄도 없습니다
 [정상] 아래에만 아메리카노 / 라떼 / 케이크 세 줄이 나옵니다

이런 실수를 합니다

중괄호를 쓰고 `return` 을 빠뜨림

화살표 뒤에 중괄호를 쓰면 그 안은 '함수 몸통' 입니다. `return` 이 없으면 `undefined` 를 돌려줍니다. 그래서 `map` 의 결과는 `[undefined, undefined, undefined]` 가 됩니다. React 는 `undefined` 를 받으면 아무것도 안 그립니다(개념01 섹션 4). 에러도 경고도 없이 목록만 통째로 사라집니다. 고치는 법: 소괄호로 감싸거나 (권장), 중괄호를 쓰려면 `return` 을 적으세요. JS자료 08단원 개념03에서 본 그 실수가 그대로 나온 것입니다. 거기서는 `undefined` 가 콘솔에 보였지만, 여기서는 화면이 빌 뿐입니다.

이런 실수를 합니다

`map` 이 아니라 `forEach`

`forEach` 는 아무것도 안 돌려줍니다. 결과는 `undefined` 입니다. 이것도 화면에 아무것도 안 그려집니다. 고치는 법: 화면을 만들 때는 항상 `map` 입니다. "새 배열이 필요하면 `map`" 이라는 08단원의 기준이 그대로 적용됩니다.

이런 실수를 합니다

객체를 통째로 화면에 넣음 (런타임 에러 — 화면이 안 그려집니다)

```
<li key={item.id}>{item}</li>
```

콘솔에 이 에러가 납니다. Objects are not valid as a React child (found: object with keys {id, name, price}). If you meant to render a collection of children, use an array instead. "객체는 화면에 넣을 수 없습니다" 라는 뜻입니다. React 가 그릴 수 있는 것은 글자·숫자·화면 조각·그것들의 배열입니다.

고치는 법: 객체 안에서 필요한 값을 꺼내 쓰세요. `{item.name}` 처럼.

에러 문구에 나온 keys {id, name, price} 를 보면
 어느 객체를 잘못 넣었는지 바로 알 수 있습니다.

이런 실수를 합니다

`key` 를 안 붙임 (조용히 넘어가지만 콘솔에 경고가 납니다)

```
{menu.map((item) => <li>{item.name}</li>)}
```

화면은 멀쩡히 나옵니다. 그래서 그냥 지나치기 쉽습니다. 하지만 콘솔에 노란 경고가 뜨고, 나중에 목록이 바뀔 때 이상하게 동작합니다. 무엇이 어떻게 이상해지는지는 개념03에서 직접 재현해 봅니다.

전체 화면 그리기

정리

- 1 목록은 `map` 으로 그립니다. JS자료 08단원의 그 `map` 입니다.
- 2 달라진 것은 하나뿐입니다. 콜백이 돌려주는 값이 **글자가 아니라 화면 조각**입니다.
- 3 React 는 화면 조각이 든 배열을 받으면 순서대로 전부 그립니다.
- 4 여러 줄짜리 화면 조각을 돌려줄 때는 **소괄호**로 감쌉니다. 중괄호를 쓰면 `return` 이 필요합니다.
- 5 `return` 을 빠뜨리거나 `forEach` 를 쓰면 **에러 없이 목록만 사라집니다**. 목록이 비면 여기부터 보세요.
- 6 한 줄이 길어지면 컴포넌트로 떼어 내고 `props` 로 값을 넘깁니다.
- 7 `map` 안에서 삼항`&&` 를 그대로 쓸 수 있습니다.

```
export default function Concept02() {
  return (
    <div> <h1>개념 02 - 리스트 그리기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
    <br /> <br />
    이 파일에는 <code>key</code> 라는 것이 계속 나옵니다. <strong>무엇인지는
    개념03에서 배웁니다.</strong> 지금은 "목록을 그릴 때 같이 적는 것" 이라고만 알고
    넘어가세요.
    <p> <div> <FirstList /> <FirstListInline /> <MenuList /> <SwitchMenu
    /> <MenuListWithComponent /> <TodoList /> <MistakeDemo /> </div> </div>
  );
}

console.log("개념02 화면을 그렸습니다");
```

F12 콘솔 개념02 화면을 그렸습니다

직접 해보기 정답

```
1) const menuNames = ["아메리카노", "라떼", "케이크", "삼각김밥"];
// 화면: 네 줄이 나옵니다
```

→ `map` 코드는 한 글자도 안 고쳤습니다. 배열이 길어지면 줄도 따라 늘어납니다.

"몇 개인지" 를 코드에 적지 않았기 때문입니다.


```
2) <li key={item.id}>
```

```
{item.name} - {item.price}원 ({item.name.length}자)
```

```
</li>
// 화면: 아메리카노 - 4000원 (5자)
// 화면: 라떼 - 4500원 (2자)
// 화면: 케이크 - 6000원 (3자)
// 화면: 삼각김밥 - 1200원 (4자)
```

→ 중괄호 안에는 값이면 무엇이든 들어갑니다. 계산식도 됩니다(02단원).

```
3) function MenuItem({ name, price }) {
```

```
  return (
    <li>
      {name} - {price}원 {price >= 4500 ? "(비쌈)" : ""}
      {price <= 1200 ? "(싸다)" : ""}
    </li>
  );
```

```
}
// 화면: 삼각김밥 - 1200원 (싸다)
```

→ 삼항 두 개를 나란히 놓았습니다. 서로 영향을 주지 않습니다.

```
{price <= 1200 && "(싸다)"} 라고 && 로 써도 같습니다.
```

```
4) <li key={todo.id} className={todo.done ? "done" : ""}>
```

```
{todo.text} {todo.done ? "(완료)" : "(할 일)"}
```

```
</li>
// 화면: 우유 사기 (완료) / 책 반납 (할 일) / 청소하기 (할 일)
```

→ 둘 중 하나를 골라야 하므로 && 대신 삼항이 맞습니다.

```
{!todo.done && "(할 일)"} 을 한 줄 더 쓰는 방법도 있습니다.
```

key

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 을 먼저 열어 두세요.

개념02에서 `map` 을 쓸 때마다 `key` 라는 것을 같이 적었습니다. “지금은 그냥 적으세요” 하고 넘어갔습니다. 이제 그게 무엇인지 봅시다.

이 파일이 05단원에서 가장 중요합니다. `key` 를 대충 쓰면 에러가 안 납니다. 화면도 멀쩡히 나옵니다.

그런데 목록이 바뀌는 순간 엉뚱한 값이 납니다. 이런 종류의 버그는 원인을 모르면 며칠도 헤맬니다.

그래서 이 파일은 ‘말로 설명’ 하지 않고 전부 눈으로 보여 줍니다. 반드시 버튼을 직접 눌러 가며 읽으세요.

key 를 빼면 무슨 일이 일어나나

섹션 1

아래 컴포넌트는 `key` 를 일부러 뺐습니다. 화면은 멀쩡히 나옵니다. 아무 문제 없어 보입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function NoKeyList() {
  const names = ["아메리카노", "라떼", "케이크"];

  return (
    <ul>
      {names.map((name) => (
        <li>{name}</li>
      ))}
    </ul>
  );
}
```

이걸 페이지가 열리자마자 그리면 콘솔이 처음부터 지저분해집니다. 그래서 버튼을 눌렀을 때만 그리도록 했습니다. ★ 콘솔을 열어 두고 아래 데모의 “key 없이 그려 보기” 를 눌러 보세요.

```
function NoKeyDemo() {
  const [show, setShow] = useState(false);

  return (
    <div className="demo"> <h3>섹션 1 – key 를 빼면 나는 경고 (일부러 냅니다)
    </h3> <p>F12 → Console 을 먼저 열고 아래 버튼을 누르세요.</p> <button onClick={() =>
```

```
{show ? "감추기" : "key 없이 그려 보기"}
  </button> <div className="output">
    {show ? <NoKeyList /> : <p>아직 안 그렸습니다. 콘솔도 깨끗합니다.</p>}
  </div> </div>
);
}
```

화면 "아직 안 그렸습니다. 콘솔도 깨끗합니다."

화면(누르면): 아메리카노 / 라떼 / 케이크 세 줄 + 콘솔에 빨간 경고

콘솔에 아래 글이 그대로 나옵니다. 직접 실행해서 옮겨 적은 것입니다.

Each child in a list should have a unique "key" prop.

Check the render method of `NoKeyList`. See <https://reactjs.org/link/warning-keys> for more information.

```
at li
at NoKeyList
at div
at div
at NoKeyDemo
at div
at App
```

(at 로 시작하는 줄 뒤에는 코드 위치가 더 붙어 나오기도 합니다)

한 줄씩 읽어 봅시다.

"Each child in a list should have a unique key prop"

→ 목록의 각 자식은 겹치지 않는 key 를 가져야 합니다.

"Check the render method of `NoKeyList`"

→ `NoKeyList` 컴포넌트를 보라는 뜻입니다.
경고가 어느 컴포넌트에서 났는지 알려 주므로 여기부터 찾으려 합니다.

at li / at NoKeyList / at div ...

→ 그 아래는 어디에 들어 있는 컴포넌트인지 알려 주는 목록입니다.
맨 위 at li 가 문제가 난 태그입니다.

★ 이 경고는 컴포넌트마다 한 번만 나옵니다. “감추기”를 눌렀다가 다시 눌러도 두 번째부터는 안 나옵니다. 다시 보고 싶으면 왼쪽에서 다른 예제를 골랐다 돌아온 뒤 다시 누르세요.

이 경고는 빨간 글씨로 나오지만 페이지가 멈추지는 않습니다. 화면도 제대로 나옵니다. 그래서 그냥 넘어가기 쉽습니다. 넘어가면 어떻게 되는지를 섹션 4에서 봅니다.

▫ 직접 해보기 1

NoKeyList 의 `<li>` 에 `key={name}` 을 붙이고 저장한 뒤 왼쪽에서 다른 예제를 골랐다 돌아와 버튼을 다시 눌러 보세요. 경고가 사라지나요?

key 는 ‘이름표’ 입니다

섹션 2

React 는 화면을 다시 그릴 때 전부 새로 만들지 않습니다. 이전에 그린 것과 이번에 그릴 것을 나란히 놓고 비교해서, 달라진 곳만 고칩니다. 01단원에서 “바뀐 부분만 찾아서 고친다” 고 한 그 일입니다.

목록에서는 비교가 까다롭습니다. 줄이 세 개에서 두 개가 됐을 때, 어느 줄이 없어진 걸까요?

반 아이들 단체 사진에 비유해 봅시다. 이름표가 없으면 “왼쪽에서 두 번째” 처럼 자리로만 부를 수 있습니다. 한 명이 빠지면 뒤에 있던 아이들이 전부 한 칸씩 당겨집니다. 자리로만 부르면 “두 번째” 가 누구인지 사진마다 달라집니다. 이름표가 있으면 누가 빠졌는지 정확히 알 수 있습니다.

비유는 여기까지입니다. 실제로 일어나는 일은 이렇습니다.

key 가 있으면

- 이전 목록과 새 목록에서 같은 key 끼리 짝을 짓는다
- 짝이 있으면 그 화면 조각을 그대로 두고 달라진 값만 고친다
- 새 목록에 없는 key 는 화면에서 지운다

key 가 없으면

- 짝을 지을 이름이 없으니 ‘몇 번째냐’ 로만 비교한다
- 첫 번째는 첫 번째끼리, 두 번째는 두 번째끼리 맞춘다
- 앞쪽이 빠지면 전부 한 칸씩 밀려서 엉뚱한 것끼리 짝이 된다

key 는 React 만 쓰는 값입니다. · 화면(HTML)에는 나가지 않습니다. key 라는 속성은 생기지 않습니다. · 컴포넌트가 props 로 받을 수도 없습니다. props.key 는 `undefined` 입니다. · 값이 필요하다면 key 와 별개로 한 번 더 넘겨야 합니다.

```
<MenuItem key={item.id} id={item.id} name={item.name} />
```

```
function KeyIsNotVisible() {  
  const menu = [  

```

```

{ id: "americano", name: "아메리카노" },
  { id: "latte", name: "라떼" },
];

return (
  <div className="demo"> <h3>섹션 2 – key 는 화면에 안 나온다
</h3> <div className="output"> <ul id="keyProof">
  {menu.map((item) => (
    <li key={item.id}>{item.name}</li>
  ))}
</ul> </div> </div>
);
}

```

화면 아메리카노 / 라떼 두 줄 (key 값인 americano · latte 는 안 보입니다)

▹ 직접 해보기 2

F12 → Elements 탭에서 위 목록의 를 찾아보세요. key="americano" 같은 속성이 붙어 있나요?

05

무엇을 key 로 쓰나

섹션 3

좋은 key 의 조건은 두 가지뿐입니다.

(1) 형제끼리 겹치지 않는다

한 목록 안에서만 유일하면 됩니다. 페이지 전체에서 유일할 필요는 없습니다.
그래서 목록 A 와 목록 B 가 같은 key 를 써도 괜찮습니다.

(2) 다시 그려도 값이 안 바뀐다

같은 항목이면 몇 번을 다시 그려도 같은 key 가 나와야 합니다.
그래야 React 가 "아, 아까 그 항목이구나" 하고 알아봅니다.

이 두 조건을 가장 잘 만족하는 것이 데이터에 들어 있는 id 입니다. 서버에서 받아 오는 데이터에는 거의 항상 id 가 있습니다(JS자료 12단원 참고).

```

const users = [
  { id: 101, name: "김민준", age: 20 },
  { id: 102, name: "이서연", age: 22 },
  { id: 103, name: "박지훈", age: 28 },
];

```

id 가 없다면? 겹치지 않는 값이 이미 있는지 먼저 찾아보세요. 아래 메뉴 이름은 서로 겹치지 않으므로 이름을 key 로 써도 됩니다.

```
const menuNames = ["아메리카노", "라떼", "케이크", "삼각김밥"];
```

겹치는 값이 있는지는 filter 로 세어 보면 됩니다(JS자료 08단원).

```
console.log(menuNames.filter((name) => name === "아메리카노").length);
```

F12 콘솔 1

1 이므로 "아메리카노" 는 하나뿐입니다. 나머지도 마찬가지입니다.

이름이 겹치는 목록이라면 이름은 key 로 쓸 수 없습니다.

```
const withDuplicate = ["아메리카노", "라떼", "아메리카노"];
console.log(withDuplicate.filter((name) => name === "아메리카노").length);
```

F12 콘솔 2

2 입니다. 같은 이름이 두 개 있습니다. 이걸 key 로 쓰면 경고가 납니다.

key 는 숫자를 넣어도 되고 문자열을 넣어도 됩니다. React 는 안에서 문자열로 바뀌어서 씁니다. key={101} 과 key="101" 은 같습니다.

```
function KeyChoiceDemo() {
  return (
    <div className="demo"> <h3>섹션 3 - 무엇을 key 로 쓰나
  </h3> <div className="output"> <p>① id 가 있으면 id 를 씁니다</p> <ul>
    {users.map((user) => (
      <li key={user.id}>
        {user.name} ({user.age}세)
      </li>
    ))}
  </ul> <p>② id 가 없고 값이 겹치지 않으면 그 값을 씁니다</p> <ul>
    {menuNames.map((name) => (
      <li key={name}>{name}</li>
    ))}
  </ul> </div> </div>
  );
}
```

화면 김민준 (20세) / 이서연 (22세) / 박지훈 (28세)
아메리카노 / 라떼 / 케이크 / 삼각김밥

⇒ 직접 해보기 3

users 배열의 key 를 user.id 대신 user.name 으로 바꿔 보세요. 지금 데이터에서는 문제가 없습니다. 왜 그럴까요?

index 를 key 로 쓰면 생기는 일 ★

섹션 4

“id 가 없으면 index 를 쓰면 되잖아요?” map 의 두 번째 인자로 index 를 받을 수 있으니(JS자료 08단원) 이렇게 쓰고 싶어집니다.

```
{todos.map((todo, index) => <li key={index}>{todo.text}</li>)}
```

이렇게 하면 경고는 사라집니다. 화면도 잘 나옵니다. 그래서 인터넷에서 이 코드를 아주 많이 보게 됩니다. 하지만 목록이 바뀌는 순간 조용히 망가집니다. 직접 확인해 봅시다.

먼저 콘솔로 index 와 id 가 어떻게 다른지 봅시다.

```
const beforeDelete = [
  { id: 1, text: "우유 사기" },
  { id: 2, text: "책 반납" },
  { id: 3, text: "청소하기" },
];
```

slice(1) 은 맨 앞 하나를 뺀 새 배열을 돌려줍니다. 원본은 그대로입니다(JS자료 06단원).

```
const afterDelete = beforeDelete.slice(1);

console.log(beforeDelete.map((todo) => todo.text));
```

F12 콘솔 ['우유 사기', '책 반납', '청소하기']

```
console.log(afterDelete.map((todo) => todo.text));
```

F12 콘솔 ['책 반납', '청소하기']

각 줄의 key 가 어떻게 되는지 두 방식으로 뽑아 봅시다.

```
console.log(beforeDelete.map((todo, index) => index));
```

F12 콘솔 [0, 1, 2]

```
console.log(afterDelete.map((todo, index) => index));
```

F12 콘솔 [0, 1]

코드	▶ F12 콘솔
console.log(beforeDelete.map((todo) => todo.id));	▶ [1, 2, 3]
console.log(afterDelete.map((todo) => todo.id));	▶ [2, 3]

여기가 핵심입니다.

id 를 key 로 · 1, 2, 3 → 2, 3

사라진 key 는 1 입니다. 우리가 지운 "우유 사기" 와 정확히 같습니다.
React 는 1번 줄만 지우고 2·3번 줄은 손도 대지 않습니다.

index 를 key 로 · 0, 1, 2 → 0, 1

사라진 key 는 2 입니다. 맨 뒤입니다.
우리는 첫 줄을 지웠는데 React 는 '마지막 줄이 없어졌다' 고 봅니다.
0번 줄은 그대로 있는데 글자만 "우유 사기" 에서 "책 반납" 으로 바뀐 것으로 봅니다.

index 는 항목을 따라다니지 않습니다. 그냥 자리 번호입니다. 항목이 자리를 옮기면 index 는 다른 항목에게 넘어갑니다.

그러면 실제로 무엇이 망가질까요? 아래 데모를 직접 보세요.

```
const START_TODOs = [  
  { id: 1, text: "우유 사기" },  
  { id: 2, text: "책 반납" },  
  { id: 3, text: "청소하기" },  
];
```

각 줄에 입력칸을 하나씩 뒀습니다. defaultValue 는 "입력칸이 처음 나타날 때 들어 있을 글자" 를 정하는 속성입니다. 그 줄의 할 일 이름을 미리 넣어 뒀습니다. 즉 왼쪽 이름표와 오른쪽 입력칸의 글자는 항상 같아야 정상입니다.

★ 입력칸을 React 로 제대로 다루는 법은 06단원에서 배웁니다. 여기서는 "그 줄이 지워졌는지 살아남았는지" 를 눈으로 보려고 쓰는 장치일 뿐입니다. 입력칸에 든 글자는 React 가 아니라 브라우저가 들고 있습니다. 그래서 그 줄이 살아남으면 글자도 그대로 남습니다.


```
function KeyBreakDemo() {
  const [todos, setTodos] = useState(START_TODOS);

  return (
    <div className="demo"> <h3>섹션 4 – index 를 key 로 쓰면 이렇게 됩니다 ★</h3> <p>
      두 목록은 데이터도 같고 코드도 같습니다. key 만 다릅니다.
    <br />
    [맨 앞 항목 지우기] 를 누르고 <strong>이름표와 입력칸을 비교</strong>하세요.
    </p> <button onClick={() => setTodos(todos.slice(1))}>맨 앞 항목 지우기
  </button> <button onClick={() => setTodos(START_TODOS)}>처음으로 되돌리기
  </button> <div className="output"> <p>① key="{index}" – 자리 번호를 key 로
  </p> <ul>
    {todos.map((todo, index) => (
      <li key={index}>
        {todo.text} <input defaultValue={todo.text} /> </li>
    ))}
  </ul> <p>② key="{todo.id}" – 항목의 id 를 key 로</p> <ul>
    {todos.map((todo) => (
      <li key={todo.id}>
        {todo.text} <input defaultValue={todo.text} /> </li>
    ))}
  </ul> </div> </div>

  );
}
```

화면 ① 우유 사기 [우유 사기] / 책 반납 [책 반납] / 청소하기 [청소하기]
 ② 우유 사기 [우유 사기] / 책 반납 [책 반납] / 청소하기 [청소하기]

화면(맨 앞 항목 지우기를 누르면): ① 책 반납 [우유 사기] ← 이름표와 입력칸이 어긋났습니다 ① 청소하기 [책 반납] ← 여기도 어긋났습니다 ② 책 반납 [책 반납] ← 맞습니다 ② 청소하기 [청소하기] ← 맞습니다

①에서 무슨 일이 일어난 걸까요?

우리가 지운 것은 “우유 사기” 입니다. 그런데 React 가 실제로 화면에서 지운 것은 마지막 줄입니다. 남은 두 줄은 그대로 두고 글자만 한 칸씩 위로 밀어 넣었습니다.

글자는 React 가 넣어 준 것이라 새 값으로 바뀌었습니다. 하지만 입력칸에 든 글자는 브라우저가 들고 있는 것이라 그대로 남았습니다. 그래서 이름표는 “책 반납” 인데 입력칸은 “우유 사기” 입니다.

②는 key 가 1·2·3 이라 1번 줄이 통째로 사라졌습니다. 2·3번 줄은 손도 안 댔으니 입력칸도 자기 것을 그대로 들고 있습니다.

★ 입력칸에 직접 글자를 쳐서 확인해 보세요.

처음으로 되돌리기 · → ①의 첫 줄 입력칸에 “여기다” 라고 쳐 넣기

→ [맨 앞 항목 지우기] → “여기다” 가 “책 반납” 줄에 남아 있습니다.

입력칸만의 문제가 아닙니다. 아래 것들이 전부 같은 이유로 어긋납니다. · 체크박스에 해 둔 체크 · 그 줄 안에서만 쓰는 state (예: 줄마다 있는 '좋아요' 개수) · 스크롤 위치, 커서 위치, 애니메이션 진행 상태

그래서 "화면이 잘 나오니까 index 를 써도 되겠지" 가 위험한 것입니다. 목록이 안 바뀔 때는 멀쩡하다가, 목록이 바뀌는 순간 조용히 어긋납니다.

▫ 직접 해보기 4

위 데모에서 [맨 앞 항목 지우기] 를 두 번 눌러 보세요. ①에 남은 한 줄의 이름표와 입력칸은 각각 무엇일까요?

그럼 index 는 절대 쓰면 안 되나

섹션 5

아닙니다. 아래 세 가지를 '전부' 만족하면 index 를 써도 됩니다.

- (1) 목록의 순서가 절대 안 바뀐다 (정렬·뒤집기 없음)
- (2) 항목을 넣거나 빼지 않는다 (특히 중간이나 앞에)
- (3) 줄 안에 입력칸이나 state 처럼 '기억해야 할 것' 이 없다

한 번 그리고 끝나는 고정 목록이 딱 여기에 해당합니다. 아래 요일 목록이 그 예입니다. 순서도 안 바뀌고 개수도 안 변합니다.

```
const days = ["월", "화", "수", "목", "금"];

function StaticListDemo() {
  return (
    <div className="demo"> <h3>섹션 5 – index 를 써도 되는 경우
  </h3> <div className="output"> <p>바뀌지 않는 고정 목록입니다</p> <ul>
    {days.map((day, index) => (
      <li key={index}>
        {index + 1}. {day}요일
      </li>
    ))}
  </ul> </div> </div>
);
}
```

화면 1. 월요일 / 2. 화요일 / 3. 수요일 / 4. 목요일 / 5. 금요일

다만 판단이 애매하면 그냥 id 를 쓰세요. "지금은 안 바뀌니까 괜찮다" 로 시작한 목록이 나중에 바뀌는 일은 아주 흔합니다. 그때 이 버그가 생기면 원인을 찾기가 정말 어렵습니다.

참고로 위 요일 목록은 값이 겹치지 않으므로 key={day} 라고 쓸 수도 있습니다. 그게 더 안전합니다. index 를 굳이 쓸 이유가 없습니다.

▹ 직접 해보기 5

아래 두 목록 중 index 를 key 로 써도 되는 것은 무엇일까요? (가) 로그인한 사람에게 보여 주는 '최근 본 상품' 목록 (나) 화면 맨 위에 늘 같은 순서로 있는 메뉴 탭 5개

자주 하는 실수

섹션 6

이 섹션에는 SyntaxError 항목이 없습니다. key 실수는 전부 조용히 지나갑니다. 그래서 더 위험합니다.

이런 실수를 합니다

key 를 아예 안 붙임

```
{menu.map((item) => <li>{item.name}</li>)}
```

섹션 1의 경고가 납니다. 화면은 나옵니다. 목록이 바뀌면 섹션 4와 똑같은 문제가 생깁니다. key 가 없으면 React 는 자리 번호로 비교하기 때문입니다. 즉 key 를 안 쓰는 것은 index 를 쓰는 것과 비슷하게 동작합니다.

이런 실수를 합니다

key 를 컴포넌트 '안쪽' 태그에 붙임

```
function MenuItem({ item }) {
```

```
  return <li key={item.id}>{item.name}</li>;  // ← 여기 붙이면 소용없습니다
```

```
}
```

...

```
{menu.map((item) => <MenuItem item={item} />)} // ← 여기에 붙여야 합니다
```

이런 실수를 합니다

경고가 그대로 납니다. “붙였는데 왜 경고가 나지?” 하고 헤매게 됩니다. key 는 ‘map 이 직접 돌려주는 것’에 붙입니다. 위 코드에서 map 이 돌려주는 것은 <MenuItem /> 입니다. 가 아닙니다.

고치는 법: {menu.map((item) => <MenuItem key={item.id} item={item} />)}

이런 실수를 합니다

겹치는 값을 key 로 씀

```
const list = ["아메리카노", "라떼", "아메리카노"];
{list.map((name) => <li key={name}>{name}</li>)}
```

콘솔에 이런 경고가 납니다. Encountered two children with the same key, **아메리카노**. Keys should be unique so that components maintain their identity across updates. "같은 key 를 가진 자식이 둘 있다" 는 뜻입니다. 화면에서 줄이 사라지거나 겹쳐 보일 수 있습니다.

고치는 법: 겹치지 않는 값을 찾으세요. 없으면 데이터에 id 를 붙이는 게 맞습니다.

이런 실수를 합니다

key={Math.random()} — 경고는 안 나는데 조용히 망가집니다

"겹치지만 않으면 되잖아" 하고 매번 새 값을 만드는 사람이 있습니다. 경고는 안 납니다. 그래서 잘한 줄 압니다. 하지만 다시 그럴 때마다 key 가 전부 새 값이 됩니다. React 는 "예전 줄은 전부 없어지고 완전히 새 줄이 생겼다" 고 판단해서 모든 줄을 지우고 새로 만듭니다.

아래 데모에서 직접 확인하세요. 두 목록의 줄마다 '좋아요' 버튼이 있습니다. 좋아요를 몇 번 누른 뒤 [화면 다시 그리기] 를 누르세요.

```

function LikeRow({ text }) {
  const [like, setLike] = useState(0);

  return (
    <span>
      {text} <button onClick={() => setLike(like + 1)}>좋아요 {like}</button> </span>
    );
}

function RandomKeyDemo() {
  const [drawCount, setDrawCount] = useState(0);

  return (
    <div className="demo">
      <h3>섹션 6 - [실수 4] key 를 매번 새로 만들면</h3>

      <p>
        각 줄의 [좋아요] 를 몇 번 누른 뒤 [화면 다시 그리기] 를 누르세요.
      <br />
      지금까지 {drawCount}번 다시 그렸습니다.
    </p>

    <button onClick={() => setDrawCount(drawCount + 1)}>화면 다시 그리기</button>

    <div className="output">
      <p>① key={`{Math.random()}`} - 매번 새 값</p>
      <ul>
        {START_TODOs.map((todo) => (
          <li key={Math.random()}>
            <LikeRow text={todo.text} />
          </li>
        ))}
      </ul>

      <p>② key={`{todo.id}`} - 항상 같은 값</p>
      <ul>
        {START_TODOs.map((todo) => (
          <li key={todo.id}> <LikeRow text={todo.text} /> </li>
        ))}
      </ul>
    </div>
  </div>
);
}

```

화면

①과 ② 모두 세 줄, 좋아요 0

화면(좋아요를 누른 뒤 화면 다시 그리기를 누르면): ① 좋아요 개수가 전부 0 으로 돌아갑니다 ← 줄이 통째로 새로 만들어졌습니다 ② 좋아요 개수가 그대로 남아 있습니다

이런 실수를 합니다

key 는 '이 줄이 아까 그 줄인지' 를 알아보는 이름표입니다. 매번 새 이름을 붙이면 React 는 매번 처음 보는 줄이라고 생각합니다. 화면은 똑같아 보이지만 안에 있던 것이 전부 날아갑니다.

고치는 법: 항목마다 고정된 값을 쓰세요. 없으면 데이터를 만들 때 id 를 넣으세요.

전체 화면 그리기

정리

- 1 map 으로 목록을 그릴 때는 각 줄에 key 를 붙입니다. 안 붙이면 콘솔에 Each child in a list should have a unique "key" prop 경고가 납니다.
- 2 key 는 React 가 이 줄이 아까 그 줄인지 알아보는 이름표입니다. 화면(HTML)에는 나가지 않습니다.
- 3 좋은 key 는 두 가지 조건뿐입니다. **형제끼리 안 겹치고, 다시 그려도 안 바뀝니다.**
- 4 데이터에 id 가 있으면 그걸 쓰세요. 가장 안전합니다.
- 5 index 를 key 로 쓰면 목록이 바뀔 때 조용히 어긋납니다. 앞 항목을 지우면 입력값·체크·state 가 엉뚱한 줄에 남습니다.
- 6 순서도 개수도 안 바뀌고 줄 안에 기억할 것이 없는 고정 목록이면 index 를 써도 됩니다. 애매하면 id 를 쓰세요.
- 7 key={Math.random()} 은 경고가 안 나지만 매번 줄을 새로 만듭니다. 절대 쓰지 마세요.

```
export default function Concept03() {
  return (
    <div> <h1>개념 03 - key</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 → Console 을
      먼저 열어 두세요.</strong> 이 파일은 콘솔을 보는 것이 절반입니다.
      <br /> <br />
      ★ 이 파일에는 <strong>일부러 경고를 내는 버튼</strong>이 하나 있습니다. 섹션
      1의 "key 없이 그려 보기" 입니다. 그 버튼을 누르기 전까지 콘솔은 깨끗합니다.
      </p> <div> <NoKeyDemo /> <KeyIsNotVisible /> <KeyChoiceDemo /> <KeyBreakDemo
      /> <StaticListDemo /> <RandomKeyDemo /> </div> </div>
    );
  }

  console.log("개념03 화면을 그렸습니다. 콘솔은 아직 깨끗합니다.");
```

F12 콘솔 개념03 화면을 그렸습니다. 콘솔은 아직 깨끗합니다.

직접 해보기 정답

```
1) <li key={name}>{name}</li>
// 화면: 아메리카노 / 라떼 / 케이크 (그대로입니다)
```

→ 경고가 사라집니다. 반드시 왼쪽에서 다른 예제를 골랐다 돌아온 뒤 확인하세요.

새로고침을 안 하면 이미 나온 경고가 콘솔에 남아 있어 헛갈립니다.
이 목록은 이름이 겹치지 않으므로 name 을 key 로 써도 됩니다.

2) 안 붙어 있습니다. Elements 탭에는 아메리카노 만 보입니다. → key 는 React 가 자기 안에서만 쓰는 값입니다. 화면(HTML)에 나가지 않습니다.

그래서 key 값을 화면에 보여 주고 싶으면 따로 넘겨야 합니다.

3) 문제가 없습니다. 세 사람의 이름이 서로 겹치지 않기 때문입니다. → 다만 '지금 데이터에서' 안 겹치는 것뿐입니다.

나중에 김민준이 한 명 더 들어오면 그날 바로 중복 key 경고가 납니다.
이름은 겹칠 수 있는 값입니다. id 가 있으면 id 를 쓰세요.

4) 이름표는 "청소하기", 입력칸은 "우유 사기" 입니다.

```
// 화면: 청소하기 [우유 사기]
```

→ 두 번 누르면 항목이 하나만 남습니다.

①은 매번 마지막 줄을 지우고 글자만 위로 밀었습니다.
그래서 맨 처음 0번 자리에 있던 입력칸이 끝까지 살아남았습니다.
②는 "청소하기 [청소하기]" 로 제대로 나옵니다.

5) (나) 입니다. → (나)는 순서도 개수도 안 바뀌는 고정 목록이라 index 를 써도 됩니다.

(가)는 새로 본 상품이 맨 앞에 들어오면서 순서가 계속 바뀝니다.
섹션 4에서 본 문제가 그대로 생깁니다. 상품 id 를 key 로 쓰세요.

걸러서 그리기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념02에서는 배열에 든 것을 전부 그렸습니다. 실제 화면은 대부분 그렇지 않습니다.

할 일 목록에서 “안 끝난 것만” 보기 메뉴판을 “싼 것부터” 정렬해서 보기 “몇 개 남았는지” 세어 보여 주기
세 가지 모두 JS자료 08단원에서 이미 배운 도구로 합니다.

`filter` 조건에 맞는 것만 남긴다
`sort` 순서를 바꾼다 (JS자료 06단원)

`.length` 개수를 센다

새로 배우는 것은 “그 결과를 화면에 어떻게 없느냐” 뿐입니다.

이 파일에서 계속 쓸 데이터입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

const todos = [
  { id: 1, text: "우유 사기", done: true },
  { id: 2, text: "책 반납", done: false },
  { id: 3, text: "청소하기", done: false },
  { id: 4, text: "메모 정리", done: true },
];

const menu = [
  { id: "americano", name: "아메리카노", price: 4000 },
  { id: "latte", name: "라떼", price: 4500 },
  { id: "cake", name: "케이크", price: 6000 },
  { id: "gimbap", name: "삼각김밥", price: 1200 },
];
```

filter 로 고르고 map 으로 그린다

섹션 1

먼저 콘솔에서 확인합시다. JS자료 08단원 개념04 그대로입니다.

```
console.log(todos.filter((todo) => !todo.done).map((todo) => todo.text));
```


F12 콘솔 ['책 반납', '청소하기']

filter 로 고르고, 그 결과에 map 을 이어 붙였습니다. 이렇게 이어 쓰는 것을 체이닝이라고 배웠습니다.

화면에서도 똑같이 합니다. map 이 돌려주는 값만 화면 조각으로 바꾸면 됩니다.

```
{todos.filter((todo) => !todo.done).map((todo) => <li ...>)}
```

순서가 중요합니다. 반드시 filter 가 먼저입니다. filter → 개수를 줄인다 map → 남은 것의 모양을 바꾼다
순서를 바꾸면 화면 조각을 걸러야 해서 조건을 쓸 수 없습니다. 섹션 5의 [실수 4] 에서 실제로 어떻게 되는지 봅시다.

왜 이 순서인지 한 번 더 생각해 봅시다. filter 의 콜백은 항목을 받아 “남길까?” 를 판단합니다. 판단하려면
todo.done 처럼 항목 ‘안의 값’ 을 볼 수 있어야 합니다. 그런데 map 을 먼저 돌리면 항목이 화면 조각으로
바뀌어 버립니다. 화면 조각에는 done 이 없습니다. 볼 수 있는 값이 사라지는 것입니다.

그래서 규칙은 이렇게 외우는 게 아니라 이렇게 이해하면 됩니다. ★ 값을 보고 판단하는 일은, 아직 값일 때
끝낸다.

참고로 filter 를 두 번 이어 써도 됩니다. 조건이 서로 다른 성격일 때 읽기 좋습니다.

```
todos.filter((t) => !t.done).filter((t) => t.text.length > 3).map(...)
```

물론 조건 하나에 && 로 묶어도 됩니다. 어느 쪽이든 결과는 같습니다.

```
function UndoneList() {
  return (
    <div className="demo"> <h3>섹션 1 – 안 끝난 것만 그리기
  </h3> <div className="output"> <p>전체 목록</p> <ul>
    {todos.map((todo) => (
      <li key={todo.id} className={todo.done ? "done" : ""}>
        {todo.text}
      </li>
    ))}
  </ul> <p>안 끝난 것만</p> <ul>
    {todos
      .filter((todo) => !todo.done)
      .map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
  </ul> </div> </div>
  );
}
```

화면	전체 목록 – 우유 사기(줄 그어짐) / 책 반납 / 청소하기 / 메모 정리(줄 그어짐)
	안 끝난 것만 – 책 반납 / 청소하기

줄이 길어져서 filter 와 map 을 각각 다른 줄에 썼습니다. 한 줄로 쭉 이어 써도 똑같이 동작합니다. 읽기 좋은 쪽을 고르세요.

key 는 filter 를 거쳐도 그대로 todo.id 를 씁니다. 걸러 낸 뒤에도 항목은 자기 id 를 그대로 들고 있기 때문입니다. 여기서 index 를 쓰면 안 되는 이유가 하나 더 생깁니다. 필터를 바꿀 때마다 같은 항목의 index 가 달라지기 때문입니다.

▣ 직접 해보기

1

위 UndoneList 에 “끝난 것만” 보여 주는 목록을 하나 더 추가해 보세요.

버튼으로 조건 갈아 끼우기

섹션 2

조건을 코드에 박아 두지 말고 state 에 담으면, 버튼으로 바꿀 수 있습니다. 04단원에서 배운 useState 를 그대로 씁니다.

여기서 중요한 것은 ‘목록을 만들어 두고 갈아 끼우는 게 아니라는’ 점입니다. 원본 배열 todos 는 하나뿐입니다. 손대지 않습니다. 바뀌는 것은 filter 조건을 담은 state 하나뿐입니다. 화면은 그릴 때마다 원본에서 다시 걸러 냅니다.

```
function FilterDemo() {
```

“all” · “todo” · “done” 중 하나를 담습니다

```
const [mode, setMode] = useState("all");
```

개념01 섹션 3에서 배운 대로 return 밖에서 골라 둡니다.

```
let shown;
if (mode === "todo") {
  shown = todos.filter((todo) => !todo.done);
} else if (mode === "done") {
  shown = todos.filter((todo) => todo.done);
} else {
  shown = todos;
}

return (
  <div className="demo"> <h3>섹션 2 – 버튼으로 조건 갈아 끼우기
</h3> <button className={mode === "all" ? "on" : ""} onClick={() => setMode("all")}>
  전체
  </button> <button className={mode === "todo" ? "on" : ""} onClick={() =>
setMode("todo")}>
  안 끝난 것
  </button> <button className={mode === "done" ? "on" : ""} onClick={() =>
```

```

    끝난 것
    </button> <div className="output"> <ul>
      {shown.map((todo) => (
        <li key={todo.id} className={todo.done ? "done" : ""}>
          {todo.text}
        </li>
      ))}
    </ul> </div> </div>
  );
}

```

화면 네 줄 모두 (전체 버튼이 파랗게 켜져 있습니다)

화면(안 끝난 것을 누르면): 책 반납 / 청소하기 두 줄 화면(끝난 것을 누르면): 우유 사기 / 메모 정리 두 줄
눌린 버튼에 className={...} 으로 on 클래스를 붙였습니다. 개념02 섹션 4에서 본 방법입니다. 지금
무엇을 보고 있는지 알려 줍니다.

★ 목록을 그리는 map 코드는 한 벌뿐입니다. 버튼마다 목록을 따로 만들지 않았습니다. “무엇을 보여 줄지”
를 state 로 두고, 화면은 그 state 를 보고 한 번만 그립니다.

이게 왜 중요한지 반대로 해 보면 알 수 있습니다. 만약 버튼마다 목록을 미리 만들어 두면 이렇게 됩니다.

```

const allList = todos;
const todoList = todos.filter((todo) => !todo.done);
const doneList = todos.filter((todo) => todo.done);

```

지금은 잘 돌아갑니다. 그런데 나중에 todos 에 항목을 추가하는 기능이 붙으면 세 목록을 전부 다시 만들어
줘야 합니다. 하나라도 빠뜨리면 화면이 어긋납니다. JS자료 13단원 종합03에서 겪었던 그 문제입니다.

반면 위 FilterDemo 는 원본 todos 하나만 맞으면 화면이 저절로 맞습니다. 걸러 낸 결과를 어디에도
저장해 두지 않았기 때문입니다.

이것을 “화면에 필요한 것을 저장하지 말고, 그럴 때마다 계산한다” 고 합니다. React 자료 전체를 관통하는
사고방식입니다. 07단원에서 다시 만납니다.

그리고 여기서 눈여겨볼 것이 하나 더 있습니다. 화면에 보이는 줄 수가 4개에서 2개로 바뀌는데, 우리는
줄을 지우는 코드를 한 줄도 쓰지 않았습니다. shown 이 달라지니 React 가 알아서 맞춰 준 것입니다.
그런데 이때 ‘어느 줄을 지울지’ 를 React 가 어떻게 알까요? key 입니다. 필터를 바꿀 때마다 같은 항목의
자리 번호가 달라지므로, 여기서 key 를 index 로 썼다면 개념03 섹션 4의 문제가 그대로 생깁니다.

▸ 직접 해보기 2

버튼을 하나 더 만들어 “제목이 네 글자 이하인 것” 만 보여 주게 해 보세요. (text.length 를 씁니다)
공백도 한 글자로 셉니다. “책 반납” 은 네 글자입니다.

정렬은 sort 로 합니다. JS자료 06단원에서 배웠습니다. 그런데 sort 에는 조심할 점이 하나 있습니다. 원본을 바꿔 버립니다. filter 나 map 과 다릅니다. 콘솔로 확인해 봅시다.

```
const nums = [3, 1, 2];
const sortedNums = nums.sort((a, b) => a - b);

console.log(sortedNums);
```

F12 콘솔 [1, 2, 3]

```
console.log(nums);
```

F12 콘솔 [1, 2, 3]

결과만 정렬된 게 아니라 원본 nums 까지 정렬됐습니다.

filter 와 map 은 이러지 않습니다. 비교해 보세요.

```
const nums2 = [3, 1, 2];
console.log(nums2.filter((n) => n > 1));
```

F12 콘솔 [3, 2]

```
console.log(nums2);
```

F12 콘솔 [3, 1, 2]

원본이 그대로입니다.

원본을 지키려면 복사해서 정렬하면 됩니다. JS자료 09단원의 스프레드로 새 배열을 만든 다음 정렬합니다.

```
const nums3 = [3, 1, 2];
const sortedCopy = [...nums3].sort((a, b) => a - b);

console.log(sortedCopy);
```

F12 콘솔 [1, 2, 3]

```
console.log(nums3);
```

F12 콘솔 [3, 1, 2]

이번에는 원본이 그대로입니다.

React 에서 이게 왜 중요할까요? 두 가지 때문입니다.

하나 · 같은 배열을 보는 다른 화면까지 바꿉니다.

화면 A 는 정렬해서 보여 주고 화면 B 는 원래 순서로 보여 준다고 합시다. A 가 원본을 정렬해 버리면 B 도 같이 정렬됩니다. B 를 만든 사람은 자기 코드를 아무리 봐도 원인을 못 찾습니다. “내 코드에는 sort 가 없는데 왜 순서가 바뀌지?” 가 되는 것입니다.

둘 · 되돌릴 방법이 없어집니다.

원래 순서는 원본에만 들어 있습니다. 그걸 덮어썼으니 어디에도 안 남습니다. 아래 데모의 [원래 순서] 버튼이 동작하는 이유가 바로 원본을 지켰기 때문입니다.

그 배열이 state 라면 문제가 더 커집니다. 화면이 아예 안 바뀌기도 합니다. 그 이야기는 07단원에서 제대로 합니다.

지금은 규칙 하나만 지키면 됩니다. ★ 화면을 그리려고 정렬할 때는 항상 [...배열] 로 복사한 다음 sort 한다.

원본을 바꾸는 메소드가 sort 만 있는 것은 아닙니다. JS자료 06단원에서 배운 push · pop · splice · reverse 도 원본을 바꿉니다. 반대로 filter · map · slice · concat 은 새 배열을 돌려줍니다. 화면을 그릴 때는 뒤쪽 것만 쓰면 안전합니다.

```
function SortDemo() {
  const [order, setOrder] = useState("none");

  let shown;
  if (order === "cheap") {
    shown = [...menu].sort((a, b) => a.price - b.price);
  } else if (order === "name") {
    shown = [...menu].sort((a, b) => a.name.localeCompare(b.name));
  } else {
    shown = menu;
  }

  return (
    <div className="demo"> <h3>섹션 3 - 정렬해서 그리기</h3> <button className={order
    === "none" ? "on" : ""} onClick={() => setOrder("none")}>
      원래 순서
    </button> <button className={order === "cheap" ? "on" : ""} onClick={() =>
    setOrder("cheap")}>
      싼 것부터
    </button> <button className={order === "name" ? "on" : ""} onClick={() =>
    setOrder("name")}>
      이름순
    </button> <div className="output"> <ul>
      {shown.map((item) => (
        <li key={item.id}>
          {item.name} - {item.price}원
        </li>
      ))}
    </ul> </div> </div>
  );
}
```

화면 아메리카노 4000 / 라떼 4500 / 케이크 6000 / 삼각김밥 1200

화면(싼 것부터를 누르면): 삼각김밥 1200 / 아메리카노 4000 / 라떼 4500 / 케이크 6000
 화면(이름순을 누르면): 라떼 / 삼각김밥 / 아메리카노 / 케이크

이름순 정렬에 쓴 localeCompare 는 JS자료 08단원 개념06에서 배운 그대로입니다. 글자를 사전 순서로 비교해 줍니다.

복사할 때 [...menu] 대신 menu.slice() 를 써도 똑같습니다. JS자료 08단원 개념06에서는 slice() 를 썼습니다. 둘 다 새 배열을 만듭니다.

★ 원래 순서 버튼을 눌러 보세요. 처음 순서가 그대로 돌아옵니다.

...menu · 로 복사해서 정렬했기 때문에 원본 menu 가 안 망가진 것입니다.

만약 menu.sort(...) 라고 썼다면 한 번 정렬한 뒤로는 “원래 순서” 를 눌러도 돌아오지 않습니다.

▹ 직접 해보기

3

“비싼 것부터” 버튼을 하나 더 만들어 보세요. (a.price - b.price 를 뒤집으면 됩니다)

개수 세기

섹션 4

개수는 filter 결과에 .length 를 붙이면 끝입니다. JS자료 08단원에서 한 그대로입니다.

```
console.log(todos.length);
```

F12 콘솔 4

```
console.log(todos.filter((todo) => todo.done).length);
```

F12 콘솔 2

```
console.log(todos.filter((todo) => !todo.done).length);
```

F12 콘솔 2

화면에서도 그대로 씁니다. 중괄호 안에는 계산식이 들어가도 되니까요(02단원).

```
function CountDemo() {
  const [mode, setMode] = useState("all");

  const doneCount = todos.filter((todo) => todo.done).length;
  const undoneCount = todos.length - doneCount;

  const shown = mode === "undone" ? todos.filter((todo) => !todo.done) : todos;

  return (
    <div className="demo"> <h3>섹션 4 - 개수 세기</h3> <button className={mode ===
    "all" ? "on" : ""} onClick={() => setMode("all")}>
      전체 보기
    </button> <button className={mode === "undone" ? "on" : ""} onClick={() =>
    setMode("undone")}>
      남은 것만 보기
    </button> <div className="output"> <p>
      전체 {todos.length}개 · 완료 {doneCount}개 · 남은 {undoneCount}개
    </p> <p>지금 보고 있는 줄: {shown.length}개</p> <ul>
      {shown.map((todo) => (
        <li key={todo.id} className={todo.done ? "done" : ""}>
```

```

        {todo.text}
      </li>
    )}
  </ul> </div> </div>
);
}

```

화면 전체 4개 · 완료 2개 · 남음 2개
 지금 보고 있는 줄: 4개

화면(남은 것만 보기를 누르면): 지금 보고 있는 줄: 2개

★ doneCount 를 useState 로 따로 만들지 않았습니다. 개수는 todos 만 있으면 언제든지 다시 셀 수 있는 값입니다. state 로 따로 들고 있으면 목록과 개수가 어긋날 수 있습니다. (목록만 고치고 개수 고치는 걸 깜빡하는 식입니다) “계산할 수 있는 값은 state 로 두지 않는다” 는 07단원에서 자세히 배웁니다.

JS자료 13단원 종합03에서 할 일 목록을 만들 때를 떠올려 보세요. 할 일을 추가하는 곳, 지우는 곳, 완료로 바꾸는 곳마다 “남은 개수” 를 고치는 줄을 하나씩 더 적어야 했습니다. 한 군데라도 빠뜨리면 목록과 숫자가 안 맞았습니다. 지금은 그런 줄이 한 줄도 없습니다. 화면을 그릴 때마다 todos 를 보고 그 자리에서 다시 세기 때문입니다.

“매번 다시 세면 느리지 않나요?” 하는 걱정은 지금 하지 않아도 됩니다. 항목이 수천 개가 넘어가면 그때 생각합니다. 그 방법은 10단원에서 배웁니다.

▣ 직접 해보기 4

CountDemo 에 “완료율” 을 보여 주세요. 완료 2개 / 전체 4개면 50% 입니다.

자주 하는 실수

섹션 5

이 섹션에는 SyntaxError 항목이 없습니다. 전부 에러 없이 조용히 틀리는 것들이라 더 조심해야 합니다.

이런 실수를 합니다

map 으로 걸러내려고 함

map 은 개수를 바꾸지 않습니다(JS자료 08단원 개념03 실수 3). 조건에 안 맞는 자리에 null 을 넣어도 개수는 그대로입니다.

```
console.log(todos.map((todo) => (todo.done ? todo.text : null)));
```

F12 콘솔 ['우유 사기', null, null, '메모 정리']

```
console.log(todos.map((todo) => (todo.done ? todo.text : null)).length);
```


F12 콘솔 4

```
console.log(todos.filter((todo) => todo.done).length);
```

F12 콘솔 2

무서운 것은 화면입니다. `null` 은 안 그려지므로(개념01 섹션 4) 화면만 보면 제대로 걸러진 것처럼 보입니다.

```
function MapFilterMistake() {
  return (
    <div className="demo"> <h3>섹션 5 - [실수 1] map 으로 걸러낸 척하기
    </h3> <div className="output"> <p>① map + 삼항 (화면은 맞아 보입니다)</p> <ul>
      {todos.map((todo) =>
        todo.done ? <li key={todo.id}>{todo.text}</li> : null
      )}
    </ul> <p>
      ① 이 만든 배열의 길이:{" "}
      {todos.map((todo) => (todo.done ? todo.text : null)).length}개 ← 틀렸습니다
    </p> <p>② filter + map</p> <ul>
      {todos
        .filter((todo) => todo.done)
        .map((todo) => (
          <li key={todo.id}>{todo.text}</li>
        ))}
    </ul> <p>
      ② 가 만든 배열의 길이: {todos.filter((todo) => todo.done).length}개 ←
      맞습니다
    </p> </div> </div>
  );
}
```

화면

- ① 우유 사기 / 메모 정리 두 줄 · 길이는 4개
- ② 우유 사기 / 메모 정리 두 줄 · 길이는 2개

이런 실수를 합니다

두 목록이 똑같아 보입니다. 그래서 ①을 쓰고도 문제를 못 느낍니다. 그런데 “몇 개 남았습니다” 를 보여 주는 순간 4개라고 나옵니다. 걸러 낼 때는 `filter` 를 쓰세요. `map` 은 모양을 바꾸는 도구입니다.

이런 실수를 합니다

원본을 정렬해 버림

```
const shown = menu.sort((a, b) => a.price - b.price);
```

섹션 3에서 본 대로 원본 menu 가 정렬됩니다. “원래 순서” 로 돌아갈 방법이 없어집니다. 그 배열을 쓰는 다른 화면도 같이 바뀝니다.

고치는 법: [...menu].sort(...) 처럼 복사한 뒤 정렬하세요.

이런 실수를 합니다

filter 에 조건이 아니라 계산식을 넣음

```
console.log(menu.filter((item) => item.price - 4000).map((item) => item.name));
```

F12 콘솔 ['라떼', '케이크', '삼각김밥']

```
console.log(menu.filter((item) => item.price >= 4000).map((item) => item.name));
```

F12 콘솔 ['아메리카노', '라떼', '케이크']

위 줄은 “4000원짜리를 뺀 목록” 이 됐습니다. item.price - 4000 은 아메리카노일 때만 0(거짓)이고 나머지는 전부 참이ращ요. filter 는 “이 값을 남길까?” 를 묻는 것입니다. 참·거짓이 나오는 식을 쓰세요. JS자료 08단원 개념04 실수 3에서 본 것과 같습니다.

이런 실수를 합니다

filter 와 map 의 순서를 바꿈

```
function OrderMistake() {  
  return (  
    <div className="demo"> <h3>섹션 5 - [실수 4] map 을 먼저 하면  
</h3> <div className="output"> <p>① map 먼저, filter 나중 (빈 목록이 됩니다)  
</p> <ul>  
      {menu  
        .map((item) => <li key={item.id}>{item.name}</li>  
        .filter((item) => item.price >= 4500))  
      </ul> <p>② filter 먼저, map 나중</p> <ul>  
      {menu  
        .filter((item) => item.price >= 4500)  
        .map((item) => (  
          <li key={item.id}>{item.name}</li>  
        ))}  
      </ul> </div> </div>
```

```
);
}
```

화면 ① 아래에는 아무 줄도 없습니다
② 라떼 / 케이크 두 줄

①에서 map 이 먼저 돌면서 항목이 전부 화면 조각으로 바뀝니다. 그다음 filter 가 받는 것은 { name, price } 객체가 아니라 화면 조각입니다. 화면 조각에는 price 가 없으므로 item.price 는 `undefined` 입니다. `undefined >= 4500` 은 `undefined` 가 숫자로 바뀌며 `NaN` 이 되고, `NaN` 과의 비교는 항상 `false` 라서 전부 걸러져 빈 목록이 됩니다. 에러도 경고도 안 납니다. 그냥 목록이 사라집니다.

고치는 법: 항상 filter 를 먼저 하세요.

"값을 보고 고르는 일" 은 값이 아직 값일 때 해야 합니다.

전체 화면 그리기

정리

- 1 걸러서 그릴 때는 `filter` 를 먼저, `map` 을 나중에 씁니다. 순서를 바꾸면 조용히 빈 목록이 됩니다.
- 2 보여 줄 조건은 **state 에 답합니다**. 원본 배열은 손대지 않고, 그릴 때마다 다시 걸러 냅니다.
- 3 `sort` 는 **원본을 바꿉니다**. `filter·map` 과 다릅니다.
- 4 그래서 정렬은 항상 `[...배열].sort(...)` 처럼 복사한 뒤에 합니다.
- 5 개수는 `filter(...).length` 로 셉니다. 따로 state 에 담아 두지 않습니다. 어긋날 수 있기 때문입니다.
- 6 `map` 은 개수를 안 바꿉니다. 걸러 낸 것처럼 보여도 길이는 그대로입니다.

```
export default function Concept04() {
  return (
    <div> <h1>개념 04 – 걸러서 그리기</h1> <p className="guide">
      왼쪽 목록에서 이 예제를 고르면 이 화면이 나옵니다. <strong>F12 →
      Console</strong> 도 함께 보세요.
      </p> <div> <UndoneList /> <FilterDemo /> <SortDemo /> <CountDemo
      /> <MapFilterMistake /> <OrderMistake /> </div> </div>
  );
}

console.log("개념04 화면을 그렸습니다");
```

F12 콘솔 개념04 화면을 그렸습니다

직접 해보기 정답

1) <p>끝난 것만</p>

```
<ul>

{todos
  .filter((todo) => todo.done)
  .map((todo) => (
    <li key={todo.id}>{todo.text}</li>
  ))}

</ul>
// 화면: 우유 사기 / 메모 정리
```

→ 조건에서 !를 뺐을 뿐입니다. done 이 true 인 것만 남습니다.

```
2) <button className={mode === "short" ? "on" : ""} onClick={() => setMode("short")}>
```

짧은 것

```
</button>
```

그리고 if 에 갈래를 하나 더 넣습니다.

```
} else if (mode === "short") {
```

```
  shown = todos.filter((todo) => todo.text.length <= 4);
```

```
}
// 화면: 책 반납 / 청소하기 두 줄
```

→ "책 반납" 과 "청소하기" 가 네 글자입니다.

"우유 사기" 와 "메모 정리" 는 다섯 글자라 빠집니다.
공백도 한 글자로 셉니다. "책 반납" 을 세 글자로 세면 결과가 빈 목록이 됩니다.

```
3) } else if (order === "expensive") {
```

```
  shown = [...menu].sort((a, b) => b.price - a.price);
```

```
}
// 화면: 케이크 6000 / 라떼 4500 / 아메리카노 4000 / 삼각김밥 1200
```

→ a 와 b 의 자리를 바꾸면 순서가 뒤집힙니다. JS자료 06단원에서 배운 그대로입니다.

```
4) <p>완료율 {Math.round((doneCount / todos.length) * 100)}%</p>  
// 화면: 완료율 50%
```

→ 나눗셈 결과가 0.5 이므로 100 을 곱해 50 으로 만들었습니다.

소수가 나올 수 있으니 `Math.round` 로 다듬었습니다(JS자료 02단원).
`todos` 가 빈 배열이면 0 으로 나누게 되어 `NaN` 이 나옵니다.
그 경우를 어떻게 막는지는 개념05에서 다룹니다.

비었을 때와 falsy 함정

RUN 실습프로젝트에서 npm run dev → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

개념04에서 filter 로 걸러 봤습니다. 그런데 조건에 맞는 게 하나도 없으면 어떻게 될까요?

장바구니를 비웠을 때, 검색 결과가 없을 때, 할 일을 다 끝냈을 때 — 목록이 비는 일은 아주 흔합니다.

오히려 화면이 처음 열릴 때는 대부분 비어 있습니다.

그런데 여기에 React 초보자가 가장 많이 빠지는 함정이 하나 있습니다. 화면에 난데없이 0 이 하나 찍히는 것입니다. 이 파일에서 그 함정을 직접 재현하고, 왜 그런지 끝까지 파고듭니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

const fullCart = [
  { id: "americano", name: "아메리카노", price: 4000 },
  { id: "cake", name: "케이크", price: 6000 },
];

const emptyCart = [];
```

빈 목록은 에러가 아니다

섹션 1

먼저 확인할 것이 있습니다. 빈 배열에 map 을 써도 에러가 나지 않습니다. 빈 배열이 나올 뿐입니다. JS자료 08단원에서 배운 그대로입니다.

코드	▶ F12 콘솔
console.log(emptyCart.map((item) => item.name));	▶ []
console.log(emptyCart.length);	▶ 0

React 도 빈 배열을 받으면 아무것도 안 그립니다. 조용히 넘어갑니다. 즉 코드는 멀쩡합니다. 문제는 '사용자가 보는 화면' 입니다.

```
function EmptyIsSilent() {
  return (
    <div className="demo"> <h3>섹션 1 - 빈 목록은 그냥 아무것도 안 나온다
    </h3> <div className="output"> <p>장바구니</p> <ul>
      {emptyCart.map((item) => (
```

05단원 · 연습문제

조건부 렌더링과 리스트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

문제에서 함께 쓰는 데이터

```
import { useState } from "react";

const menu = [
  { id: "americano", name: "아메리카노", price: 4000, soldOut: false },
  { id: "latte", name: "라떼", price: 4500, soldOut: true },
  { id: "cake", name: "케이크", price: 6000, soldOut: false },
  { id: "gimbap", name: "삼각김밥", price: 1200, soldOut: true },
];

const users = [
  { id: 101, name: "김민준", age: 20 },
  { id: 102, name: "이서연", age: 22 },
  { id: 103, name: "박지훈", age: 28 },
];

const todos = [
  { id: 1, text: "우유 사기", done: true },
  { id: 2, text: "책 반납", done: false },
  { id: 3, text: "청소하기", done: false },
];
```

05

문제 1 (개념01 삼항)

`isOpen` 이 `true` 면 “영업 중입니다”, `false` 면 “준비 중입니다” 가 화면에 나오게 하세요.

기대 화면 영업 중입니다
 `isOpen` 을 `false` 로 바꿔 저장하면 “준비 중입니다” 가 나와야 합니다.
 아무것도 안 나오면 중괄호 `{}` 를 안 쓴 것입니다.

```
function Q1() {
  const isOpen = true;

  return (
    <div className="demo"> <h3>문제 1 – 삼항 연산자</h3> <div className="output">{/*
TODO: 여기에 코드를 쓰세요 */}</div> </div>
    );
}
```

문제 2 (개념01 &&)

newCount 가 0보다 클 때만 "새 알림 3개" 처럼 보여 주세요. 0일 때는 아무것도 안 나와야 합니다.

기대 화면 새 알림 3개
newCount 를 0 으로 바꿔 저장하면 아무것도 안 나와야 합니다.
0 으로 바꿨는데 화면에 0 이 보이면 && 왼쪽에 개수를 그대로 쓴 것입니다.

```
function Q2() {
  const newCount = 3;

  return (
    <div className="demo"> <h3>문제 2 – && 로 있을 때만 보이기
</h3> <div className="output">{/* TODO: 여기에 코드를 쓰세요 */}</div> </div>
    );
}
```

문제 3 (개념01 변수에 담아 두기)

point 에 따라 아래 셋 중 하나가 나오게 하세요. 100점 이상 → <p>VIP 회원입니다</p> 50점 이상 →
<p>일반 회원입니다</p> 그 밖 → <p>신규 회원입니다</p>

return 밖에서 if 로 골라 badge 변수에 담고, return 안에서는 {badge} 만 쓰세요.

기대 화면 일반 회원입니다
"신규 회원입니다" 가 나오면 if 의 순서나 부등호를 확인하세요.

```
function Q3() {
  const point = 70;
  let badge;
```


여기에 if 문을 쓰세요

```
return (
  <div className="demo"> <h3>문제 3 - 변수에 담아 두기
</h3> <div className="output"> <p>이서연님 · {point}점</p>
    {badge}
  </div> </div>
);
}
```

문제 4 (개념01 일찍 return)

ProfileCard 는 user 가 없으면 <p>회원 정보가 없습니다</p> 만 돌려주고 끝나야 합니다. 함수 맨 위에서 확인하고 먼저 return 하세요.

확인하는 방법: 아래 Q4 의 user={users[0]} 을 user={null} 로 바꿔 저장하세요.

기대 화면 지금은 "김민준 (20세)" 가 나옵니다.
 user={null} 로 바꾸고 일찍 return 을 썼다면
 → 회원 정보가 없습니다
 일찍 return 을 안 썼다면 화면이 통째로 비고 콘솔에
 TypeError: Cannot read properties of null (reading 'name')
 이 납니다. 확인했으면 users[0] 으로 되돌리세요.

```
function ProfileCard({ user }) {
```

여기에 일찍 return 하는 코드를 쓰세요

```
return (
  <p>
    {user.name} ({user.age}세)
  </p>
);
```

```

}

function Q4() {
  return (
    <div className="demo"> <h3>문제 4 - 일찍
return</h3> <div className="output"> <ProfileCard user={users[0]} /> </div> </div>
  );
}

```

문제 5 (개념02 map)

fruits 의 각 값을 한 줄씩으로 그리세요. key 도 빠뜨리지 마세요. 이 배열은 값이 서로 겹치지 않습니다.

기대 화면 사과 / 포도 / 딸기 세 줄
아무 줄도 안 나오면 return 을 빠뜨렸거나 forEach 를 쓴 것입니다.
콘솔에 key 경고가 나면 key 를 안 붙인 것입니다.

```

function Q5() {
  const fruits = ["사과", "포도", "딸기"];

  return (
    <div className="demo"> <h3>문제 5 - map 으로 목록 그리기
</h3> <div className="output"> <ul>{/* TODO: 여기에 코드를 쓰세요 */}
</ul> </div> </div>
  );
}

```

문제 6 (개념02 객체 배열)

users 를 "김민준 (20세)" 형태로 한 줄씩 그리세요. key 는 id 를 쓰세요.

기대 화면 김민준 (20세) / 이서연 (22세) / 박지훈 (28세)
"[object Object]" 가 나오거나 콘솔에
"Objects are not valid as a React child" 에러가 나면
객체를 통째로 넣은 것입니다. 안에서 값을 꺼내 쓰세요.

```
function Q6() {
  return (
    <div className="demo"> <h3>문제 6 - 객체 배열 그리기
  </h3> <div className="output"> <ul>{/* TODO: 여기에 코드를 쓰세요 */}
  </ul> </div> </div>
  );
}
```

문제 7 (개념02 컴포넌트에 넘기기)

아래 MenuRow 컴포넌트를 써서 menu 를 그리세요. MenuRow 는 name 과 price 를 props 로 받습니다. MenuRow 는 고치지 마세요. key 를 어디에 붙여야 하는지 주의하세요.

기대 화면 아메리카노 - 4000원 / 라떼 - 4500원 /
 케이크 - 6000원 / 삼각김밥 - 1200원
 key 를 붙였는데도 콘솔에 key 경고가 나면
 MenuRow 안쪽 에 붙인 것입니다.

```
function MenuRow({ name, price }) {
  return (
    <li>
      {name} - {price}원
    </li>
  );
}

function Q7() {
  return (
    <div className="demo"> <h3>문제 7 - 컴포넌트에 넘기기
  </h3> <div className="output"> <ul>{/* TODO: 여기에 코드를 쓰세요 */}
  </ul> </div> </div>
  );
}
```

문제 8 (개념03 key 고르기)

orders 는 오늘 들어온 주문입니다. 같은 메뉴를 두 번 시킬 수 있습니다. 주문 번호와 메뉴 이름을 "1번 · 아메리카노" 형태로 한 줄씩 그리세요.

무엇을 key 로 쓸지 잘 고르세요. name 을 key 로 쓰면 어떻게 될까요? 궁금하면 일부러 name 을 넣어 보고 콘솔을 확인한 뒤 고쳐도 좋습니다.

기대 화면 1번 · 아메리카노 / 2번 · 라떼 / 3번 · 아메리카노
 콘솔에 "Encountered two children with the same key" 가 나오면
 겹치는 값을 key 로 쓴 것입니다.

```
function Q8() {
  const orders = [
    { id: 1, name: "아메리카노" },
    { id: 2, name: "라떼" },
    { id: 3, name: "아메리카노" },
  ];

  return (
    <div className="demo"> <h3>문제 8 – 무엇을 key 로 쓸까
  </h3> <div className="output"> <ul>{/* TODO: 여기에 코드를 쓰세요 */}
  </ul> </div> </div>
  );
}
```

문제 9 (개념03 index 를 key 로 쓰면)

아래 목록은 key 로 index 를 쓰고 있습니다.

맨 앞 항목 지우기 · 를 눌러 보세요. 이름표와 입력칸이 어긋납니다.

key 를 고쳐서 어긋나지 않게 하세요. (한 곳만 고치면 됩니다)

기대 화면 고치기 전 – 지우면 "책 반납 [우유 사기]" 처럼 어긋납니다
 고친 뒤 – 지우면 "책 반납 [책 반납]" 으로 맞습니다
 고쳤는데도 어긋나면 왼쪽에서 다른 예제를 골랐다 돌아오기를 안 한
 것입니다.

```
function Q9() {
  const [list, setList] = useState(todos);

  return (
    <div className="demo">
      <h3>문제 9 – index 를 key 로 쓰면</h3>

      <button onClick={() => setList(list.slice(1))}>맨 앞 항목 지우기</button>
      <button onClick={() => setList(todos)}>처음으로 되돌리기</button>

      <div className="output">
        <ul>
          {list.map((todo, index) => (
```

아래 key 를 고치세요

```

    <li key={index}>
      {todo.text} <input defaultValue={todo.text} />
    </li>
  )}
</ul>
</div>
</div>
);
}

```

05

문제 10 (개념04 filter + map)

menu 에서 품절이 아닌 것만(soldOut 이 false) 골라 이름을 그리세요.

기대 화면 아메리카노 / 케이크 두 줄
네 줄이 다 나오면 filter 를 안 쓴 것입니다.
라떼와 삼각김밥이 나오면 조건을 뒤집어 쓴 것입니다.

```

function Q10() {
  return (
    <div className="demo"> <h3>문제 10 - 걸러서 그리기
  </h3> <div className="output"> <ul>{/* TODO: 여기에 코드를 쓰세요 */}
  </ul> </div> </div>
  );
}

```

문제 11 (개념04 정렬)

menu 를 쓴 것부터 정렬해서 그리세요. 단, 원본 menu 의 순서는 바뀌면 안 됩니다.

기대 화면 정렬한 목록 - 삼각김밥 / 아메리카노 / 라떼 / 케이크
원본 확인 - 아메리카노 / 라떼 / 케이크 / 삼각김밥
아래 '원본 확인' 목록의 순서까지 바뀌면
복사하지 않고 원본을 정렬한 것입니다.

```
function Q11() {
  return (
    <div className="demo"> <h3>문제 11 – 원본을 지키며 정렬하기
  </h3> <div className="output"> <p>정렬한 목록</p> <ul>{/* TODO: 여기에 코드를 쓰세요
  */}</ul> <p>원본 확인</p> <ul>
    {menu.map((item) => (
      <li key={item.id}>{item.name}</li>
    ))}
  </ul> </div> </div>
  );
}
```

문제 12 (개념05 비었을 때와 0 함정)

cart 가 비었을 때 아래 두 가지가 되게 하세요. (1) “장바구니가 비었습니다” 안내 문구가 나온다 (2) 화면에 0 이 찍히지 않는다 아래 코드는 지금 0 이 찍힙니다. 두 곳을 고쳐야 합니다.

기대 화면 장바구니가 비었습니다
대괄호 안에 0 이 남아 있으면 && 왼쪽을 안 고친 것입니다.

```
function Q12() {
  const cart = [];

  return (
    <div className="demo"> <h3>문제 12 – 비었을 때 처리</h3> <div className="output">
      {/* TODO: 비었을 때 안내 문구가 나오게 하세요 */}

      {/* TODO: 아래 줄을 고쳐 0 이 안 나오게 하세요 */}
      <p>[{cart.length} && <span>담긴 상품이 있습니다</span>]</p> </div> </div>
    );
  }
}
```

문제 13 [응용] (개념04 + 개념02)

버튼 세 개로 목록을 갈아 끼우세요.

전체 · → menu 전부

판매중 · → soldOut 이 false 인 것만

품절 · → soldOut 이 `true` 인 것만

지금 눌린 버튼에는 `className="on"` 이 붙어 파랗게 보여야 합니다. `mode state` 는 이미 만들어 두었습니다.

기대 화면 처음에는 네 줄 모두, [전체] 버튼이 파랗습니다

판매중 · → 아메리카노 / 케이크 두 줄

품절 · → 라떼 / 삼각김밥 두 줄

버튼 색이 안 바뀌면 `className` 을 안 붙인 것입니다.
목록이 안 바뀌면 `shown` 을 안 쓰고 `menu` 를 그대로 그린 것입니다.

```
function Q13() {
  const [mode, setMode] = useState("all");
```

`mode` 에 따라 `shown` 을 정하세요

```
let shown = menu;

return (
  <div className="demo"> <h3>문제 13 [응용] - 필터 버튼</h3>

  {/* TODO: 버튼 세 개를 만드세요 */}

  <div className="output"> <ul>
    {shown.map((item) => (
      <li key={item.id}>{item.name}</li>
    ))}
  </ul> </div> </div>
);
}
```

문제 14 [도전] (개념02~05 전부)

아래 두 버튼을 한 화면에서 함께 동작하게 만드세요. 글자는 바뀌지 않습니다. ① [품절 감추기] — 누르면 품절 상품이 사라지고, 다시 누르면 돌아온다 ② [싼 것부터] — 누르면 가격순으로 정렬되고, 다시 누르면

원래 순서로 돌아온다 두 버튼 모두 지금 켜져 있으면 className="on" 이 붙어 파랗게 보여야 합니다. 그리고 아래 세 가지도 지켜야 합니다. ㉓ 지금 보이는 줄이 몇 개인지 "N개 보이는 중" 으로 보여 준다 ㉔ 보여 줄 것이 하나도 없으면 "조건에 맞는 메뉴가 없습니다" 를 대신 보여 준다 ㉕ 원본 menu 의 순서는 절대 바뀌면 안 된다

순서에 주의하세요. 거르는 것이 먼저이고 정렬이 나중입니다. (문제 13을 먼저 풀고 오세요. 버튼 만드는 방법이 같습니다)

기대 화면 처음 → 4개 보이는 중 · 아메리카노 / 라떼 / 케이크 / 삼각김밥
 품질 감추기 → 2개 보이는 중 · 아메리카노 - 4000원 / 케이크 - 6000원
 + 싼 것부터 → 2개 보이는 중 · 아메리카노 - 4000원 / 케이크 - 6000원
 품질 감추기 끄기 → 4개 보이는 중 ·
 삼각김밥 1200 / 아메리카노 4000 / 라떼 4500 / 케이크 6000
 두 버튼을 여러 번 번갈아 눌러도 결과가 같아야 합니다.

싼 것부터 · 를 꺾을 때 원래 순서가 안 돌아오면 원본을 정렬한 것입니다.

문제 11의 '원본 확인' 목록 순서까지 바뀌었다면 확실히 그렇습니다. ㉔를 확인하려면 menu 의 soldOut 을 전부 true 로 바꿔 보세요.

```
function Q14() {
  const [hideSoldOut, setHideSoldOut] = useState(false);
  const [sortByPrice, setSortByPrice] = useState(false);
```

hideSoldOut 과 sortByPrice 를 보고 shown 을 만드세요

```
let shown = menu;

return (
  <div className="demo"> <h3>문제 14 [도전] - 거르기 + 정렬 + 개수 + 빈 상태</h3>

  { /* TODO: 버튼 두 개를 만드세요 */ }

  <div className="output">
    { /* TODO: 개수와 목록을 보여 주세요. 비었으면 안내 문구를 대신 보여 주세요 */ }
  </div> </div>
);
```


문제 15 (에러 확인 - 맨 마지막)

아래 BadList 는 key 를 일부러 뺐습니다. F12 → Console 을 열고 [key 없이 그려 보기] 버튼을 누르세요.

화면에는 세 줄이 멀쩡히 나옵니다. 그런데 콘솔에는 무엇이 나올까요? 읽어 보고 아래 빈칸을 채워 보세요.

경고의 첫 줄은? _____ 어느 컴포넌트를 보라고 하나?

_____ 고치려면 무엇을 해야 하나?

그리고 하나 더: 버튼을 눌러 감췄다가 다시 눌러 보세요. 경고가 또 나오나요? 왜 그럴까요?

```
function BadList() {
  const names = ["아메리카노", "라떼", "케이크"];

  return (
    <ul>
      {names.map((name) => (
        <li>{name}</li>
      ))}
    </ul>
  );
}

function Q15() {
  const [show, setShow] = useState(false);

  return (
    <div className="demo"> <h3>문제 15 - 에러 확인</h3> <button onClick={() =>
setShow(!show)}>
      {show ? "감추기" : "key 없이 그려 보기"}
    </button> <div className="output">
      {show ? <BadList /> : <p>아직 안 그렸습니다. 콘솔도 깨끗합니다.</p>}
    </div> </div>
  );
}
```

전체 화면 그리기 (고치지 마세요) _____

```
export default function Exercises() {
  return (
    <div> <h1>05단원 연습문제 - 조건부 렌더링과 리스트</h1> <p className="guide">
      아래 TODO 자리에 코드를 쓰고 저장하면 화면이 바로 바뀝니다. 브라우저를
      <strong>새로고침</strong>하세요. <strong>F12 → Console</strong> 도 함께 보세요. <br
```

```
/> 1~12는 기본, 13은 [응용], 14는 [도전], 15는 에러 확인입니다.
```

```
<br /> <br />
```

각 문제의 컴포넌트는 이미 만들어져 있습니다. 지우지 말고 `<code>{"{"/ * TODO */{"}}</code>` 자리만 채우세요. 문제를 안 푼 상태에서는 회색 상자가 비어 있는 것이 정상입니다.

```
</p> <div> <Q1 /> <Q2 /> <Q3 /> <Q4 /> <Q5 /> <Q6 /> <Q7 /> <Q8 /> <Q9 /> <Q10 /> <Q11 /> <Q12 /> <Q13 /> <Q14 /> <Q15 /> </div> </div>
);
}
```

05단원 마무리 조건부 렌더링과 리스트

자주 넘어지는 자리

- 1 `key` 를 빼면 나는 경고를 직접 보여 주기 (버튼을 눌러야 나옵니다) |
- 2 `index` 를 `key` 로 쓰면 맨 앞 항목을 지웠을 때 입력값이 엉뚱한 줄에 남음 | 말로는 절대 이해 못 합니다
- 3 `{items.length && ...}` 가 빈 목록에서 화면에 0 을 찍음 |

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

01단원 연습문제 정답.jsx 정답 React 시작하기

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 눌러 보세요. F12 → Console 도 함께.

먼저 스스로 풀어 본 다음에 보세요. 답만 보는 것보다, 왜 그렇게 되는지 아래 설명을 같이 읽는 편이 남습니다.

```
import Summary from "../_ui/Summary.jsx";
import 그려진뒤 from "../_ui/그려진뒤.js";
```

문제 1

```
function Q1() {
  return <h3>안녕하세요</h3>;
}
```

화면 문제 1 상자에 "안녕하세요" 가 굵고 큰 글씨로 보입니다.

두 가지만 지키면 됩니다. - 이름이 대문자 Q 로 시작합니다. - `return` 으로 화면 설명을 돌려줍니다. `return` 을 빼면 조용히 아무것도 안 나옵니다.

문제 2

```
function Q2() {
  return <p>아메리카노 4000원</p>;
}
```

화면 문제 2 상자에 "아메리카노 4000원" 이 보입니다.

문제 3

```
function Q3() {
```

← q3 를 Q3 로 (첫 글자 대문자)

```
return <p>케이크 6000원</p>;
}
```

화면 문제 3 상자에 "케이크 6000원" 이 보입니다.

쓰는 곳도 함께 <Q3 /> 로 고쳐야 합니다. 한쪽만 고치면 그대로 빕니다.

왜 조용했을까요? React 는 태그 이름의 첫 글자로 구별합니다. 소문자로 시작 → 브라우저가 아는 HTML 태그로 봅니다 대문자로 시작 → 내가 만든 컴포넌트로 봅니다 q3 는 브라우저가 모르는 태그지만, 그래도 에러를 내지는 않습니다. 그냥 <q3></q3> 라는 빈 태그를 만들고 끝냅니다.

문제 4

```
function Q4가게이름() {
  return <h3>동네카페</h3>;
}

function Q4오늘메뉴() {
  return <p>오늘의 메뉴: 라떼</p>;
}

function Q4카드() {
```

className 은 화면 모양을 정하는 이름입니다. 02단원에서 배웁니다.

```
return (
  <div className="output"> <Q4가게이름 /> <Q4오늘메뉴 /> </div>
);
}
```

화면 문제 4 상자에 "동네카페" 와 "오늘의 메뉴: 라떼" 가 보입니다.

컴포넌트 안에서 다른 컴포넌트를 쓰는 것이 React 의 기본 조립 방식입니다. 03단원에서 이 이야기를 제대로 합니다.

문제 5

답: React 가 그림을 그릴 자리입니다.

index.html 이 브라우저에 도착한 순간 그 div 는 비어 있습니다. src/main.jsx 가 그 자리를 맡고 (createRoot), 무엇을 그릴지 넘깁니다(render). 그때부터 그 안은 React 가 통째로 관리합니다. 그래서 페이지 소스(Ctrl+U)에는 글자가 하나도 없는데 화면에는 잔뜩 보입니다.

문제 6

답: main.jsx 에 1번 / src 전체에도 1번입니다.

이유: createRoot 는 “이 자리를 내가 맡겠다” 는 선언입니다.

자리는 하나뿐이니 맡는 것도 한 번입니다.
화면이 아무리 많아져도, 13단원 종합 프로젝트에서도 한 번입니다.
예제 파일 안에 보이는 createRoot 는 전부 주석 속 설명입니다.

같은 자리에 두 번 부르면 이런 경고가 납니다.
You are calling ReactDOMClient.createRoot() on a container
that has already been passed to createRoot() before.

문제 7

```
그려진뒤("#q7", (자리) => {
  console.log("문제 7 상자 안 글자:", 자리.innerText);
```

F12 콘솔 문제 7 상자 안 글자: 여기는 문제 7 자리입니다

```
});
```

고치기 전에는 “(아직 없음)” 이 찍혔습니다.

render 는 “이렇게 그려 주세요” 라는 부탁일 뿐이고, 화면은 잠깐 뒤에 만들어집니다. 이 파일이 실행되는 시점에는 아직 그 자리가 없습니다. 그려진뒤 는 그 자리가 생길 때까지 잠깐씩 확인하다가 한 번 실행해 줍니다.

문제 8

답: ① 은 글자가 그대로 남고, ② 는 사라집니다.

왜: ① React 는 새 화면 설명과 직전 설명을 비교해 달라진 곳만 고칩니다.

바뀐 것은 숫자 하나뿐이라 그 글자만 고치고 입력칸은 건드리지 않습니다.
처음 만든 그 입력칸을 계속 씁니다.

② innerHTML 에 새 문자열을 넣으면 안에 있던 요소가 전부 버려집니다.

글자를 치던 그 입력칸은 사라지고 빈 입력칸이 새로 생깁니다.
커서·스크롤·체크 상태도 같이 날아갑니다.

문제 9

```
const q9목록 = [];

function Q9() {
  return (
    <div className="output">
      <p>담긴 개수: {q9목록.length}</p>
      {/* onClick 은 04단원에서 배웁니다 */}
      <button
        onClick={() => {
          q9목록.push("아메리카노");
          console.log("담긴 개수:", q9목록.length);

```

F12 콘솔 담긴 개수: 1

```
    })
  >
    담기
  </button>
</div>
);
}
```

화면(누르면): 숫자가 계속 0 입니다. 몇 번을 눌러도 그대로입니다.

이게 정답입니다. 고장이 아닙니다. 배열은 진짜로 늘어납니다. 콘솔의 숫자가 그 증거입니다. 다만 React 에게 “다시 그려라” 라고 말한 적이 없습니다. 그 말을 하는 방법이 04단원의 useState 입니다.

문제 10

```
function Q10카드() {
```

className — 02단원에서 배웁니다

```
  return (
    <div className="output"> <p>아메리카노 4000원</p> </div>
  );
}
```

화면 같은 카드가 세 개 위아래로 보입니다.

함수 하나를 만들어 두고 세 번 썼습니다. 복사해서 붙여 넣지 않았습니다. 카드 모양을 바꾸고 싶으면 Q10카드 한 곳만 고치면 세 개가 다 바뀝니다. 개념01에서 말한 “고칠 곳이 한 곳으로 모인다”가 이것입니다.

문제 11

```
let q11처음input = null;

그려진뒤("#q11", (자리) => {
  function 그리기() {
    자리.innerHTML = "<p>직접 그린 줄</p><p>입력칸: <input /></p>";
  }

  그리기(); // 첫 화면
  q11처음input = 자리.querySelector("input");

  setTimeout(() => {
    그리기(); // 같은 내용으로 한 번 더 const 지금input =
    자리.querySelector("input");
    console.log("문제 11 같은 입력칸인가?", 지금input === q11처음input);
  }, 1000);
});
```

F12 콘솔 문제 11 같은 입력칸인가? false

화면 입력칸에 글자를 쳐 두면 1초 뒤에 사라집니다.

내용이 똑같은데도 false 입니다. innerHTML 은 “같은지 비교” 를 하지 않습니다. 무조건 다 버리고 다시 만듭니다. React 는 반대로 “무엇이 달라졌나” 부터 봅니다. 그래서 true 가 나옵니다.

문제 12

(가) console.log(없는이름);

어디에: F12 → Console (빨간 줄). 브라우저에도 빨간 상자가 뜹니다. 메시지: Uncaught ReferenceError: 없는이름 is not defined → 문법은 맞습니다. 그래서 터미널은 조용합니다.

실행하다가 없는 이름을 만나서 터진 것이라 브라우저 쪽 일입니다.

(나) const 잘못 = React.createElement("h1", null, "안녕");

어디에: (가)와 같습니다. 콘솔과 빨간 상자. 메시지: Uncaught ReferenceError: React is not defined
 왜: JSX 만 쓸 때는 React 를 안 불러와도 됩니다. Vite 가 알아서 넣어 줍니다.

그런데 React 라는 이름을 손으로 적으면 그때는 불러와야 합니다.
`import React from "react";`

(다) `function` 망가짐({

답: 터미널이 멈춥니다. 브라우저에는 아무것도 안 바뀌거나 덮개가 씌워집니다.

괄호가 안 맞으면 파일을 읽는 단계에서 실패합니다.
 읽지도 못했으니 브라우저까지 갈 것이 없습니다.
 → (가)·(나)와 결정적으로 다른 점입니다.
 "브라우저는 조용한데 화면이 안 바뀐다" 면 터미널을 보세요.

이 연습문제에서 확인한 것

- 1 컴포넌트는 이름 첫 글자가 대문자(또는 한글)여야 합니다. 소문자면 에러 없이 화면만 빕니다.
- 2 `createRoot` 는 프로젝트 전체에 한 번, `src/main.jsx` 에만 있습니다. 우리 파일은 `export default` 만 내놓습니다.
- 3 `render` 는 부탁입니다. 부른 직후에 그 자리를 읽으면 아직 비어 있습니다.
- 4 React 는 달라진 곳만 고칩니다. `innerHTML` 은 안을 통째로 버리고 새로 만듭니다.
- 5 데이터만 바꾸면 화면은 안 바뀝니다. 다시 그리라고 말하는 방법이 04단원 `useState` 입니다.
- 6 문법이 틀리면 터미널, 실행 중에 터지면 브라우저입니다. 화면이 안 바뀌면 터미널부터 보세요.

```

export default function ExerciseAnswers() {
  return (
    <div>
      <h1>01단원 연습문제 정답 – React 시작하기</h1>

      <p className="guide">
        먼저 스스로 풀어 본 다음에 보세요. 직접 눌러 보면서{" "}
        <strong>F12 → Console</strong> 도 함께 확인하세요.
      <br />
      <br />
        5·6·8·12번은 코드가 아니라 <strong>말로 답하는 문제</strong>입니다. 위 주석에
        답과 이유를 적어 두었습니다.
      </p>

      <div className="demo">
        <h3>문제 1</h3>
        <div className="output">
          <Q1 />
        </div>

        <h3>문제 2</h3>
        <div className="output">
          <Q2 />
        </div>

        <h3>문제 3</h3>
        <div className="output">
          <Q3 />
        </div>

        <h3>문제 4</h3>
        <Q4카드 />

        <h3>문제 7</h3>
        <div className="output" id="q7">
          여기는 문제 7 자리입니다
        </div>
      </div>
    </div>
  )
}

```

```
        </div>

        <h3>문제 9</h3> <Q9 /> <h3>문제 10 - 응용</h3> <Q10카드 /> <Q10카드
/> <Q10카드 /> <h3>문제 11 - 도전</h3> <div className="output" id="q11" /> <h3>문제
12 - 에러 확인</h3> <p>화면에 그릴 것이 없습니다. 위 주석의 답을 읽으세요.</p>
        </div>

    </div>
  );
}
```

02단원 연습문제 정답.jsx 정답 JSX

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

준비물

```
import React from "react";
import Summary from "../_ui/Summary.jsx";
import * as Babel from "@babel/standalone";

const user = { name: "김민준", age: 20 };
const menu = ["아메리카노", "라떼", "케이크"];
const price = 4000;
const isOpen = true;
const stock = 0;
const homepage = "https://example.com";

function toJs(jsxCode) {
  return Babel.transform(jsxCode, {
    presets: [["react", { runtime: "classic" }]],
    generatorOpts: { jsescOption: { minimal: true }, concise: true },
  }).code;
}
```

문제 1

```
const title = <h2>오늘의 메뉴</h2>;

console.log(title.type);
```

F12 콘솔 h2

```
console.log(title.props);
```

F12 콘솔 { children: '오늘의 메뉴' }

JSX 는 값입니다. 그래서 변수에 담깁니다. 따옴표로 감싸면 그냥 문자열이 되어 `type` 이 `undefined` 가 됩니다.

문제 2

```
console.log(toJs("<p>라떼 4500원</p>"));
```

```
F12 콘솔    /*#__PURE__*/React.createElement("p", null, "라떼 4500원");
```

태그 이름이 첫 번째 인자, 속성이 두 번째(없으니 `null`), 태그 사이의 글자가 세 번째 인자로 들어갑니다.

문제 3

```
const answer3 = <p>{user.name}님 안녕하세요</p>;
```

```
console.log(answer3.props.children);
```

```
F12 콘솔    ['김민준', '님 안녕하세요']
```

중괄호에서 꺼낸 값과 그냥 글자가 배열로 나뉘어 담깁니다. 중괄호를 빼면 `children` 이 통째로 'user.name님 안녕하세요' 라는 문자열이 됩니다.

문제 4

```
const answer4 = <p>아메리카노 두 잔은 {price * 2}원입니다</p>;
```

```
console.log(answer4.props.children);
```

```
F12 콘솔    ['아메리카노 두 잔은 ', 8000, '원입니다']
```

8000 에 따옴표가 없습니다. 계산이 먼저 끝나서 숫자로 들어갔기 때문입니다. 8000 을 직접 적으면 `price` 가 바뀌었을 때 화면이 따라오지 않습니다.

문제 5

```
const answer5 = <p>{user.name}님은 {user.age >= 19 ? "성인" : "미성년"}입니다</p>;
```

```
console.log(user.age >= 19 ? "성인" : "미성년");
```

```
F12 콘솔    성인
```

중괄호 안에는 '식' 만 들어갑니다. `if` 문은 값을 남기지 않아 못 들어갑니다. 삼항 연산자는 값 하나가 남으므로 들어갑니다.

if 문을 꼭 쓰고 싶으면 중괄호 밖에서 미리 변수에 담아 두면 됩니다.

```
let grade;
if (user.age >= 19) {
  grade = "성인";
} else {
  grade = "미성년";
}
console.log(grade);
```

F12 콘솔 성인

이 방법도 정답입니다. <p>{user.name}님은 {grade}입니다</p> 로 쓰면 됩니다.

문제 6

```
const answer6 = (
  <p>
    메뉴 {menu.length}개: {menu.join(", ")}
  </p>
);

console.log(menu.length);
```

F12 콘솔 3

```
console.log(menu.join(", "));
```

F12 콘솔 아메리카노, 라떼, 케이크

배열을 그냥 {menu} 로 넣으면 사이에 아무것도 안 넣고 이어 붙입니다. “아메리카노라떼케이크” 가 되므로 join 으로 사이를 정해 줘야 합니다.

문제 7

```
const answer7 = <p className="output on">파란 상자</p>;

console.log(answer7.props.className);
```

F12 콘솔 output on

class 는 자바스크립트 예약어라 className 을 씁니다. 클래스 두 개는 className 을 두 번 쓰지 않고 한 문자열에 띄어쓰기로 이어 씁니다.

문제 8

```
const answer8 = (
  <p style={{ color: "#c00", backgroundColor: "#fff8dc" }}>노란 바탕에 빨간 글자</p>
);

console.log(answer8.props.style);
```

```
F12 콘솔    { color: '#c00', backgroundColor: '#fff8dc' }
```

중괄호가 두 개인 이유입니다. 바깥 중괄호 — “여기는 자바스크립트다” 안쪽 중괄호 — “이건 객체다” 객체를 미리 만들어 두면 중괄호가 하나로 줄어듭니다. 이것도 정답입니다.

```
const answer8Style = { color: "#c00", backgroundColor: "#fff8dc" };
console.log(answer8Style.backgroundColor);
```

```
F12 콘솔    #fff8dc
```

```
<p style={answer8Style}>노란 바탕에 빨간 글자</p>
```

문제 9

```
const answer9 = <a href={homepage}>홈페이지로 가기</a>;

console.log(answer9.props.href);
```

```
F12 콘솔    https://example.com
```

따옴표를 같이 쓰면(href="{homepage}") "{homepage}" 라는 글자가 주소가 됩니다. 따옴표와 중괄호는 둘 중 하나만 씁니다.

문제 10

```
const answer10 = <button disabled={stock === 0}>담기</button>;

console.log(answer10.props.disabled);
```

```
F12 콘솔    true
```

stock 이 0 이므로 stock === 0 이 true 가 되어 버튼이 잠깁니다. disabled="false" 라고 쓰면 안 됩니다. 글자 "false" 는 참으로 취급됩니다.

```
console.log(Boolean("false"));
```

F12 콘솔 true

문제 11

```
const answer11 = (  
  <> <h4>아메리카노</h4> <p>4000원</p> </>  
);  
  
console.log(answer11.type === React.Fragment);
```

F12 콘솔 true

두 태그를 나란히 두면 [SyntaxError] 가 납니다. 하나로 묶어야 합니다. <div> 로 감싸도 화면은 같지만 실제 화면에 div 가 하나 남습니다. "감싸는 태그가 남으면 안 된다" 는 조건이 있었으니 Fragment 가 답입니다.

문제 12 [응용]

```
const answer12 = (  
  <div className="output"> <h4>{user.name}님의 주문</h4> <p>아메리카노  
{price.toLocaleString()}원</p> <p>세 잔 합계 {(price * 3).toLocaleString()}원  
</p> </div>  
);  
  
console.log(price.toLocaleString());
```

F12 콘솔 4,000

```
console.log((price * 3).toLocaleString());
```

F12 콘솔 12,000

여러 줄이라 소괄호로 감쌌습니다. 소괄호가 없으면 const 는 동작하지만 이 모양 그대로 return 뒤에 쓰면 undefined 가 됩니다. (개념05)

toLocaleString 은 숫자에 세 자리마다 쉼표를 넣어 줍니다. 숫자 하나에 바로 붙일 때는 (4000).toLocaleString() 처럼 소괄호가 필요합니다. 4000.toLocaleString() 은 소수점으로 오해받아 에러가 납니다.

문제 13 [도전]

```
const answer13 = (
  <> <p style={{ color: isOpen ? "green" : "gray" }}>{isOpen ? "영업 중" : "준비 중"}
</p> <p>{menu.length}종류를 팝니다</p> </>
);

console.log(isOpen ? "green" : "gray");
```

F12 콘솔 green

```
console.log(answer13.props.children[0].props.style);
```

F12 콘솔 { color: 'green' }

이 문제가 어려운 이유는 삼항 연산자가 두 군데에 들어가기 때문입니다. 글자 자리 — 태그 사이의 종괄호 안
색깔 자리 — style 객체의 값 자리

둘 다 '값이 하나 남는 자리' 라서 삼항 연산자가 들어갑니다. style 안쪽은 그냥 객체이므로, 객체의 값
자리에 무엇이든 넣을 수 있습니다.

감싸는 태그가 남으면 안 되므로 <> </> 를 썼습니다.

문제 14 (에러 확인)

```
const badLine = <p>{user}</p>;
ReactDOM.createRoot(document.querySelector("#root")).render(badLine);
```

콘솔의 빨간 글씨 첫 줄: Objects are not valid as a React child (found: object with keys {name, age}). If you meant to render a collection of children, use an array instead.

왜 이렇게 되나: 종괄호 안에 넣은 user 는 객체입니다. React 는 객체를 화면에 그릴 방법이 없습니다.
객체를 글자로 바꾸면 아래처럼 되기 때문입니다.

```
console.log(String(user));
```

F12 콘솔 [object Object]

화면에 [object Object] 라고 나와도 아무 도움이 안 됩니다. 그래서 React 는 조용히 이상하게 그리는 대신
에러를 내서 알려 줍니다.

고치는 방법: (1) 필요한 속성만 꺼내서 넣습니다 — 대부분 이쪽입니다.

```
<p>{user.name}</p>
```

(2) 통째로 보고 싶으면 글자로 만들어서 넣습니다.

```
<p>{JSON.stringify(user)}</p>
```

```
console.log(user.name);
```

```
F12 콘솔    김민준
```

```
console.log(JSON.stringify(user));
```

```
F12 콘솔    {"name":"김민준","age":20}
```

배열은 에러가 안 납니다. 하나씩 이어서 그림니다. 그래서 에러 문구가 “여러 개를 그리려던 거면 배열을 쓰세요” 라고 알려 준 것입니다.

화면에 늘어놓기

```
function App() {  
  return (  
    <div>  
      <div className="output">  
        <h4>문제 3</h4>  
        {answer3}  
      </div>  
      <div className="output">  
        <h4>문제 4</h4>  
        {answer4}  
      </div>  
      <div className="output">  
        <h4>문제 5</h4>  
        {answer5}  
      </div>  
      <div className="output">  
        <h4>문제 6</h4>  
        {answer6}  
      </div>  
      <div className="output">  
        <h4>문제 7</h4>  
        {answer7}  
      </div>  
      <div className="output">  
        <h4>문제 8</h4>  
        {answer8}  
      </div>  
      <div className="output">  
        <h4>문제 9</h4>  
        {answer9}  
      </div>  
      <div className="output">  
        <h4>문제 10</h4>  
        {answer10}  
      </div>  
      <div className="output">
```

```

    <h4>문제 11</h4>
    {answer11}
  </div>
  <div className="output"> <h4>문제 12 [응용]</h4>
    {answer12}
  </div> <div className="output"> <h4>문제 13 [도전]</h4>
    {answer13}
  </div>
</div>
);
}

```

화면 문제 3부터 13까지의 답이 흰 상자에 하나씩 담겨 나옵니다.
 문제 7은 파란 상자, 문제 8은 노란 바탕에 빨간 글자,
 문제 10의 답기 버튼은 회색으로 잠겨 있고, 문제 13은 초록 글씨입니다.

이 연습문제에서 확인한 것

- 1 JSX 는 값입니다. 변수에 담기고 `type:props` 를 가집니다.
- 2 값을 넣을 때는 중괄호를 씁니다. 중괄호 안에는 식만 들어갑니다.
- 3 `class` 는 `className`, CSS 이름은 카멜케이스, `style` 에는 객체를 넣습니다.
- 4 속성 값에 변수를 넣을 때는 중괄호만 씁니다. 따옴표와 같이 쓰면 글자가 됩니다.
- 5 태그 두 개는 하나로 감쌉니다. 화면에 남기기 싫으면 `<>...</>` 를 씁니다.
- 6 여러 줄 JSX 는 소괄호로 감쌉니다. `return` 뒤에서 줄을 바꾸지 않습니다.
- 7 객체는 화면에 못 넣습니다. 속성을 꺼내거나 글자로 바꿔서 넣습니다.

```

export default function ExerciseAnswers() {
  return (
    <div> <h1>02단원 연습문제 정답 - JSX</h1> <p className="guide">
      먼저 스스로 풀어 본 다음에 보세요. 화면과 함께 <strong>F12 → Console</strong>
      도 확인하세요.
    </p> <div className="demo"> <h3>정답이 만든 화면</h3> <div id="root"> <App
  /> </div> </div> </div>
  );
}

```

03단원 연습문제 정답.jsx 정답 컴포넌트와 props

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

문제 1

```
function Hello() {
  return <p>안녕하세요, 김민준입니다</p>;
}
```

컴포넌트는 화면을 돌려주는 함수입니다. 새 문법은 없습니다. 이름을 `hello` 라고 쓰면 React 가 HTML 태그로 읽어서 화면에서 사라집니다.

문제 2

원래 코드는 이랬습니다.

```
function header() { ... }      ← 소문자로 시작
<header />                    ← 쓰는 쪽도 소문자
```

`header` 는 실제로 있는 HTML 태그 이름입니다. 그래서 React 는 “빈 `<header>` 태그를 만들라” 로 읽었고, 우리가 만든 함수는 한 번도 불리지 않았습니니다. 에러도 안 났습니니다.

이름을 대문자로 시작하게 바꾸고, 쓰는 쪽도 함께 바꿉니다.

```
function ShopTitle() {
  return <h4>동네 카페</h4>;
}
```

문제 3

```
function Coffee() {
  return <p>아메리카노</p>;
}
```

```
function Cake() {
  return <p>케이크</p>;
}

function MenuList() {
  return (
    <div> <Coffee /> <Cake /> </div>
  );
}
```

돌려주는 것은 항상 하나여야 하므로 <div> 로 감쌌습니다. 02단원에서 배운 <> </> 로 감싸도 똑같이 동작합니다.

```
return (
  <> <Coffee /> <Cake /> </>
);
```

문제 4

```
function Welcome(props) {
  return <p>{props.name}님 환영합니다</p>;
}
```

넘기는 쪽은 <Welcome name="이서연" /> 입니다. 넘긴 것이 { name: '이서연' } 이라는 객체 하나가 되어 props 로 옵니다. 개념03을 배웠으니 function Welcome({ name }) 으로 써도 됩니다.

문제 5

```
function PriceTag(props) {
  return (
    <div>
      {props.price}원
      <br />
      500원 더하면 {props.price + 500}원
    </div>
  );
}
```

price={4000} 처럼 중괄호로 넘겨야 숫자입니다.

price="4000" 으로 넘기면 화면 첫 줄은 똑같이 4000원으로 보이지만 둘째 줄이 4000500원이 됩니다. 문자열끼리 이어 붙었기 때문입니다.

문제 6·7

문제 6 · props 로 받은 모습입니다.

```
function MenuItem(props) {
  return (
    <p>
      {props.name} - {props.price}원
    </p>
  );
}
```

문제 7 · 매개변수 자리에서 구조분해한 모습입니다. 아래가 최종 답입니다.

```
function MenuItem({ name, price }) {
  return (
    <p>
      {name} - {price}원
    </p>
  );
}
```

넘기는 쪽은 한 글자도 안 바뀝니다. 받는 쪽 괄호 안만 바뀝니다. 구조분해한 뒤에도 props.name 을 쓰면 ReferenceError: props is not defined 가 납니다. { name } 이라고 쓰면 props 라는 이름 자체가 만들어지지 않기 때문입니다.

문제 8

```
function Greeting({ name = "손님" }) {
  return <p>{name}님, 어서 오세요.</p>;
}
```

기본값은 name 이 undefined 일 때만 쓰입니다. (JS자료 09단원 개념02) <Greeting name="" /> 처럼 빈 문자열을 넘기면 기본값이 안 쓰이고 "님, 어서 오세요." 가 됩니다. 안 넘긴 것처럼 보여서 헷갈리기 쉽습니다.

문제 9

```
function Box({ children }) {  
  return <div className="output">{children}</div>;  
}
```

태그 사이에 넣은 "오늘도 좋은 하루" 가 children 으로 들어옵니다. {children} 을 안 쓰면 흰 칸만 나오고 글자는 사라집니다. 에러는 안 납니다.

문제 10

```
function TitleBox({ title, children }) {  
  return (  
    <div className="output"> <h4>{title}</h4>  
      {children}  
    </div>  
  );  
}
```

짧은 값(title)은 속성으로, 화면 덩어리는 children 으로 받았습니다. 둘 다 결국 props 입니다. `Object.keys(props)` 를 찍어 보면 ['title', 'children'] 입니다.

문제 11

```
function NoticeLine({ text }) {  
  return (  
    <div> <strong>[알림]</strong> {text}  
    </div>  
  );  
}  
  
function BigNotice() {  
  return (  
    <div className="output"> <NoticeLine text="오전 8시에 문을 엽니다"  
/> <NoticeLine text="오후 10시에 문을 닫습니다" /> <NoticeLine text="케이크는  
6000원입니다" /> </div>  
  );  
}
```

세 줄에서 달라지는 것은 글자 하나뿐이므로 그것만 props 로 뺐습니다.

알림 · 표시는 세 줄이 모두 같으므로 NoticeLine 안에 그대로 뒀습니다.

이제 [알림] 을 [공지] 로 바꾸려면 NoticeLine 한 곳만 고치면 됩니다.

props 이름을 text 대신 message 나 content 라고 지어도 맞습니다.

문제 12 [응용]

```
const americano = { name: "아메리카노", price: 4000 };
const latte = { name: "라떼", price: 4500 };

function OrderCard({ item, count = 1 }) {
  const { name, price } = item; // 받은 객체에서 다시 꺼냅니다
  const total = price * count; // props 를 읽어서 새 값을 만듭니다
  return (
    <div className="output">
      {name} {count}개 - 합계 {total}원
    </div>
  );
}
```

객체를 통째로 넘길 때는 <OrderCard item={americano} /> 처럼 중괄호가 필요합니다.

item="americano" 라고 쓰면 글자가 넘어가서 name 과 price 가 undefined 가 됩니다.

total 은 props 를 '읽어서' 새로 만든 값입니다. props 자체를 고친 것이 아닙니다. item.price = ... 처럼 안쪽 값을 고치면 에러 없이 원본까지 바뀝니다. (개념05)

매개변수 자리에서 한 번에 꺼낼 수도 있습니다. 되지만 읽기는 조금 어렵습니다.

```
function OrderCard({ item: { name, price }, count = 1 }) { ... }
```

문제 13 [도전]

```
function Receipt({ title, total = 0, children }) {
  return (
    <div className="output"> <h4>{title}</h4>
      {children}
    <div> <strong>합계 {total}원</strong> </div> </div>
  );
}
```

쓰는 쪽은 이렇게 됩니다.

```
<Receipt title="영수증" total={16500}>
  <OrderCard item={americano} count={3} /> <OrderCard item={latte} />
</Receipt>
```

여기서 children 은 OrderCard 두 개입니다. 두 개라서 배열로 들어옵니다. 그래도 {children} 한 줄이면 React 가 알아서 둘 다 그려 줍니다.

상자(Receipt) 는 안에 무엇이 들어오는지 모릅니다. 제목과 합계, 테두리만 담당합니다. 그래서 주문 목록이 아니라 다른 내용을 넣어도 그대로 동작합니다.

합계 16500 은 12000 + 4500 을 손으로 계산해 넘긴 값입니다. 목록에서 합계를 자동으로 구하는 것은 05단원(map)과 07단원에서 배웁니다.

문제 14 (에러 확인)

```
function ChangeProps(props) {
  props.name = "바꿈";
  return <p>{props.name}</p>;
}
```

주석을 풀면 이렇게 됩니다.

화면 이 파일의 화면이 통째로 빡니다. 문제 14 칸만 비는 것이 아닙니다.

React 는 그리는 도중에 에러가 나면 화면 전체를 지워 버립니다.

F12 콘솔 TypeError: Cannot assign to read only property 'name' of object '#<Object>'

그 아래에 "The above error occurred in the <ChangeProps> component" 도 함께 나옵니다.

왜 이렇게 되냐: props 는 부모가 자식에게 준 값입니다. 자식이 마음대로 고치면 부모가 아는 값과 화면에 보이는 값이 달라지고, 누가 고쳤는지 찾을 수 없습니다. 그래서 React 는 개발 중에 props 객체를 아예 못 고치게 잠가 둡니다. 잠긴 값에 대입하려고 하면 위 TypeError 가 납니다.

1 그냥 읽어서 새 변수를 만든다 — **const** newName = props.name + "님";

2 값이 바뀌어야 하는 화면이라면 04단원의 state 를 씁니다.

★ props 안에 든 '객체' 를 고치는 것은 에러조차 나지 않습니다. props.item.price = 0 은 조용히 통과하고 원본 객체까지 바꿔 버립니다. 이쪽이 더 위험합니다. 개념05 화면 ㉔ 에서 확인했습니다.

화면에 그리기

```

export default function ExerciseAnswers() {
  return (
    <div>
      <h1>03단원 연습문제 정답 - 컴포넌트와 props</h1>

      <p className="guide">
        먼저 스스로 풀어 본 다음에 보세요. 화면과 <strong>F12 → Console</strong> 을
        함께 확인하세요.
      <br />
      <br />
      정답이 하나뿐인 문제는 거의 없습니다. 화면이 기대 결과와 같다면 맞은
      것입니다. 아래 풀이는 이 단원에서 배운 방법으로 쓴 것입니다.
    </p>

    <>
      <div className="demo">
        <h3>문제 1</h3>
        <div className="output">
          <Hello />
        </div>
      </div>

      <div className="demo">
        <h3>문제 2</h3>
        <div className="output">
          <ShopTitle />
        </div>
      </div>

      <div className="demo">
        <h3>문제 3</h3>
        <div className="output">
          <MenuList />
        </div>
      </div>

      <div className="demo">
        <h3>문제 4</h3>

```

```

        <div className="output">
            <Welcome name="이서연" />
        </div>
    </div>

    <div className="demo"> <h3>문제
5</h3> <div className="output"> <PriceTag price={4000}
/> </div> </div> <div className="demo"> <h3>문제 6 .
7</h3> <div className="output"> <MenuItem name="아메리카노" price={4000}
/> <MenuItem name="라떼" price={4500} /> <MenuItem name="케이크" price={6000}
/> </div> </div> <div className="demo"> <h3>문제
8</h3> <div className="output"> <Greeting name="박지훈" /> <Greeting
/> </div> </div> <div className="demo"> <h3>문제 9</h3> <Box>오늘도 좋은 하루
</Box> </div> <div className="demo"> <h3>문제 10</h3>

        <TitleBox title="오늘의 메뉴">
            <p>아메리카노 4000원</p>
        </TitleBox>
    </div>

    <div className="demo"> <h3>문제 11</h3> <BigNotice
/> </div> <div className="demo"> <h3>문제 12 [응용]</h3> <OrderCard item=
{americano} count={3} /> <OrderCard item={latte}
/> </div> <div className="demo"> <h3>문제 13 [도전]
</h3> <Receipt title="영수증" total={16500}> <OrderCard item={americano} count={3}
/> <OrderCard item={latte} /> </Receipt> </div> <div className="demo"> <h3>문제 14
(에러 확인)</h3> <div className="output">{ /* <ChangeProps name="김민준" /> */}
</div> </div>

    </>
</div>
);
}

```

04단원 연습문제 정답.jsx 정답 state와 이벤트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

직접 눌러 보세요. F12 → Console 도 함께.

문제 1

```
import { useState } from "react";
import Summary from "../_ui/Summary.jsx";

function Q1() {
  function handleClick() {
    console.log("안녕하세요, 김민준님!");
  }
}
```

F12 콘솔 안녕하세요, 김민준님!

```
    }

    return (
      <div className="demo"> <h3>문제 1 – 버튼에 이벤트 붙이기
</h3> <button id="q1Btn" onClick={handleClick}>
      인사하기
    </button> </div>
    );
  }
}
```

`onClick={handleClick}` 입니다. 괄호를 붙이면 안 됩니다.
`onClick={handleClick()}` 로 쓰면 화면을 그릴 때 한 번 실행되고,

그 결과인 `undefined` 가 `onClick` 에 들어가 버튼이 아무 반응도 안 합니다.

문제 2

```
function Q2() {
  function greet(name) {
    console.log(`${name}님 반갑습니다`);
  }
}
```

F12 콘솔 김민준님 반갑습니다
 이서연님 반갑습니다

```

    }

    return (
      <div className="demo"> <h3>문제 2 - 인자 넘기기</h3> <button id="q2BtnA" onClick=
{() => greet("김민준")}>
        김민준
      </button> <button id="q2BtnB" onClick={() => greet("이서연")}>
        이서연
      </button> </div>
    );
  }

```

값을 넘겨야 하니 괄호가 필요합니다. 그래서 화살표 함수로 한 겹 감쌌습니다. onClick 이 받는 것은 화살표 함수이고, greet("김민준") 은 그 안에 들어 있어 누를 때 비로소 실행됩니다.

onClick={greet("김민준")} 로 쓰면 페이지를 여는 순간 두 줄이 다 찍힙니다.

문제 3

```

function Q3() {
  const [count, setCount] = useState(0);

  return (
    <div className="demo"> <h3>문제 3 - 카운터 만들기
</h3> <div className="output" id="q3Out">
      담은 개수: {count}
    </div> <button id="q3Btn" onClick={() => setCount(count + 1)}>
      +1
    </button> </div>
  );
}

```

화면(누르면): 담은 개수: 1

let count = 0 으로 만들고 count++ 를 했다면 콘솔의 값만 올라가고 화면은 0에서 멈춥니다. 화면을 다시 그리라고 알려 주는 것이 set 함수입니다.

문제 4

```

function Q4() {
  const [cake, setCake] = useState(6);

  return (

```

05단원 연습문제 정답.jsx 정답 조건부 렌더링과 리스트

RUN 실습프로젝트에서 `npm run dev` → 왼쪽 목록에서 이 예제를 고르세요.

F12 → Console 도 함께 보세요.

```
import { useState } from "react";

console.log("정답 파일을 열었습니다. 콘솔은 아직 깨끗합니다.");
```

F12 콘솔 정답 파일을 열었습니다. 콘솔은 아직 깨끗합니다.

문제에서 함께 쓰는 데이터

```
const menu = [
  { id: "americano", name: "아메리카노", price: 4000, soldOut: false },
  { id: "latte", name: "라떼", price: 4500, soldOut: true },
  { id: "cake", name: "케이크", price: 6000, soldOut: false },
  { id: "gimbap", name: "삼각김밥", price: 1200, soldOut: true },
];

const users = [
  { id: 101, name: "김민준", age: 20 },
  { id: 102, name: "이서연", age: 22 },
  { id: 103, name: "박지훈", age: 28 },
];

const todos = [
  { id: 1, text: "우유 사기", done: true },
  { id: 2, text: "책 반납", done: false },
  { id: 3, text: "청소하기", done: false },
];
```

문제 1

```
function Q1() {
  const isOpen = true;

  return (
    <div className="demo"> <h3>문제 1 – 삼항 연산자</h3> <div className="output">
      {isOpen ? <p>영업 중입니다</p> : <p>준비 중입니다</p>}
    </div> </div>
  );
}
```

화면 영업 중입니다

JSX 중괄호 안에는 값만 들어갑니다. if 는 문장이라 못 씁니다. 삼항은 둘 중 하나를 '값' 으로 돌려주므로 들어갈 수 있습니다. {isOpen ? "영업 중입니다" : "준비 중입니다"} 처럼 글자만 골라도 맞습니다.

문제 2

```
function Q2() {
  const newCount = 3;

  return (
    <div className="demo"> <h3>문제 2 – && 로 있을 때만 보이기
  </h3> <div className="output">
    {newCount > 0 && <p>새 알림 {newCount}개</p>}
  </div> </div>
  );
}
```

화면 새 알림 3개

★ 왼쪽이 newCount 가 아니라 newCount > 0 인 것이 핵심입니다. {newCount && ...} 라고 쓰면 0일 때 화면에 0 이 찍힙니다(개념05). && 는 왼쪽이 거짓이면 '왼쪽 값' 을 그대로 돌려주고, React 는 숫자 0 을 화면에 그리기 때문입니다.

문제 3

```
function Q3() {
  const point = 70;
  let badge;
```


부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.